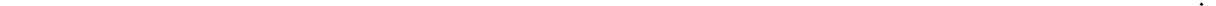


جزوه جامع درس علوم شناختی محاسباتی

از نظریه‌های شناختی تا مدل‌های محاسباتی زبان و گفتار

تهیه شده بر اساس رونوشت جلسات کلاس

بهار ۱۴۰۵



فهرست مطالب

۱	اول مبانی علوم شناختی محاسباتی
۳	۱ چارچوب درس و مسئله اصلی
۳	۱.۱ علوم شناختی چیست؟
۳	۲.۱ از علوم شناختی تا علوم شناختی محاسباتی
۴	۳.۱ نسبت علوم شناختی محاسباتی با هوش مصنوعی
۵	۴.۱ تفاوت پرسش شناختی و پرسش یادگیری ماشین
۵	۵.۱ نظریه شناختی، مدل محاسباتی و اعتبارسنجی
۶	۶.۱ مغز، نواحی کارکردی و احتیاط در تفسیر
۶	۷.۱ کاربرد علوم شناختی و محدوده این درس
۶	۸.۱ شکست مدل های بزرگ در مهارت های شناختی
۷	۹.۱ تغییرات Adversarial و Counterfactual
۷	۱۰.۱ نقش داده انسانی و همبستگی خطا
۸	۱۱.۱ مسیر پژوهشی درس
۹	۲ نظریه های کلاسیک پردازش گفتار
۹	۱.۲ تشخیص کلمه گفتاری
۹	۲.۲ Cohort Model
۱۰	۱.۲.۲ فعال سازی نامزدهای واژگانی
۱۰	۲.۲.۲ پردازش پایین به بالا
۱۰	۳.۲.۲ محدودیت های مدل
۱۰	۳.۲ TRACE Model
۱۱	۱.۳.۲ تعامل داده صوتی و زمینه

۱۱	 پردازش بالابه پایین	۲.۳.۲
۱۱	 مقایسه با Cohort	۳.۳.۲
۱۱	 اعتبارسنجی نظریه‌های شناختی	۴.۲

دوم معماری شناختی، حافظه و بار شناختی ۱۳

۱۵	 معماری شناختی	۳
۱۵	 چرا معماری شناختی لازم است؟	۱.۳
۱۵	 مثال جارو رباتیک	۲.۳
۱۶	 راه حل اول: دوری از اشیای پرریسک	۱.۲.۳
۱۶	 راه حل دوم: تعریف تابع ریسک	۲.۲.۳
۱۶	 راه حل سوم: یادگیری از تجربه و بازخورد	۳.۲.۳
۱۶	 دو مسیر برای بهبود مدل	۴.۲.۳
۱۷	 مازول‌های اصلی در یک معماری شناختی	۳.۳
۱۷	 ادراک و مازول بینایی	۱.۳.۳
۱۷	 حافظه و بافرهای کاری	۲.۳.۳
۱۷	 تصمیم‌گیری و انتخاب عمل	۳.۳.۳
۱۸	 مازول حرکتی	۴.۳.۳
۱۸	 نگاشت معماری شناختی به مغز	۴.۳
۱۸	 ACT-R	۵.۳
۱۹	 حافظه اعلانی	۱.۵.۳
۲۰	 حافظه رویه‌ای	۲.۵.۳
۲۰	 Pattern Matching	۳.۵.۳
۲۰	 Production Execution	۴.۵.۳
۲۱	 اعتبارسنجی معماری شناختی	۶.۳

۴ حافظه در انسان و مدل ۲۳

۲۳	 حافظه حسی، کوتاه‌مدت و کاری	۱.۴
۲۳	 حافظه بلندمدت	۲.۴
۲۴	 حافظه اعلانی	۱.۲.۴

۲۴	حافظه رویه‌ای	۲.۲.۴
۲۵	انتقال اطلاعات از حافظه کوتاه مدت به بلندمدت	۳.۴
۲۵	احساس شدید	۱.۳.۴
۲۵	تعجب و غیرمنتظره بودن	۲.۳.۴
۲۵	تمایز و برجستگی	۳.۳.۴
۲۵	علامت گذاری و ارتباط سازی	۴.۳.۴
۲۶	اختلالات و شواهد شناختی	۴.۴
۲۶	استرس و بازیابی حافظه	۱.۴.۴
۲۶	پارکینسون و حافظه رویه‌ای	۲.۴.۴
۲۶	حافظه در مدل های زبانی	۵.۴
۲۷	پارامترها به عنوان حافظه بلندمدت	۱.۵.۴
۲۷	Prompt و Context Window به عنوان حافظه کاری	۲.۵.۴
۲۷	RAG به عنوان حافظه بیرونی	۳.۵.۴
۲۹	۵ نظریه بار شناختی و حافظه کاری	
۲۹	Cognitive Load Theory	۱.۵
۲۹	ظرفیت محدود حافظه کاری	۲.۵
۳۰	عدد 2 ± 7	۳.۵
۳۰	Chunking	۴.۵
۳۰	تعریف Chunk	۱.۴.۵
۳۱	نقش دانش قبلی در Chunking	۲.۴.۵
۳۱	تفاوت های فردی	۳.۴.۵
۳۱	کاربردهای Chunking	۵.۵
۳۱	Multi-Hop Reasoning به عنوان آزمون حافظه کاری	۶.۵
۳۲	محدودیت حافظه کاری در مدل های زبانی	۷.۵
۳۲	Context Saturation	۱.۷.۵
۳۲	Attentional Residue	۲.۷.۵
۳۳	تفاوت دو نوع فشار	۳.۷.۵
۳۳	طراحی آزمایش برای سنجش بار شناختی مدل	۸.۵

۳۳	تکامل آزمون‌های هوش مصنوعی	۹.۵
۳۴	راهکارهای شناختی	۱۰.۵
۳۴	Divide and Conquer	۱.۱۰.۵
۳۴	Sub-goal و مدیریت سلسله‌مراتبی حافظه	۲.۱۰.۵
۳۵	مثال Blocks World	۳.۱۰.۵
۳۵	مثال Gripper	۴.۱۰.۵
۳۶	محدودیت‌های Chunking	۵.۱۰.۵
۳۷	ترنسفورمر، توجه و حافظه کاری	۶
۳۷	حافظه در Transformer	۱.۶
۳۷	Contextual Embedding	۲.۶
۳۸	حل ارجاع	۱.۲.۶
۳۸	رفع ابهام معنای واژه	۲.۲.۶
۳۸	Value و Key، Query	۳.۶
۳۹	Self-Attention از دید حافظه کاری	۴.۶
۳۹	آیا افزایش طول Context کافی است؟	۵.۶
۳۹	کاهش یا حذف توجه به اطلاعات کم‌اهمیت	۶.۶
۴۰	Local Attention	۷.۶
۴۰	ترکیب پنجره‌های محلی	۱.۷.۶
۴۱	Mixture of Experts و Multi-Head Attention	۸.۶
۴۱	جمع‌بندی	۹.۶
۴۳	سوم سوگیری، پیش‌بینی و توجه انتخابی	
۴۵	سوگیری شناختی	۷
۴۵	تعریف Cognitive Bias	۱.۷
۴۶	نمونه‌های اصلی سوگیری	۲.۷
۴۶	Confirmation Bias	۱.۲.۷
۴۶	Anchoring Bias	۲.۲.۷

۴۶	Halo Effect	۳.۲.۷
۴۷	آیا سوگیری همیشه بد است؟	۳.۷
۴۷	تبیین سوگیری با نظریهٔ بار شناختی	۴.۷
۴۷	تصمیم‌گیری Greedy	۱.۴.۷
۴۸	تعمیم از حافظهٔ بلندمدت	۲.۴.۷
۴۸	کاهش سوگیری با کنترل بار شناختی	۳.۴.۷
۴۸	تبیین سوگیری با نظریهٔ دوفراپندگی	۵.۷
۴۹	System 1	۱.۵.۷
۴۹	System 2	۲.۵.۷
۴۹	فعال‌سازی رفتار شبیه System 2 در مدل‌های زبانی	۳.۵.۷
۴۹	Self-Critique و Tree of Thought، Chain of Thought	۶.۷
۵۱	Predictive Coding و مدل بیزی ذهن	۸
۵۱	مغز به‌عنوان سیستم پیش‌بین	۱.۸
۵۲	Prediction Error	۲.۸
۵۲	Posterior و Likelihood، Prior	۳.۸
۵۳	مثال کارآگاه	۱.۳.۸
۵۳	Precision	۴.۸
۵۳	Precision Prior	۱.۴.۸
۵۳	مثال رسانه و تخصص	۲.۴.۸
۵۴	Precision Input	۳.۴.۸
۵۴	ترکیب ورودی و باور قبلی	۴.۴.۸
۵۴	بازنویسی فرمول با خطای پیش‌بینی	۵.۴.۸
۵۵	چه زمانی باور به‌روزرسانی نمی‌شود؟	۵.۸
۵۵	حالت اول: خطای پیش‌بینی صفر باشد	۱.۵.۸
۵۵	حالت دوم: Precision Input صفر یا بسیار کم باشد	۲.۵.۸
۵۵	حالت سوم: Precision Prior بسیار بیشتر باشد	۳.۵.۸
۵۶	کاهش Confirmation Bias با چارچوب بیزی	۶.۸
۵۶	ارتباط Predictive Coding با مدل‌های زبانی	۷.۸

۵۷	RAG	۱.۷.۸
۵۷	Few-shot Prompting	۲.۷.۸
۵۷	Negative Prompting	۳.۷.۸
۵۷	Role Playing	۴.۷.۸
۵۷	تفاوت Predictive Coding با شبکه‌های عصبی رایج	۸.۸
۵۸	خطای محلی در مغز، خطای سراسری در شبکه	۱.۸.۸
۵۸	جهت جریان اطلاعات	۲.۸.۸
۵۹	آیا وزن‌های شبکه همان Prior هستند؟	۳.۸.۸
۵۹	مقایسه مفهومی با GAN	۹.۸
۶۱	توجه انتخابی	۹
۶۲	جایگاه توجه انتخابی در معماری شناختی	۱.۹
۶۲	تفاوت توجه شناختی با Attention در مدل‌های زبانی	۲.۹
۶۲	مسئله گلوگاه توجه	۳.۹
۶۳	Broadbent's Filter Theory	۴.۹
۶۳	Early Selection	۱.۴.۹
۶۳	محدودیت‌های انتخاب زود هنگام	۲.۴.۹
۶۳	Late Selection	۵.۹
۶۴	انتخاب پس از پردازش معنایی	۱.۵.۹
۶۴	محدودیت‌های انتخاب دیر هنگام	۲.۵.۹
۶۴	Perceptual Load Theory	۶.۹
۶۵	بار ادراکی بالا	۱.۶.۹
۶۵	بار ادراکی پایین	۲.۶.۹
۶۵	Treisman's Attenuation Theory	۷.۹
۶۵	فیلتر تضعیفی	۱.۷.۹
۶۶	آستانه و محرک‌های مهم	۲.۷.۹
۶۶	پردازش سری، موازی و جابه‌جایی توجه	۸.۹

چهارم زبان، گفتار و مدل‌های شنیداری ۶۷

۱۰ از توجه انتخابی تا دستیار صوتی ۶۹

۶۹	نگاشت نظریه‌های توجه به سیستم‌های گفتاری	۱.۱۰
۶۹	Wake Word در انسان و ماشین	۲.۱۰
۷۰	مرحله انتخاب زود هنگام در دستیار صوتی	۳.۱۰
۷۰	مرحله انتخاب دیر هنگام و فهم معنا	۴.۱۰
۷۰	چرا گفتار معمولاً به متن تبدیل می‌شود؟	۵.۱۰
۷۱	سه جزء اصلی سیستم گفتاری	۶.۱۰
۷۱	ASR	۱.۶.۱۰
۷۱	NLU	۲.۶.۱۰
۷۱	TTS	۳.۶.۱۰

۱۱ مبانی پردازش سیگنال گفتار ۷۳

۷۳	تولید گفتار در انسان	۱.۱۱
۷۳	واج، واکه و فورمنت	۲.۱۱
۷۴	موج صوتی خام	۳.۱۱
۷۴	نمونه برداری و قانون نایکوئیست	۴.۱۱
۷۴	فریم بندی، پنجره گذاری و هم پوشانی	۵.۱۱
۷۵	تبدیل فوریه و طیف فرکانسی	۶.۱۱
۷۵	اسپکتروگرام	۷.۱۱
۷۵	MFCC	۸.۱۱
۷۶	Mel Scale	۱.۸.۱۱
۷۶	Mel Filter Bank	۲.۸.۱۱
۷۶	خروجی عددی و کاربرد در طبقه بندی	۳.۸.۱۱

۱۲ مدل‌های خود نظارتی گفتار ۷۹

۷۹	استخراج ویژگی قطعی و یادگرفتنی	۱.۱۲
۸۰	Self-Supervised Learning	۲.۱۲
۸۰	wav2vec 2.0	۳.۱۲

۸۱	 ورودی خام	۱.۳.۱۲
۸۱	 CNN Feature Encoder	۲.۳.۱۲
۸۱	 Transformer Encoder	۳.۳.۱۲
۸۱	 Masking	۴.۳.۱۲
۸۲	 Codebook و Quantization	۵.۳.۱۲
۸۲	 Contrastive Loss	۶.۳.۱۲
۸۲	 Fine-tuning و Pretraining	۷.۳.۱۲

۸۵	Encoder-Decoder معماری و Whisper	۱۳
۸۵	 Whisper ورودی و خروجی	۱.۱۳
۸۵	 چرا معماری Encoder-Decoder؟	۲.۱۳
۸۶	 بخش Encoder	۳.۱۳
۸۶	 Self-Attention بدون ماسک	۱.۳.۱۳
۸۶	 بازنمایی صوتی زمینه‌مند	۲.۳.۱۳
۸۶	 بخش Decoder	۴.۱۳
۸۷	 Masked Self-Attention	۱.۴.۱۳
۸۷	 Cross-Attention	۲.۴.۱۳
۸۷	 تولید خودرگرسیو متن	۳.۴.۱۳
۸۷	 ارتباط مدل‌های شنیداری با توجه انتخابی	۵.۱۳
۸۸	 بازگشت کوتاه به Predictive Coding و GAN	۶.۱۳
۸۸	 پردازنده‌های نورومورفیک	۷.۱۳

۹۱ پنجم پاداش، یادگیری تقویتی و شبکه‌های اسپایکی

۹۳	سیستم پاداش و یادگیری تقویتی	۱۴
۹۳	 پاداش در مغز	۱.۱۴
۹۴	 دوپامین و VTA	۲.۱۴
۹۴	 مسیرهای دوپامینرژیک	۳.۱۴
۹۴	 Mesolimbic Pathway	۱.۳.۱۴

۹۵	Mesocortical Pathway	۲.۳.۱۴
۹۵	اختلال‌های مرتبط با سیستم پاداش	۴.۱۴
۹۵	پیری	۱.۴.۱۴
۹۵	افسردگی	۲.۴.۱۴
۹۶	اعتیاد	۳.۴.۱۴
۹۶	ارتباط سیستم پاداش با Predictive Coding	۵.۱۴
۹۷	Reward Prediction Error	۶.۱۴
۹۷	Free Energy Principle و کاهش Surprise	۷.۱۴
۹۹	مدل‌های محاسباتی یادگیری تقویتی	۱۵
۹۹	مسئله Reinforcement Learning	۱.۱۵
۱۰۰	تفاوت با یادگیری نظارت‌شده و بدون ناظر	۲.۱۵
۱۰۰	Temporal Difference Learning	۳.۱۵
۱۰۱	Discount Factor	۴.۱۵
۱۰۱	Value Function	۵.۱۵
۱۰۲	Q Function	۶.۱۵
۱۰۳	Q-Learning	۷.۱۵
۱۰۴	محیط‌های تصادفی و عدم قطعیت	۸.۱۵
۱۰۴	Exploitation و Exploration	۹.۱۵
۱۰۵	Active RL و Passive RL	۱۰.۱۵
۱۰۷	مبانی شبکه‌های عصبی اسپایکی	۱۶
۱۰۷	چرا شبکه‌های اسپایکی؟	۱.۱۶
۱۰۷	تفاوت ANN و SNN	۲.۱۶
۱۰۸	نورون زیستی	۳.۱۶
۱۰۸	دندریت	۱.۳.۱۶
۱۰۸	Soma	۲.۳.۱۶
۱۰۸	آکسون	۳.۳.۱۶
۱۰۸	سیناپس	۴.۳.۱۶
۱۰۹	اسپایک و زمان‌محور بودن محاسبه	۴.۱۶

۱۰۹		مدل LIF	۵.۱۶
۱۰۹		Integration	۱.۵.۱۶
۱۰۹		Fire	۲.۵.۱۶
۱۱۰		Leak	۳.۵.۱۶
۱۱۰		Refractory Period	۴.۵.۱۶
۱۱۰		مصرف انرژی و پردازش رویدادمحور	۶.۱۶
۱۱۱		یادگیری در شبکه‌های اسپایکی	۱۷
۱۱۱		Hebbian Learning قانون	۱.۱۷
۱۱۲		STDP	۲.۱۷
۱۱۲		LTP	۱.۲.۱۷
۱۱۲		LTD	۲.۲.۱۷
۱۱۲		نقش اختلاف زمانی	۳.۲.۱۷
۱۱۳		Reward-Modulated STDP	۳.۱۷
۱۱۴		از SNN پایه به Deep SNN	۴.۱۷
۱۱۵		الهام زیستی از ساختار چندلایه مغز	۵.۱۷
۱۱۵		مصرف انرژی و انگیزه Deep SNN	۶.۱۷
۱۱۶		محدودیت‌های STDP در شبکه‌های عمیق	۷.۱۷
۱۱۶		Surrogate Gradient	۸.۱۷
۱۱۶		مسئله مشتق‌پذیری اسپایک	۱.۸.۱۷
۱۱۶		تابع جایگزین	۲.۸.۱۷
۱۱۷		تفاوت با Activation Function معمولی	۳.۸.۱۷
۱۱۷		Deep SNN در Feedforward	۹.۱۷
۱۱۸		نقش reset	۱.۹.۱۷
۱۱۸		Surrogate Gradient با Backpropagation	۱۰.۱۷
۱۱۸		مثال عددی	۱.۱۰.۱۷
۱۱۹		محلی بودن، آموزش سراسری و ارزش عملی SNN	۱۱.۱۷
۱۲۰		کاربرد STDP در برابر Surrogate Gradient	۱۲.۱۷

۱۲۱	ششم مدل‌های اسپایکی مدرن
۱۲۳	۱۸ SpikeFormer و Spiking Transformer
۱۲۳	۱.۱۸ چرا Transformer در SNN؟
۱۲۴	۲.۱۸ مسئله کدگذاری ورودی
۱۲۴	۱.۲.۱۸ کدگذاری تصویر
۱۲۴	۲.۲.۱۸ نکته محاسباتی در کدگذاری
۱۲۵	۳.۲.۱۸ کدگذاری متن
۱۲۵	۴.۲.۱۸ کدگذاری با ایده Integration
۱۲۶	۳.۱۸ Attention در شبکه اسپایکی
۱۲۶	۱.۳.۱۸ Query، Key و Value اسپایکی
۱۲۶	۲.۳.۱۸ حذف Softmax
۱۲۷	۳.۳.۱۸ تغییر ترتیب ضرب‌ها و کاهش پیچیدگی
۱۲۷	۴.۳.۱۸ تفسیر Attention اسپایکی
۱۲۸	۴.۱۸ معماری کلی SpikeFormer
۱۲۸	۵.۱۸ نقش LIF
۱۲۸	۶.۱۸ MLP در Transformer اسپایکی
۱۲۹	۷.۱۸ Residual Connection
۱۲۹	۸.۱۸ جمع‌بندی SpikeFormer
۱۳۱	۱۹ SpikeBERT و تقطیر دانش
۱۳۱	۱.۱۹ چرا متن برای مدل اسپایکی دشوار است؟
۱۳۱	۱.۱.۱۹ تراکم اطلاعات در متن
۱۳۲	۲.۱.۱۹ وابستگی‌های بلندمدت
۱۳۲	۳.۱.۱۹ ماهیت نمادین و انتزاعی زبان
۱۳۲	۲.۱۹ Knowledge Distillation
۱۳۳	۳.۱۹ Student و Teacher
۱۳۳	۴.۱۹ تبدیل Embedding پیوسته به اسپایک
۱۳۴	۵.۱۹ بازگرداندن اسپایک‌ها به نمایش پیوسته
۱۳۴	۶.۱۹ Projection و هم‌ترازسازی بازنمایی

۱۳۵	Feature Loss	۷.۱۹
۱۳۵	دو مرحله آموزش در SpikeBERT	۸.۱۹
۱۳۵	انتقال دانش عمومی	۱.۸.۱۹
۱۳۶	آموزش مخصوص وظیفه	۲.۸.۱۹
۱۳۶	Cross Entropy Loss	۹.۱۹
۱۳۷	نقش Time Step	۱۰.۱۹
۱۳۷	رابطه Time Step و دقت	۱.۱۰.۱۹
۱۳۷	محدودیت های Time Step زیاد	۲.۱۰.۱۹
۱۳۸	بازگشت کوتاه به تمرین Spiking Attention	۱۱.۱۹
۱۴۱	۲۰ RWKV، SpikeGPT و تولید متن اسپایکی	
۱۴۱	تفاوت طبقه بندی و تولید	۱.۲۰
۱۴۲	چرا تبدیل مستقیم Transformer کافی نیست؟	۲.۲۰
۱۴۲	شباهت RNN و GPT	۳.۲۰
۱۴۲	مشکل RNN های کلاسیک	۴.۲۰
۱۴۳	معماری RWKV	۵.۲۰
۱۴۳	Receptance	۱.۵.۲۰
۱۴۳	Weight یا عامل فراموشی	۲.۵.۲۰
۱۴۳	Key	۳.۵.۲۰
۱۴۳	Value	۴.۵.۲۰
۱۴۴	ایده State در RWKV	۶.۲۰
۱۴۵	تفاوت Time Step در RWKV و SNN	۷.۲۰
۱۴۵	معماری کلی SpikeGPT	۸.۲۰
۱۴۶	Time-Mix	۱.۸.۲۰
۱۴۶	Channel-Mix	۲.۸.۲۰
۱۴۶	R-FFN	۳.۸.۲۰
۱۴۶	پیوستگی محاسبات میانی و اسپایکی بودن مدل	۹.۲۰
۱۴۷	مصرف انرژی و محدودیت اجرای SNN روی GPU	۱۰.۲۰

هفتم کدگذاری عصبی، هم‌ترازی مغز و استدلال ۱۴۹

۲۱ کدگذاری و رمزگشایی عصبی ۱۵۱

۱.۲۱ Neural Decoding و Neural Encoding ۱۵۱

۲.۲۱ نظریه‌های اصلی Neural Coding ۱۵۱

۱.۲.۲۱ Grandmother Cell ۱۵۲

۲.۲.۲۱ Population Coding ۱۵۲

۳.۲.۲۱ Sparse Coding ۱۵۳

۳.۲۱ ترکیب نظریه‌های کدگذاری در مدل‌های امروزی ۱۵۳

۴.۲۱ نسبت Symbolic AI با نظریه‌های کدگذاری ۱۵۴

۵.۲۱ Brain-to-Text ۱۵۴

۱.۵.۲۱ فاز آموزش ۱۵۵

۲.۵.۲۱ نقش CNN ۱۵۵

۳.۵.۲۱ نقش Transformer ۱۵۵

۴.۵.۲۱ نقش Language Model ۱۵۶

۵.۵.۲۱ فاز آزمون ۱۵۶

۶.۲۱ کاربرد Brain-to-Text و مقایسه با Neuralink ۱۵۶

۷.۲۱ مقایسه روش‌های ثبت فعالیت مغزی ۱۵۶

۱.۷.۲۱ fMRI ۱۵۷

۲.۷.۲۱ EEG ۱۵۷

۳.۷.۲۱ ایمپلنت‌های مغزی ۱۵۷

۸.۲۱ چالش‌های Brain-to-Text ۱۵۷

۱.۸.۲۱ Plasticity ۱۵۸

۲.۸.۲۱ تعمیم‌پذیری بین افراد ۱۵۸

۳.۸.۲۱ Multimodality ۱۵۸

۹.۲۱ مسیر آینده: سنسورهای پوشیدنی و سخت‌افزار Neuromorphic ۱۵۸

۱۰.۲۱ ترجیح مکانی، قانون Hebb و مقداردهی اولیه ۱۵۹

۲۲ Brain Score و Brain Alignment ۱۶۱

۱.۲۲ مسئله بازنمایی ۱۶۱

۱۶۱	۲.۲۲	آیا مدل واقعاً می‌فهمد؟
۱۶۲	۳.۲۲	ایده اصلی Brain Alignment
۱۶۲	۴.۲۲	طراحی آزمایش هم‌ترازی
۱۶۳	۱.۴.۲۲	تعریف لایه‌های میانی مغز
۱۶۳	۲.۴.۲۲	انتخاب Task
۱۶۳	۳.۴.۲۲	تعریف Score
۱۶۳	۵.۲۲	مقاله Brain Score
۱۶۴	۱.۵.۲۲	اجزای آزمایش
۱۶۴	۲.۵.۲۲	مدل Mapping
۱۶۴	۳.۵.۲۲	مرحله آموزش و آزمون
۱۶۵	۴.۵.۲۲	اهمیت Next Word Prediction
۱۶۵	۶.۲۲	Brain Parcellation
۱۶۵	۱.۶.۲۲	Voxel
۱۶۵	۲.۶.۲۲	Parcel
۱۶۵	۳.۶.۲۲	Region of Interest
۱۶۵	۴.۶.۲۲	سیگنال BOLD
۱۶۶	۷.۲۲	روش‌های Parcellation
۱۶۶	۱.۷.۲۲	بر اساس آناتومی
۱۶۶	۲.۷.۲۲	Functional Parcellation
۱۶۶	۳.۷.۲۲	Atlas و Localizer
۱۶۷	۸.۲۲	شبکه‌های هفت‌گانه Yeo
۱۶۷	۱.۸.۲۲	Visual Network
۱۶۷	۲.۸.۲۲	Somatomotor Network
۱۶۷	۳.۸.۲۲	Dorsal Attention Network
۱۶۷	۴.۸.۲۲	Ventral Attention Network
۱۶۷	۵.۸.۲۲	Frontoparietal Network
۱۶۸	۶.۸.۲۲	Default Mode Network
۱۶۸	۷.۸.۲۲	Limbic Network
۱۶۸	۹.۲۲	فعالیت شبکه‌ها و Resting-State fMRI

۱۶۸	همبستگی در Parcellation	۱۰.۲۲
۱۶۹	Task-Negative و Task-Positive	۱۱.۲۲
۱۶۹	بازگشت به Brain Alignment	۱۲.۲۲
۱۶۹	خط پژوهشی آینده: استنتاج و مغالطه	۱۳.۲۲
۱۷۱	استدلال، قیاس و هم‌ترازی خطای انسان و مدل	۲۳
۱۷۱	چرا پاسخ درست کافی نیست؟	۱.۲۳
۱۷۱	یادگیری از خطای انسان	۲.۲۳
۱۷۲	Pareidolia و تشخیص الگو	۳.۲۳
۱۷۲	System 1، System 2 و تفکر انتقادی	۴.۲۳
۱۷۳	انواع Reasoning	۵.۲۳
۱۷۳	Syllogism	۶.۲۳
۱۷۳	ساختار قیاس	۱.۶.۲۳
۱۷۳	Mode A	۲.۶.۲۳
۱۷۴	Mode I	۳.۶.۲۳
۱۷۴	Mode E	۴.۶.۲۳
۱۷۴	Mode O	۵.۶.۲۳
۱۷۴	Figure	۶.۶.۲۳
۱۷۴	Valid Conclusion	۷.۶.۲۳
۱۷۵	استفاده از قیاس برای Brain Alignment	۷.۲۳
۱۷۵	نتایج انسانی در Syllogistic Reasoning	۸.۲۳
۱۷۶	از فرم صوری به زبان طبیعی	۹.۲۳
۱۷۶	دیتاست Avicenna	۱۰.۲۳
۱۷۶	تفاوت Syllogism و Entailment	۱۱.۲۳
۱۷۷	مغالطه‌ها و استدلال‌های روزمره	۱۲.۲۳
۱۷۷	مقایسه مدل‌های زبانی و انسان در قیاس	۱۳.۲۳
۱۷۸	حافظه، آمار و استدلال	۱۴.۲۳

بخش اول

مبانی علوم شناختی محاسباتی

فصل ۱

چارچوب درس و مسئله اصلی

۱.۱ علوم شناختی چیست؟

علوم شناختی فقط مطالعه مغز به معنای زیستی و کالبدشناختی آن نیست. پرسش اصلی علوم شناختی این است که انسان چگونه می تواند کنش (Act) داشته باشد: چگونه می بیند، می شنود، حرف می زند، فکر می کند، تصمیم می گیرد، قضاوت می کند، یاد می گیرد، خطا می کند و گاهی دچار اختلال می شود. بنابراین موضوع علوم شناختی، فهم سازوکارهایی است که رفتار و تجربه انسانی را ممکن می کنند.

در این نگاه، «انسان» فقط فرد بزرگسال سالم و متوسط نیست. نظریه شناختی باید بتواند درباره کودک، سالمند، فرد بسیار توانمند، فرد دارای ناتوانی یا اختلال، و فردی با عملکرد عادی توضیحی منسجم بدهد. اگر نظریه ای فقط برای یک گروه محدود جواب بدهد، هنوز نظریه عمومی خوبی درباره شناخت انسان نیست.

استاد برای توضیح این نکته از تعبیر «ماژول شناختی» استفاده کرد. منظور از ماژول در اینجا یک قطعه سخت افزاری جدا و دقیقاً مرزبندی شده نیست؛ بلکه مجموعه ای از فرایندها و سازوکارهاست که باعث می شود انسان بتواند ادراک، زبان، حافظه، تصمیم گیری و عمل را انجام دهد. علوم شناختی می کوشد این سازوکارها را توصیف، آزمایش و در نهایت مدل کند.

۲.۱ از علوم شناختی تا علوم شناختی محاسباتی

در علوم شناختی، ممکن است یک نظریه در سطح کلامی یا مفهومی توضیح دهد که انسان چگونه یک مهارت را انجام می دهد. علوم شناختی محاسباتی یک گام جلوتر می رود و می پرسد آیا می توان آن نظریه را به شکل صوری، محاسباتی و قابل پیاده سازی بیان کرد یا نه.

پس هدف فقط این نیست که بگوییم «انسان از زمینه استفاده می کند» یا «حافظه کاری محدود است». هدف این است که بتوانیم این ادعاها را به مدل، الگوریتم، معماری، آزمایش و پیش بینی تبدیل کنیم. یک نظریه شناختی زمانی برای علوم شناختی محاسباتی مفیدتر می شود که بتوان آن را در قالبی اجراپذیر یا دست کم آزمون پذیر بازنویسی کرد.

به همین دلیل، درس از نظریه های کلاسیک شناختی شروع می کند، اما در آنجا متوقف نمی شود. مسیر درس این است

که هر نظریه شناختی را به یک مسئله محاسباتی وصل کند: آیا این نظریه می تواند ضعف یک مدل هوش مصنوعی را توضیح دهد؟ آیا می تواند راهی برای طراحی معماری بهتر، داده بهتر یا ارزیابی بهتر پیشنهاد کند؟

۳.۱ نسبت علوم شناختی محاسباتی با هوش مصنوعی

مسیر دانشی درس را می توان به صورت تقریبی چنین دید:

Artificial Intelligence → Machine Learning → Deep Learning
→ Computational Cognitive Science.

در هوش مصنوعی کلاسیک، با روش هایی مانند جست و جو، brute force، BFS، الگوریتم ژنتیک و simulated annealing روبه رو می شویم. در یادگیری ماشین، مسئله یادگیری از داده و استخراج ویژگی برجسته می شود. در یادگیری عمیق، مدل ها بازنمایی های پیچیده تر و معماری های عمیق تر می سازند. در علوم شناختی محاسباتی، پرسش تازه ای اضافه می شود: کدام مهارت شناختی در این مدل ها غایب است و چگونه می توان آن را وارد مدل کرد؟

در بسیاری از دوره های رایج علوم شناختی محاسباتی، نمونه اصلی یا case study از حوزه بینایی و پردازش تصویر انتخاب می شود. این انتخاب تاریخی و آموزشی قابل فهم است، چون هم علوم اعصاب و روان شناسی شناختی داده های کلاسیک زیادی درباره بینایی دارند و هم هوش مصنوعی در دوره ای رشد بسیار مهمی را از بینایی ماشین تجربه کرده است. با این حال، در این درس تمرکز اصلی روی زبان، گفتار، استدلال و مدل های زبانی است؛ یعنی حوزه هایی مانند:

- پردازش زبان طبیعی (Natural Language Processing / NLP)

- استدلال (Reasoning)

- پردازش گفتار (Speech Processing)

- مدل های زبانی (Language Models)

- ترنسفورمر (Transformer)

لازم نیست دانشجو پیش از این درس حتماً درس مستقل NLP گذرانده باشد؛ مدل ها و معماری های لازم در طول درس مرور می شوند. اما چون وزن اصلی مثال ها و پروژه ها روی زبان و گفتار است، دانشجویی که خط پژوهشی کاملاً دور از این حوزه ها دارد باید از ابتدا نسبت این درس را با مسیر پژوهشی خود روشن کند.

ساختار منابع و شیوه ارائه

این درس کتاب محور نیست و یک کتاب واحد به عنوان منبع اصلی ندارد. محتوای درس عمدتاً مقاله محور است و میان دو نوع منبع رفت و برگشت می کند: مقاله های کلاسیک علوم شناختی که نظریه های پایه را معرفی می کنند، و مقاله های مدرن در مرز هوش مصنوعی و علوم شناختی که ضعف ها، ارزیابی ها یا راهکارهای جدید مدل های محاسباتی را بررسی می کنند.

۴.۱ تفاوت پرسش شناختی و پرسش یادگیری ماشین

در یادگیری ماشین، مسئله معمولاً به شکل زیر صورت‌بندی می‌شود:

برای این ورودی، چگونه خروجی درست را پیش‌بینی کنیم؟

در چنین چارچوبی، معیارهایی مانند recall, precision, accuracy یا معیارهای مشابه اهمیت زیادی دارند. اگر مدل ورودی را بگیرد و خروجی درست بدهد، از نگاه مهندسی ممکن است موفق تلقی شود؛ حتی اگر ندانیم دقیقاً در درون مدل چه اتفاقی افتاده است.

اما در علوم شناختی، پرسش اصلی کمی متفاوت است:

انسان چگونه این کار را انجام می‌دهد؟

بنابراین درست بودن خروجی کافی نیست. یک مدل شناختی باید تا حدی درباره فرایند، زمان واکنش، الگوی خطا، تفاوت‌های فردی و اختلالات نیز توضیح بدهد. اگر مدلی در یک آزمون امتیاز بالا بگیرد، اما خطاهایش هیچ شباهتی به خطاهای انسانی نداشته باشد، شاید ابزار مهندسی خوبی باشد، اما الزاماً مدل شناختی خوبی نیست.

همین تفاوت باعث می‌شود علوم شناختی محاسباتی پلی میان دو دغدغه باشد: از یک طرف می‌خواهد مدل محاسباتی بسازد، و از طرف دیگر نمی‌خواهد فقط به خروجی نهایی قانع شود. مدل باید از نظر رفتاری و در صورت امکان از نظر فرایندی با شواهد انسانی قابل مقایسه باشد.

۵.۱ نظریه شناختی، مدل محاسباتی و اعتبارسنجی

استاد برای توضیح تعدد نظریه‌های علوم شناختی از استعاره فیل استفاده کرد. گویی پدیده شناخت انسان مانند فیلی بزرگ است و هر نظریه فقط بخشی از آن را لمس می‌کند: یک نظریه شنیدن را توضیح می‌دهد، نظریه‌ای دیگر حافظه را، نظریه‌ای دیگر توجه را، و نظریه‌ای دیگر تصمیم‌گیری یا خطا را. هیچ نظریه‌ای به تنهایی تمام ذهن انسان را توضیح نمی‌دهد.

یک نظریه شناختی خوب باید چند ویژگی داشته باشد:

- رفتار درست انسان را توضیح دهد.
- خطاهای انسان را نیز توضیح دهد، نه اینکه فقط پاسخ‌های درست را مدل کند.
- فقط به یک سن، یک وضعیت یا یک گروه محدود وابسته نباشد.
- درباره تفاوت‌های فردی و اختلالات شناختی حرفی برای گفتن داشته باشد.
- پیش‌بینی تولید کند و با آزمایش‌های مستقل سنجیده شود.

اعتبارسنجی در علوم شناختی معمولاً با یک مثال قانع‌کننده تمام نمی‌شود. باید آزمایش طراحی شود، داده انسانی جمع‌آوری شود، تحلیل آماری انجام شود و نظریه در موقعیت‌های مستقل دوباره سنجیده شود. اگر نظریه‌ای می‌گوید انسان در تشخیص گفتار ابتدا چند نامزد واژگانی را فعال می‌کند، باید بتواند توضیح دهد چرا در یک موقعیت نویزی یا مبهم، فرد به جای یک کلمه، کلمه دیگری را می‌شنود.

نکته مهم این است که نظریه‌های شناختی معمولاً به شکل صفر و یکی کنار گذاشته نمی‌شوند. بسیاری از آن‌ها بخشی از پدیده را خوب توضیح می‌دهند، اما بعداً نظریه‌های کامل‌تر یا دقیق‌تر می‌آیند و بخش‌های جاافتاده را اضافه می‌کنند. بنابراین تکامل نظریه‌ها در علوم شناختی بیشتر شبیه اصلاح، گسترش و دقیق‌تر کردن است تا صرفاً رد کامل نظریه قبلی.

۶.۱ مغز، نواحی کارکردی و احتیاط در تفسیر

نظریه‌های شناختی باید با شواهد مغزی نیز تا حدی سازگار باشند، اما نباید مغز را به مجموعه‌ای از جعبه‌های کاملاً جدا و بدون هم‌پوشانی تبدیل کرد. قشر مغز نواحی مختلفی دارد که با کارکردهایی مانند بینایی، شنوایی، گفتار، تعادل، هماهنگی، تصمیم‌گیری و استدلال مرتبط‌اند. برای نمونه، نواحی پس‌سری بیشتر با پردازش بینایی، نواحی گیجگاهی با بخش‌هایی از پردازش شنیداری و زبان، و نواحی پیشانی با کارکردهایی مانند برنامه‌ریزی، قضاوت و کنترل شناختی ارتباط دارند.

با این حال، این نگاشت‌ها مرزهای کاملاً تیز ندارند. کارکردهای شناختی معمولاً حاصل شبکه‌هایی از نواحی مغزی‌اند، نه فقط یک نقطه منفرد. پس وقتی می‌گوییم «این ناحیه با گفتار مرتبط است» یا «این ناحیه با استدلال مرتبط است»، منظور یک رابطه غالب و تقریبی است، نه مالکیت انحصاری آن ناحیه بر یک توانایی.

۷.۱ کاربرد علوم شناختی و محدوده این درس

یکی از کاربردهای مهم علوم شناختی، فهم اختلالات و بیماری‌هاست. اگر بفهمیم یک اختلال شناختی یا رشدی، مانند بخشی از طیف اوتیسم، با چه سازوکارهایی در ادراک، زبان، تعامل اجتماعی یا پیش‌بینی محیط مرتبط است، می‌توانیم مداخله‌های بهتر و انسانی‌تری طراحی کنیم. البته این درس مستقیماً درس درمان اختلالات یا نوروسایکولوژی بالینی نیست.

هدف این درس ساختن پلی میان علوم شناختی و هوش مصنوعی است. بنابراین از نظریه‌های شناختی استفاده می‌کنیم تا ضعف‌های مدل‌های محاسباتی را بهتر بفهمیم، آزمایش‌های دقیق‌تری طراحی کنیم و در صورت امکان، مدل‌هایی بسازیم که از نظر شناختی معنادارتر باشند.

۸.۱ شکست مدل‌های بزرگ در مهارت‌های شناختی

مدل‌های زبانی بزرگ در بسیاری از وظایف زبانی و دانشی عملکرد چشمگیری دارند، اما این موفقیت به تنهایی نشان نمی‌دهد که آن‌ها مهارت‌های شناختی را به شکل انسانی و عمومی یاد گرفته‌اند. یکی از مسیرهای مهم ارزیابی این مدل‌ها، طراحی آزمون‌هایی است که شباهت سطحی به داده‌های آموزشی را کم می‌کند و از مدل می‌خواهد رابطه انتزاعی را در موقعیتی تازه تشخیص دهد.

نمونه مهم، آزمون‌های استدلال قیاسی (Analogical Reasoning) با تغییرات پادواقعی (Counterfactual) است. اگر مدل در قیاس‌های آشنا خوب عمل کند، مثلاً رابطه جمع بستن در «book به books» و «girl به girls» را تشخیص

دهد، هنوز معلوم نیست مفهوم انتزاعی رابطه را فهمیده یا فقط الگوهای آشنا را بازتولید کرده است. برای آزمون سخت‌تر، می‌توان الفبا، نمادها، رنگ‌ها یا ترتیب‌ها را عوض کرد و دید آیا مدل هنوز رابطه «بعدی بودن»، «معکوس بودن» یا «هم‌رده بودن» را منتقل می‌کند یا نه.

در یادداشت جلسه، مثال‌هایی از الفبای ساختگی، داستان‌هایی با زمینه‌های متفاوت اما theme مشترک، و جایگزینی دسته‌ها با نمادهایی مانند ضربدر یا علامت تعجب مطرح شد. نکته مشترک همه این مثال‌ها این است که مدل نباید فقط به شباهت ظاهری تکیه کند؛ باید رابطه عمیق‌تر را تشخیص دهد. پژوهش‌های جدید درباره قیاس در مدل‌های زبانی دقیقاً در همین نقطه بحث دارند: برخی نتایج افت شدید مدل‌ها را در نسخه‌های پادواقعی نشان می‌دهند، و برخی کارهای دیگر ادعا می‌کنند مدل‌های قوی‌تر یا طراحی‌های دقیق‌تر می‌توانند بخشی از این تعمیم را انجام دهند. بنابراین صورت دقیق ادعا باید محتاطانه باشد: قیاس انتزاعی و پادواقعی هنوز یکی از آزمون‌های مهم و چالش‌برانگیز برای سنجش شناختی مدل‌های زبانی است.

۹.۱ تغییرات Adversarial و Counterfactual

وقتی ترتیب الفبا را عوض می‌کنیم، نمادهای تازه تعریف می‌کنیم، یا یک رابطه آشنا را در محیطی ناآشنا می‌گذاریم، یک تغییر counterfactual ساخته‌ایم. اگر تغییر طوری طراحی شود که مدل را از الگوهای سطحی و حفظ‌شده دور کند، می‌تواند نقش adversarial نیز داشته باشد؛ یعنی برای آشکار کردن ضعف مدل به کار رود.

برای انسان، تعمیم یک رابطه انتزاعی به محیط تازه معمولاً طبیعی‌تر است. البته انسان هم همیشه بی‌خطا نیست، اما اگر رابطه را فهمیده باشد، می‌تواند آن را از الفبای واقعی به الفبای ساختگی منتقل کند. مدل‌هایی که بیشتر بر همبستگی‌های آماری آشنا تکیه دارند، ممکن است با همین تغییر کوچک دچار افت شدید شوند. این افت همان جایی است که پرسش شناختی آغاز می‌شود: کدام مهارت انتزاعی یا سازوکار تعمیم در مدل ضعیف است؟

۱۰.۱ نقش داده انسانی و همبستگی خطا

یکی از ایده‌های اصلی جلسه اول این بود که برای ارزیابی شناختی مدل‌ها، فقط نباید پرسیم آیا مدل task را حل کرده است یا نه. باید پرسیم الگوی عملکرد مدل چقدر با الگوی عملکرد انسان همبستگی دارد. دو معیار را باید از هم جدا کرد:

۱. آیا مدل در انجام وظیفه موفق است؟

۲. آیا سختی‌ها و خطاهای مدل با سختی‌ها و خطاهای انسان correlation دارد؟

این تمایز به هدف ما بستگی دارد. اگر هدف ساختن ابزاری باشد که خطاهای انسان را جبران کند، شاید نخواهیم مدل همان خطاهای انسان را تکرار کند. اما حتی در این حالت هم دانستن نسبت خطای مدل و انسان مهم است: ابتدا باید بفهمیم آیا مدل و انسان در یک خطای شناختی مشترک‌اند یا نه، سپس می‌توانیم با fine-tuning، few-shot learning، تغییر داده، تغییر معماری یا روش‌های مشابه آن خطا را کاهش دهیم.

اگر هدف مدل‌سازی شناخت انسان باشد، همبستگی با خطای انسانی اهمیت بیشتری پیدا می‌کند. مدلی که فقط

پاسخ‌های درست بیشتری می‌دهد، اما هیچ شباهتی به الگوی خطای انسان ندارد، الزاماً توضیح‌دهنده شناخت انسان نیست. به همین دلیل، در علوم شناختی محاسباتی گاهی خطاها به اندازه پاسخ‌های درست مهم‌اند.

در بحث هوش مصنوعی عمومی (Artificial General Intelligence / AGI) نیز همین مسئله دیده می‌شود. برخی چارچوب‌های جدید ارزیابی، به جای سنجش یک مهارت واحد، مجموعه‌ای از توانایی‌های شناختی را بررسی می‌کنند. یکی از چارچوب‌های روان‌سنجی مهم برای دسته‌بندی توانایی‌های شناختی انسان، نظریه کتل-هورن-کرول (Cattell-Horn-Carroll / CHC) است که توانایی‌ها را در سطوح گسترده و محدود طبقه‌بندی می‌کند. استفاده از چنین چارچوب‌هایی برای ارزیابی مدل‌های هوش مصنوعی به این معناست که مدل فقط در یک بنچمارک خاص سنجیده نشود، بلکه پروفایل شناختی آن در چند توانایی مختلف بررسی شود.

۱۱.۱ مسیر پژوهشی درس

مسیر پژوهشی پیشنهادی جلسه اول چنین بود:

۱. یک ضعف یا خطای شناختی در مدل پیدا کنیم.
 ۲. بررسی کنیم آیا این ضعف با خطای انسان همبستگی دارد یا نه.
 ۳. خطای انسانی را با یک نظریه شناختی توضیح دهیم.
 ۴. داده، آزمایش یا توصیف مناسب برای سنجش آن طراحی کنیم.
 ۵. مدل را با روش‌هایی مانند prompting، few-shot learning، fine-tuning، RAG یا تغییر معماری اصلاح کنیم.
 ۶. نشان دهیم اصلاح مدل فقط افزایش سطحی عملکرد نیست، بلکه با تحلیل شناختی مسئله ارتباط دارد.
- اگر بدون مرحله شناختی فقط مدل را بزرگ‌تر کنیم یا روی داده بیشتری آموزش دهیم، ممکن است بهبود مهندسی به دست آید، اما سهم علوم شناختی روشن نیست. ارزش پژوهشی زمانی بیشتر می‌شود که ابتدا ضعف مدل در قالب یک مهارت شناختی توضیح داده شود و سپس راهکار محاسباتی دقیقاً همان ضعف را هدف بگیرد.

نکات اجرایی جلسه اول

دانشجویان باید در گروه تعاملی درس در Bale عضو شوند. از هر دانشجو خواسته شد یک بیوگرافی یا رزومه کوتاه برای استاد بفرستد: دانشگاه قبلی، معدل، استاد راهنما، خط پژوهشی، سابقه پژوهشی و علائق مرتبط با درس. هدف این کار تنظیم سطح و مسیر درس بر اساس ترکیب واقعی کلاس است. همچنین فایل‌های Excalidraw هر جلسه در اختیار دانشجویان قرار می‌گیرد تا بتوانند ساختار تصویری بحث‌ها را دنبال کنند.

فصل ۲

نظریه‌های کلاسیک پردازش گفتار

۱.۲ تشخیص کلمه گفتاری

برای آشنایی با شکل نظریه‌های شناختی، جلسه اول از پردازش گفتار شروع کرد. مسئله این است: انسان چگونه از جریان پیوسته صدا به کلمه می‌رسد؟ ورودی شنیداری معمولاً نویزی، ناقص و وابسته به زمینه است. با این حال، انسان در بیشتر موقعیت‌های روزمره خیلی سریع کلمات را تشخیص می‌دهد و حتی گاهی پیش از کامل شدن کلمه، حدس نسبتاً دقیقی درباره آن دارد.

این مسئله برای علوم شناختی مهم است، چون فقط یک مسئله مهندسی گفتار نیست. تشخیص کلمه نشان می‌دهد ذهن چگونه میان ورودی حسی، حافظه واژگانی، انتظار قبلی و زمینه معنایی تعامل برقرار می‌کند. دو نظریه کلاسیک که در جلسه معرفی شدند، Cohort Model و TRACE Model بودند.

۲.۲ Cohort Model

مدل کوهورت (Cohort Model) از نظریه‌های کلاسیک تشخیص واژه گفتاری است. ایده اصلی آن این است که با شنیدن بخش آغازین یک کلمه، مجموعه‌ای از کلمات ممکن در ذهن فعال می‌شوند. این مجموعه اولیه را می‌توان گروه نامزدها یا cohort دانست. با رسیدن هر بخش صوتی جدید، نامزدهایی که دیگر با ورودی سازگار نیستند حذف می‌شوند تا در نهایت یک گزینه یا گزینه غالب انتخاب شود.

برای مثال، اگر در رستوران صدایی با «ب» شروع شود، کلماتی مانند «برنج» یا «بریانی» می‌توانند فعال شوند. اگر صدای بعدی «ر» باشد، مجموعه نامزدها محدودتر می‌شود. این فرایند از نظر شهودی شبیه autocomplete است: چند حرف اول، فهرستی از پیشنهادها را فعال می‌کند و هرچه اطلاعات بیشتری وارد شود، فهرست محدودتر می‌شود.

۱.۲.۲ فعال‌سازی نامزدهای واژگانی

در این مدل، ذهن منتظر پایان کامل کلمه نمی‌ماند. همان بخش‌های آغازین ورودی می‌توانند واژه‌های سازگار را فعال کنند. بنابراین تشخیص واژه فرایندی زمان‌مند است، نه عملی که فقط در انتهای کلمه رخ دهد. هرچه ورودی صوتی جلوتر می‌رود، رقابت میان نامزدها تغییر می‌کند و برخی گزینه‌ها کنار گذاشته می‌شوند.

این نکته به فهم خطاهای شنیداری نیز کمک می‌کند. اگر ورودی نویزی باشد، یا اگر چند کلمه آغاز مشابهی داشته باشند، ممکن است واژه‌ای نادرست برای مدت کوتاهی فعال شود یا حتی به‌عنوان پاسخ نهایی انتخاب شود. نظریه خوب باید بتواند چنین خطاهایی را توضیح دهد.

۲.۲.۲ پردازش پایین‌به‌بالا

در نسخه ساده‌شده Cohort Model، جریان اصلی اطلاعات پایین‌به‌بالا (Bottom-Up) است:

انتخاب کلمه → کلمات ممکن → صدا

یعنی ورودی شنیداری تعیین می‌کند کدام کلمات فعال شوند و کدام کلمات حذف شوند. این نگاه برای توضیح نقش مهم آغاز کلمه و ترتیب زمانی صداها مفید است.

۳.۲.۲ محدودیت‌های مدل

محدودیت مدل کوهورت این است که انسان همیشه مانند autocomplete ساده عمل نمی‌کند. در زندگی واقعی، زمینه مکالمه، انتظار قبلی، موقعیت، دانش واژگانی و معنای جمله روی شنیدن اثر می‌گذارند. اگر بخشی از صدا حذف شود یا واضح نباشد، ذهن انسان گاهی با اتکا به زمینه آن را کامل می‌کند.

پس مدل کوهورت بخش مهمی از پدیده را توضیح می‌دهد، اما برای توضیح نقش زمینه و انتظار کافی نیست. همین جاست که نظریه بعدی، یعنی TRACE، اهمیت پیدا می‌کند.

TRACE Model ۳.۲

مدل TRACE (TRACE Model) یک مدل تعاملی‌تر برای ادراک گفتار است. این مدل در سنت مدل‌های اتصال‌گرا و interactive activation قرار می‌گیرد و می‌کوشد نشان دهد سطوح مختلف پردازش، مانند ویژگی‌های صوتی، واج‌ها و کلمات، چگونه هم‌زمان روی یکدیگر اثر می‌گذارند.

در TRACE، شنیدن فقط حرکت از صدا به کلمه نیست. کلمات فعال‌شده و انتظار ذهنی نیز می‌توانند روی تفسیر صدا اثر بگذارند. بنابراین اگر ورودی ناقص یا مبهم باشد، زمینه می‌تواند به تفسیر درست کمک کند.

۱.۳.۲ تعامل داده صوتی و زمینه

فرض کنید در رستوران هستیم و صدایی نزدیک به «برنج» می‌شنویم، اما بخشی از آن به دلیل نویز محیط واضح نیست. در چنین موقعیتی، ذهن فقط به سیگنال صوتی خام تکیه نمی‌کند؛ دانستن اینکه در رستوران هستیم و موضوع گفت‌وگو احتمالاً غذاست، روی انتخاب نهایی اثر می‌گذارد. این همان نقشی است که زمینه یا context در مدل تعاملی پیدا می‌کند.

۲.۳.۲ پردازش بالابه‌پایین

تفاوت اصلی TRACE با مدل پایین‌به‌بالای ساده این است که در آن جریان بالابه‌پایین (Top-Down) نیز وجود دارد:

تفسیر صدا → انتظار و زمینه

البته این به معنی حذف نقش ورودی صوتی نیست. مدل تعاملی می‌گوید ورودی حسی و انتظار ذهنی هم‌زمان در تفسیر گفتار نقش دارند. این نگاه برای توضیح پدیده‌هایی مناسب‌تر است که در آن‌ها انسان با وجود ورودی ناقص، کلمه درست را تشخیص می‌دهد.

۳.۳.۲ مقایسه با Cohort

مقایسه ساده دو مدل چنین است:

- در Cohort Model، آنچه می‌شنویم گزینه‌ها را فعال و محدود می‌کند.
 - در TRACE Model، آنچه می‌شنویم و آنچه انتظار داریم بشنویم، هر دو در انتخاب نهایی نقش دارند.
- پس TRACE را نباید صرفاً رد کامل Cohort دانست. مدل کوهورت ایده مهم فعال‌شدن زود هنگام نامزدهای واژگانی را برجسته می‌کند؛ TRACE این تصویر را با افزودن تعامل میان سطوح پردازش و نقش زمینه کامل‌تر می‌کند.

۴.۲ اعتبارسنجی نظریه‌های شناختی

از این دو مثال می‌توان یک درس کلی گرفت: نظریه شناختی باید بتواند هم رفتار عادی و هم خطا را توضیح دهد. اگر فردی به جای «برنج» واژه دیگری بشنود، نظریه باید روشن کند این خطا از کجا آمده است: شباهت صوتی، نویز، زمینه، انتظار قبلی، رقابت واژگانی یا ترکیبی از این عوامل.

اعتبارسنجی چنین نظریه‌هایی نیازمند آزمایش‌های متنوع است. باید افراد مختلف، شرایط شنیداری مختلف، واژه‌های مختلف و زمینه‌های مختلف بررسی شوند. همچنین یک نظریه معتبر باید فقط پس از دیدن داده توضیح نسازد، بلکه پیش‌بینی‌هایی تولید کند که بتوان آن‌ها را در آزمایش‌های بعدی سنجید.

این شیوه فکر کردن برای کل درس مهم است. هر جا با یک مدل هوش مصنوعی روبه‌رو می‌شویم، باید پرسیم: آیا مدل فقط خروجی درست می‌دهد، یا می‌توان فرایند و خطاهایش را نیز با یک نظریه شناختی توضیح داد؟

بخش دوم

معماری شناختی، حافظه و بار شناختی

فصل ۳

معماری شناختی

۱.۳ چرا معماری شناختی لازم است؟

جلسه اول هنوز وارد جزئیات معماری‌های شناختی نشد، اما با یک مثال نشان داد چرا صرفاً داشتن حسگر، قانون و خروجی درست کافی نیست. اگر بخواهیم مدل هوش مصنوعی به مهارتی نزدیک شود که انسان در زندگی روزمره به سادگی انجام می‌دهد، باید بدانیم چه اجزایی در فرایند شناختی دخیل‌اند: ادراک، حافظه، دانش عامه، پیش‌بینی پیامد، تصمیم‌گیری، عمل و دریافت بازخورد.

معماری شناختی دقیقاً برای همین لازم می‌شود. معماری شناختی فقط فهرستی از قابلیت‌ها نیست؛ بلکه باید روشن کند اطلاعات از کجا وارد می‌شود، در کجا نگه داشته می‌شود، با کدام دانش قبلی ترکیب می‌شود، چگونه تصمیم ساخته می‌شود و نتیجه عمل چگونه به سیستم برمی‌گردد.

۲.۳ مثال جارو رباتیک

مثال کلاس، جاروبرقی رباتیک بود. یک ربات ساده ممکن است چند حسگر و چند قانون داشته باشد:

- اگر به گردوغبار رسید، آن را جمع کند.
- اگر مانعی دید، مسیرش را تغییر دهد.
- اگر به یک شیء کوچک رسید، آن را کنار بزند.

این قواعد در بسیاری از موقعیت‌های ساده کافی‌اند. اما اگر ربات به فنجان پر از مایع برسد، مسئله تغییر می‌کند. انسان معمولاً فنجان را بدون فکر هل نمی‌دهد، چون چند رابطه علی و پیامدی را می‌فهمد: فنجان ممکن است پر باشد؛ اگر هل داده شود، مایع می‌ریزد؛ اگر مایع بریزد، زمین خیس می‌شود؛ خیس شدن زمین نامطلوب است؛ پس نباید فنجان را مانند یک مانع معمولی کنار زد.

این مثال نشان می‌دهد مسئله فقط تشخیص شیء نیست. ربات باید چیزی شبیه دانش عامه، پیش‌بینی پیامد، ارزیابی ریسک و انتخاب عمل داشته باشد. بدون این مهارت‌ها، سیستم ممکن است در معیارهای ساده مهندسی خوب به نظر

برسد، اما در موقعیت‌های انسانی و پر جزئیات رفتار نامناسبی تولید کند.

۱.۲.۳ راه حل اول: دوری از اشیای پرریسک

یک راه ساده این است که ربات اشیای پرریسک را نادیده بگیرد یا از آن‌ها دور شود. این راه حل خسارت را کم می‌کند، اما مسئله را کامل حل نمی‌کند. اگر ربات هر شیء مشکوکی را رها کند، ممکن است بخش‌هایی از محیط هرگز تمیز نشوند. بنابراین با یک trade-off روبه‌رو هستیم: کاهش خطر در برابر ناقص ماندن وظیفه.

۲.۲.۳ راه حل دوم: تعریف تابع ریسک

راه دیگر این است که برای حرکت دادن هر شیء، یک تابع ریسک تعریف کنیم. مدل می‌تواند بپرسد: شیء چیست؟ ظرف است یا نه؟ شکستگی است یا نه؟ داخل آن مایع هست یا نه؟ احتمال ریختن چقدر است؟ پیامد ریختن چیست؟ این مسیر از نظر مهندسی ممکن است مفید باشد، اما اگر فقط به افزودن دائمی if-else‌های تازه منجر شود، سیستم شکننده و پرهزینه می‌شود. هر موقعیت تازه ممکن است قانون تازه‌ای بخواهد.

۳.۲.۳ راه حل سوم: یادگیری از تجربه و بازخورد

راه سوم این است که ربات از تجربه و feedback یاد بگیرد. کودک نیز از ابتدا همه قوانین فیزیکی و اجتماعی را به صورت صریح ندارد؛ تجربه می‌کند، پیامدها را می‌بیند و از بازخورد مثبت یا منفی یاد می‌گیرد. اگر فنجان بیفتد و آب بریزد، این تجربه می‌تواند در رفتارهای بعدی اثر بگذارد.

از نگاه علوم شناختی محاسباتی، پرسش مهم این است که این یادگیری فقط حفظ نمونه‌ها نباشد، بلکه به فهم روابط علی و تعمیم به موقعیت‌های تازه نزدیک شود.

۴.۲.۳ دو مسیر برای بهبود مدل

از این مثال دو مسیر کلی برای بهبود مدل‌های هوش مصنوعی به دست می‌آید:

۱. مدل را بزرگ‌تر کنیم و قوانین یا داده‌های بیشتری به آن بدهیم.

۲. یک مهارت یا ماژول شناختی به مدل اضافه کنیم.

مسیر اول می‌تواند در عمل مفید باشد، اما همیشه خطر دارد که فقط به انباشتن قوانین سطحی تبدیل شود. مسیر دوم می‌پرسد کدام توانایی شناختی کم است: فهم علیت، حافظه، توجه، پیش‌بینی پیامد، ارزیابی ریسک یا یادگیری از بازخورد؟ این همان نقطه‌ای است که علوم شناختی به طراحی مدل هوش مصنوعی وصل می‌شود.

۳.۳ مازول‌های اصلی در یک معماری شناختی

از مثال جارو رباتیک می‌توان اجزای اولیه یک معماری شناختی ساده را استخراج کرد. ربات فقط یک ورودی و خروجی ندارد؛ میان ورودی و خروجی چند سطح پردازش لازم است. ابتدا باید محیط دیده یا حس شود، سپس وضعیت فعلی در حافظه موقت قرار بگیرد، بعد دانش مرتبط از حافظه بلندمدت بازیابی شود، سپس یک قاعده یا مهارت مناسب انتخاب شود و در نهایت دستور حرکتی تولید گردد.

معماری شناختی یعنی همین سازمان‌دهی یکپارچه فرایندها. اگر چند نظریه شناختی جداگانه داشته باشیم، معماری مشخص می‌کند این نظریه‌ها چگونه در یک سیستم واحد با هم تعامل می‌کنند: اطلاعات از کجا وارد می‌شود، کدام مازول آن را پردازش می‌کند، چه چیزی در حافظه می‌ماند، چه چیزی بازیابی می‌شود و عمل چگونه تولید می‌شود.

۱.۳.۳ ادراک و مازول بینایی

ماژول بینایی یا ادراکی، ورودی محیط را به بازنمایی قابل استفاده برای سیستم تبدیل می‌کند. در مثال ربات، این مازول باید تشخیص دهد که یک فنجان وجود دارد، داخل آن احتمالاً آب است و فنجان در مسیر حرکت قرار گرفته است.

در نگاشت تقریبی به مغز، پردازش بینایی با نواحی بینایی، به‌ویژه لوب پس‌سری، ارتباط دارد. البته در مغز واقعی، بینایی فقط در یک نقطه جداگانه انجام نمی‌شود و مسیرهای گسترده‌تری برای تشخیص شیء، مکان، حرکت و معنا درگیرند. اما برای سطح فعلی معماری، می‌توان مازول بینایی را ورودی ادراکی سیستم دانست.

۲.۳.۳ حافظه و بافرهای کاری

اطلاعاتی که از محیط می‌آید باید موقتاً نگه داشته شود. در معماری‌های شناختی معمولاً از مفهوم بافر (Buffer) استفاده می‌شود: فضای محدودی که وضعیت فعلی سیستم را در دسترس سایر مازول‌ها قرار می‌دهد. بافر را می‌توان به حافظه کوتاه‌مدت یا حافظه کاری نزدیک دانست، هرچند در معماری‌هایی مانند ACT-R معنای فنی دقیق‌تری دارد.

برای مثال، پس از دیدن فنجان، محتوای بافر می‌تواند چیزی شبیه این باشد: «فنجانی در مسیر است» و «مایع داخل فنجان دیده می‌شود». این اطلاعات هنوز به‌تنهایی تصمیم ایجاد نمی‌کند؛ فقط وضعیت فعلی محیط را برای بازیابی دانش و انتخاب عمل آماده می‌کند.

۳.۳.۳ تصمیم‌گیری و انتخاب عمل

پس از قرار گرفتن وضعیت فعلی در بافر، سیستم باید تصمیم بگیرد چه عملی مناسب است. این تصمیم از ترکیب چند منبع به دست می‌آید: وضعیت فعلی، دانش اعلانی درباره اشیاء و پیامدها، و قواعد یا مهارت‌های رویه‌ای.

اگر سیستم بداند فنجان شکستی است، آب ممکن است بریزد و خیس شدن زمین نامطلوب است، آنگاه یک قاعده رویه‌ای می‌تواند فعال شود: اگر فنجان پر از آب در مسیر است، با آن مثل مانع معمولی برخورد نکن. این مرحله همان جایی است که تفاوت یک مدل صرفاً واکنشی با یک مدل شناختی‌تر آشکار می‌شود.

۴.۳.۳ ماژول حرکتی

ماژول حرکتی تصمیم مفهومی را به عمل فیزیکی تبدیل می‌کند. «تغییر مسیر بده» باید به دستورهای مانند کاهش سرعت، چرخش، حرکت بازو، گرفتن آرام فنجان یا بازگشت به مسیر قبلی تبدیل شود.

در نگاشت مغزی، کنترل حرکت با بخش‌هایی از لوب پیشانی و مدارهای حرکتی مرتبط است. باید میان frontal lobe به عنوان ناحیه وسیع تر و prefrontal cortex به عنوان بخشی که بیشتر با برنامه‌ریزی، استدلال، کنترل اجرایی و تصمیم‌گیری مرتبط است تمایز گذاشت.

۴.۳ نگاشت معماری شناختی به مغز

یکی از شروط مهم معماری شناختی این است که اجزای آن دست‌کم به صورت تقریبی با مغز قابل نگاشت باشند. اگر معماری یک ماژول بینایی، ماژول حرکتی، حافظه اعلانی و حافظه رویه‌ای دارد، باید بتوان توضیح داد این اجزا با چه شبکه‌ها یا نواحی مغزی ارتباط دارند.

این نگاشت نباید ساده‌انگارانه باشد. مغز شبکه‌ای و تعاملی عمل می‌کند و هیچ مرز کاملاً تیزی میان همه کارکردها وجود ندارد. اما نگاشت تقریبی برای اعتبار شناختی مدل مهم است. برای نمونه:

جزء معماری	نگاشت تقریبی مغزی
ماژول بینایی	نواحی بینایی، به ویژه لوب پس سری
ماژول حرکتی	نواحی حرکتی در لوب پیشانی و مدارهای مرتبط
تصمیم‌گیری و کنترل شناختی	نواحی پیش‌پیشانی
حافظه اعلانی	هیپوکامپ و ساختارهای لوب گیجگاهی میانی
حافظه رویه‌ای	عقدده‌های قاعده‌ای و مدارهای قشری-زیرقشری مرتبط

این جدول یک نقشه آموزشی است، نه ادعای جداسازی کامل. هرکدام از این کارکردها با شبکه‌های گسترده‌تر و تعامل میان چند ناحیه انجام می‌شوند.

۵.۳ ACT-R

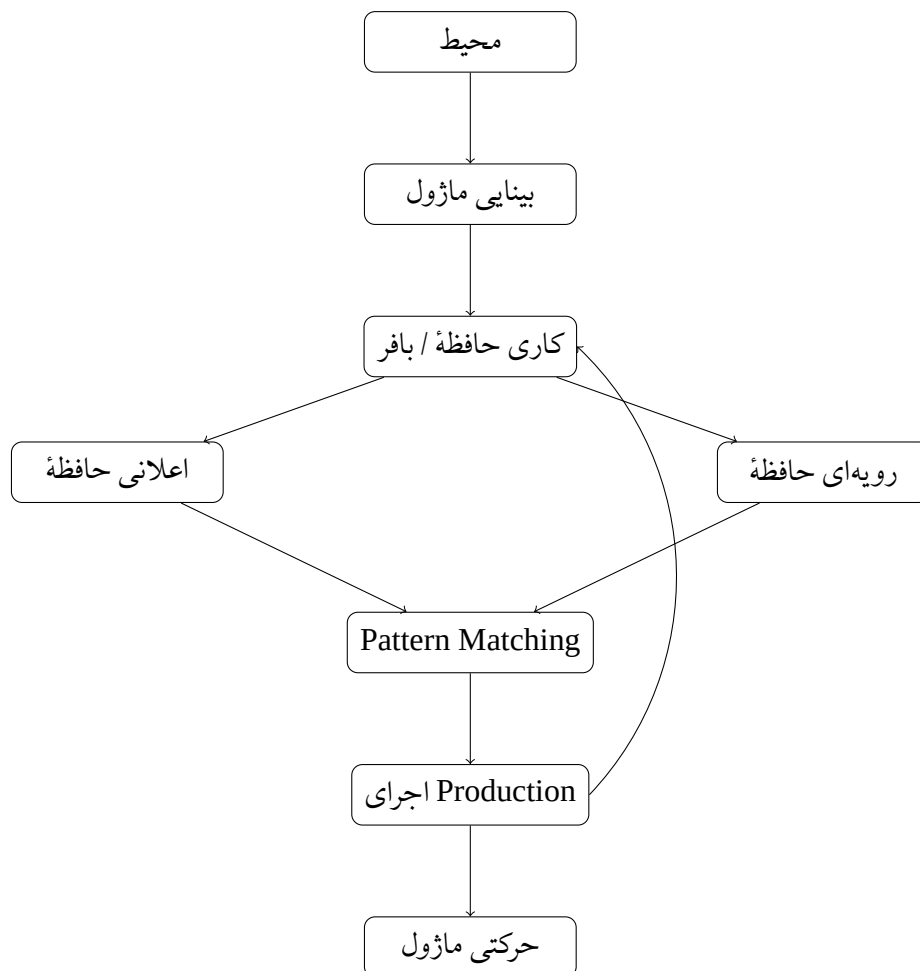
ACT-R (Adaptive Control of Thought-Rational) یکی از معماری‌های شناختی اثرگذار است که عمدتاً با کارهای John R. Anderson و همکارانش شناخته می‌شود. ریشه‌های تاریخی آن به نظریه‌های ACT در دهه ۱۹۷۰ برمی‌گردد، اما خود ACT-R در نسخه‌های بعدی توسعه یافته و همچنان در مدل‌سازی شناختی استفاده می‌شود.

ایده کلی ACT-R این است که شناخت انسان را می‌توان با چند جزء اصلی مدل کرد: ماژول‌ها، بافرها، حافظه اعلانی، حافظه رویه‌ای و سازوکار تطبیق الگو. در این معماری، دانش اعلانی معمولاً به شکل chunkها و دانش رویه‌ای به شکل production ruleها نمایش داده می‌شود. وضعیت لحظه‌ای سیستم در محتوای بافرها دیده می‌شود و pattern matcher قاعده‌ای را پیدا می‌کند که با وضعیت فعلی سازگار است.

در مثال جارو رباتیک، مسیر ساده شده چنین است:

Environment → Visual Module → Buffer

→ Memory Retrieval → Production → Motor Module.



۱.۵.۳ حافظه اعلانی

حافظه اعلانی در ACT-R محل دانش صریح و قابل بیان است. در مثال ربات، وقتی بافر شامل «فنجان» و «آب» است، حافظه اعلانی می تواند دانشی مانند این را بازیابی کند:

- فنجان می تواند شکستنی باشد.
- آب یک مایع است.
- برخورد با فنجان می تواند باعث ریختن آب شود.
- ریختن آب زمین را خیس می کند.

این نوع دانش برای توضیح پذیری نیز مهم است. اگر سیستم مسیرش را عوض کند، می توان گفت: «فنجان پر از آب در مسیر بود و برخورد با آن می توانست باعث ریختن آب شود.» این بخش به بحث interpretability در هوش مصنوعی

نزدیک می‌شود، چون تصمیم مدل به دانش قابل بیان متصل می‌شود.

۲.۵.۳ حافظه رویه‌ای

حافظه رویه‌ای در ACT-R شامل productionها یا قواعدی از جنس اگر-آنگاه است. این قواعد شبیه مهارت‌هایی‌اند که می‌گویند در یک وضعیت خاص چه عملی مناسب است:

اگر فنجان پر از آب در مسیر بود، مسیر را تغییر بده.

یا:

اگر مانع شکستی در مسیر بود، سرعت را کم کن و آرام جابه‌جایش کن.

در انسان، بسیاری از مهارت‌ها پس از تمرین زیاد سریع و کم‌هزینه اجرا می‌شوند. حافظه رویه‌ای نیز دقیقاً به همین جنبه «چگونه انجام دادن» مربوط است، نه فقط «چه چیزی دانستن».

۳.۵.۳ Pattern Matching

Pattern Matching مرحله‌ای است که در آن وضعیت فعلی بافرها و اطلاعات بازیابی شده با قواعد حافظه رویه‌ای مقایسه می‌شود. اگر وضعیت فعلی با شرط یک production سازگار باشد، آن قاعده نامزد اجرا می‌شود.

برای مثال:

- بافر: فنجان پر از آب در مسیر است.
- حافظه اعلانی: فنجان شکستی است و آب ممکن است بریزد.
- حافظه رویه‌ای: اگر مانع پرخطر در مسیر بود، با آن مثل مانع معمولی برخورد نکن.
- نتیجه: قاعده تغییر مسیر یا جابه‌جایی آرام فعال می‌شود.

در ACT-R، شناخت به صورت توالی‌ای از فعال‌شدن و اجرای productionها پیش می‌رود. بنابراین فرایند فقط یک خروجی نهایی نیست؛ زنجیره‌ای از وضعیت‌ها و تغییر وضعیت‌هاست.

۴.۵.۳ Production Execution

پس از انتخاب قاعده، production اجرا می‌شود. خروجی این مرحله باید بتواند وضعیت بافرها را تغییر دهد و در نهایت به دستور حرکتی تبدیل شود. برای ربات، «مسیر را تغییر بده» باید به دستورهای مانند کاهش سرعت، چرخش چرخ‌ها یا فعال‌کردن بازو تبدیل شود.

نکته مهم جلسه این بود که خروجی اجرای production باید به حافظه کوتاه‌مدت یا بافر برگردد. سیستم باید بداند چه تصمیمی گرفته و چه عملی را شروع کرده است. این بازگشت باعث می‌شود توالی عمل‌ها حفظ شود، عمل بعدی با عمل قبلی سازگار باشد و رفتار سیستم از هم گسسته نشود.

۶.۳ اعتبارسنجی معماری شناختی

اعتبارسنجی معماری شناختی فقط با مقایسه خروجی انجام نمی‌شود. ممکن است انسان و مدل هر دو به پاسخ درست برسند، اما مسیر رسیدن آن‌ها کاملاً متفاوت باشد. در علوم شناختی، خود مسیر اهمیت دارد.

برای اعتبارسنجی، می‌توان یک task مشخص طراحی کرد و آن را هم به انسان و هم به مدل داد. در انسان، ابزارهایی مانند fMRI، EEG، زمان واکنش یا eye-tracking می‌توانند اطلاعاتی درباره توالی و زمان‌بندی فعالیت‌ها بدهند. در مدل نیز می‌توان دید کدام ماژول در هر مرحله فعال شده است.

اگر مدل ابتدا ماژول بینایی را فعال کند، سپس وضعیت را در بافر قرار دهد، بعد حافظه اعلانی و رویه‌ای را درگیر کند و در نهایت عمل حرکتی تولید کند، باید بررسی شود آیا داده انسانی نیز جریان مشابهی را نشان می‌دهد یا نه. هرچه توالی فعال‌شدن اجزا، زمان‌بندی و الگوی خطا با انسان سازگارتر باشد، معماری از نظر شناختی معتبرتر است.

فاز اول پروژه معماری شناختی

دانشجویان باید یک مجموعه داده مناسب برای مدل‌سازی معماری شناختی پیدا کنند. مجموعه داده خوب فقط ورودی و خروجی ندارد؛ بهتر است درباره فرایند میان ورودی و خروجی نیز چیزی نشان دهد، مثل توالی عمل‌ها، زمان واکنش، مسیر نگاه، گزارش حل مسئله، EEG، fMRI یا هر نشانه‌ای که بتوان آن را با فعال‌شدن ماژول‌های مدل مقایسه کرد.

خروجی مورد انتظار یک سند کوتاه است که این موارد را توضیح دهد: دیتاست چیست، ورودی انسان چه بوده، خروجی یا رفتار انسان چه بوده، آیا سیگنال مغزی/رفتاری/زمانی وجود دارد یا نه، آیا همان ورودی را می‌توان به مدل داد یا نه، چه taskی با دیتاست انجام می‌شود، و چگونه می‌توان انسان و مدل را مقایسه کرد. مثال‌های مناسب می‌توانند شامل داستان شنیداری همراه با fMRI و زمان‌بندی کلمات، EEG در حل مسئله ریاضی، داده eye-tracking در تصمیم‌گیری پزشکی، بازی‌های دارای توالی عمل انسانی، n-back برای حافظه، یا داده‌های زمان واکنش در تصمیم‌گیری باشند. تمرین می‌تواند دو نفره انجام شود. در رونوشت جلسه، ددلاین دقیق کمی نامطمئن است، اما تحویل برای جلسه بعدی/یکشنبه مطرح شده بود.

فصل ۴

حافظه در انسان و مدل

۱.۴ حافظه حسی، کوتاه مدت و کاری

حافظه در معماری شناختی فقط یک محل ذخیره سازی ساده نیست. اگر قرار است مدل مهارت شناختی داشته باشد، باید روشن شود چه نوع اطلاعاتی را موقت نگه می دارد، چه دانشی را در بلندمدت ذخیره می کند و چگونه میان این دو سطح رفت و برگشت انجام می دهد.

در جلسه دوم، حافظه ابتدا به صورت ساده به حافظه کوتاه مدت و بلندمدت تقسیم شد. حافظه کوتاه مدت شبیه یک بافر یا cache محدود است. اطلاعات تازه ابتدا وارد این فضا می شوند و ممکن است خیلی زود از بین بروند.

مثلاً اگر کسی آدرس اتاقي را برای لحظه ای بشنود، ممکن است همان روز از آن استفاده کند، اما اگر ماه بعد دوباره به آن نیاز داشته باشد، احتمالاً باید دوباره بپرسد. این مثال نشان می دهد حضور موقت یک اطلاعات در ذهن به معنی ورود پایدار آن به حافظه بلندمدت نیست.

در ادبیات جدید، باید میان حافظه کوتاه مدت (Short-Term Memory) و حافظه کاری (Working Memory) تمایز گذاشت. حافظه کوتاه مدت بیشتر بر نگهداری موقت تأکید دارد، در حالی که حافظه کاری علاوه بر نگهداری، پردازش فعال اطلاعات فعلی را نیز شامل می شود. در این مرحله از درس، این دو مفهوم نزدیک به هم استفاده شدند، اما در فصل های بعدی دقیق تر به حافظه کاری بر می گردیم.

۲.۴ حافظه بلندمدت

اطلاعاتی که با تمرین، تکرار، استفاده یا تجربه مهم تثبیت می شوند، می توانند وارد حافظه بلندمدت شوند. حافظه بلندمدت یک مخزن یکنواخت نیست و انواع مختلفی دارد. در جلسه دوم، دو دسته اصلی برجسته شدند: حافظه اعلانی و حافظه رویه ای.

۱.۲.۴ حافظهٔ اعلانی

حافظهٔ اعلانی (Declarative Memory) مربوط به دانشی است که می‌توان آن را به صورت صریح بیان کرد: واقعیت‌ها، مفاهیم، رویدادها و اطلاعاتی که می‌توان درباره‌شان گزارش داد. مثال‌ها:

- دانستن اینکه آب زمین را خیس می‌کند.

- دانستن اینکه فنجان ممکن است شکستنی باشد.

- دانستن آدرس یک مکان.

- دانستن یک تعریف یا مفهوم درسی.

از نظر مغزی، حافظهٔ اعلانی به‌ویژه با هیپوکامپ و ساختارهای لوب گیجگاهی میانی ارتباط دارد، هرچند ذخیره و بازیابی دانش در شبکه‌های گسترده‌تری انجام می‌شود. بنابراین بهتر است نگوییم همهٔ حافظهٔ اعلانی «در هیپوکامپ ذخیره می‌شود»؛ دقیق‌تر این است که هیپوکامپ و نواحی پیرامونی برای شکل‌گیری و بازیابی بسیاری از خاطرات اعلانی نقش کلیدی دارند.

در مقایسهٔ آموزشی با مدل‌های زبانی، می‌توان گفت مدل‌هایی مانند ChatGPT در بازیابی و ترکیب دانش زبانی و واقعیت‌های آموخته‌شده قوی به نظر می‌رسند؛ بنابراین از این جهت به حافظهٔ اعلانی تشبیه می‌شوند. این تشبیه کامل نیست، چون مدل زبانی حافظهٔ انسانی ندارد، اما برای فهم تفاوت «دانستن» و «انجام دادن» مفید است.

۲.۲.۴ حافظهٔ رویه‌ای

حافظهٔ رویه‌ای (Procedural Memory) مربوط به مهارت‌ها، عادت‌ها و روش‌های انجام کار است. مثال‌ها:

- راه رفتن

- رانندگی

- آشپزی

- ترمز گرفتن

- حرکت دادن یک ابزار یا بازوی ربات

حافظهٔ رویه‌ای بیشتر با «چگونه انجام دادن» سروکار دارد. ممکن است فردی نتواند به یاد بیاورد دیشب چه غذایی خورده، اما همان لحظه بتواند آشپزی کند. این جدایی نسبی نشان می‌دهد دانستن یک واقعیت و اجرای یک مهارت یکسان نیستند.

از نظر مغزی، حافظهٔ رویه‌ای با عقده‌های قاعده‌ای، به‌ویژه مدارهای مرتبط با striatum، و نیز مدارهای حرکتی و یادگیری عادت ارتباط دارد. در مثال هوش مصنوعی، خودروی خودران می‌تواند تا حدی به حافظهٔ رویه‌ای تشبیه شود، چون بخشی از مهارت رانندگی را اجرا می‌کند. البته خودروی خودران هم دانش اعلانی، ادراک و تصمیم‌گیری دارد؛ پس این فقط یک تشبیه آموزشی است.

دقت درباره Chain of Thought

Chain of Thought را نباید مستقیماً معادل حافظه رویه ای دانست. این روش بیشتر به آشکارسازی یا هدایت گام های استدلال کمک می کند و به بازیابی دانش و استدلال صریح نزدیک تر است. ممکن است در اجرای یک روش مرحله به مرحله اثر بگذارد، اما خودش مدل کامل حافظه رویه ای انسان نیست.

۳.۴ انتقال اطلاعات از حافظه کوتاه مدت به بلند مدت

اطلاعات معمولاً با تمرین، تکرار و استفاده وارد حافظه بلند مدت می شوند. اما برخی شرایط می توانند این انتقال را بسیار سریع تر یا قوی تر کنند.

۱.۳.۴ احساس شدید

اگر فرد هنگام دریافت یک اطلاعات یا تجربه، احساس شدیدی داشته باشد، احتمال ماندگاری آن بیشتر می شود. تصادف، تولد فرزند، یک خبر بسیار خوشحال کننده یا یک تجربه ترسناک می تواند همراه با جزئیات زیادی در حافظه بماند: زمان، مکان، فصل، وضعیت هوا یا حتی صحنه های فرعی.

۲.۳.۴ تعجب و غیرمنتظره بودن

چیزی که فرد را غافلگیر کند، راحت تر در حافظه می ماند. به همین دلیل در آموزش، تجربه غیرمنتظره، آزمایش عملی یا مثال متفاوت می تواند یادگیری را تقویت کند. البته غافلگیری باید به فهم موضوع وصل شود؛ صرفاً عجیب بودن کافی نیست.

۳.۳.۴ تمایز و برجستگی

اطلاعاتی که از زمینه معمول جدا می شوند، شانس بیشتری برای ماندن دارند. اگر فردی ظاهر یا رفتاری بسیار متمایز داشته باشد، احتمال دارد نام یا تصویر او بهتر در ذهن بماند. در آموزش نیز برجسته سازی نکته مهم می تواند به ماندگاری کمک کند.

۴.۳.۴ علامت گذاری و ارتباط سازی

روش هایی مانند علامت گذاری، ساخت مثال، تصویرسازی ذهنی، تجسم سازی و پیدا کردن ارتباط میان مفاهیم کمک می کنند اطلاعات جدید با ساختارهای قبلی ذهن پیوند بخورد. این پیوندها مسیر بازیابی را تقویت می کنند و احتمال انتقال به حافظه بلند مدت را بالا می برند.

از منظر آموزشی، این پرسش خودش یک خط پژوهشی مهم است: چگونه می‌توان یادگیری را طوری طراحی کرد که اطلاعات مهم سریع‌تر، عمیق‌تر و پایدارتر وارد حافظه بلندمدت شوند؟

۴.۴ اختلالات و شواهد شناختی

تقسیم‌بندی حافظه فقط زمانی ارزش نظری دارد که بتواند داده‌های انسانی، از جمله اختلالات، را توضیح دهد. اگر می‌گوییم حافظه کوتاه‌مدت، بلندمدت، اعلانی و رویه‌ای از هم متمایزند، باید بتوانیم نشان دهیم در برخی شرایط یکی از این بخش‌ها بیشتر آسیب می‌بیند و دیگری بهتر باقی می‌ماند.

۱.۴.۴ استرس و بازیابی حافظه

استرس می‌تواند بازیابی اطلاعات اعلانی را مختل کند. فرد ممکن است چیزی را بداند، اما در لحظه فشار نتواند آن را توضیح دهد یا به یاد آورد. این حالت را می‌توان به اختلال در بازیابی factها یا توضیح‌های صریح ربط داد.

اگر استرس روی خود عمل صحبت‌کردن، لکنت، کنترل تنفس یا اجرای مهارت اثر بگذارد، بخشی از مسئله می‌تواند با جنبه‌های رویه‌ای نیز مرتبط شود. بنابراین بهتر است استرس را فقط به یک نوع حافظه محدود نکنیم؛ اثر آن به نوع وظیفه و سطح درگیری سیستم بستگی دارد.

۲.۴.۴ پارکینسون و حافظه رویه‌ای

در بیماری پارکینسون، به‌ویژه در مراحل اولیه، اختلالات حرکتی و مهارت‌های رویه‌ای برجسته‌اند. لرزش، کندی حرکت، دشواری در شروع حرکت و تغییر در راه رفتن می‌تواند باعث شود فرد ابتدا تصور کند مشکل فقط از پا، کمر یا عضله است؛ در حالی که بخش مهمی از مسئله به مدارهای عصبی کنترل حرکت و یادگیری رویه‌ای مربوط است.

این مثال با نقش عقده‌های قاعده‌ای در حرکت، عادت و یادگیری رویه‌ای سازگار است. البته پارکینسون فقط یک اختلال ساده حافظه رویه‌ای نیست و در طول زمان می‌تواند جنبه‌های شناختی دیگری را نیز درگیر کند. بنابراین باید آن را به‌عنوان شاهدهی برای تعامل سیستم‌های حرکتی، رویه‌ای و شناختی فهمید، نه به‌عنوان جداسازی کامل یک ماژول از بقیه.

۵.۴ حافظه در مدل‌های زبانی

در جلسه دوم، اشاره شد که مفاهیم حافظه شناختی می‌توانند برای فکر کردن درباره مدل‌های زبانی مفید باشند. این نگاشت استعاری است و نباید آن را یکی‌گرفتن مستقیم مدل زبانی با مغز انسان دانست.

۱.۵.۴ پارامترها به‌عنوان حافظه بلندمدت

وزن‌ها و پارامترهای آموزش‌دیده مدل را می‌توان به حافظه بلندمدت تشبیه کرد. دانشی که مدل در طول آموزش از داده‌ها جذب کرده، در این پارامترها توزیع شده است. وقتی مدل به پرسشی پاسخ می‌دهد، بخشی از این دانش آموخته‌شده را فعال و ترکیب می‌کند.

۲.۵.۴ Prompt و Context Window به‌عنوان حافظه کاری

Prompt و context window را می‌توان به حافظه کاری یا بافر فعلی مدل تشبیه کرد. اطلاعاتی که در متن ورودی قرار دارد، در همان تعامل جاری در دسترس مدل است. اگر متن طولانی، پرنویز یا متناقض شود، مدل ممکن است در نگهداری و استفاده درست از اطلاعات مهم دچار مشکل شود؛ مسئله‌ای که در فصل‌های بعد با عنوان بار شناختی و محدودیت حافظه کاری بررسی می‌شود.

۳.۵.۴ RAG به‌عنوان حافظه بیرونی

Retrieval-Augmented Generation یا RAG را می‌توان شبیه حافظه بیرونی دانست. مدل به‌جای تکیه کامل بر پارامترهای خودش، اطلاعات مرتبط را از منبعی بیرونی بازیابی می‌کند و سپس بر اساس آن پاسخ می‌دهد. این سازوکار به‌ویژه وقتی مهم است که دانش لازم بسیار خاص، تازه، حجیم یا نیازمند ارجاع دقیق باشد.

فصل ۵

نظریه بار شناختی و حافظه کاری

تا اینجا حافظه به عنوان یکی از ماژول‌های اصلی معماری شناختی معرفی شد. در این فصل، تمرکز از «انواع حافظه» به محدودیت‌های عملی حافظه کاری منتقل می‌شود. پرسش اصلی این است: وقتی اطلاعات زیاد، پراکنده یا مزاحم باشند، انسان و مدل چگونه اف‌ت می‌کنند؟

۱.۵ Cognitive Load Theory

نظریه بار شناختی (Cognitive Load Theory) بر این ایده تکیه دارد که حافظه کاری انسان محدود است. یادگیری، حل مسئله و استدلال وقتی دشوار می‌شوند که مقدار اطلاعاتی که باید هم‌زمان نگه داشته و پردازش شود از ظرفیت مؤثر حافظه کاری فراتر برود.

این نظریه در آموزش اهمیت ویژه‌ای دارد، چون طراحی بد محتوا می‌تواند بار اضافه ایجاد کند. اگر اسلاید، تمرین، منو، رابط کاربری یا توضیح آموزشی هم‌زمان اطلاعات زیادی را بدون ساختار مناسب ارائه کند، بخشی از توان شناختی دانشجو صرف مدیریت آشفتگی می‌شود، نه فهم مفهوم اصلی.

در ادبیات کلاسیک آموزش، معمولاً میان چند نوع بار شناختی تمایز گذاشته می‌شود: دشواری ذاتی خود مطلب، بار اضافی ناشی از شیوه ارائه، و باری که صرف ساختن طرحواره مفهومی مفید می‌شود. در این درس، تمرکز اصلی روی ایده عمومی‌تر است: حافظه کاری محدود است و طراحی محاسباتی یا آموزشی باید این محدودیت را جدی بگیرد.

۲.۵ ظرفیت محدود حافظه کاری

حافظه کاری فقط محل نگهداری موقت نیست؛ محل پردازش فعال اطلاعات فعلی نیز هست. وقتی شماره تلفنی را برای چند ثانیه در ذهن نگه می‌داریم تا آن را وارد گوشی کنیم، از حافظه کاری استفاده کرده‌ایم. اگر هم‌زمان کسی با ما حرف بزند، عددها طولانی باشند یا الگوی آشنایی نداشته باشند، خطا بیشتر می‌شود.

برای بررسی ظرفیت حافظه کاری می‌توان آزمایش‌های گوناگون طراحی کرد: نمایش سریع چند تصویر و پرسیدن تعداد

یا نوع آیتم‌ها، پخش چند صدا و سنجش تشخیص آن‌ها، ارائه رشته‌های عددی با طول‌های مختلف، یا دادن چند مزه و درخواست دسته‌بندی. وجه مشترک این آزمایش‌ها این است که با افزایش تعداد یا پیچیدگی آیتم‌ها، عملکرد افت می‌کند. همین ایده برای مدل‌های زبانی نیز قابل ترجمه است. به جای نمایش تصویر یا رشته عدد به انسان، می‌توان prompt‌هایی طراحی کرد که اطلاعات طولانی، پراکنده، چندمرحله‌ای یا همراه با نویز دارند و سپس دید آیا مدل همچنان می‌تواند اطلاعات مرتبط را نگه دارد و از آن‌ها استفاده کند یا نه.

۳.۵ عدد 7 ± 2

مقاله مشهور جورج میلر با عنوان The Magical Number Seven, Plus or Minus Two در سال ۱۹۵۶ یکی از نقاط کلاسیک بحث درباره ظرفیت حافظه کوتاه‌مدت است. برداشت رایج از این مقاله این است که انسان می‌تواند حدود 7 ± 2 واحد را در حافظه کوتاه‌مدت نگه دارد.

اما این عدد نباید به عنوان قانون ثابت همه انسان‌ها و همه موقعیت‌ها فهمیده شود. پژوهش‌های بعدی نشان داده‌اند ظرفیت مؤثر به نوع ماده، طول کلمات، آشنایی فرد با محتوا، روش اندازه‌گیری، سن، تمرین و راهبردهای سازمان‌دهی بستگی دارد. پیام اصلی برای این درس، خود عدد هفت نیست؛ پیام اصلی این است که حافظه کاری محدود است و واحد مهم در بسیاری از موقعیت‌ها chunk است.

دقت درباره عدد میلر

عدد 7 ± 2 یک بیان کلاسیک و آموزشی مفید است، اما ظرفیت حافظه کاری را نمی‌توان همیشه با یک عدد ثابت توضیح داد. در بسیاری از بحث‌های جدید، ظرفیت مؤثر کمتر و وابسته‌تر به نوع اطلاعات و راهبردهای فرد در نظر گرفته می‌شود.

۴.۵ Chunking

Chunking راهی برای کاهش بار حافظه کاری است. انسان به جای نگه داشتن تعداد زیادی جزء جداگانه، آن‌ها را به واحدهای معنادار بزرگ‌تر تبدیل می‌کند. به این ترتیب، حافظه کاری به جای چندین جزء پراکنده، چند واحد سازمان‌یافته را نگه می‌دارد.

۱.۴.۵ تعریف Chunk

چانک (Chunk) یعنی گروهی از اطلاعات کوچک‌تر که برای فرد به صورت یک واحد معنادار در می‌آیند. برای مثال:

1 3 6 9

اگر این چهار رقم جداگانه دیده شوند، چهار آیتم‌اند. اما اگر فرد آن‌ها را سال 1369 یا بخشی از یک الگوی آشنا بداند، کل رشته می‌تواند یک chunk شود.

بنابراین اندازه واقعی حافظه کاری فقط به تعداد خام نشانه‌ها بستگی ندارد؛ به این بستگی دارد که ذهن چگونه آن نشانه‌ها را سازمان‌دهی می‌کند.

۲.۴.۵ نقش دانش قبلی در Chunking

Chunking به حافظه بلندمدت وابسته است. ما برای فشرده‌سازی اطلاعات جدید از دانش قبلی، الگوهای آشنا، معنا، تصویر ذهنی و تداعی‌های قبلی استفاده می‌کنیم. کسی که یک رشته عددی را به تاریخ تولد، شماره ماشین، وزن ورزشی یا الگوی ریاضی ربط می‌دهد، آن را بهتر از کسی نگه می‌دارد که فقط رقم‌ها را جداگانه حفظ می‌کند.

به همین دلیل تخصص می‌تواند ظرفیت مؤثر حافظه کاری را تغییر دهد. متخصص شطرنج ممکن است چیدمان صفحه را به چند الگوی معنادار تبدیل کند، در حالی که فرد مبتدی همان صفحه را مجموعه‌ای از مهره‌های پراکنده می‌بیند.

۳.۴.۵ تفاوت‌های فردی

تفاوت افراد در حافظه کاری فقط به ظرفیت خام محدود نمی‌شود. افراد در نوع و کیفیت chunking نیز تفاوت دارند. یک نفر چانک تصویری می‌سازد، دیگری چانک زبانی، دیگری از داستان یا تداعی استفاده می‌کند و فردی دیگر الگوی عددی یا فضایی پیدا می‌کند.

بنابراین نظریه خوب باید بتواند توضیح دهد چرا دو نفر با داده ظاهراً یکسان عملکرد متفاوت دارند. تفاوت می‌تواند از اندازه چانک‌ها، نوع چانک‌ها، توانایی اتصال اطلاعات جدید به حافظه بلندمدت، یا راهبردهای سازمان‌دهی ذهنی ناشی شود.

۵.۵ کاربردهای Chunking

Chunking فقط یک مفهوم آزمایشگاهی نیست. در طراحی آموزشی، طراحی اسلاید، طراحی رابط کاربری، طراحی منوها، بازی‌ها، شماره تلفن‌ها و سازمان‌دهی اطلاعات پیچیده از همین اصل استفاده می‌شود. اگر کاربر یا دانشجو با تعداد زیادی آیتیم بی‌ساختار روبه‌رو شود، حافظه کاری او سریع‌تر اشباع می‌شود. اما اگر همان اطلاعات در گروه‌های معنادار، مرحله‌های کوچک‌تر یا ساختارهای قابل پیش‌بینی ارائه شود، پردازش ساده‌تر می‌شود.

۶.۵ Multi-Hop Reasoning به عنوان آزمون حافظه کاری

استدلال چندمرحله‌ای (Multi-Hop Reasoning) وظیفه‌ای است که پاسخ نهایی در آن از یک گام ساده به دست نمی‌آید. مدل یا انسان باید چند قطعه اطلاعات را نگه دارد، رابطه میان آن‌ها را بفهمد، نتیجه میانی بسازد و از نتیجه میانی برای گام بعدی استفاده کند.

در یک مثال ساده:

- انسان‌ها فانی‌اند.
 - سقراط انسان است.
 - پس سقراط فانی است.
 - اگر فانی‌ها معمولاً تا سن خاصی عمر کنند، نتیجه بعدی نیز به این زنجیره وابسته می‌شود.
- یا در پرسش خانوادگی «پدر همسر برادرِ مادر من چه کسی است؟» باید چند رابطه پشت سر هم پردازش شود. هر گام باید تا گام بعدی در حافظه کاری فعال بماند. همین ویژگی، multi-hop reasoning را برای سنجش بار شناختی در انسان و مدل مناسب می‌کند.

۷.۵ محدودیت حافظه کاری در مدل‌های زبانی

در مدل‌های زبانی، معادل آزمایش حافظه کاری را می‌توان با طراحی prompt‌های دشوار انجام داد. هدف این است که نشان دهیم:

۱. مدل نیز در استفاده از زمینه فعلی محدودیت دارد.
 ۲. با افزایش بار اطلاعاتی، عملکرد افت می‌کند.
 ۳. نویز، پراکندگی شواهد و تغییر موضوع می‌توانند بازیابی اطلاعات مرتبط را مختل کنند.
 ۴. سپس می‌توان با الهام از انسان، راهکارهایی مانند chunking، خلاصه‌سازی، تقسیم مسئله یا مدیریت حافظه را بررسی کرد.
- در سال‌های اخیر، برخی کارها همین ایده را با عنوان بار شناختی محاسباتی در مدل‌های زبانی بررسی کرده‌اند. یکی از مسیرهای مطرح، سنجش مدل‌ها در multi-hop reasoning همراه با دست‌کاری مقدار نویز یا ترتیب وظایف است.

۱.۷.۵ Context Saturation

Context Saturation زمانی رخ می‌دهد که اطلاعات مهم در میان حجم زیادی از اطلاعات نامرتبط یا کم‌ارتباط پخش شود. در این حالت، مسئله فقط طول context window نیست؛ مسئله این است که مدل باید از میان مقدار زیادی نویز، شواهد اصلی را پیدا کند.

ساختار آزمایش می‌تواند چنین باشد: ابتدا یک مسئله چندمرحله‌ای داده می‌شود، سپس مقدار زیادی متن نامرتبط یا اطلاعات مزاحم میان شواهد مرتبط اضافه می‌شود، و در نهایت عملکرد مدل اندازه‌گیری می‌شود. اگر با افزایش نویز، نرخ موفقیت کاهش یابد، می‌توان گفت زمینه مدل از نظر شناختی اشباع شده است.

۲.۷.۵ Attentional Residue

Attentional Residue به باقی ماندن اثر وظیفه یا موضوع قبلی روی وظیفه فعلی اشاره دارد. در تجربه عملی با مدل‌های

زبانی، گاهی مدل سؤال جدید را با زمینه قبلی قاطی می‌کند، یا هنوز در چارچوب مسئله قبلی پاسخ می‌دهد. در چنین حالتی، پاک کردن مکالمه، بازنویسی prompt یا جدا کردن وظایف می‌تواند کمک کند. در طراحی آزمایش، می‌توان ابتدا چند بخش نامرتبط یا یک وظیفه مزاحم به مدل داد، سپس مسئله اصلی را مطرح کرد و دید آیا مدل می‌تواند توجه خود را از اثر باقی مانده قبلی جدا کند یا نه.

۳.۷.۵ تفاوت دو نوع فشار

- هر دو پدیده با نویز و مزاحمت سروکار دارند، اما چیدمان آن‌ها متفاوت است:
 - در Context Saturation، اطلاعات مرتبط در میان نویز پخش می‌شود.
 - در Attentional Residue، وظیفه یا نویز قبلی روی وظیفه بعدی سایه می‌اندازد.
- در هر دو حالت، افت عملکرد مدل نشانه‌ای از محدودیت در مدیریت زمینه، توجه و حافظه کاری است.

۸.۵ طراحی آزمایش برای سنجش بار شناختی مدل

برای نشان دادن محدودیت حافظه کاری در مدل زبانی، فقط طولانی کردن prompt کافی نیست. باید طراحی آزمایش طوری باشد که عامل‌های مزاحم قابل کنترل و قابل تفسیر باشند. چند روش مطرح شده در کلاس:

- افزایش طول prompt
 - اضافه کردن اطلاعات زیاد اما نامرتبط
 - پخش کردن شواهد مرتبط در بخش‌های دور از هم
 - اضافه کردن نویز میان اطلاعات مهم
 - ارجاع دادن به بخش‌های قبلی مکالمه
 - ایجاد تداخل میان موضوع قبلی و موضوع فعلی
 - دادن یک کد یا دستور و سپس تغییر خواسته در ادامه prompt
- در هر طراحی، باید روشن باشد چه چیزی اندازه‌گیری می‌شود: بازیابی اطلاعات مرتبط، حفظ نتیجه میانی، مقاومت در برابر نویز، یا توانایی تغییر توجه از وظیفه قبلی به وظیفه جدید.

۹.۵ تکامل آزمون‌های هوش مصنوعی

این بحث در ادامه تکامل ارزیابی هوش مصنوعی قرار می‌گیرد. در ابتدا آزمون‌هایی مانند Turing Test برجسته بودند. بعدتر در یادگیری ماشین، معیارهایی مانند precision, accuracy و recall روی test set اهمیت پیدا کردند. با پیشرفت

مدل‌ها، مسئله تعمیم خارج از توزیع (Out-of-Distribution Generalization) مطرح شد.

اکنون یکی از مسیرهای مهم، آزمون‌های شناختی است. به جای اینکه فقط پیرسیم مدل روی یک بنچمارک چند درصد امتیاز می‌گیرد، می‌پرسیم: آیا مدل مانند انسان محدودیت حافظه کاری دارد؟ آیا خطاهای مدل با خطاهای انسان همبستگی دارد؟ آیا در مهارت‌هایی مانند استدلال، حافظه، برنامه‌ریزی و انتزاع ضعف نظام‌مند دارد؟ آیا می‌توان با الهام از نظریه‌های شناختی این ضعف را کاهش داد؟

۱۰.۵ راهکارهای شناختی

وقتی محدودیت حافظه کاری شناسایی شد، مرحله بعد طراحی راهکار است. در انسان، راهکارهایی مانند chunking، تقسیم مسئله، ساختن نشانه‌های بازبایی، خلاصه‌سازی و استفاده از دانش قبلی به کاهش بار شناختی کمک می‌کنند. در مدل‌های زبانی نیز می‌توان از ایده‌های مشابه استفاده کرد: تقسیم مسئله به زیرمسئله، خلاصه کردن شواهد، ساختن حافظه بیرونی، یا طراحی prompt‌هایی که اطلاعات را مرحله‌به‌مرحله سازمان‌دهی می‌کنند.

۱.۱۰.۵ Divide and Conquer

Divide and Conquer یعنی مسئله بزرگ را به چند مسئله کوچک‌تر تقسیم کنیم. این ایده وقتی مفید است که نتیجه هر بخش بتواند به صورت خلاصه و قابل استفاده برای گام بعدی نگه داشته شود. اگر مدل مجبور نباشد همه جزئیات خام را هم‌زمان در context نگه دارد، فشار حافظه کاری کمتر می‌شود. از نظر الگوریتمی، این ایده را می‌توان با رابطه کلی زیر نمایش داد:

$$T(n) = aT(n/b) + f(n)$$

که در آن a تعداد زیرمسئله‌ها، n/b اندازه هر زیرمسئله، و $f(n)$ هزینه شکستن مسئله و ترکیب پاسخ‌هاست. پیام آموزشی این فرمول این است که تقسیم درست مسئله می‌تواند هزینه را به شدت کم کند؛ برای مثال، در بعضی مسائل به جای هزینه‌های مرتبه $O(n^2)$ می‌توان به ساختارهایی نزدیک‌تر به $O(n \log n)$ رسید. البته این مقایسه قانون عمومی همه مسائل نیست و به ساختار دقیق مسئله و الگوریتم بستگی دارد.

مثال ساده کلاس خانه‌تکانی بود. اگر کل خانه یک‌باره موضوع کار باشد، همه جزئیات هم‌زمان روی حافظه کاری فشار می‌آورند. اما اگر خانه به اتاق‌ها، اتاق به کمد و کمد و کمد و هر کدو به چند دسته کوچک‌تر تقسیم شود، در هر لحظه فقط یک بخش فعال است. پس از تمام شدن هر بخش، لازم نیست همه حرکت‌های کوچک در حافظه بماند؛ کافی است نتیجه آن بخش به عنوان یک وضعیت خلاصه نگه داشته شود.

۲.۱۰.۵ Sub-goal و مدیریت سلسله‌مراتبی حافظه

در وظایف طولانی، هر sub-goal می‌تواند نقش یک چانک سطح بالاتر را داشته باشد. مدل ابتدا یک زیرهدف را حل می‌کند، خروجی مهم آن را نگه می‌دارد، سپس به زیرهدف بعدی می‌رود. این ایده در agent‌های بلندمدت مهم است، چون مدل باید بداند چه چیزی از مرحله قبلی را نگه دارد و چه جزئیاتی را می‌تواند کنار بگذارد.

در مقاله HiAgent این ایده به شکل صریح به مدیریت حافظه کاری agent های زبانی وصل می شود. در راهبرد ساده، همه جفت های «عمل-مشاهده» در طول انجام کار وارد زمینه مدل می شوند. این کار اطلاعات را کامل نگه می دارد، اما در وظایف بلندمدت زمینه را پر از جزئیات تکراری و کم اهمیت می کند. راه حل پیشنهادی این است که مدل ابتدا زیرهدف بسازد، عمل های مربوط به همان زیرهدف را دنبال کند، و وقتی زیرهدف کامل شد، جزئیات اجرایی آن را با یک مشاهده خلاصه جایگزین کند.

به این ترتیب، حافظه کاری به جای یک فهرست بلند از همه حرکت ها، چیزی شبیه این دارد:

$$(g_1, s_1), (g_2, s_2), \dots, (g_k, \text{زیرهدف فعلی})$$

که در آن g_i زیرهدف و s_i خلاصه وضعیت پس از انجام آن زیرهدف است. اگر بعداً جزئیات یک زیرهدف گذشته لازم شود، می توان آن بخش را دوباره بازیابی کرد؛ اما حالت پیش فرض این نیست که همه جزئیات همیشه در حافظه فعال بمانند.

۳.۱۰.۵ مثال Blocks World

در Blocks World چند بلوک روی میز یا روی یکدیگر قرار دارند و هدف این است که به وضعیت نهایی مشخصی برسیم؛ مثلاً بلوک آبی روی بلوک نارنجی قرار گیرد. برای رسیدن به این هدف، مسئله را می توان به چند زیرهدف شکست:

۱. بلوک آبی clear شود؛ یعنی چیزی روی آن نباشد.

۲. بلوک نارنجی clear شود.

۳. بلوک آبی روی بلوک نارنجی گذاشته شود.

نکته شناختی این نیست که زیرهدف در برنامه ریزی رباتیک چیز تازه ای است؛ زیرهدف ها از قبل در planning وجود داشته اند. نکته این است که در اینجا زیرهدف به عنوان واحد حافظه نیز تفسیر می شود. پس از کامل شدن زیرهدف «بلوک نارنجی clear است»، لازم نیست همه حرکت هایی که به این وضعیت منجر شده اند در حافظه فعال بمانند. نتیجه فشرده زیرهدف برای ادامه حل مسئله کافی است.

۴.۱۰.۵ مثال Gripper

در Gripper، ربات باید اشیایی را بین اتاق ها جابه جا کند. در نتایج گزارش شده HiAgent، چانک سازی سلسله مراتبی در بعضی وظایف مانند Blocksworld سود آشکارتری داشت، اما در Gripper بهبود موفقیت ایجاد نکرد و حتی شاخص پیشرفت کمی افت کرد، هرچند مصرف زمینه کمتر شد. تفسیر آموزشی این است که chunking وقتی بیشترین فایده را دارد که مسئله واقعاً ساختار سلسله مراتبی و وابستگی میان زیرهدف ها داشته باشد. اگر کارها بسیار مستقل و خرد باشند، تقسیم اضافه ممکن است فقط هزینه مدیریتی بسازد.

۵.۱۰.۵ محدودیت‌های Chunking

Chunking همیشه راه حل کامل نیست. اگر مسئله ذاتاً ساختار تقسیم پذیر نداشته باشد، یا اگر تقسیم بندی غلط انجام شود، چانک‌ها می‌توانند اطلاعات مهم را پنهان کنند. همچنین در مدل‌های زبانی، ساختن چانک معمولاً به طراحی بیرونی، prompting یا ماژول حافظه نیاز دارد؛ مدل همیشه خودش نمی‌داند چه چیزی را باید خلاصه کند و چه چیزی را باید نگه دارد.

تفاوت مهم انسان و مدل نیز همین جاست. انسان‌ها در بسیاری از موقعیت‌ها خودشان راهبرد می‌سازند: موضوع را دسته‌بندی می‌کنند، چیزهای کم‌اهمیت را کنار می‌گذارند، و تشخیص می‌دهند چه چیزی باید در حافظه بماند. در بسیاری از agentهای فعلی، ما هنوز باید صریحاً بگوییم زیرهدف چیست، چه چیزی چانک شود، چه چیزی فراموش شود و چه چیزی بازیابی شود. بنابراین راهبرد شناختی به مدل الهام می‌دهد، اما پیاده‌سازی آن هنوز به طراحی محاسباتی دقیق نیاز دارد.

فصل ۶

ترنسفورمر، توجه و حافظه کاری

۱.۶ حافظه در Transformer

برای پیوند دادن بحث حافظه کاری با معماری‌های امروزی، باید ببینیم در Transformer چه چیزی را می‌توان به صورت تقریبی معادل حافظه بلندمدت و کوتاه‌مدت دانست. این تشبیه کامل و زیستی نیست، اما برای تحلیل مدل مفید است. حافظه بلندمدت مدل در وزن‌ها و پارامترهای آن قرار دارد. دانشی که مدل در زمان آموزش از متن، گفتار، تصویر یا داده‌های دیگر آموخته، در همین پارامترها کدگذاری شده است. برای مثال، اینکه مدل واژه data را به چه حوزه‌هایی نزدیک می‌داند، یا چه رابطه‌هایی میان کلمات، ساختارهای نحوی و مفاهیم یاد گرفته است، محصول آموزش و پارامترهاست. در مقابل، حافظه کوتاه‌مدت یا حافظه کاری به پردازش ورودی فعلی مربوط است: prompt، جمله، متن، مکالمه یا دنباله‌ای که در زمان inference پیش روی مدل قرار دارد. بخشی از معماری که بیشترین ارتباط را با این پردازش دارد، لایه attention است؛ زیرا مدل در آنجا تصمیم می‌گیرد هر توکن، برای ساختن بازنمایی فعلی خود، باید به کدام توکن‌های دیگر توجه کند.

تشبیه، نه همسانی کامل

وقتی می‌گوییم پارامترها شبیه حافظه بلندمدت و attention شبیه حافظه کاری است، منظور یک نگاشت دقیق عصبی نیست. مدل زبانی مغز ندارد و حافظه انسانی نیز صرفاً ماتریس وزن نیست. این تشبیه فقط کمک می‌کند نقش «دانش آموخته‌شده» و «پردازش زمینه فعلی» را جدا کنیم.

۲.۶ Contextual Embedding

مدل در ابتدا برای هر token یک بازنمایی اولیه دارد. این بازنمایی از دانش آموخته‌شده مدل و جدول‌های embedding یا لایه‌های ورودی می‌آید. اما هدف Transformer این نیست که یک معنای ثابت برای هر کلمه نگه دارد؛ هدف این است که بازنمایی کلمه با توجه به زمینه فعلی تغییر کند. به این بازنمایی وابسته به زمینه بردار زمینه‌مند (Contextual Embedding) می‌گوییم.

برای مثال، واژه «شیر» در فارسی می‌تواند به حیوان، نوشیدنی یا وسیله باز و بسته کردن آب اشاره کند. اگر مدل فقط یک بردار ثابت برای «شیر» داشته باشد، نمی‌تواند این تفاوت‌ها را به خوبی نشان دهد. اما پس از لایه‌های self-attention، بازنمایی «شیر» در «شیر در جنگل زندگی می‌کند» باید با بازنمایی آن در «شیر را در لیوان ریخت» متفاوت شود.

همین نکته برای واژه data نیز برقرار است. بسته به زمینه، data ممکن است به پایگاه داده، یادگیری ماشین، مصورسازی، آمار، حریم خصوصی یا سیگنال آزمایشگاهی نزدیک شود. Contextual embedding یعنی مدل معنای عملی توکن را در همان متن خاص بسازد، نه اینکه فقط به معنای میانگین و ثابت آن تکیه کند.

۱.۲.۶ حل ارجاع

حل ارجاع (Coreference Resolution) یعنی مدل بفهمد چند عبارت مختلف به یک موجودیت واحد اشاره می‌کنند. در جمله «علی رفت مدرسه. او کتابش را برداشت»، مدل باید تشخیص دهد «او» و «ش» به علی برمی‌گردند. در سطح بازنمایی، انتظار داریم بردارهای مربوط به این ارجاع‌ها به اطلاعاتی درباره «علی» وصل شوند، نه اینکه ضمیرها جدا و بی‌ارتباط پردازش شوند.

این کار نمونه روشنی از حافظه کاری در پردازش زبان است. مدل باید موجودیتی را که در جمله قبلی آمده نگه دارد، ضمیر را در جمله بعدی ببیند، و میان آن‌ها پیوند بسازد.

۲.۲.۶ رفع ابهام معنای واژه

رفع ابهام معنای واژه (Word Sense Disambiguation) یعنی تشخیص اینکه یک واژه چند معنا در متن فعلی کدام معنا را دارد. برای «شیر»، نشانه‌هایی مانند «جنگل»، «یال»، «لیوان»، «کلسیم»، «آب» یا «باز کردن» معنا را تعیین می‌کنند. مدل باید این نشانه‌ها را در زمینه پیدا کند و بازنمایی واژه را به معنای درست نزدیک کند.

پس contextual embedding فقط یک اصطلاح فنی در NLP نیست؛ همان نقطه‌ای است که مدل از حافظه بلندمدت خود، یعنی دانش آماری و معنایی آموخته‌شده، برای پردازش وضعیت فعلی استفاده می‌کند.

۳.۶ Value و Key، Query

در self-attention، بازنمایی هر توکن با چند ماتریس وزنی به سه بردار تبدیل می‌شود:

- پرس‌وجو (Query): این توکن دنبال چه اطلاعاتی است؟
- کلید (Key): این توکن چه نشانه‌ای برای یافتن خود ارائه می‌کند؟
- مقدار (Value): اگر به این توکن توجه شود، چه اطلاعاتی منتقل می‌شود؟

اگر بردار اولیه یک توکن مثلاً ابعاد 1×768 داشته باشد، مدل با ضرب آن در ماتریس‌های آموخته‌شده، بردارهای Q ، K و V می‌سازد. سپس query یک توکن با key توکن‌های دیگر مقایسه می‌شود. هرچه شباهت یا سازگاری آن‌ها بیشتر

باشد، وزن توجه بیشتر می شود. در شکل ساده، وزن ها با ضرب داخلی و سپس softmax ساخته می شوند:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

که d_k بعد بردارهای کلید است. ضرب نهایی در V باعث می شود بازنمایی جدید هر توکن از ترکیب اطلاعات توکن هایی ساخته شود که برای آن مهم تر بوده اند.

۴.۶ Self-Attention از دید حافظه کاری

Self-attention را می توان به صورت آموزشی بخشی از سازوکار حافظه کاری مدل دانست، چون روی ورودی فعلی کار می کند. پیش از attention، مدل بازنمایی هایی دارد که عمدتاً از حافظه بلندمدت و جایگاه توکن ها آمده اند. پس از attention، هر توکن بازنمایی تازه ای دارد که به متن فعلی وابسته است.

برای مثال، در جمله ای که چند بار از «شیر» با معنای متفاوت استفاده شده، attention باید نشانه های اطراف هر نمونه را جمع کند تا بازنمایی مناسب بسازد. در مسئله چندمرحله ای، attention باید میان قطعه های دور از هم پیوند برقرار کند. بنابراین اگر مدل در بازیابی شواهد مرتبط یا نادیده گرفتن اطلاعات نامرتبب ضعیف داشته باشد، این ضعف را می توان تا حدی در نحوه مدیریت توجه و زمینه جست و جو کرد.

البته در decoder-only transformerهای خودرگرسیو، هر توکن فقط به توکن های قبلی توجه می کند، چون تولید متن نباید به آینده دسترسی داشته باشد. در encoder-only ها، مانند مدل های خانواده BERT، توجه معمولاً دوسویه است و هر توکن می تواند از توکن های قبل و بعد استفاده کند. پس نوع ترنسفورمر بر شکل حافظه کاری محاسباتی اثر می گذارد.

۵.۶ آیا افزایش طول Context کافی است؟

یک پاسخ ساده به محدودیت حافظه کاری مدل این است که پنجره زمینه را بزرگ تر کنیم. این کار در بسیاری از کاربردها مفید است، اما به تنهایی مسئله را حل نمی کند. انسان نیز اگر حافظه خام بیشتری داشته باشد، بدون راهبرد سازمان دهی الزاماً بهتر فکر نمی کند. مسئله اصلی فقط «جا داشتن اطلاعات» نیست؛ مسئله این است که مدل بتواند اطلاعات مرتبط را بازیابی کند، اطلاعات نامرتبب را کنار بگذارد و مسیر استدلال را حفظ کند.

اگر prompt طولانی باشد اما پر از نویز، تکرار یا موضوع های نامرتبب شود، افزایش طول زمینه حتی می تواند بار پردازشی بیشتری ایجاد کند. بنابراین هدف بهتر، افزایش بی قید حافظه نیست؛ هدف مدیریت توجه و بازیابی (Attention and Retrieval Management) است.

۶.۶ کاهش یا حذف توجه به اطلاعات کم اهمیت

یک ایده معماری این است که ارتباط هایی با attention score پایین حذف یا صفر شوند تا مدل روی موارد مهم تر تمرکز کند. این ایده از نظر شهودی جذاب است، اما ساده نیست.

اول، وزن توجه نسبی است. ممکن است یک توکن نسبت به توکن دوم وزن کمی داشته باشد، اما نسبت به توکن سوم وزن بالایی داشته باشد. بنابراین «کم بودن» یک وزن در یک رابطه به تنهایی نشان نمی‌دهد آن توکن یا مسیر اطلاعاتی بی‌اهمیت است.

دوم، وابستگی‌ها می‌توانند زنجیره‌ای باشند. اگر A به B مربوط باشد و B به C ، ممکن است A از راه B به C وابسته شود، حتی اگر توجه مستقیم A به C کوچک باشد. حذف خام وزن‌های کوچک می‌تواند این مسیرهای غیرمستقیم را خراب کند.

پس کاهش توجه باید با طراحی دقیق انجام شود: با پنجره‌های محلی، الگوهای پراکنده، بازیابی هدفمند، خلاصه‌سازی، یا لایه‌هایی که اجازه دهند اطلاعات مهم در سطح بالاتر دوباره ترکیب شود.

۷.۶ Local Attention

Local Attention یعنی هر توکن به جای مقایسه با همه توکن‌های دنباله، فقط به یک پنجره محلی از توکن‌های نزدیک خود توجه کند. این کار از نظر محاسباتی مهم است، چون self-attention کامل نسبت به طول دنباله هزینه درجه دوم دارد؛ اگر طول دنباله n باشد، مقایسه همه با همه تقریباً از مرتبه $O(n^2)$ است.

با محلی کردن توجه، هزینه و مصرف حافظه کاهش می‌یابد و مدل مجبور نیست همه ارتباط‌های ممکن را هم‌زمان بررسی کند. از نظر مفهومی، این ایده به divide and conquer شباهت دارد: ورودی به بخش‌های کوچک‌تر تقسیم می‌شود، در هر بخش پردازش محلی انجام می‌گیرد، و سپس لایه‌های بعدی یا سازوکارهای مکمل باید اطلاعات بخش‌ها را با هم ترکیب کنند.

اما local attention خطر مهمی دارد: وابستگی‌های دوربرد ممکن است از دست بروند. اگر پاسخ یک سؤال به جمله‌ای در ابتدای متن و جمله‌ای در انتهای متن وابسته باشد، پنجره محلی ساده ممکن است این دور را به هم وصل نکند. به همین دلیل، مدل‌های عملی معمولاً فقط به پنجره محلی خام تکیه نمی‌کنند؛ ممکن است از پنجره‌های لغزان، توکن‌های جهانی، الگوهای پراکنده یا لایه‌های سلسله‌مراتبی برای عبور اطلاعات بین بخش‌ها استفاده کنند.

مثال کلاسی این بود که استاد به جای نگاه هم‌زمان به کل کلاس، ابتدا با یک بخش تعامل کند و سپس به بخش‌های دیگر برود. کل کلاس حذف نشده است؛ فقط توجه به صورت مرحله‌ای و محلی اعمال می‌شود. در مدل نیز local attention به معنی حذف متن نیست، بلکه به معنی محدود کردن میدان توجه در هر گام و نیاز به ترکیب بعدی است.

۱.۷.۶ ترکیب پنجره‌های محلی

اگر در هر پنجره محلی عناصر مهم پیدا شوند، هنوز باید مشخص شود چگونه این عناصر با عناصر مهم پنجره‌های دیگر ترکیب می‌شوند. یک راه ساده این است که از بازنمایی‌های خلاصه، میانگین‌ها یا توکن‌های تجمیعی استفاده شود. راه دقیق‌تر این است که لایه بالاتری ساخته شود که بازنمایی‌های محلی را دوباره با هم مقایسه کند. بدون چنین مرحله‌ای، مدل ممکن است فقط چند نگاه محلی جداگانه داشته باشد و بازنمایی منسجم کل متن را نسازد.

۸.۶ Mixture of Experts و Multi-Head Attention

Mixture of Experts و Multi-head attention را نباید مستقیماً معادل راهکار حافظه کاری دانست، اما هر دو می‌توانند به استفاده بهتر از ظرفیت مدل کمک کنند.

در multi-head attention، مدل چند مجموعه جدا از Q ، K و V دارد. هر head می‌تواند جنبه‌ای متفاوت از ورودی را برجسته کند: یک سر ممکن است رابطه‌های نحوی را دنبال کند، سر دیگر رابطه‌های معنایی، سر دیگر ارجاع‌های موجودیت‌ها، و سر دیگر نشانه‌های موقعیت یا ساختار جمله را. سپس خروجی سرها با هم ترکیب می‌شود. این کار شبیه نگاه کردن از چند زاویه است، نه الزاماً بزرگ‌تر کردن حافظه کاری.

Mixture of Experts نیز معمولاً به این معناست که بخش‌هایی از مدل یا «متخصص‌ها» برای نمونه‌ها یا توکن‌های مختلف فعال می‌شوند. هدف اصلی آن افزایش ظرفیت مؤثر یا تخصصی کردن پردازش است، بدون اینکه همیشه همه پارامترها فعال شوند. این ایده می‌تواند غیرمستقیم به مدیریت اطلاعات کمک کند، اما مسئله اصلی حافظه کاری، یعنی نگهداری و بازیابی هدفمند اطلاعات زمینه فعلی، همچنان به طراحی توجه، حافظه، خلاصه‌سازی و راهبردهای استدلال وابسته است.

۹.۶ جمع‌بندی

در این فصل دو مسیر برای کاهش محدودیت حافظه کاری مدل‌ها از هم جدا شد. مسیر اول شناختی و الگوریتمی بود: chunking، زیرهدف، خلاصه‌سازی و تقسیم مسئله. مسیر دوم معماری بود: بررسی اینکه در Transformer، توجه چگونه بازنمایی زمینه‌مند می‌سازد و چه تغییرهایی مانند local attention می‌توانند فشار محاسباتی و حافظه‌ای را کم کنند.

نکته مشترک هر دو مسیر این است که حافظه مؤثر فقط به مقدار فضای خام بستگی ندارد. آنچه اهمیت دارد، سازمان‌دهی، انتخاب، بازیابی و فشرده‌سازی اطلاعات است.

بخش سوم

سوگیری، پیش‌بینی و توجه انتخابی

فصل ۷

سوگیری شناختی

یادداشت اجرایی جلسه

این جلسه پس از یک وقفه و در شرایطی برگزار شد که دسترسی به منابع آنلاین و ادامه پروژه قبلی دشوار شده بود. استاد توضیح داد که درس علوم شناختی محاسباتی به بحث کلاسی وابسته است و اگر فقط به ارائه یک طرفه تبدیل شود، بخش مهمی از کیفیت خود را از دست می‌دهد. بنابراین قرار شد تا جای ممکن مسیر درس با گفت‌وگو، جلسه آنلاین یا محتوای ضبط‌شده ادامه پیدا کند. درباره پروژه نیز گفته شد که به دلیل محدودیت دسترسی به اینترنت بین‌الملل، احتمال تغییر یا جایگزینی آن با کار مطالعاتی وجود دارد.

در ابتدای جلسه، استاد نکته‌ای عمومی‌تر درباره انتخاب خط پژوهشی در تحصیلات تکمیلی مطرح کرد. در این سطح، دانشجوی فقط یک تمرین درسی انجام نمی‌دهد؛ بلکه آرام‌آرام انتخاب می‌کند در چه حوزه‌ای مسئله ببیند، خطر بپذیرد و دانش تولید کند. برخی حوزه‌ها، به‌ویژه حوزه‌های حساس یا دفاعی، ممکن است از نگاه بخشی از جامعه دانشگاهی «لیست سیاه» تلقی شوند و به همین دلیل پژوهشگران جوان از آن‌ها فاصله بگیرند. استاد هشدار داد که اگر یک نسل از پژوهشگران به شکل محافظه‌کارانه از حوزه‌هایی مانند هوافضا، هسته‌ای، موشکی، نانو یا حتی هوش مصنوعی دور شود، نسل‌های بعدی نیز مسیر پژوهشی محدودتری خواهند داشت و کشور بیشتر به مصرف‌کننده محصول دانش دیگران تبدیل می‌شود.

این بحث مستقیماً موضوع فنی فصل نیست، اما به مسیر کلی درس مربوط است: علوم شناختی محاسباتی فقط مرور نظریه‌ها نیست؛ انتخاب مسئله، انتخاب مدل و انتخاب نوع کاربرد نیز بخشی از کار علمی است. در این درس، کیس‌استادی اصلی Transformerها و مدل‌های زبانی بزرگ است. پیش از این، معماری شناختی، نظریه بار شناختی، محدودیت حافظه کاری، توجه و راهکارهای کنترل بار شناختی بررسی شد. موضوع این جلسه، ویژگی دیگری از ذهن انسان بود: سوگیری شناختی (Cognitive Bias).

۱.۷ تعریف Cognitive Bias

سوگیری شناختی (Cognitive Bias) یعنی انحراف نظام‌مند در فرایند تفکر، استدلال، قضاوت یا تصمیم‌گیری که باعث می‌شود فرد به نتیجه‌ای نادرست، نادقیق یا غیربهبه‌ن برسد. نکته مهم این است که سوگیری شناختی معمولاً فقط

یک واکنش خام احساسی نیست. فرد اغلب برای تصمیم خود دلیل می‌آورد، اما همان دلیل آوردن و وزن‌دهی به شواهد تحت تأثیر سوگیری قرار گرفته است.

برای مثال، ممکن است فرد بگوید «من این خبر را قبول دارم چون با چند تجربه قبلی‌ام سازگار است». ظاهر جمله عقلانی است، اما اگر فرد فقط تجربه‌های سازگار را دیده و تجربه‌های ناسازگار را حذف کرده باشد، استدلال او سوگیرانه است. بنابراین در سوگیری شناختی، مشکل معمولاً در مسیر پردازش اطلاعات رخ می‌دهد: انتخاب شواهد، وزن‌دهی به آنها، تفسیرشان و تصمیم نهایی.

۲.۷ نمونه‌های اصلی سوگیری

در کلاس سه نمونهٔ پرکاربرد بررسی شد: *confirmation bias*، *anchoring bias* و *halo effect*. این نمونه‌ها برای درس مهم‌اند، چون هرکدام را می‌توان هم در انسان و هم در رفتار خروجی مدل‌های زبانی مطالعه کرد.

۱.۲.۷ Confirmation Bias

سوگیری تأییدی (Confirmation Bias) یعنی فرد باور قبلی خود را معیار اصلی جست‌وجو و تفسیر اطلاعات قرار دهد. او داده‌هایی را بیشتر می‌بیند، انتخاب می‌کند یا جدی می‌گیرد که باور قبلی‌اش را تأیید کنند، و داده‌های ناسازگار را کم‌وزن‌تر یا بی‌اهمیت‌تر می‌بیند.

در زبان مدل‌سازی، می‌توان گفت فرد یک مدل ذهنی قبلی دارد و دادهٔ جدید می‌توانست وزن‌های آن مدل را به‌روزرسانی کند، اما به‌روزرسانی رخ نمی‌دهد یا بسیار ضعیف رخ می‌دهد. این سوگیری فقط در باورهای سیاسی یا اجتماعی دیده نمی‌شود؛ در خواندن مقاله، تحلیل داده، انتخاب فرضیه و حتی داوری پژوهشی هم ممکن است رخ دهد.

۲.۲.۷ Anchoring Bias

سوگیری لنگر (Anchoring Bias) زمانی رخ می‌دهد که اولین داده، اولین عدد، اولین برداشت یا اولین چارچوب ارائه‌شده وزن بیش از حدی در قضاوت بعدی پیدا کند. پس از شکل‌گیری این لنگر، فرد ممکن است اطلاعات بعدی را هم حول همان نقطه تفسیر کند.

استاد روی نقش «اولین‌ها» تأکید کرد: اولین دیدار، اولین برداشت از یک فرد، نخستین خبری که دربارهٔ یک موضوع می‌شنویم، یا اولین عددی که در یک مذاکره مطرح می‌شود. این موارد می‌توانند نقطهٔ شروع قضاوت را تعیین کنند، حتی اگر بعداً اطلاعات کافی برای اصلاح آن وجود داشته باشد.

۳.۲.۷ Halo Effect

اثر هاله‌ای (Halo Effect) یعنی یک ویژگی مثبت یا برجستهٔ فرد به ویژگی‌های دیگر او تعمیم داده شود. اگر کسی ظاهر مرتب، جذابیت اجتماعی یا شهرت زیادی داشته باشد، ممکن است ناخودآگاه او را باهوش‌تر، منطقی‌تر، اخلاقی‌تر یا

قابل اعتمادتر تصور کنیم؛ در حالی که این ویژگی‌ها از نظر منطقی لزوماً به هم وابسته نیستند.

پدیدهٔ celebrity را نیز می‌توان تا حدی با همین اثر توضیح داد. یک فرد ممکن است در هنر، ورزش، زیبایی یا اجرای رسانه‌ای ویژگی برجسته‌ای داشته باشد، اما مخاطب اعتبار آن ویژگی را به تحلیل سیاسی، قضاوت اخلاقی یا داوری علمی او نیز تعمیم دهد. این تعمیم سریع برای ذهن کم‌هزینه است، اما می‌تواند تصمیم را از شواهد واقعی جدا کند.

۳.۷ آیا سوگیری همیشه بد است؟

سوگیری‌های شناختی همیشه به معنای نقص کامل ذهن نیستند. بسیاری از آن‌ها احتمالاً به دلیل کارکردهای تطبیقی در روند تکامل باقی مانده‌اند: تصمیم‌گیری سریع، کاهش مصرف انرژی، واکنش فوری به خطر، و استفاده از نشانه‌های کم برای رسیدن به قضاوت قابل قبول.

بنابراین مسئله این نیست که ذهن باید در همهٔ موقعیت‌ها مثل یک الگوریتم کامل و بی‌نقص عمل کند. در زندگی روزمره، تصمیم سریع و تقریبی گاهی بهتر از تصمیم کامل اما دیر هنگام است. مشکل وقتی آغاز می‌شود که همان راهبرد سریع در تصمیم‌های مهم، پیچیده یا پرهزینه به کار رود؛ جایی که خطای کوچک می‌تواند پیامد بزرگ داشته باشد.

۴.۷ تبیین سوگیری با نظریهٔ بار شناختی

نظریهٔ بار شناختی (Cognitive Load Theory) می‌گوید حافظهٔ کاری انسان محدود است. اگر ذهن می‌توانست همهٔ اطلاعات محیط، همهٔ شاخه‌های ممکن تصمیم و کل فضای حالت را هم‌زمان نگه دارد و ارزیابی کند، تصمیم‌گیری می‌توانست بهینه‌تر باشد. اما در واقعیت، چنین ظرفیتی وجود ندارد.

به همین دلیل ذهن ناچار است انتخاب کند: به بعضی داده‌ها توجه کند، بعضی را حذف کند، برخی را در قالب chunk فشرده کند و برای برخی نشانه‌ها وزن بیشتری بگذارد. این انتخاب‌ها همیشه خطا نیستند، اما وقتی انتخاب یا وزن‌دهی نامتناسب شود، سوگیری ایجاد می‌شود.

۱.۴.۷ تصمیم‌گیری Greedy

در کلاس، این مسئله با تعبیر greedy توضیح داده شد. ذهن به جای جست‌وجوی کامل همهٔ مسیرها، یک مسیر یا چند نشانه در دسترس را انتخاب می‌کند و بر اساس همان تصمیم می‌گیرد. این تصمیم ممکن است feasible باشد، اما الزاماً optimal نیست.

برای مثال، در confirmation bias سیاست انتخاب می‌تواند چنین باشد: «به باور قبلی وزن زیاد بده و شواهد ناسازگار را کم‌وزن کن». در anchoring bias سیاست انتخاب چنین است: «به اولین داده یا اولین برداشت وزن زیاد بده». در هر دو حالت، محدودیت حافظهٔ کاری باعث می‌شود ذهن همهٔ داده‌ها را با وزن برابر بررسی نکند.

۲.۴.۷ تعمیم از حافظه بلندمدت

یکی از مشارکت‌های کلاسی این بود که ما درباره گروه‌هایی از افراد یا موقعیت‌ها، میانگینی از ویژگی‌ها در حافظه بلندمدت داریم. وقتی با فرد یا موقعیت تازه‌ای روبه‌رو می‌شویم، به جای پردازش همه ویژگی‌های جدید، آن میانگین قبلی را سریع فعال می‌کنیم و به مورد تازه تعمیم می‌دهیم.

این کار می‌تواند مفید باشد، چون با حداقل اطلاعات، بیشترین حدس ممکن را می‌سازد. اما اگر میانگین قبلی نامعتبر باشد، یا تفاوت‌های فرد جدید نادیده گرفته شود، همین راهبرد به کلیشه، قضاوت نادرست یا تصمیم سوگیرانه تبدیل می‌شود.

۳.۴.۷ کاهش سوگیری با کنترل بار شناختی

اگر بخشی از سوگیری از محدودیت حافظه کاری بیاید، راهکارهای کنترل بار شناختی می‌توانند احتمال سوگیری را کم کنند:

- ساختارمند کردن اطلاعات؛
- chunking و گروه‌بندی معنادار؛
- استفاده از چک‌لیست؛
- شکستن مسئله پیچیده به مسئله‌های کوچک‌تر؛
- divide and conquer؛
- مدیریت توجه؛
- پرهیز از تصمیم‌گیری در حالت داده نامنظم و پراکنده.

این راهکارها سوگیری را تضمینی حذف نمی‌کنند، اما در تصمیم‌های مهم باعث می‌شوند ذهن فقط به اولین نشانه در دسترس تکیه نکند.

۵.۷ تبیین سوگیری با نظریه دوفرآیندی

نظریه دوفرآیندی (Dual Process Theory) یکی از چارچوب‌های مشهور برای توضیح قضاوت و تصمیم‌گیری است. در روایت آموزشی رایج، این نظریه میان دو سبک پردازش تمایز می‌گذارد: System 1 و System 2. کتاب Thinking, Fast and Slow دنیل کانمن این تمایز را برای مخاطب عمومی بسیار مشهور کرد.

دقت درباره دو سیستم

System 1 و System 2 را نباید دو جعبه جدا و دقیق در مغز دانست. این‌ها نام‌هایی آموزشی برای دو سبک پردازش‌اند: یکی سریع‌تر و شهودی‌تر، دیگری کندتر و تحلیلی‌تر. همچنین همه سوگیری‌ها فقط از System 1 نمی‌آیند؛ پردازش تحلیلی نیز اگر با فرض‌های غلط یا داده سوگیرانه شروع شود، می‌تواند نتیجه نادرست بسازد.

System 1 ۱.۵.۷

System 1 سریع، شهودی، کم هزینه و مناسب واکنش فوری است. بسیاری از توانایی های روزمره انسان به همین سبک پردازش وابسته اند: تشخیص چهره، تشخیص فرد از نشانه های ناقص، فهم سریع موقعیت، جهت یابی، حدس زدن حالت عاطفی دیگران و واکنش فوری در خطر.

به همین دلیل، System 1 ذاتاً بد نیست. اگر برای هر تصمیم کوچک مجبور بودیم تحلیل کامل انجام دهیم، زندگی روزمره کند و پرهزینه می شد. مشکل وقتی رخ می دهد که مسئله پیچیده، اطلاعات ناقص، پیامد مهم، یا نیازمند تحلیل دقیق باشد، اما ذهن همچنان با پاسخ سریع و شهودی تصمیم بگیرد.

System 2 ۲.۵.۷

System 2 کندتر، تحلیلی تر، منطقی تر و پرهزینه تر است. این سبک پردازش با reasoning مرحله به مرحله، بررسی مفروضات، جمع آوری شواهد، نقد پاسخ اولیه و تعریف معیارهای تصمیم ارتباط دارد.

از منظر این نظریه، سوگیری شناختی اغلب زمانی تشدید می شود که پاسخ سریع System 1 بدون ورود System 2 نهایی شود. راهکار کاهش سوگیری این است که در تصمیم های مهم مکث کنیم، اولین پاسخ را قطعی ندانیم، هنگام خستگی یا عصبانیت تصمیم نهایی نگیریم، شواهد بیشتری جمع کنیم، معیارها را صریح کنیم و پاسخ اولیه را نقد کنیم.

۳.۵.۷ فعال سازی رفتار شبیه System 2 در مدل های زبانی

در Transformer یا LLM چیزی به معنای انسانی System 1 و System 2 وجود ندارد. با این حال، خروجی مدل گاهی شبیه پاسخ سریع و شهودی است و گاهی شبیه استدلال تحلیلی. سؤال عملی این است: چگونه می توان مدل را به سمت خروجی تحلیلی تر و کمتر شتاب زده برد؟

یکی از راه ها prompt engineering است. می توان از مدل خواست مسئله را مرحله به مرحله حل کند، مفروضات را پیش از پاسخ نهایی بررسی کند، معیارهای پاسخ را مشخص کند، پاسخ اولیه را نقد کند و سپس اصلاح کند. Few-shot prompting نیز اگر فقط شامل مثال های ساده نباشد و موارد مرزی و دشوار را نشان دهد، مدل را از تکیه بر الگوی سطحی دورتر می کند.

Self-critique نیز همین نقش را دارد. ابتدا مدل پاسخ اولیه می دهد؛ سپس در مرحله بعد از آن خواسته می شود پاسخ خود را نقد کند، خطاها یا ضعف ها را پیدا کند و نسخه اصلاح شده بسازد. پاسخ اولیه در این تشبیه شبیه System 1 است و نقد و اصلاح، رفتاری شبیه System 2 ایجاد می کند.

Self-Critique و Tree of Thought, Chain of Thought ۶.۷

Chain of Thought یعنی مدل به جای دادن پاسخ نهایی فوری، مسئله را در چند گام حل کند. این روش در بسیاری از مسائل استدلالی می تواند کمک کند، چون مدل را وادار می کند رابطه های میانی را آشکارتر و منظم تر کند. البته

در کاربردهای واقعی لازم نیست همیشه همه زنجیره استدلال خام به کاربر نمایش داده شود؛ مهم این است که فرایند پاسخ‌دهی مدل فقط یک حدس فوری نباشد.

Tree of Thought یک گام فراتر می‌رود. در این روش، مدل فقط یک مسیر استدلالی را دنبال نمی‌کند، بلکه چند شاخه ممکن را می‌سازد، آن‌ها را ارزیابی می‌کند و سپس مسیر بهتر را ادامه می‌دهد. این ایده از نظر شناختی شبیه کاهش اتکای ذهن به اولین مسیر فعال شده است: اگر چند مسیر بررسی شوند، احتمال اینکه پاسخ فقط اسیر لنگر اولیه یا چارچوب اول شود کمتر می‌شود.

در کنار این‌ها، self-critique و مرحله بازبینی پاسخ برای کاهش خطا و سوگیری مهم‌اند. مجموعه این روش‌ها با هدف مشترک به کار می‌روند: مدل از پاسخ سریع، سطحی و حساس به framing فاصله بگیرد و خروجی دقیق‌تر، تحلیلی‌تر و کمتر سوگیرانه تولید کند.

فصل ۸

Predictive Coding و مدل بیزی ذهن

یادداشت اجرایی جلسه

در آغاز جلسه، استاد وضعیت صدا و تصویر را بررسی کرد و به دلیل شلوغی دانشکده، احتمال وجود نویز در صدا را توضیح داد. درباره پرسشنامه زمان کلاس نیز گفته شد که پرسشنامه قبلی اشتباهاً برای درس دیگری قرار گرفته بود و چون زمان پیشنهادی با کلاس دیگری تداخل داشت، احتمالاً تغییر زمان عملی نیست. ممکن است پرسشنامه تازه‌ای برای بازه‌های عصر، مانند یکشنبه و سه‌شنبه، ایجاد شود.

در جلسه قبل، بحث سوگیری شناختی از سه مسیر دنبال شد: نظریه بار شناختی، نظریه دوفراپندگی، و Predictive Coding. هدف این جلسه این بود که Predictive Coding دقیق‌تر بررسی شود و سپس ببینیم تا چه حد می‌توان این نظریه را به Transformerها، مدل‌های زبانی و شبکه‌های عصبی فعلی نگاشت.

۱.۸ مغز به عنوان سیستم پیش‌بین

تصویر ساده و قدیمی از ادراک این است که محیط بیرونی داده تولید می‌کند، قشرهای حسی آن را از پایین به بالا پردازش می‌کنند، و در نهایت ادراک یا تصمیم ساخته می‌شود. کدگذاری پیش‌بینانه (Predictive Coding) این تصویر را کامل‌تر می‌کند: مغز فقط گیرنده منفعل داده نیست؛ مغز دائماً بر اساس دانش قبلی، انتظار و پیش‌بینی می‌سازد.

در این نظریه، مغز شبیه یک Generative Model عمل می‌کند. از بالا به پایین، بر اساس دانش قبلی پیش‌بینی می‌کند که در محیط چه اتفاقی در حال رخ دادن است. از پایین به بالا، ورودی‌های حسی از قشرهای اولیه دریافت می‌شوند. در هر لحظه اختلاف میان پیش‌بینی و ورودی حسی محاسبه می‌شود.

پس در هر لحظه دو جریان اطلاعاتی داریم:

- جریان بالا به پایین (Top-down): پیش‌بینی بر اساس دانش قبلی؛
- جریان پایین به بالا (Bottom-up): ورودی حسی از محیط.

اگر پیش‌بینی و ورودی حسی با هم اختلاف داشته باشند، prediction error ایجاد می‌شود. اگر اختلافی وجود نداشته

باشد، سیستم در حالت تعادل باقی می ماند و نیازی به تغییر مدل ذهنی نیست.

۲.۸ Prediction Error

خطای پیش بینی (Prediction Error) اختلاف میان چیزی است که سیستم پیش بینی کرده و چیزی که از ورودی دریافت شده است. در صورت ساده:

$$\text{Prediction Error} = \text{Input} - \text{Prediction}$$

اگر ورودی حسی را با r یا μ_i و پیش بینی یا دانش قبلی را با μ_p نمایش دهیم، داریم:

$$\text{Error} = r - \mu_p$$

یا:

$$\text{Error} = \mu_i - \mu_p$$

اگر این خطا صفر باشد، ورودی جدید با پیش بینی قبلی سازگار است و چیزی برای اصلاح وجود ندارد. اگر خطا وجود داشته باشد، سیستم باید یا باور قبلی را تغییر دهد یا ورودی جدیدی بگیرد تا ابهام کمتر شود.

۳.۸ Posterior، Likelihood و Prior

برای دقیق تر کردن بحث، استاد دانش قبلی را به صورت یک توزیع در نظر گرفت؛ مثلاً یک توزیع نرمال زنگوله ای. این توزیع دو ویژگی مهم دارد:

- میانگین μ ؛
- واریانس σ^2 .

وقتی می گوییم یک مدل ذهنی قوی است، یعنی میانگین آن مشخص و برجسته است، واریانس آن کم است، و عدم قطعیت کمی دارد. در مقابل، وقتی مدل ذهنی ضعیف است، واریانس زیاد است، قطعیت پایین است و فرد کمتر به آن دانش قبلی تکیه می کند.

فرمول کلی بیز در زبان درس چنین نوشته شد:

$$\text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

Prior دانش قبلی ماست. Likelihood شواهد جدیدی است که از محیط می گیریم. Posterior باور به روز شده پس از ترکیب دانش قبلی و شواهد جدید است.

۱.۳.۸ مثال کارآگاه

برای توضیح prior و likelihood، مثال کارآگاه مطرح شد. فرض کنید کارآگاه پیش از ورود به صحنه جرم اطلاعاتی درباره مظنون‌ها دارد: یکی از مظنون‌ها پیرمردی است که ظاهراً چند هفته در بیمارستان بوده و مظنون دیگر کودکی است که شواهد قبلی بیشتر به سمت او اشاره می‌کنند. در این حالت، prior کارآگاه به شدت به سمت کودک است.

اما در صحنه جرم، کارآگاه ردپای بزرگی پیدا می‌کند؛ مثلاً ساینز ۴۳. این شواهد جدید، یعنی likelihood، بیشتر با پیرمرد سازگار است تا کودک. اگر prior کارآگاه خیلی قوی باشد، ممکن است بگوید کودک کفش شخص دیگری را پوشیده است. یعنی با اینکه شواهد جدید علیه باور قبلی است، کارآگاه تلاش می‌کند همان باور قبلی را حفظ کند. این مثال نشان می‌دهد فقط احتمال‌ها کافی نیستند؛ وزن یا اطمینان هر بخش نیز مهم است.

۴.۸ Precision

Precision یعنی میزان اطمینان یا وزنی که به یک منبع اطلاعاتی می‌دهیم. در این بحث دو نوع precision داریم:

- اطمینان به دانش قبلی (Precision Prior)؛

- اطمینان به ورودی یا شواهد جدید (Precision Input).

اگر precision prior زیاد باشد، فرد به دانش قبلی خود بسیار اعتماد دارد. اگر precision input زیاد باشد، فرد به ورودی جدید یا شواهد فعلی اعتماد زیادی دارد.

۱.۴.۸ Precision Prior

Precision prior زمانی زیاد می‌شود که دانش قبلی بر اساس شواهد زیاد شکل گرفته باشد، در طول زمان طولانی تأیید شده باشد، با تجربه‌های متعدد سازگار باشد، یا در حوزه‌ای باشد که فرد در آن تخصص دارد.

مثال کلاس، تابش نور خورشید از بالا بود. از کودکی بارها تجربه کرده‌ایم که نور خورشید از بالا می‌تابد، بنابراین precision این prior بسیار بالاست و یک مشاهده تصادفی یا حرف یک نفر به راحتی آن را تغییر نمی‌دهد.

Precision prior وقتی کم است که اطلاعات قبلی کم باشد، داده‌ها پراکنده باشند، فرد تجربه یا تخصص کافی نداشته باشد، یا برای اولین بار با موضوعی مواجه شده باشد. در این حالت، چون prior ضعیف است، فرد ممکن است بیش از حد به اولین ورودی‌ها تکیه کند. این خود می‌تواند منبعی برای سوگیری شناختی باشد.

۲.۴.۸ مثال رسانه و تخصص

استاد مثالی درباره رسانه‌ها زد. گاهی گفته می‌شود فرد نباید هر رسانه‌ای را دنبال کند، چون اگر precision prior او پایین باشد، اطلاعات رسانه می‌تواند به راحتی روی قضاوت او اثر بگذارد و باعث سوگیری شود. اما اگر فرد در یک حوزه

متخصص باشد، می‌تواند رسانه‌ها و دیدگاه‌های مختلف را بررسی کند، چون prior او قوی‌تر است و به‌سادگی دچار انحراف نمی‌شود.

۳.۴.۸ Precision Input

Precision input زمانی زیاد است که فرد به شواهد فعلی اعتماد زیادی داشته باشد؛ مثلاً خودش چیزی را با دقت دیده باشد، منبع اطلاعات قابل اعتماد باشد، شواهد مختلف یکدیگر را تأیید کنند، فرد از نظر جسمی و ذهنی در وضعیت خوبی باشد، یا وقت کافی برای مشاهده و دقت داشته باشد. وقتی کسی می‌گوید «نه، من مطمئنم که خودم دیدم»، یعنی precision input او بالاست.

Precision input زمانی کم است که فرد خسته باشد، عجله داشته باشد، در وضعیت جسمی یا ذهنی مناسبی نباشد، مشاهده مبهم باشد، یا منبع اطلاعات قابل اعتماد نباشد. مثلاً اگر فرد خیلی خسته باشد و چیزی ببیند، ممکن است خودش بگوید «شاید اشتباه دیدم». در این حالت، precision input پایین است.

۴.۴.۸ ترکیب ورودی و باور قبلی

وقتی هم prior و هم input داریم، هرکدام با precision خود در باور نهایی اثر می‌گذارند. فرمول میانگین posterior به‌صورت زیر نوشته شد:

$$\mu_{\text{post}} = \frac{\pi_p \mu_p + \pi_i \mu_i}{\pi_p + \pi_i}$$

در این فرمول:

- μ_{post} : میانگین باور به‌روزشده؛
- μ_p : میانگین prior؛
- μ_i : میانگین input؛
- π_p : precision prior؛
- π_i : precision input.

این فرمول شبیه میانگین‌گیری وزنی است. هر منبع اطلاعاتی به‌اندازهٔ precision خود در خروجی نهایی اثر می‌گذارد.

۵.۴.۸ بازنویسی فرمول با خطای پیش‌بینی

فرمول بالا بازنویسی شد تا ارتباط آن با prediction error روشن‌تر شود:

$$\mu_{\text{post}} = \mu_p + \frac{\pi_i}{\pi_p + \pi_i} (\mu_i - \mu_p)$$

در اینجا:

$$(\mu_i - \mu_p)$$

همان prediction error است و عبارت:

$$\frac{\pi_i}{\pi_p + \pi_i}$$

وزن خطاست. پس باور جدید برابر است با:

$$\text{Prior} + \text{Weight} \times \text{Prediction Error}$$

۵.۸ چه زمانی باور به روزرسانی نمی شود؟

طبق فرمول، prior زمانی به روزرسانی نمی شود که $\mu_{\text{post}} = \mu_p$ باشد. این اتفاق در چند حالت رخ می دهد.

۱.۵.۸ حالت اول: خطای پیش بینی صفر باشد

اگر:

$$\mu_i - \mu_p = 0$$

ورودی جدید با پیش بینی قبلی کاملاً سازگار است. در این حالت خطایی وجود ندارد، پس چیزی برای اصلاح کردن نیست و سیستم در تعادل می ماند. مثال کلاس این بود که فرد به لپ تاپ نگاه می کند و چیزی خلاف انتظار رخ نمی دهد؛ پس مغز نیازی به اصلاح مدل ذهنی ندارد.

۲.۵.۸ حالت دوم: Precision Input صفر یا بسیار کم باشد

اگر:

$$\pi_i \approx 0$$

فرد به ورودی جدید تقریباً هیچ اعتمادی ندارد. در این حالت حتی اگر ورودی جدید با prior ناسازگار باشد، اثر چندانی روی باور نهایی ندارد.

۳.۵.۸ حالت سوم: Precision Prior بسیار بیشتر باشد

اگر:

$$\pi_p \gg \pi_i$$

آنگاه:

$$\frac{\pi_i}{\pi_p + \pi_i} \approx 0$$

در نتیجه:

$$\mu_{\text{post}} \approx \mu_p$$

یعنی باور قبلی تقریباً بدون تغییر باقی می ماند. این همان حالتی است که در confirmation bias دیده می شود: فرد آن قدر به مدل ذهنی قبلی خود اعتماد دارد که شواهد جدید را نادیده می گیرد.

۶.۸ کاهش Confirmation Bias با چارچوب بیزی

وقتی prediction error وجود دارد، سیستم باید به نوعی خطا را کم کند. دو راه کلی وجود دارد. راه اول تغییر دادن prior است؛ یعنی فرد باور یا مدل ذهنی قبلی خود را به روزرسانی کند. برای مثال، اگر قبلاً فکر می کرد محیط امن است اما شواهدی خلاف آن ببیند، ممکن است مدل ذهنی خود را تغییر دهد و بگوید شاید اینجا خطرناک باشد.

راه دوم تغییر دادن input است؛ یعنی فرد ورودی جدیدی بگیرد تا ابهام رفع شود: سر خود را برگرداند، نور را روشن کند، دوباره نگاه کند، از زاویه دیگری بررسی کند یا داده جدید جمع کند. در مثال جسمی روی زمین، فرد ممکن است نور را روشن کند تا بفهمد آن جسم چیست. در این حالت، input تغییر می کند تا prediction error کاهش یابد.

اگر ورودی جدیدی در دسترس نباشد یا نتوانیم input را تغییر دهیم، معمولاً prior غالب می شود. در مثال کارآگاه، اگر شواهد جدید کافی پیدا نشود، کارآگاه بر اساس همان اطلاعات قبلی قضاوت می کند.

برای کاهش سوگیری تأییدی، باید کاری کنیم اثر بیش از حد prior کم شود. طبق فرمول، دو راه اصلی وجود دارد:

۱. کاهش precision prior؛ یعنی مدل کمتر به پیش فرض های قبلی خود تکیه کند.

۲. افزایش precision input؛ یعنی ورودی جدید دقیق، واضح و قابل اعتماد باشد.

اگر:

$$\pi_i \gg \pi_p$$

آنگاه:

$$\frac{\pi_i}{\pi_p + \pi_i} \approx 1$$

پس:

$$\mu_{\text{post}} = \mu_p + (\mu_i - \mu_p)$$

و در نتیجه:

$$\mu_{\text{post}} = \mu_i$$

یعنی باور نهایی تقریباً کاملاً بر اساس input شکل می گیرد، نه prior. در این حالت، سوگیری تأییدی کاهش می یابد، چون مدل مجبور می شود به شواهد جدید توجه کند.

۷.۸ ارتباط Predictive Coding با مدل های زبانی

در ادامه، بحث به LLM ها و Transformer ها وصل شد. اگر بخواهیم در یک مدل زبانی سوگیری تأییدی را کم کنیم، باید کاری کنیم مدل کمتر بر prior داخلی خود، یعنی وزن ها و پارامترهای آموخته شده، تکیه کند و بیشتر به input فعلی توجه کند.

در این چارچوب، تکنیک‌های prompting را می‌توان به‌عنوان روش‌هایی برای افزایش precision input یا کاهش اثر precision prior تحلیل کرد.

۱.۷.۸ RAG

Retrieval-Augmented Generation یا RAG یعنی به مدل اطلاعات بیرونی و بازیابی شده داده شود. این اطلاعات نقش input با precision بالا را دارد. اگر مدل مجبور شود بر اساس RAG پاسخ دهد، اثر prior عمومی مدل کمتر می‌شود و پاسخ بیشتر بر اساس داده ورودی تولید می‌شود.

۲.۷.۸ Few-shot Prompting

در few-shot prompting، چند نمونه مشخص به مدل داده می‌شود. این نمونه‌ها سیگنال ورودی را واضح‌تر می‌کنند و واریانس آن را کاهش می‌دهند. در نتیجه precision input بالا می‌رود و مدل کمتر به پیش‌فرض‌های عمومی خود تکیه می‌کند.

۳.۷.۸ Negative Prompting

در negative prompting می‌توان به مدل گفت از برخی فرض‌ها، سوگیری‌ها یا جهت‌گیری‌ها اجتناب کند. برای مثال: «فرضیات خودت را نادیده بگیر»، «از سوگیری فرهنگی پرهیز کن»، یا «فقط بر اساس شواهد ارائه‌شده پاسخ بده». این کار به‌نوعی precision prior را کاهش می‌دهد یا دست‌کم اثر برخی priorهای ناخواسته را محدود می‌کند.

۴.۷.۸ Role Playing

در role playing، به مدل یک نقش مشخص داده می‌شود؛ مثلاً «تو یک قاضی بی‌طرف هستی». این کار مدل را از فضای عمومی پاسخ‌دهی وارد یک فضای تخصصی‌تر و کنترل‌شده‌تر می‌کند. در نتیجه، prior عمومی مدل ضعیف‌تر می‌شود و مدل بیشتر مطابق نقش تعیین‌شده رفتار می‌کند.

۸.۸ تفاوت Predictive Coding با شبکه‌های عصبی رایج

در بخش پایانی جلسه، تفاوت predictive coding با شبکه‌های عصبی فعلی بررسی شد. شبکه‌های عصبی رایج معمولاً به این صورت توضیح داده می‌شوند:

۱. ورودی دریافت می‌کنند.

۲. محاسبات feedforward انجام می‌دهند.

۳. خروجی تولید می کنند.

۴. خطا با loss function محاسبه می شود.

۵. خطا با backpropagation به عقب منتقل می شود.

۶. وزن ها با gradient descent به روزرسانی می شوند.

اگر شبکه ای با لایه ورودی، لایه پنهان و لایه خروجی داشته باشیم، هر نورون ورودی با وزن هایی به نورون های بعدی وصل است. محاسبه ای از این نوع انجام می شود:

$$h = a_1w_1 + a_2w_2 + b$$

سپس خروجی تولید می شود و با مقدار هدف مقایسه می شود. برای مثال، با خطای میانگین مربعات:

$$\text{Loss} = (y - \hat{y})^2$$

بعد با backpropagation محاسبه می شود هر وزن چقدر باید تغییر کند تا خطا کم شود.

۱.۸.۸ خطای محلی در مغز، خطای سراسری در شبکه

در شبکه های عصبی فعلی، خطا معمولاً در انتهای شبکه محاسبه می شود؛ یعنی پس از اینکه ورودی از کل شبکه عبور کرد و خروجی تولید شد. این خطا سپس به عقب منتقل می شود و وزن ها اصلاح می شوند. بنابراین خطا در این شبکه ها عمدتاً global یا end-to-end است.

اما در predictive coding، هر بخش از مغز یا هر سطح از پردازش می تواند به صورت محلی اختلاف بین پیش بینی و ورودی خود را محاسبه کند. لازم نیست همه چیز تا انتهای یک سلسله مراتب برود و سپس از بالا دستور اصلاح صادر شود. استاد این تفاوت را این گونه توضیح داد: انگار همه بخش ها خودشان را مسئول خطا می دانند، نه اینکه منتظر بمانند خطا فقط در انتهای شبکه مشخص شود.

۲.۸.۸ جهت جریان اطلاعات

در شبکه های عصبی فعلی، جریان اصلی اطلاعات معمولاً از پایین به بالا یا از ورودی به خروجی است:

Input → Hidden Layers → Output

اما در مغز طبق predictive coding، علاوه بر جریان bottom-up، جریان top-down هم وجود دارد. مغز دائماً از سطوح بالاتر پیش بینی تولید می کند و آن را با ورودی های حسی پایین تر مقایسه می کند.

پس تفاوت دوم این است: در شبکه عصبی، جریان اطلاعات عمدتاً bottom-up است، اما در مغز هم bottom-up داریم و هم top-down. در شبکه عصبی، پیش بینی معمولاً پس از دریافت ورودی و عبور از شبکه تولید می شود؛ اما در مغز، پیش بینی از قبل و به صورت مداوم در حال تولید است و سپس با ورودی مقایسه می شود.

۳.۸.۸ آیا وزن‌های شبکه همان Prior هستند؟

در بحث دانشجویان، این سؤال مطرح شد که آیا وزن‌هایی که شبکه عصبی تاکنون یاد گرفته، می‌تواند prior آن محسوب شود؟ پاسخ استاد مثبت بود: پارامترها و وزن‌های مدل، دانش قبلی یا prior مدل محسوب می‌شوند و مدل هنگام پاسخ‌دهی بر اساس همین وزن‌ها عمل می‌کند. اما تفاوت اینجاست که شبکه‌های عصبی فعلی معمولاً مانند مغز، در هر لحظه و به صورت محلی prediction error را محاسبه و اصلاح نمی‌کنند.

۹.۸ مقایسه مفهومی با GAN

یکی از دانشجویان پرسید آیا در GAN چیزی شبیه جریان top-down یا predictive coding وجود دارد، چون GAN یک بخش تولیدکننده و یک بخش مقایسه‌کننده یا تشخیص‌دهنده دارد. استاد گفت این ایده قابل تأمل است، چون در GAN یک بخش generative وجود دارد و ممکن است در نگاه اول شبیه بخش top-down در predictive coding به نظر برسد.

اما استاد تأکید کرد که مطمئن نیست این نگاهت دقیق باشد و نیاز به فکر و بررسی بیشتر دارد. احتمالاً نمی‌توان به سادگی خروجی بخش adversarial یا generator را معادل جریان top-down در مغز دانست. قرار شد این موضوع بیشتر بررسی شود.

اعلام پایانی

استاد گفت احتمالاً برای Predictive Coding تمرینی در سامانه درس قرار می‌دهد. درباره ارزیابی درس نیز توضیح داده شد که با توجه به شرایط، احتمالاً ارزیابی بیشتر به سمت امتحان کتبی می‌رود؛ در امتحان کتبی بیشتر مفاهیم پرسیده می‌شوند و دانشجویان باید بتوانند آن‌ها را توضیح دهند. اگر دسترسی به مقالات و شرایط مناسب فراهم شود، ممکن است کار مطالعاتی یا ارائه نیز بخشی از نمره باشد. درباره دلایل‌ها نیز گفته شد که احتمالاً همه در یک بازه زمانی تنظیم می‌شوند و شاید تمرین‌های بیشتری اضافه شود.

فصل ۹

توجه انتخابی

در آغاز این جلسه، استاد مسیر بحث‌های قبل را مرور کرد. در معماری ACT، انسان با محیط بیرونی در تعامل است. اطلاعات محیط از راه ماژول‌های حسی مانند بینایی و شنوایی دریافت می‌شود و بخشی از آن وارد حافظه کوتاه‌مدت می‌شود. حافظه کوتاه‌مدت با حافظه بلندمدت در تعامل است: از آن اطلاعات را retrieve می‌کند و در مقابل، تجربه‌هایی که بارها در حافظه کوتاه‌مدت پردازش و به حافظه بلندمدت ارجاع داده شوند، به تدریج در حافظه بلندمدت تثبیت می‌شوند. این همان فرایند learning است. هر دو حافظه نیز ممکن است در گذر زمان دچار forgetting شوند. در زبان بیزی، حافظه بلندمدت را می‌توان شبیه prior knowledge دانست. ورودی‌های فعلی محیط که از راه حافظه کوتاه‌مدت وارد پردازش می‌شوند، شبیه likelihood هستند. ترکیب این دو یک posterior می‌سازد و این باور به‌روزشده می‌تواند در طول زمان دانش قبلی را تغییر دهد:

Long-Term Memory $\approx$ Prior Knowledge,

Short-Term Input $\approx$ Likelihood,

Updated Belief $\approx$ Posterior.

همچنین بحث حافظه کاری و Cognitive Load یادآوری شد. حافظه کاری محل پردازش‌های فعال ذهنی است، اما ظرفیت محدودی دارد. به همین دلیل، راهکارهایی مانند chunking اهمیت پیدا می‌کنند؛ مثلاً یک شماره تلفن را به چند قطعه کوچک‌تر تقسیم می‌کنیم تا فشار کمتری به حافظه کاری وارد شود.

از اینجا بحث اصلی جلسه آغاز شد: ما در محیطی بسیار بزرگ و پر از اطلاعات زندگی می‌کنیم. حس‌های مختلف دائماً از محیط داده می‌گیرند و این داده‌ها ابتدا در سطح حافظه حسی قرار می‌گیرند. حافظه حسی ظرفیت زیادی دارد، اما اطلاعات در آن تنها برای زمانی بسیار کوتاه باقی می‌ماند. وقتی وارد یک محیط می‌شویم، هم‌زمان صداها، چهره‌ها، اشیا، بوها و حس‌های بدنی مختلفی را دریافت می‌کنیم؛ با این حال همه این اطلاعات وارد حافظه کوتاه‌مدت نمی‌شود. هم ظرفیت حافظه محدود است و هم توان پردازش.

Selective Attention یعنی انتخاب بخشی از اطلاعات محیط از میان حجم بزرگی از ورودی‌ها، به گونه‌ای که همان بخش بتواند از حافظه حسی عبور کند، وارد حافظه کوتاه‌مدت شود و در پردازش آگاهانه نقش بگیرد.

۱.۹ جایگاه توجه انتخابی در معماری شناختی

مسیر کلی پردازش در این بحث چنین است: محیط اطلاعات زیادی تولید می‌کند؛ سیستم‌های حسی این اطلاعات را دریافت می‌کنند؛ همهٔ این اطلاعات در سطح حافظهٔ حسی ثبت می‌شود؛ سپس فقط بخشی از آن‌ها انتخاب می‌شود و وارد حافظهٔ کوتاه‌مدت می‌شود. در حافظهٔ کوتاه‌مدت، پردازش معنایی و تصمیم‌گیری انجام می‌شود و در نهایت یک Action یا عمل تولید می‌شود.

بنابراین مسئلهٔ اصلی توجه انتخابی این است که از میان حجم زیادی از اطلاعات محیط، چه چیزی انتخاب می‌شود و چگونه وارد حافظهٔ کوتاه‌مدت می‌شود.

۲.۹ تفاوت توجه شناختی با Attention در مدل‌های زبانی

وقتی در مدل‌هایی مانند ChatGPT دربارهٔ attention صحبت می‌کنیم، معمولاً منظور توجه مدل به بخش‌های مختلف prompt است. اما توجه انتخابی در این فصل به مرحله‌ای قبل‌تر مربوط می‌شود.

در یک چت‌بات معمولی، کاربر خودش محیط سیستم را مشخص کرده است؛ prompt همان محیطی است که مدل باید بر اساس آن پاسخ دهد. بنابراین سیستم لازم نیست از کل محیط فیزیکی چیزی را انتخاب کند. اما در سامانه‌هایی مانند دستیارهای صوتی، از جمله Siri یا Alexa، مسئله متفاوت است. این سامانه‌ها در محیط واقعی قرار دارند و دائماً صداهای اطراف را می‌شنوند. آن‌ها باید از میان همهٔ صداها، بخش مرتبط را انتخاب کنند.

برای نمونه، وقتی کاربر عبارت‌هایی مانند Hey Siri یا Alexa را می‌گوید، سامانه باید از میان صداهای محیط تشخیص دهد که کلمهٔ فعال‌ساز گفته شده است. پس بحث Selective Attention بیشتر به سیستم‌هایی مربوط است که واقعاً با محیط بیرونی در تعامل‌اند و باید از میان داده‌های محیطی، بخش مربوط به خود را انتخاب کنند.

۳.۹ مسئلهٔ گلوگاه توجه

برای فهم توجه انتخابی، مثال رایج مهمانی مطرح شد. در یک مهمانی افراد مختلفی اطراف ما هستند و هر کدام در حال صحبت‌اند؛ اما ما معمولاً فقط می‌توانیم روی حرف یک نفر تمرکز کنیم. اگر دوستان روبه‌روی ما نشسته باشد و شماره تلفنی را بگویید، می‌توانیم آن شماره را وارد حافظهٔ کوتاه‌مدت کنیم، آن را chunk کنیم و بعداً از آن استفاده کنیم. اما اگر هم‌زمان چند نفر دیگر هم شماره تلفن بگویند، معمولاً نمی‌توانیم همهٔ آن‌ها را هم‌زمان پردازش کنیم.

این تجربه نشان می‌دهد در مسیر پردازش اطلاعات یک bottleneck یا گلوگاه وجود دارد. اطلاعات زیادی در محیط حاضر است، اما فقط بخشی از آن می‌تواند از این گلوگاه عبور کند و وارد پردازش آگاهانه شود.

۴.۹ Broadbent's Filter Theory

نخستین نظریه‌ای که در این جلسه دربارهٔ توجه انتخابی مطرح شد، Filter Theory از Broadbent بود. این نظریه در سال ۱۹۵۸ مطرح شد و به آن Early Selection نیز گفته می‌شود.

۱.۴.۹ Early Selection

طبق نظریهٔ Early Selection، انتخاب خیلی زود اتفاق می‌افتد؛ یعنی پیش از آنکه معنا را درک کنیم. ابتدا اطلاعات بر اساس ویژگی‌های فیزیکی یا آکوستیکی فیلتر می‌شوند. در مثال مهمانی، ممکن است صدای دوستان انتخاب شود چون نزدیک‌تر است، بلندتر صحبت می‌کند، صدایش برای ما آشناست، جهت صدا مشخص است، یا می‌دانیم با او در حال گفت‌وگو هستیم.

پس از انتخاب یک ورودی، تازه پردازش معنایی انجام می‌شود. یعنی ابتدا مشخص می‌کنیم به کدام صدا گوش بدهیم و سپس معنای حرف او را می‌فهمیم. مسیر پردازش در این نگاه چنین است:

۱. دریافت ورودی‌های متعدد از محیط؛

۲. انتخاب یک ورودی بر اساس ویژگی‌های فیزیکی یا آکوستیکی؛

۳. حذف یا نادیده گرفتن ورودی‌های دیگر؛

۴. پردازش معنایی ورودی انتخاب‌شده؛

۵. ورود آن به حافظهٔ کوتاه‌مدت؛

۶. تصمیم‌گیری و انجام عمل.

در این مدل، فیلتر قبل از درک معنا قرار دارد.

۲.۴.۹ محدودیت‌های انتخاب زود هنگام

نظریهٔ Early Selection بخشی از تجربهٔ ما را توضیح می‌دهد؛ مثلاً اینکه چرا وقتی با یک نفر صحبت می‌کنیم، معمولاً حرف بقیه را نمی‌فهمیم. اما با یک تجربهٔ مهم مشکل دارد: گاهی در مهمانی مشغول صحبت با یک نفر هستیم، اما اگر کسی در گوشه‌ای از اتاق اسم ما را صدا بزند، متوجه می‌شویم؛ حتی اگر صدا آرام باشد.

اگر طبق این نظریه ورودی‌های دیگر حذف شده باشند، روشن نیست چگونه توانسته‌ایم اسم خود را از میان آن‌ها تشخیص دهیم. همین مسئله زمینه را برای نظریهٔ دیگری فراهم کرد.

۵.۹ Late Selection

Late Selection پس از نظریهٔ برادبنت مطرح شد؛ از جمله در کار Deutsch & Deutsch در سال ۱۹۶۳. در این

نگاه، فیلتر بعد از پردازش معنایی قرار دارد.

۱.۵.۹ انتخاب پس از پردازش معنایی

طبق Late Selection، ذهن ابتدا ورودی‌های مختلف را تا سطح معنا پردازش می‌کند و سپس تصمیم می‌گیرد کدام ورودی مهم‌تر است و باید وارد حافظه کوتاه‌مدت شود. در مثال مهمانی، اگر کسی اسم ما را بگوید، چون معنای آن برای ما مهم است، آن ورودی از فیلتر عبور می‌کند.

مسیر پردازش در این مدل چنین است:

۱. دریافت ورودی‌های متعدد از محیط؛

۲. پردازش معنایی همه یا بخش زیادی از ورودی‌ها؛

۳. انتخاب ورودی مهم‌تر بر اساس معنا؛

۴. ورود ورودی انتخاب‌شده به حافظه کوتاه‌مدت؛

۵. تصمیم‌گیری و عمل.

در این مدل، فیلتر بعد از درک معنا قرار دارد.

۲.۵.۹ محدودیت‌های انتخاب دیر هنگام

Late Selection می‌تواند توضیح دهد چرا وقتی کسی اسم ما را صدا می‌زند، آن را تشخیص می‌دهیم. اما مشکل اصلی آن این است که اگر ذهن بخواهد همه ورودی‌های محیط را تا سطح معنایی پردازش کند، بار پردازشی بسیار سنگینی ایجاد می‌شود.

در هر لحظه با حجم بزرگی از اطلاعات روبه‌رو هستیم. اگر قرار باشد همه این اطلاعات از نظر معنایی تحلیل شوند، مغز باید پردازش بسیار زیادی انجام دهد. این با تجربه واقعی ما نیز سازگار نیست، زیرا ما همیشه معنای همه صداها و اتفاق‌های اطراف را نمی‌فهمیم و بسیاری از آن‌ها اصلاً وارد آگاهی ما نمی‌شوند. بنابراین Late Selection نیز به‌تنهایی کافی نیست.

۶.۹ Perceptual Load Theory

برای جمع کردن دو دیدگاه قبل، Perceptual Load Theory مطرح شد. این نظریه در میانه دهه ۱۹۹۰ مطرح شد و می‌گوید هر دو حالت Early Selection و Late Selection در ذهن انسان وجود دارند. مغز انسان پویاست و فقط از یک سازوکار ثابت استفاده نمی‌کند. بسته به شرایط، ممکن است انتخاب زود هنگام یا انتخاب دیر هنگام غالب شود.

عامل اصلی این تغییر، Perceptual Load یا بار ادراکی است: میزان فشاری که یک کار از نظر ادراکی و توجهی به ذهن وارد می‌کند. اگر کاری سخت، جدید، خطرناک یا پیچیده باشد، بار ادراکی بالایی دارد. اگر کاری ساده، تکراری یا آشنا

باشد، بار ادراکی پایینی دارد.

۱.۶.۹ بار ادراکی بالا

طبق Perceptual Load Theory، وقتی بار ادراکی زیاد است، ذهن از Early Selection استفاده می‌کند. یعنی زود انتخاب می‌کند و بیشتر ورودی‌های دیگر را کنار می‌گذارد، چون ظرفیت پردازشی کافی برای تحلیل همه ورودی‌ها وجود ندارد.

برای نمونه، رانندگی در شب، در جاده جدید، در شرایط خطرناک، در ترافیک پیچیده، یا در وضعیت آب‌وهوایی بد بار ادراکی بالایی دارد. در چنین شرایطی، بیشتر ظرفیت توجه صرف رانندگی می‌شود. ممکن است کسی در ماشین حرف بزند، اما بعداً بگوییم: «اصلاً متوجه نشدم چه گفتی.»

۲.۶.۹ بار ادراکی پایین

وقتی بار ادراکی کم است، ذهن ظرفیت اضافه دارد و می‌تواند ورودی‌های بیشتری را تا سطح معنا پردازش کند. در این حالت Late Selection امکان‌پذیرتر می‌شود. برای مثال، رانندگی در یک مسیر آشنا و ساده بار ادراکی کمتری دارد؛ در چنین حالتی می‌توانیم رانندگی کنیم، موسیقی گوش بدهیم و تا حدی با دیگران صحبت کنیم.

بنابراین بحث Early Selection و Late Selection را نباید به شکل «یکی درست است و دیگری غلط» دید. هر دو می‌توانند درست باشند، اما در شرایط متفاوت. در بار ادراکی بالا، انتخاب زود هنگام غالب می‌شود؛ در بار ادراکی پایین، انتخاب دیر هنگام امکان بیشتری پیدا می‌کند. سیستم توجه انسان انعطاف‌پذیر است و بسته به وضعیت محیط و دشواری کار، بین این دو حالت جابه‌جا می‌شود.

Treisman's Attenuation Theory ۷.۹

نظریه دیگری که در فضای بحث انتخاب زود هنگام و دیر هنگام مطرح شد، نظریه Treisman است. این نظریه با عنوان Attenuation Theory شناخته می‌شود و می‌توان آن را نظریه «تضعیف» نامید.

ایده اصلی این نظریه آن است که فیلتر توجه را نباید کاملاً صفر و یکی در نظر بگیریم. در نظریه Broadbent، فیلتر تا حدی شبیه یک دروازه صفر و یکی بود: یک پیام عبور می‌کند و بقیه حذف می‌شوند. اما Treisman می‌گوید اطلاعات حذف نمی‌شوند، بلکه تضعیف یا تقویت می‌شوند.

۱.۷.۹ فیلتر تضعیفی

طبق نظریه Treisman، همه ورودی‌ها تا حدی از فیلتر عبور می‌کنند، اما با شدت‌های متفاوت. در مهمانی، صدای دوستان ممکن است با شدت زیاد عبور کند، صدای افراد دیگر با شدت کم عبور کند، و صدای کسی که اسم ما را می‌گوید، با وجود ضعیف بودن، به دلیل اهمیتش فعال شود.

پس فیلتر در این نظریه به جای حذف کامل اطلاعات، آن‌ها را وزن‌دهی می‌کند. برخی ورودی‌ها تقویت و برخی تضعیف می‌شوند.

۲.۷.۹ آستانه و محرک‌های مهم

مفهوم مهم دیگر در نظریه Treisman، Threshold یا آستانه است. برخی کلمات یا محرک‌ها آستانه پایین‌تری دارند؛ یعنی برای فعال شدن به انرژی یا شدت زیادی نیاز ندارند. اسم خودمان معمولاً چنین وضعی دارد، چون آن را سال‌ها شنیده‌ایم و برای ما آشنا و مهم است. حتی اگر کسی اسممان را آرام یا مبهم بگوید، ممکن است آن را تشخیص دهیم. در این نظریه، اطلاعات حذف نمی‌شود؛ بخشی از اطلاعات تضعیف و بخشی تقویت می‌شود. محرک‌هایی که آستانه پایین دارند، راحت‌تر فعال می‌شوند و پس از عبور از آستانه، پردازش معنایی انجام می‌شود.

این ایده را می‌توان به سیستم‌های هوشمند نیز ربط داد. دستیارهای صوتی معمولاً یک wake word دارند؛ مانند Hey Siri یا Alexa. سامانه باید در میان همه صداهای محیط، این عبارت خاص را تشخیص دهد. این عبارت برای سیستم مانند محرکی با آستانه پایین طراحی شده است: سیستم دائماً همه مکالمات را به صورت عمیق تحلیل نمی‌کند، اما نسبت به wake word حساس است. وقتی آن را تشخیص داد، فعال می‌شود و وارد پردازش جدی‌تر می‌شود. این تشخیص می‌تواند بر اساس ویژگی‌های آکوستیکی، الگوی صوتی، یا در سامانه‌های پیچیده‌تر همراه با پردازش معنایی انجام شود.

۸.۹ پردازش سری، موازی و جابه‌جایی توجه

در بحث توجه انتخابی، یک نکته مهم این است که نمی‌توانیم همه کارهای پیچیده را هم‌زمان به صورت موازی انجام دهیم. مثلاً اگر از ما بخواهند هم‌زمان دو محاسبه ۳۷×۱۷ و ۸۱×۹ را انجام دهیم، نمی‌توانیم واقعاً هر دو را هم‌زمان و موازی حل کنیم. ممکن است خیلی سریع بین آن‌ها جابه‌جا شویم، اما این بیشتر شبیه task switching است، نه پردازش موازی واقعی.

کارهای ساده ممکن است به ظاهر هم‌زمان انجام شوند، اما کارهایی که به تحلیل، محاسبه، تصمیم‌گیری یا درک معنایی سطح بالا نیاز دارند، معمولاً باید به صورت سری پردازش شوند: ابتدا یک ورودی پردازش می‌شود و سپس ورودی بعدی. به همین دلیل، سیستم توجه نیازمند سازوکاری است که مشخص کند در هر لحظه کدام ورودی باید از گلوگاه توجه عبور کند.

جمع‌بندی جلسه این بود که توجه انتخابی توضیح می‌دهد چگونه از میان حجم عظیم اطلاعات محیط، فقط بخشی انتخاب می‌شود و وارد حافظه کوتاه‌مدت و پردازش آگاهانه می‌شود. چهار نظریه اصلی مطرح شد: Broadbent's Filter Theory یا Early Selection که انتخاب را پیش از درک معنا می‌داند؛ Late Selection که انتخاب را پس از پردازش معنایی قرار می‌دهد؛ Perceptual Load Theory که هر دو حالت را وابسته به بار ادراکی می‌داند؛ و man's Attenuation Theory که فیلتر توجه را صفر و یکی نمی‌بیند، بلکه آن را سازوکاری برای تضعیف، تقویت و عبور محرک‌های دارای آستانه پایین می‌داند.

بخش چهارم

زبان، گفتار و مدل‌های شنیداری

فصل ۱۰

از توجه انتخابی تا دستیار صوتی

استاد جلسه را با مرور بحث توجه انتخابی آغاز کرد. از منظر شناختی، دو حالت اصلی مطرح شده بود: Early Selection و Late Selection. در Early Selection، وقتی بار ادراکی زیاد است، سیستم در همان مراحل اولیه پردازش بخشی از ورودی را انتخاب می‌کند و بقیه را کنار می‌گذارد. در Late Selection، وقتی بار ادراکی کم‌تر است، پردازش عمیق‌تری انجام می‌شود و انتخاب می‌تواند در سطح بالاتر، مثلاً پس از درک معنا، رخ دهد.

پرسش اصلی این جلسه این بود: اگر یک ماشین بخواهد به گفتار توجه کند، چگونه این کار را انجام می‌دهد؟ هدف، نگاشت مفاهیم شناختی توجه انتخابی به مدل‌های محاسباتی پردازش گفتار و به‌ویژه ASR یا تشخیص خودکار گفتار بود. تمرکز روی سامانه‌هایی مانند دستیارهای صوتی، از جمله Siri و Alexa بود؛ سامانه‌هایی که معمولاً در محیط به‌صورت always-on حضور دارند و باید از میان صداهای محیط، بخش مرتبط را انتخاب کنند.

۱.۱۰ نگاشت نظریه‌های توجه به سیستم‌های گفتاری

اگر یک دستیار صوتی بخواهد همیشه همه صداهای محیط را به متن تبدیل کند و سپس روی متن پردازش معنایی انجام دهد، هزینه محاسباتی و مصرف انرژی بسیار زیادی خواهد داشت. به همین دلیل، این سامانه‌ها معمولاً دو مرحله دارند: مرحله سبک‌تر برای تشخیص نشانه فعال‌ساز، و مرحله سنگین‌تر برای فهم دستور کاربر.

در این نگاشت، تشخیص اولیه کلمه فعال‌ساز به Early Selection نزدیک است، چون سیستم هنوز وارد فهم معنا نمی‌شود. پس از تشخیص آن، پردازش عمیق‌تر آغاز می‌شود و این مرحله به Late Selection نزدیک‌تر است.

۲.۱۰ Wake Word در انسان و ماشین

در دستیارهای صوتی، کلمه یا عبارت فعال‌ساز را wake word یا wake-up word می‌نامند؛ مانند Hey Siri یا Alexa. سیستم باید از میان صداهای محیط تشخیص دهد که آیا این عبارت گفته شده است یا نه.

استاد این پرسش را مطرح کرد که wake word انسان‌ها چیست. پاسخ مطرح‌شده این بود که نام خود فرد می‌تواند

چنین نقشی داشته باشد. اگر در محیطی شلوغ اسم ما گفته شود، ممکن است توجه ما به آن جلب شود. ابتدا یک نشانه آکوستیکی یا کلمه خاص توجه را فعال می‌کند و سپس پردازش عمیق‌تر آغاز می‌شود.

۳.۱۰ مرحله انتخاب زود هنگام در دستیار صوتی

در مرحله انتخاب زود هنگام، سیستم هنوز وارد درک معنا نمی‌شود و فقط به دنبال تشخیص یک یا چند کلمه خاص است. در این مرحله، سیستم همه گفتار را به متن تبدیل نمی‌کند، Natural Language Understanding انجام نمی‌دهد، و تنها به ویژگی‌هایی توجه می‌کند که نشان دهند wake word گفته شده است.

پس این مرحله سبک‌تر و کم‌مصرف‌تر است. ایده اصلی این است که برای انتخاب اولیه لازم نیست همه صداهای محیط از مسیر کامل فهم زبان عبور کنند؛ خود سیگنال صوتی و ویژگی‌های آکوستیکی آن می‌تواند برای تشخیص اولیه کافی باشد.

۴.۱۰ مرحله انتخاب دیر هنگام و فهم معنا

پس از تشخیص wake word، سیستم فعال می‌شود و وارد پردازش عمیق‌تر می‌گردد. در این مرحله معمولاً گفتار به متن تبدیل می‌شود، متن پردازش می‌شود، intent تشخیص داده می‌شود، slot filling انجام می‌شود و سپس task اجرا می‌شود.

برای نمونه، اگر کاربر بگوید: «برای من بلیت تهران به مشهد ساعت ۸ صبح بخر»، سیستم باید intent را خرید بلیت تشخیص دهد و slot‌هایی مانند مبدأ، مقصد و ساعت را استخراج کند. در اینجا سیستم وارد سطحی از Natural Language Understanding یا NLU شده است.

۵.۱۰ چرا گفتار معمولاً به متن تبدیل می‌شود؟

در این جلسه تأکید اصلی بر این بود که همه پردازش‌های گفتاری از ابتدا نیازمند تبدیل کامل گفتار به متن نیستند. برای Early Selection در ماشین، می‌توان از ویژگی‌های آکوستیکی خود موج استفاده کرد و مثلاً wake word را تشخیص داد. تبدیل گفتار به متن بیشتر در مرحله بعدی اهمیت پیدا می‌کند؛ یعنی وقتی سامانه باید معنای دستور، نیت کاربر و جزئیات لازم برای اجرای عمل را بفهمد.

با این حال، در معماری رایج، مسیر معمول چنین است:

Speech → Text → NLP Task → Action.

برای نمونه، کاربر می‌گوید: «برای من فلان پرواز را رزرو کن.» سیستم گفتار را به متن تبدیل می‌کند، مدل NLU متن را تحلیل می‌کند، intent را مثلاً Flight Booking تشخیص می‌دهد، slot‌هایی مانند مبدأ، مقصد، تاریخ و ساعت را استخراج می‌کند و سپس عمل مناسب را انجام می‌دهد.

استاد توضیح داد که این متن میانی از نظر نظری همیشه الزامی نیست. در بعضی کارهای سبک یا محدود، می توان مستقیماً از بردارهای گفتار یا ویژگی های صوتی استفاده کرد و کاری مانند طبقه بندی انجام داد. اما در بسیاری از سامانه ها تبدیل گفتار به متن نگه داشته می شود، چون متن ساختار مشخص تری دارد، مدل های NLP معمولاً روی متن عملکرد بهتری دارند، داده آموزشی متن بیشتر است، و اتصال متن به دیتابیس ها، موتورهای جست و جو، سرویس های نرم افزاری، API ها و اپلیکیشن ها ساده تر است.

استاد همچنین اشاره کرد که حتی برای انسان نیز در بسیاری از فرمان ها، متن می تواند پردازش روشن تری ایجاد کند. متن فاصله بین کلمات، علائم نگارشی، جداسازی کلمات و املاهای مشخص تری دارد و می تواند ابهام تلفظی را کاهش دهد. برای مثال، واژه هایی مانند «حیات» و «حیات» در گفتار ممکن است شبیه شنیده شوند، اما در متن و با توجه به بافت جمله، ابهام کم تر می شود.

۶.۱۰ سه جزء اصلی سیستم گفتاری

در مثال دستیار صوتی، سه مؤلفه پردازشی به صورت ضمنی مطرح شدند: تشخیص گفتار، فهم زبان و اجرای پاسخ یا عمل. تمرکز این جلسه بر دو مؤلفه اول بود: نخست تشخیص یا انتخاب نشانه صوتی، و سپس فهم معنایی دستور.

۱.۶.۱۰ ASR

ASR یا Automatic Speech Recognition به تبدیل گفتار به نمایش قابل پردازش، معمولاً متن، مربوط است. در مرحله پس از فعال شدن دستیار صوتی، گفتار کاربر باید به شکلی تبدیل شود که سامانه بتواند روی آن پردازش زبانی انجام دهد.

۲.۶.۱۰ NLU

NLU یا Natural Language Understanding مرحله ای است که در آن سیستم معنای دستور را تحلیل می کند. در مثال خرید بلیت، تشخیص اینکه کاربر قصد خرید بلیت دارد و استخراج مبدأ، مقصد و زمان، نمونه ای از همین مرحله است.

۳.۶.۱۰ TTS

TTS یا Text-to-Speech تبدیل متن به گفتار است. در دستیار صوتی، سیستم پس از فهمیدن درخواست کاربر ممکن است پاسخ دهد یا سؤال تکمیلی بپرسد. در این حالت، خروجی متنی دوباره به گفتار تبدیل می شود:

Text → Speech.

تمرکز فعلی درس بیشتر روی ASR و NLU است، نه TTS.

فصل ۱۱

مبانی پردازش سیگنال گفتار

برای فهم بهتر پردازش گفتار در ماشین، استاد بخشی از جلسه را به مسیر تولید گفتار در انسان و تبدیل سیگنال صوتی به ویژگی‌های عددی اختصاص داد. نکته اصلی این بود که ماشین در ابتدا متن یا معنا دریافت نمی‌کند، بلکه با موج صوتی خام روبه‌رو است. این موج باید به داده دیجیتال و سپس به ویژگی‌هایی تبدیل شود که برای طبقه‌بندی و پردازش قابل استفاده باشند.

۱.۱۱ تولید گفتار در انسان

گفتار انسان از تعامل چند بخش بدن تولید می‌شود: دیافراگم، ریه، نای، حنجره، تارهای صوتی، دهان، فک، زبان، بینی و دندان‌ها. فشار روی دیافراگم باعث خروج هوا از ریه می‌شود. هوا از مجرای صوتی عبور می‌کند و در مسیر عبور، توسط حنجره، تارهای صوتی، زبان، دهان، بینی و سایر اجزا شکل می‌گیرد. خروجی نهایی، موج صوتی است. این موج فقط حامل معنای جمله نیست. اطلاعات دیگری نیز در آن وجود دارد؛ از جمله هویت گوینده، جنسیت، لهجه، احساسات، ویژگی‌های فیزیکی مجرای صوتی و زیر یا بم بودن صدا. برای نمونه، اگر مجرای صوتی کوتاه‌تر باشد، صدا معمولاً زیرتر است و اگر بلندتر باشد، صدا بم‌تر می‌شود.

۲.۱۱ واج، واکه و فورمنت

گفتار را می‌توان در چند سطح بررسی کرد: کلمه، هجا و واج. واج‌ها به دو دسته اصلی تقسیم می‌شوند: صامت‌ها و مصوت‌ها یا واکه‌ها. در فارسی معمولاً شش واکه در نظر گرفته می‌شود: آ، ا، ای و او. در زبان‌های دیگر، تعداد واکه‌ها می‌تواند متفاوت باشد.

محل قرارگیری زبان هنگام تولید واکه‌ها اهمیت دارد. جایگاه‌های مختلف زبان باعث تولید صداهای متفاوت می‌شوند. هدف استاد از این توضیح این بود که نشان دهد خود موج صوتی، پیش از تبدیل شدن به متن، اطلاعات فیزیکی زیادی در خود دارد.

در اسپکتروگرام گفتار، معمولاً نوارهای تیره‌ای دیده می‌شود که formant نام دارند. Formant ها ناحیه‌هایی هستند که انرژی فرکانسی در آن‌ها بیشتر است و معمولاً با نام‌هایی مانند F1، F2، F3 و F4 مشخص می‌شوند. این ناحیه‌ها درباره کیفیت آوایی گفتار اطلاعات می‌دهند و می‌توانند برای تشخیص تفاوت‌های گفتاری مفید باشند.

در پاسخ به یک پرسش کلاسی، استاد توضیح داد که formant ها فقط بر اساس ترتیب دیده‌شدن نام‌گذاری نمی‌شوند، بلکه به محدوده‌های فرکانسی مشخص مربوط‌اند. اگر در محدوده F1 چیزی دیده نشود و نخستین نوار تیره در محدوده F3 باشد، آن نوار را F3 در نظر می‌گیریم، نه اینکه صرفاً چون اولین نوار دیده‌شده است، F1 نامیده شود.

۳.۱۱ موج صوتی خام

ماشین در ابتدا فقط یک موج صوتی خام دریافت می‌کند. این موج معمولاً روی دو محور قابل نمایش است: محور افقی زمان را نشان می‌دهد و محور عمودی شدت یا دامنه صدا را. هرچه صدا بلندتر باشد، دامنه موج از محور افقی فاصله بیشتری می‌گیرد.

موج صوتی همچنین دارای فرکانس است. نوسان سریع‌تر به فرکانس بالاتر و نوسان کندتر به فرکانس پایین‌تر مربوط می‌شود. اما ماشین نمی‌تواند مستقیماً با موج آنالوگ کار کند؛ برای پردازش، به عدد و بردار نیاز دارد. بنابراین گام نخست، تبدیل موج آنالوگ به داده دیجیتال است.

۴.۱۱ نمونه‌برداری و قانون نایکوئیست

برای دیجیتال کردن موج، از آن نمونه‌برداری می‌کنیم؛ یعنی در زمان‌های مختلف از روی موج نقطه‌هایی برمی‌داریم. هر نقطه نشان می‌دهد که در آن لحظه شدت صدا چقدر بوده است. هرچه نرخ نمونه‌برداری بیشتر باشد، موج با دقت بیشتری ثبت می‌شود. اگر نرخ نمونه‌برداری کم باشد، ممکن است تغییرات مهم موج از دست برود.

طبق قانون Nyquist، نرخ نمونه‌برداری باید حداقل دو برابر بیشترین فرکانس سیگنال باشد. برای مثال، اگر یک سیگنال فرکانسی برابر با ۴۰۰ هرتز داشته باشد، باید دست‌کم با نرخ ۸۰۰ نمونه در ثانیه از آن نمونه‌برداری کنیم. دلیل ساده آن این است که برای ثبت حداقل قله و کف موج، باید دست‌کم دو نمونه در هر چرخه داشته باشیم.

۵.۱۱ فریم‌بندی، پنجره‌گذاری و هم‌پوشانی

پس از نمونه‌برداری، داده صوتی به مجموعه‌ای از اعداد تبدیل می‌شود. اما پردازش کل موج به صورت یک‌جا مناسب نیست. همان‌طور که در پردازش متن، متن را به جمله، کلمه یا توکن تقسیم می‌کنیم، در پردازش صوت نیز سیگنال را به بخش‌های کوچک‌تر تقسیم می‌کنیم. این بخش‌ها frame نام دارند.

برای جدا کردن فریم‌ها از یک window استفاده می‌شود. پنجره روی موج حرکت می‌کند و هر بار بخشی از موج را انتخاب می‌کند. اگر پنجره ساده باشد، لبه‌های آن ناگهانی و تیز هستند. در پردازش گفتار معمولاً از Hamming

Window استفاده می‌شود؛ در این پنجره، مرکز فریم اهمیت بیشتری دارد، لبه‌های فریم وزن کم‌تری می‌گیرند و اثر تغییرات ناگهانی در مرز فریم‌ها کاهش می‌یابد.

فریم‌ها معمولاً با هم هم‌پوشانی دارند. یعنی فریم دوم دقیقاً پس از پایان فریم اول شروع نمی‌شود، بلکه بخشی از آن با فریم اول مشترک است. دلیل این کار حفظ پیوستگی سیگنال و جلوگیری از ازدست‌رفتن تغییرات مهم در مرز فریم‌هاست. اگر تغییر مهمی دقیقاً بین دو فریم رخ دهد و فریم‌ها هم‌پوشانی نداشته باشند، ممکن است آن تغییر از دست برود.

۶.۱۱ تبدیل فوریه و طیف فرکانسی

تا اینجا سیگنال در حوزه زمان بررسی شد؛ یعنی می‌دانستیم در هر لحظه شدت صدا چقدر است. اما موج صوتی ترکیبی از فرکانس‌های مختلف است. برای تجزیه موج به مؤلفه‌های فرکانسی از Fourier Transform استفاده می‌شود. تبدیل فوریه نشان می‌دهد موج اصلی از چه طیف‌های فرکانسی تشکیل شده است و شدت هر فرکانس چقدر است.

۷.۱۱ اسپکتروگرام

نتیجه اعمال تبدیل فوریه روی فریم‌های سیگنال صوتی، spectrogram است. اسپکتروگرام تصویری از تغییرات فرکانسی صدا در طول زمان است. در آن، یک محور زمان را نشان می‌دهد، محور دیگر فرکانس را، و شدت یا انرژی هر فرکانس با تیرگی یا روشنی نمایش داده می‌شود.

اسپکتروگرام کمک می‌کند بدون ورود به سطح معنا، الگوهای آکوستیکی گفتار را ببینیم. اسپکتروگرام گفتار با اسپکتروگرام تنفس یا نویز متفاوت است: گفتار انسان الگوهای فرکانسی مشخص‌تری دارد، در حالی که تنفس یا نویز ساختار متفاوتی ایجاد می‌کند. بنابراین حتی بدون تبدیل گفتار به متن، می‌توان برخی تفاوت‌ها را از روی ویژگی‌های آکوستیکی تشخیص داد؛ موضوعی که برای حذف نویز یا تشخیص گفتار از غیرگفتار مفید است.

استاد همچنین به تفاوت صدای زنان و مردان در اسپکتروگرام اشاره کرد. به طور کلی، صدای مردان معمولاً بم‌تر و صدای زنان معمولاً زیرتر است. صدای بم بیشتر در فرکانس‌های پایین‌تر انرژی دارد و صدای زیر در فرکانس‌های بالاتر نمود بیشتری پیدا می‌کند.

برای تشخیص wake word نیز می‌توان از همین ایده استفاده کرد. Wake word می‌تواند الگوی اسپکتروگرامی خاصی داشته باشد. نام یک فرد، حتی وقتی توسط افراد مختلف ادا می‌شود، ممکن است تفاوت‌هایی داشته باشد، اما همچنان در یک دسته آکوستیکی مشابه قرار گیرد. پس می‌توان با ویژگی‌هایی مانند اسپکتروگرام و formant‌ها طبقه‌بندی کرد که آیا صدای شنیده‌شده wake word است یا نه.

۸.۱۱ MFCC

تا این مرحله، اسپکتروگرام هنوز یک تصویر است. اما بسیاری از مدل‌های محاسباتی به نمایش عددی و برداری نیاز دارند. بنابراین باید اسپکتروگرام را به مجموعه‌ای از ویژگی‌های عددی تبدیل کنیم. اینجا مفهوم MFCC مطرح می‌شود.

MFCC مخفف Mel-Frequency Cepstral Coefficients است و می‌توان آن را «ضرایب کپسترال فرکانس مل» نامید. هدف MFCC استخراج ویژگی‌هایی از سیگنال صوتی است که برای پردازش گفتار مفید باشند. مسیر کلی چنین است:

۱. دریافت موج صوتی؛
۲. نمونه‌برداری؛
۳. فریم‌بندی؛
۴. اعمال window؛
۵. تبدیل فوریه و ساخت اسپکتروگرام؛
۶. عبور دادن اسپکتروگرام از Mel Filter Bank؛
۷. استخراج ضرایب MFCC.

۱.۸.۱۱ Mel Scale

Mel Scale تلاشی برای نزدیک‌تر کردن نمایش فرکانسی به ادراک انسان است. گوش انسان تغییرات فرکانس‌های پایین را دقیق‌تر تشخیص می‌دهد، اما در فرکانس‌های بالا حساسیت کم‌تری به تفاوت‌های جزئی دارد. بنابراین در نمایش مل، فرکانس‌ها به شکلی بازنمایی می‌شوند که با شنیدن انسان سازگارتر باشد.

۲.۸.۱۱ Mel Filter Bank

برای محاسبه MFCC از Mel Filter Bank استفاده می‌شود. این بانک مجموعه‌ای از فیلترهای مثلثی است که روی طیف فرکانسی اعمال می‌شوند. فیلترها با هم هم‌پوشانی دارند؛ در فرکانس‌های پایین فشرده‌ترند و در فرکانس‌های بالا بازتر و کم‌تراکم‌تر می‌شوند.

دلیل این طراحی آن است که شنوایی انسان نسبت به فرکانس‌های پایین حساس‌تر است و جزئیات بیشتری را در آن محدوده درک می‌کند. بنابراین در فرکانس‌های پایین رزولوشن بیشتری نگه داشته می‌شود و در فرکانس‌های بالا رزولوشن کم‌تر می‌شود.

۳.۸.۱۱ خروجی عددی و کاربرد در طبقه‌بندی

پس از اعمال Mel Filter Bank و پردازش‌های تکمیلی، اسپکتروگرام به مجموعه‌ای از ضرایب عددی تبدیل می‌شود. خروجی MFCC چیزی شبیه یک ماتریس یا مجموعه‌ای از بردارهاست که می‌توان روی آن‌ها پردازش انجام داد. این نوع خروجی برای مدل‌های یادگیری ماشین مناسب است.

استاد برای روشن تر شدن کاربرد MFCC مثالی مطرح کرد: فرض کنید بخواهیم از روی صدای افراد تشخیص دهیم که آیا فرد مبتلا به کووید است یا نه. برای این کار می توان دیتاستی شامل صدای افراد مبتلا و غیرمبتلا ساخت. نوع داده ها می تواند سرفه، تنفس یا گفتار معمولی باشد. سپس از هر نمونه صوتی MFCC استخراج می شود.

پرسش بعدی این است که چگونه بفهمیم MFCC افراد مبتلا و غیرمبتلا واقعاً با هم فرق دارد. برای بررسی تفاوت میان MFCC ها می توان از معیارهایی مانند similarity و correlation استفاده کرد. اگر MFCC های دو گروه بسیار شبیه یا همبسته باشند، احتمالاً نمی توان آن ها را خوب از هم جدا کرد. اما اگر مثلاً MFCC سرفه افراد مبتلا با MFCC سرفه افراد غیرمبتلا همبستگی کمی داشته باشد، می توان گفت این ویژگی برای طبقه بندی مفید است و شاید بتواند به تشخیص کمک کند.

جمع بندی جلسه این بود که مسیر پردازش صوت از موج خام تا ویژگی های قابل استفاده برای طبقه بندی چنین پیش می رود: دریافت موج صوتی خام، نمونه برداری و دیجیتال سازی، فریم بندی، پنجره گذاری همراه با هم پوشانی، تبدیل فوریه، ساخت اسپکتروگرام، بررسی formant ها، اعمال Mel Filter Bank، استخراج MFCC و استفاده از ویژگی ها برای تشخیص wake word یا طبقه بندی های دیگر. ایده اصلی برای Early Selection در ماشین این است که بدون ورود به درک معنا نیز می توان از ویژگی های آکوستیکی موج استفاده کرد.

فصل ۱۲

مدل‌های خودنظارتی گفتار

در ادامه بحث مدل‌های شنیداری، استاد دوباره تفاوت Early Selection و Late Selection را یادآوری کرد. در Early Selection، پردازش در سطح ویژگی‌های فیزیکی و آکوستیکی انجام می‌شود و سیستم هنوز وارد فهم معنا، تبدیل گفتار به متن یا تحلیل معنایی نمی‌شود. در Late Selection، سیستم وارد فاز درک معنا می‌شود؛ یعنی گفتار باید تا حدی فهمیده شود و معمولاً به متن تبدیل شود تا روی آن تسک‌های NLP انجام شود.

با این حال، ورودی مدل برای رسیدن به معنا همیشه الزاماً متن نیست. ورودی می‌تواند متن، اسپکتروگرام، MFCC، موج خام، یا بردارهای استخراج‌شده از CNN باشد. بنابراین ممکن است مدل مستقیماً از نمایش صوتی به یک نمایش معنایی برسد، هرچند در معماری‌های رایج، تبدیل گفتار به متن همچنان مسیر مهمی است.

۱.۱۲ استخراج ویژگی قطعی و یادگرفتنی

برای پردازش گفتار، معمولاً یکی از نمایش‌هایی مانند waveform، spectrogram یا MFCC استفاده می‌شود. تولید اسپکتروگرام یا MFCC معمولاً یک فرایند ریاضی و deterministic است: فاز آموزشی ندارد، بر اساس فرمول انجام می‌شود، و اگر ورودی ثابت باشد، خروجی نیز ثابت است. برای نمونه، با تبدیل فوریه می‌توان waveform را به اسپکتروگرام تبدیل کرد.

در برابر این روش‌های قطعی، می‌توان موج خام را به یک شبکه عصبی داد تا خود شبکه ویژگی‌ها را یاد بگیرد:

Raw Waveform → CNN → Learned Features.

در این حالت، وزن‌های شبکه در ابتدا تصادفی اند، شبکه با داده آموزش می‌بیند، و فیلترها و kernel‌های CNN به تدریج یاد می‌گیرند چه ویژگی‌هایی مهم‌اند. گاهی ورودی CNN خود موج خام است و گاهی ابتدا موج به اسپکتروگرام تبدیل می‌شود و سپس اسپکتروگرام وارد CNN می‌شود. اگر داده کم باشد یا تسک سخت‌تر باشد، استفاده از پیش‌پردازش‌هایی مانند اسپکتروگرام می‌تواند مفید باشد، چون نمایش اولیه بهتری از سیگنال فراهم می‌کند.

۲.۱۲ Self-Supervised Learning

موضوع اصلی این بخش، مدل‌های Self-Supervised Learning در پردازش گفتار بود؛ به خصوص مدل‌هایی مانند wav2vec و HuBERT. ایده اصلی این مدل‌ها این است که داده خام زیاد است، اما داده برچسب‌دار کم‌تر است. برای مثال، صوت خام و متن خام زیاد داریم، اما صوتی که دقیقاً برچسب‌گذاری شده باشد یا رونوشت دقیق داشته باشد، کم‌تر است.

در یادگیری خودنظارتی، مدل از خود داده خام یک تسک آموزشی می‌سازد. در متن، نمونه معروف آن BERT است: بخشی از جمله mask می‌شود و مدل باید با توجه به context حدس بزند کلمه پنهان‌شده چیست. در گفتار نیز ایده مشابهی استفاده می‌شود: بخشی از فریم‌های صوتی mask می‌شوند و مدل باید با توجه به فریم‌های اطراف، بخش پنهان‌شده را پیش‌بینی کند.

فرایند کلی چنین است:

۱. مقدار زیادی صوت خام وارد مدل می‌شود؛
۲. بخش‌هایی از صوت mask می‌شوند؛
۳. مدل تلاش می‌کند بخش پنهان‌شده را از روی context پیش‌بینی کند؛
۴. یک loss function تعریف می‌شود؛
۵. اختلاف میان پیش‌بینی مدل و مقدار واقعی محاسبه می‌شود؛
۶. با backpropagation وزن‌های مدل اصلاح می‌شوند؛
۷. این فرایند تا کاهش loss و بهبود عملکرد ادامه پیدا می‌کند.

هدف نهایی این نیست که مدل فقط همان فریم پنهان‌شده را حدس بزند؛ هدف اصلی یادگیری نمایش مناسبی از گفتار است. پس از pretraining، مدل می‌تواند برای ورودی صوتی یک بردار یا مجموعه‌ای از بردارها تولید کند. این بردارها همان embedding یا representation گفتار هستند و بعداً می‌توان آن‌ها را برای تسک‌هایی مانند Speech-to-Text، طبقه‌بندی، تشخیص گوینده، intent detection و سایر تسک‌های گفتاری به کار برد.

۳.۱۲ wav2vec 2.0

در مدل‌هایی مانند wav2vec، ورودی معمولاً waveform خام است، نه اسپکتروگرام. مسیر کلی به این صورت است:

Raw Waveform → CNN Feature Encoder

→ Transformer Encoder

→ Contextual Representations.

۱.۳.۱۲ ورودی خام

ورودی مدل یک دنباله عددی است که همان waveform سیگنال صوتی است. در بعضی معماری‌ها ممکن است ابتدا اسپکتروگرام یا MFCC محاسبه شود، اما در wav2vec معمولاً موج خام به مدل داده می‌شود.

۲.۳.۱۲ CNN Feature Encoder

Waveform وارد CNN می‌شود و CNN از روی سیگنال خام ویژگی استخراج می‌کند. فیلترهای مختلف روی ورودی اعمال می‌شوند؛ هر فیلتر یک الگو یا ویژگی خاص را کشف می‌کند. در لایه‌های ابتدایی، ویژگی‌ها سطح پایین‌ترند و در لایه‌های عمیق‌تر، ویژگی‌ها انتزاعی‌تر و نزدیک‌تر به معنا می‌شوند.

در CNN، مقادیری که یاد گرفته می‌شوند، همان اعداد داخل فیلترها و kernelها هستند. این مقادیر ابتدا تصادفی مقداردهی می‌شوند، در طول آموزش تغییر می‌کنند، با backpropagation به‌روزرسانی می‌شوند و در نهایت یاد می‌گیرند چه الگوهایی در داده صوتی مهم‌اند.

در شبکه‌های کانولوشنی، معمولاً هرچه جلوتر می‌رویم تعداد فیلترها بیشتر می‌شود، ویژگی‌ها انتزاعی‌تر می‌شوند و مدل عمیق‌تر می‌شود. بعد از اعمال فیلترها، معمولاً pooling انجام می‌شود. هدف pooling کاهش بُعد، کاهش پیچیدگی مدل، کمک به جلوگیری از overfitting و فشرده‌سازی اطلاعات مهم است. در نهایت خروجی CNN به بردار یا دنباله‌ای از بردارهای ویژگی تبدیل می‌شود.

۳.۳.۱۲ Transformer Encoder

خروجی CNN وارد Transformer Encoder می‌شود. هدف Transformer این است که از ویژگی‌های محلی، نمایش وابسته به context بسازد. برای هر فریم، فقط خود آن فریم مهم نیست؛ رابطه آن با فریم‌های قبل و بعد نیز اهمیت دارد. Transformer این کار را با self-attention انجام می‌دهد.

استاد پرسید که این Transformer از چه نوعی است: Encoder، Decoder یا Encoder-Decoder. پاسخ درست Encoder بود، چون در این مرحله مدل قرار نیست چیزی تولید کند؛ فقط قرار است ورودی را بفهمد و encode کند.

۴.۳.۱۲ Masking

بخشی از فریم‌ها mask می‌شوند و مدل باید آن‌ها را با توجه به context پیش‌بینی کند. در متن، مدل می‌تواند کلمه پنهان‌شده را از میان واژگان محدود حدس بزند؛ اما در صوت، فریم‌ها در فضای پیوسته قرار دارند. واحدهای صوتی مثل کلمات مرزهای کاملاً مشخص و گسسته ندارند و هر بخش صوتی می‌تواند به بردارهای بسیار متنوعی نگاشت شود. بنابراین در صوت نیاز داریم چیزی شبیه واژگان بسازیم.

۵.۳.۱۲ Codebook و Quantization

Quantization یعنی بردن یک نمایش پیوسته به یک نمایش گسسته و محدود. برای اینکه در صوت نیز چیزی شبیه vocabulary داشته باشیم، از codebook استفاده می‌شود. Codebook مانند یک دیکشنری برای واحدهای صوتی است.

در این فرایند، یک بردار صوتی داریم و یک codebook شامل چند بردار نماینده. بررسی می‌کنیم بردار صوتی به کدام بردار codebook نزدیک‌تر است و همان بردار را به عنوان نماینده انتخاب می‌کنیم. به این ترتیب، به جای کار کردن با فضای پیوسته و بی نهایت حالت، آن را به چند واحد گسسته محدود می‌کنیم.

استاد برای توضیح codebook از مثال موقعیت مکانی استفاده کرد: ابتدا می‌گوییم «در ایرانم»، بعد دقیق‌تر «در تهرانم»، و بعد دقیق‌تر «در یک منطقه خاص از تهرانم». به همین شکل، quantization می‌تواند چند سطح داشته باشد و codebook می‌تواند لایه لایه یا level-based باشد. مقادیر اولیه codebook نیز معمولاً تصادفی‌اند و در فرایند آموزش یاد گرفته می‌شوند.

استاد تأکید کرد که quantization وارد مسیر اصلی Transformer نمی‌شود. خروجی CNN وارد Transformer می‌شود، روی Transformer representation های غنی و پیوسته کار می‌کند، و contextual representation ها quantize نمی‌شوند. Quantization بیشتر برای محاسبه loss و هدف آموزشی استفاده می‌شود. دلیلش این است که اگر از همان ابتدا نمایش‌ها را quantize کنیم، بخشی از اطلاعات آکوستیکی از بین می‌رود.

۶.۳.۱۲ Contrastive Loss

در wav2vec، یکی از ایده‌های اصلی استفاده از contrastive learning است. در این روش، مدل باید بردار زمینه‌ای مربوط به بخش پنهان شده را به نمونه مثبت نزدیک کند و از نمونه‌های منفی دور کند:

دورتر $\rightarrow$ Negative Samples, نزدیک‌تر $\rightarrow$ Correct Quantized Vector

تابع هزینه بر اساس همین نزدیکی و دوری محاسبه می‌شود.

اگر خروجی‌ها کاملاً پیوسته باقی بمانند، یادگیری ممکن است سخت‌تر شود. Quantization باعث می‌شود هدف آموزشی گسسته‌تر و مشخص‌تر شود. در نتیجه، ورودی مدل می‌تواند غنی و پیوسته بماند، اما هدف آموزشی محدودتر و پایدارتر باشد.

۷.۳.۱۲ Fine-tuning و Pretraining

Pretraining مرحله‌ای است که در آن مدل از داده خام گفتار، ساختارهای صوتی را یاد می‌گیرد و یک representation خوب از waveform یا گفتار می‌سازد. بعد از pretraining، معمولاً fine-tuning انجام می‌شود. در fine-tuning داده برچسب‌دار داریم، اما معمولاً مقدار آن کم‌تر است، و کل مدل یا بخشی از مدل برای یک تسک خاص تنظیم می‌شود؛ مثلاً برای Speech-to-Text یا طبقه‌بندی.

مسیر کلی چنین است:

Large Raw Speech Data → Self-Supervised Pretraining

→ Speech Representations

→ Fine-tuning on Labeled Data.

جمع‌بندی فنی استاد از معماری wav2vec چنین بود: ورودی مدل موج خام است؛ CNN سیگنال خام را به نمایش فشرده‌تری از ویژگی‌ها تبدیل می‌کند؛ بخشی از فریم‌ها mask می‌شوند؛ Transformer Encoder با self-attention نمایش وابسته به زمینه می‌سازد؛ خروجی‌های مشخصی quantize می‌شوند تا واحدهای گسسته صوتی ساخته شود؛ مدل با contrastive loss بردار درست را از میان نمونه‌های مثبت و منفی تشخیص می‌دهد؛ و پس از pretraining، با دادهٔ برچسب‌دار برای یک تسک خاص fine-tune می‌شود.

فصل ۱۳

Whisper و معماری Encoder-Decoder

در جلسه قبل، مدل‌های شنیداری خودنظارتی مانند wav2vec بررسی شدند. در آن مدل‌ها داده صوتی خام زیاد است، اما برچسب مستقیم کم است؛ بنابراین مدل تلاش می‌کند بدون رونوشت دقیق، بازنمایی مناسبی از صوت یاد بگیرد. در این جلسه، مدل Whisper به عنوان نمونه‌ای متفاوت بررسی شد: مدلی که برخلاف wav2vec به صورت supervised آموزش داده می‌شود.

۱.۱۳ ورودی و خروجی Whisper

در Whisper ورودی مدل صوت و خروجی مدل متن است. چون آموزش آن نظارت شده است، داده آموزشی به شکل جفت‌های صوت و متن در اختیار مدل قرار می‌گیرد؛ یعنی برای هر فایل صوتی، متن متناظر آن نیز وجود دارد. طبق توضیح کلاس، داده آموزشی Whisper حدود ۶۸۰ هزار ساعت صوت و متن جفت شده بوده است. این داده‌ها چندزبانه‌اند و لهجه‌های مختلف را دربرمی‌گیرند. بخش مهمی از آن‌ها نیز از محیط‌های واقعی و نسبتاً نویزی جمع‌آوری شده‌اند؛ بنابراین ممکن است صدای پس‌زمینه، مکالمات محیطی یا صدای فیلم‌ها در آن‌ها وجود داشته باشد. همین تنوع باعث می‌شود Whisper نسبت به نویز، زبان‌های مختلف و تفاوت لهجه‌ها مقاوم‌تر یا robust باشد.

۲.۱۳ چرا معماری Encoder-Decoder؟

در مدل‌های خودنظارتی مانند wav2vec هدف اصلی استخراج یک بازنمایی مناسب از صوت خام یا فریم‌های صوتی بود؛ همان چیزی که در ادبیات مدل‌ها representation یا embedding نامیده می‌شود. به همین دلیل بخش اصلی معماری در آن مدل‌ها Encoder بود: مدلی که ورودی را می‌گیرد و آن را به نمایش فشرده‌تر و معنادارتری تبدیل می‌کند. اما در Whisper مسئله فقط فهمیدن صوت نیست. مدل باید از روی صوت، متن متناظر را نیز تولید کند. پس دو کار هم‌زمان لازم است:

Speech Understanding → Audio Representation

Text Generation → Transcript.

به همین دلیل معماری مناسب برای Whisper معماری Encoder-Decoder است. Encoder صوت را می‌فهمد و به بازنمایی تبدیل می‌کند؛ Decoder با تکیه بر آن بازنمایی، متن را مرحله به مرحله تولید می‌کند.

۳.۱۳ بخش Encoder

Encoder وظیفه دارد صوت ورودی را دریافت کند و آن را به یک بازنمایی معنایی یا embedding تبدیل کند. ورودی یک مدل شنیداری می‌تواند شکل‌های مختلفی داشته باشد؛ مانند موج خام، spectrogram، mel-spectrogram یا MFCC. در Whisper ورودی اصلی به شکل mel-spectrogram است.

۱.۳.۱۳ Self-Attention بدون ماسک

در Encoder از self-attention استفاده می‌شود، چون مدل باید بخش‌های مختلف صوت را نسبت به یکدیگر بررسی کند. در این فرایند، فریم‌های صوتی با هم مقایسه می‌شوند و مدل یاد می‌گیرد هر فریم با کدام فریم‌های دیگر ارتباط بیشتری دارد.

نکته مهم این است که self-attention در Encoder ماسک ندارد. دلیل آن ساده است: کل صوت از ابتدا در اختیار مدل است. بنابراین مدل می‌تواند همه فریم‌های صوتی را هم‌زمان ببیند و میان همه آن‌ها رابطه برقرار کند. این وضعیت با Decoder فرق دارد، چون Decoder متن را مرحله به مرحله می‌سازد و در هر مرحله هنوز توکن‌های آینده را ندارد.

۲.۳.۱۳ بازنمایی صوتی زمینه‌مند

خروجی Encoder یک contextualized embedding است؛ یعنی نمایش هر بخش از صوت با توجه به کل زمینه صوت ساخته می‌شود. برای مثال اگر عبارت «پردازش زبان طبیعی» در صوت گفته شود، بخش مربوط به «زبان» نباید جدا از «پردازش» و «طبیعی» فهمیده شود. بازنمایی زمینه‌مند باعث می‌شود مدل هر بخش از ورودی را در نسبت با بخش‌های دیگر تفسیر کند.

۴.۱۳ بخش Decoder

Decoder وظیفه دارد از روی embedding صوتی ساخته شده توسط Encoder، متن متناظر را تولید کند. این تولید متن به صورت autoregressive انجام می‌شود؛ یعنی مدل توکن‌ها را یکی یکی تولید می‌کند و هر خروجی جدید، بخشی از ورودی مرحله بعد می‌شود.

۱.۴.۱۳ Masked Self-Attention

در Decoder از masked self-attention استفاده می‌شود. دلیل ماسک شدن این است که هنگام تولید یک توکن، مدل نباید به توکن‌های آینده دسترسی داشته باشد. برای نمونه، وقتی مدل هنوز فقط «پردازش» را تولید کرده است، نباید از قبل بداند کلمه بعدی «زبان» است.

بنابراین بخشی از ماتریس attention ماسک می‌شود تا مدل فقط به توکن‌های قبلی و توکن فعلی توجه کند. جمع‌بندی این تفاوت چنین است:

Encoder : Full Self-Attention over audio frames

Decoder : Masked Self-Attention over generated text.

۲.۴.۱۳ Cross-Attention

Decoder علاوه بر masked self-attention، یک بخش مهم دیگر نیز دارد: cross-attention. در cross-attention مدل میان دو چیز رابطه برقرار می‌کند: بازنمایی صوتی خروجی Encoder و متنی که تا این لحظه تولید شده است.

هدف این است که مدل برای تولید توکن بعدی، هم بداند تا این لحظه چه متنی ساخته و هم به صوت اصلی توجه کند. بنابراین cross-attention نوعی نگاه هم‌زمان به صوت و متن تولیدشده است و خروجی آن به مدل کمک می‌کند توکن بعدی را انتخاب کند.

۳.۴.۱۳ تولید خودرگرسیو متن

تولید خودرگرسیو را می‌توان به صورت زنجیره‌ای از تصمیم‌های متنی دید. ابتدا فقط توکن شروع در اختیار مدل است. سپس مدل نخستین کلمه را تولید می‌کند؛ بعد با داشتن آن کلمه، کلمه بعدی را تولید می‌کند؛ و همین روند تا پایان متن ادامه پیدا می‌کند. برای مثال:

پردازش زبان طبیعی → پردازش زبان → پردازش Start

پس معماری Whisper را می‌توان این گونه خلاصه کرد: Encoder از mel-spectrogram یک بازنمایی صوتی زمینه‌مند می‌سازد. Decoder روی متن تولیدشده masked self-attention انجام می‌دهد. سپس با cross-attention به خروجی Encoder نگاه می‌کند و توکن بعدی متن را می‌سازد.

۵.۱۳ ارتباط مدل‌های شنیداری با توجه انتخابی

در پایان جلسه، استاد دوباره به هدف اصلی درس برگشت: اینکه بتوانیم از نظریه‌های شناختی برای طراحی مدل‌های محاسباتی بهتر ایده بگیریم. تا اینجا درباره Early Selection، Late Selection و بار ادراکی صحبت شده بود. پرسش این است که از این نظریه‌ها چه ایده‌ای می‌توان برای طراحی یک سیستم شنیداری یا دستیار صوتی بهتر گرفت.

یکی از ایده‌ها این است که دستیار صوتی بتواند بار ادراکی محیط را اندازه بگیرد و بر اساس آن میان پردازش سبک و سنگین جابه‌جا شود. اگر بار ادراکی زیاد باشد:

High Perceptual Load → Early Selection

یعنی سیستم فقط پردازش‌های سبک و ویژگی‌های فیزیکی را انجام دهد. اگر بار ادراکی کم باشد:

Low Perceptual Load → Late Selection

یعنی سیستم وارد پردازش معنایی، Speech-to-Text و NLU شود.

این ایده می‌تواند در معماری‌های واقعی دستیارهای صوتی مهم باشد، چون در مهندسی همیشه با موازنه دقت و هزینه محاسباتی روبه‌رو هستیم. هدف درس نیز همین پل زدن میان نظریه شناختی و مدل محاسباتی است: نه آن‌قدر وارد جزئیات فنی معماری‌ها شویم که هدف شناختی فراموش شود، و نه آن‌قدر در سطح نظریه شناختی بمانیم که ارتباط با مدل‌های هوش مصنوعی قطع شود.

در جمع‌بندی این بخش، استاد تأکید کرد که بحث‌های شناختی مانند توجه انتخابی و بار ادراکی هنوز به شکل جدی وارد بسیاری از مدل‌های شنیداری فعلی نشده‌اند. اگر قرار باشد یک دستیار صوتی با مبنای شناختی طراحی شود، باید بتوان مفاهیمی مانند آستانه بار ادراکی، انتخاب زودهنگام و انتخاب دیرهنگام را در خود معماری یا سیاست پردازش سیستم تعریف کرد.

۶.۱۳ بازگشت کوتاه به Predictive Coding و GAN

در پایان بحث مدل‌های شنیداری، پرسشی درباره شباهت Predictive Coding و GAN مطرح شد. شباهت ظاهری این دو در این است که هر دو نوعی فرایند تولید و ارزیابی دارند. در GAN، یک Generator داده ساختگی تولید می‌کند و یک Discriminator تشخیص می‌دهد داده واقعی است یا ساختگی. در Predictive Coding نیز سیستم دائماً پیش‌بینی تولید می‌کند، آن را با واقعیت مقایسه می‌کند، خطای پیش‌بینی را محاسبه می‌کند و سپس اصلاح انجام می‌دهد.

با این حال تفاوت مهمی وجود دارد. در شبکه‌های عصبی رایج، خطا معمولاً سراسری محاسبه می‌شود: ورودی وارد شبکه می‌شود، خروجی نهایی با مقدار واقعی مقایسه می‌شود و loss به دست می‌آید. سپس خطا با روش backpropagation در کل شبکه پخش می‌شود. اما در Predictive Coding ایده این است که خطا می‌تواند محلی‌تر باشد؛ یعنی هر بخش یا هر نورون بتواند پیش‌بینی خود را با ورودی مقایسه کند و اصلاح محلی انجام دهد. این تفاوت یکی از نقاطی است که مدل‌های الهام‌گرفته از مغز را از شبکه‌های عصبی رایج جدا می‌کند.

۷.۱۳ پردازنده‌های نورومورفیک

در ادامه همین بحث، به پردازنده‌های Neuromorphic اشاره شد. اکوسیستم فعلی هوش مصنوعی عمدتاً حول GPU، گرایان و روش backpropagation شکل گرفته است. در مقابل، سخت‌افزارهای نورومورفیک تلاش می‌کنند بعضی ویژگی‌های محاسبه عصبی را شبیه‌سازی کنند: نورون، spike، سیناپس و پردازش محلی.

این پردازنده‌ها هنوز جایگزین عمومی GPU نشده‌اند، چون مهاجرت از اکوسیستم فعلی فقط زمانی منطقی است که مزیت عملی و جدی آن‌ها روشن شود. با این حال، یکی از پیشنهادهای پروژه پایانی می‌تواند بررسی یا پیاده‌سازی مجازی یک پردازنده نورومورفیک باشد؛ البته دامنه چنین پروژه‌ای باید دقیق و قابل اجرا تعریف شود.

بخش پنجم

پاداش، یادگیری تقویتی و شبکه‌های اسپایکی

فصل ۱۴

سیستم پاداش و یادگیری تقویتی

پس از پایان بحث مدل‌های شنیداری، کلاس وارد موضوع سیستم‌های پاداش در مغز و ارتباط آن با Reinforcement Learning شد. نقطه شروع این بحث این است که یک عامل در محیط قرار دارد، در هر لحظه در یک وضعیت یا state است، عملی یا action انجام می‌دهد و از یک وضعیت به وضعیت دیگر منتقل می‌شود. به این تغییر وضعیت transition گفته می‌شود.

هر transition می‌تواند با پاداش یا تنبیه همراه باشد. پاداش ممکن است مثبت، منفی، کم، زیاد یا خنثی باشد. این پاداش‌ها نقش reinforcer دارند: باعث می‌شوند عامل یاد بگیرد کدام رفتارها را بیشتر تکرار کند و کدام رفتارها را کمتر انجام دهد.

هدف این جلسه ورود سنگین به معادلات فیزیکی یا ریاضی نبود. ابتدا قرار بود سازوکار کلی مغز در یادگیری از طریق بازخورد فهمیده شود و سپس این بحث به فضای Reinforcement Learning وصل شود.

در معماری شناختی‌ای که پیش‌تر درباره آن صحبت شده بود، عامل از محیط محرک‌های حسی دریافت می‌کند. این محرک‌ها، برای مثال از مسیر ماژول بینایی یا شنیداری، وارد سیستم می‌شوند. سپس با فرایندهایی مانند توجه انتخابی، بخشی از اطلاعات محیط انتخاب می‌شود. اطلاعات فعلی وارد حافظه کوتاه‌مدت می‌شود؛ سیستم از حافظه بلندمدت اطلاعات مرتبط را بازیابی می‌کند؛ با ترکیب اطلاعات فعلی و دانش قبلی، یک الگو استخراج می‌شود؛ و بر اساس آن، عمل انتخاب و از مسیر ماژول حرکتی اجرا می‌شود.

پس از انجام عمل، عامل از زمان t به زمان $t + 1$ می‌رود و به وضعیت جدیدی منتقل می‌شود. در همین فاصله، پاداش یا تنبیه دریافت می‌شود. در این نقطه باید چیزی شبیه یک utility module یا ماژول ارزش‌گذاری به معماری اضافه شود؛ ماژولی که بررسی کند عمل انجام‌شده خوب بود، بد بود، یا تفاوت خاصی با انتظار قبلی نداشت. هدف این ماژول این است که پس از هر تجربه، مدل قبلی یا prior model به‌روزرسانی شود.

۱.۱۴ پاداش در مغز

در انسان، پاداش معمولاً به صورت حس رضایت، لذت یا موفقیت تجربه می‌شود. غذا خوردن، موفق شدن در یک کار، دریافت پیام مهم یا دیدن چیزی که قبلاً تجربه خوبی با آن داشته‌ایم، نمونه‌هایی از تجربه‌های پاداش‌دهنده‌اند.

این تجربه‌ها می‌توانند با آزاد شدن دوپامین همراه شوند. نکته مطرح شده در کلاس این بود که دوپامین در واکنش به یک تغییر حالت معنادار آزاد می‌شود؛ تغییری که برای سیستم مهم یا آموخته شده است. برای مثال خاموش شدن چراغ در روز ممکن است تجربه خاصی نباشد، اما دریافت یک پیامک به دلیل انتظار یا تجربه آموخته شده می‌تواند برای فرد پاداش دهنده باشد.

۲.۱۴ دوپامین و VTA

وقتی دوپامین آزاد می‌شود، در مغز از مسیرهایی عبور می‌کند که به آن‌ها مسیرهای دوپامینرژیک یا dopaminergic pathways گفته می‌شود. در بحث پاداش، یکی از نقاط مهم تولید دوپامین ناحیه Ventral Tegmental Area یا VTA است.

در چارچوب این جلسه، کارکرد مهم VTA محاسبه خطای پیش‌بینی پاداش است: سیستم چه پاداشی را پیش‌بینی کرده بود، در واقعیت چه پاداشی دریافت شد، و اختلاف این دو چقدر است. وقتی پاداش واقعی بهتر از انتظار باشد، VTA با ترشح دوپامین مرتبط می‌شود. بنابراین دوپامین صرفاً نشانه «لذت» نیست؛ بلکه در این بحث بیشتر به غافلگیری مثبت و خطای پیش‌بینی پاداش مربوط است.

به همین دلیل، هر چیز خوب یا لذت بخشی الزاماً همیشه دوپامین زیادی ایجاد نمی‌کند. اگر تجربه‌ای کاملاً قابل پیش‌بینی شده باشد، ممکن است غافلگیری کمی ایجاد کند و در نتیجه پاسخ دوپامینی آن نیز کم‌تر شود.

۳.۱۴ مسیرهای دوپامینرژیک

چهار مسیر اصلی دوپامینرژیک در مغز مطرح می‌شود، اما در بحث پاداش این جلسه تمرکز بر دو مسیر بود:

$$VTA \rightarrow \begin{cases} \text{Mesolimbic Pathway} \\ \text{Mesocortical Pathway} \end{cases}$$

۱.۳.۱۴ Mesolimbic Pathway

مسیر Mesolimbic از VTA به سمت سیستم لیمبیک می‌رود. این مسیر با احساس رضایت، شرطی شدن، تقویت رفتار، اعتیاد و میل به تکرار رفتار ارتباط دارد.

برای مثال اگر فرد بعد از کار چای بنوشد و این کار برای او تجربه خوشایندی باشد، سیستم Mesolimbic می‌تواند به این الگو شرطی شود. در نتیجه، بعد از کار میل به نوشیدن چای افزایش پیدا می‌کند. به زبان ساده، این مسیر به مغز می‌گوید: این رفتار قبلاً خوب بوده است؛ دوباره انجامش بده.

در توضیح این جلسه، مسیر Mesolimbic از VTA به نواحی لیمبیک، به ویژه Nucleus Accumbens، می‌رسد. این مسیر بیشتر با تقویت یا تضعیف ارزش محرک‌ها در حافظه بلندمدت ارتباط دارد. وقتی خطای پیش‌بینی پاداش مثبت است، این مسیر کمک می‌کند محرک یا عمل مربوطه در حافظه با ارزش بیشتری ثبت شود.

۲.۳.۱۴ Mesocortical Pathway

مسیر Mesocortical نیز از VTA شروع می‌شود، اما به سمت Prefrontal Cortex یا قشر پیش‌پیشانی می‌رود. قشر پیش‌پیشانی با تصمیم‌گیری، تحلیل، ارزیابی پیامدها، کنترل رفتار، بررسی ارزش تلاش و فکر کردن به نتایج بلندمدت ارتباط دارد.

مثلاً وقتی فرد کیک می‌بیند، مسیر Mesolimbic ممکن است او را به سمت خوردن کیک سوق دهد، چون کیک قبلاً تجربه‌ای لذت‌بخش بوده است. اما مسیر Mesocortical پیامدها را ارزیابی می‌کند: آیا خوردن کیک منطقی است؟ آیا فرد گرسنه است؟ آیا رژیم دارد؟ آیا نتیجه بلندمدت این رفتار خوب است؟ در این بیان استعاری، Mesolimbic بیشتر نقش گاز و Mesocortical بیشتر نقش تحلیل‌گر یا ترمز را دارد.

۴.۱۴ اختلال‌های مرتبط با سیستم پاداش

در ادامه جلسه، چند پدیده با همین چارچوب توضیح داده شد. هدف این بخش تشخیص پزشکی نیست؛ بلکه نشان دادن این است که نظریه پاداش، دوپامین و خطای پیش‌بینی چگونه می‌تواند برای فهم بعضی تغییرات رفتاری به کار رود.

۱.۴.۱۴ پیری

در پیری، سیستم پاداش معمولاً کم‌تر دچار غافلگیری می‌شود. فرد تجربه‌های زیادی کسب کرده و بسیاری از محرک‌ها برای او قابل پیش‌بینی شده‌اند. بنابراین خطای پیش‌بینی پاداش اغلب به صفر نزدیک‌تر است، دوپامین کم‌تر ترشح می‌شود، به‌روزرسانی مدل ذهنی کم‌تر رخ می‌دهد و میل به تجربه چیزهای تازه یا exploration کاهش می‌یابد.

از این دید، یکی از راه‌های ایجاد حال بهتر برای فرد سالمند می‌تواند فراهم کردن تجربه‌های تازه و کمتر تجربه‌شده باشد؛ مانند سفرهای جدید، فعالیت‌های جدید یا موقعیت‌هایی که پیش‌تر در زندگی فرد وجود نداشته‌اند.

۲.۴.۱۴ افسردگی

افسردگی نیز در کلاس با همین زبان توضیح داده شد. در این حالت، سیستم پاداش به‌درستی فعال نمی‌شود و فرد ممکن است تجربه کند که چیزی خوشحالش نمی‌کند، کارهایی که قبلاً لذت‌بخش بودند دیگر جذاب نیستند، یا انجام دادن هیچ کاری ارزش ندارد.

در این چارچوب، فرد ممکن است در حالت قطعیت بماند و عمل تازه‌ای انجام ندهد. وقتی تجربه تازه‌ای رخ ندهد، خطای پیش‌بینی پاداش نیز صفر می‌ماند. اما این با یادگیری سالم فرق دارد. در یادگیری سالم، فرد محیط را تجربه می‌کند تا مدل ذهنی‌اش دقیق‌تر شود و خطا کاهش یابد؛ اما در این توصیف از افسردگی، فرد با انجام ندادن عمل و تجربه نکردن، به‌صورت ناسالم به خطای نزدیک به صفر می‌رسد.

۳.۴.۱۴ اعتیاد

اعتیاد یکی از مثال‌های اصلی سوءاستفاده از سیستم پاداش است. در ابتدای مصرف، محرک اعتیادآور می‌تواند خطای پیش‌بینی پاداش مثبت ایجاد کند:

$$RPE > 0$$

چون تجربه قوی‌تر یا متفاوت‌تر از انتظار قبلی است. اما پس از تکرار، سیستم یاد می‌گیرد و آن تجربه برای مغز عادی‌تر می‌شود؛ بنابراین خطای پیش‌بینی پاداش به سمت صفر می‌رود:

$$RPE \rightarrow 0$$

در اینجا چرخه معیوب شروع می‌شود. فرد برای اینکه دوباره خطای مثبت و پاسخ دوپامینی ایجاد کند، دوز را بالا می‌برد. با افزایش دوز دوباره غافلگیری ایجاد می‌شود، اما پس از مدتی مغز دوباره سازگار می‌شود و خطا کاهش می‌یابد. چرخه کلی چنین است: تجربه اولیه خطای مثبت ایجاد می‌کند؛ سیستم یاد می‌گیرد؛ خطا به صفر نزدیک می‌شود؛ دوز افزایش می‌یابد؛ دوباره خطای مثبت ایجاد می‌شود؛ و مغز دوباره سازگار می‌شود.

مغز در برابر این چرخه منفعل نمی‌ماند. یکی از واکنش‌های دفاعی آن کاهش حساسیت به دوپامین است. در نتیجه، فرد نه تنها از محرک اعتیادآور کم‌تر لذت می‌برد، بلکه ممکن است از محرک‌های عادی زندگی نیز لذت کم‌تری تجربه کند.

در کلاس برای این پرسش که چرا همه افراد وارد چرخه اعتیاد نمی‌شوند، چند عامل مطرح شد. برخی افراد اصلاً در معرض محرکی قرار نمی‌گیرند که خطای مثبت شدید ایجاد کند. آگاهی و محیط نیز مهم‌اند، چون شناخت پیامدهای اعتیاد می‌تواند از ورود به چرخه جلوگیری کند. عامل دیگر تفاوت‌های فردی، ژنتیک و قدرت مسیر Mesocortical است. اگر این مسیر در کنترل، راهبرد و ارزیابی پیامدهای بلندمدت قوی‌تر عمل کند، فرد بهتر می‌تواند در برابر پاداش فوری مقاومت کند.

مثال مطرح‌شده Marshmallow Test بود: آزمایشی که در آن به کودک گفته می‌شود اگر اکنون شیرینی یا شکلات را نخورد، بعداً پاداش بیشتری می‌گیرد. برخی کودکان می‌توانند صبر کنند و برخی نه. این مثال برای توضیح خودکنترلی و توانایی دیدن پیامدهای بلندمدت استفاده شد.

۵.۱۴ ارتباط سیستم پاداش با Predictive Coding

در این بخش پرسیده شد کدام نظریه شناختی می‌تواند این فرایند را توضیح دهد. پاسخ اصلی Predictive Coding بود. در کدگذاری پیش‌بینانه، سیستم همواره میان دانش قبلی یا prior knowledge و واقعیت مشاهده‌شده مقایسه انجام می‌دهد. آنچه برای یادگیری مهم است، صرفاً خود واقعیت خام نیست؛ بلکه خطای میان پیش‌بینی و واقعیت است.

در سیستم پاداش نیز همین اتفاق می‌افتد. عامل پیش از عمل، انتظاری از پاداش دارد. سپس عمل انجام می‌دهد و پاداش واقعی را دریافت می‌کند. اختلاف میان پاداش واقعی و پاداش پیش‌بینی شده همان چیزی است که باید یاد گرفته شود.

۶.۱۴ Reward Prediction Error

مفهوم مرکزی این بخش Reward Prediction Error یا خطای پیش‌بینی پاداش است:

$$\text{Reward Prediction Error} = \text{Actual Reward} - \text{Predicted Reward}$$

یا به شکل خلاصه:

$$\text{RPE} = R_{\text{actual}} - R_{\text{predicted}}$$

اگر پاداش واقعی بیشتر از پاداش پیش‌بینی شده باشد، خطا مثبت است:

$$\text{RPE} > 0$$

یعنی نتیجه بهتر از انتظار بوده است. در این حالت سیستم غافلگیر می‌شود و عمل یا محرک مربوطه تقویت می‌شود. مثال کلاس رستورانی بود که فرد انتظار متوسطی از آن دارد، اما تجربه واقعی بسیار بهتر از انتظار اوست. در این حالت، ارزش آن رستوران در ذهن افزایش پیدا می‌کند.

اگر پاداش واقعی تقریباً برابر با پیش‌بینی قبلی باشد، خطا نزدیک به صفر است:

$$\text{RPE} = 0$$

در این حالت اتفاق تازه‌ای برای یادگیری رخ نمی‌دهد و مدل قبلی تقریباً ثابت می‌ماند.

اگر پاداش واقعی کمتر از پاداش پیش‌بینی شده باشد، خطا منفی است:

$$\text{RPE} < 0$$

یعنی نتیجه بدتر از انتظار بوده است. در این حالت سیستم باید یاد بگیرد که ارزش آن عمل یا محرک را کاهش دهد.

۷.۱۴ Surprise و Free Energy Principle و کاهش

در ادامه، Free Energy Principle مطرح شد. بر اساس این نگاه، انسان فقط به دنبال دریافت پاداش بیشتر نیست؛ بلکه تلاش می‌کند عدم قطعیت و خطای پیش‌بینی را کاهش دهد. سیستم شناختی محیط را تجربه می‌کند تا مدل ذهنی‌اش دقیق‌تر شود و اختلاف میان پیش‌بینی و واقعیت کم‌تر شود.

هدف نهایی این است که خطای پیش‌بینی پاداش به صفر نزدیک شود:

$$\text{RPE} \rightarrow 0$$

یعنی سیستم به حالتی برسد که پاداش واقعی با پاداش پیش‌بینی شده همخوان شود. در این حالت مدل ذهنی به‌روزرسانی شده و غافلگیری زیادی رخ نمی‌دهد.

فصل ۱۵

مدل‌های محاسباتی یادگیری تقویتی

از دید محاسباتی، سیستم پاداش انسان شباهت زیادی به Reinforcement Learning دارد. در یادگیری تقویتی، عامل در محیط قرار دارد، عملی انجام می‌دهد، از یک وضعیت به وضعیت دیگر می‌رود، پاداش دریافت می‌کند و بر اساس آن پاداش یاد می‌گیرد در آینده چه رفتاری انجام دهد.

۱.۱۵ مسئله Reinforcement Learning

مسئله اصلی Reinforcement Learning یادگیری از طریق تعامل با محیط است. عامل برخلاف مدل نظارت‌شده، از ابتدا برچسب درست هر تصمیم را ندارد؛ باید عملی انجام دهد و پیامد آن را ببیند. اگر پاداش دریافت‌شده مطلوب باشد، احتمال تکرار آن رفتار بیشتر می‌شود و اگر پیامد نامطلوب باشد، احتمال آن رفتار کاهش می‌یابد.

یادگیری تقویتی وقتی اهمیت ویژه پیدا می‌کند که محیط آنالین باشد؛ یعنی عامل مدل آماده و دقیقی از محیط ندارد و باید تجربه را نمونه‌به‌نمونه به‌دست آورد. در هر گام، عامل یک state را مشاهده می‌کند، یک action انجام می‌دهد، به وضعیت جدیدی منتقل می‌شود و از محیط reward یا punishment دریافت می‌کند. سپس مدل خود را بر اساس همین تجربه به‌روزرسانی می‌کند.

حالت مهم دیگر، محیط تصادفی یا stochastic است. در چنین محیطی، اگر در یک وضعیت مشخص یک عمل مشخص انجام شود، لزوماً همیشه به یک وضعیت مشخص نمی‌رسیم. گذار کلی چنین است:

$$S \xrightarrow{A} S'$$

اما S' می‌تواند چند حالت مختلف داشته باشد؛ برای مثال ممکن است با احتمال ۵/۰ به S'_1 و با احتمال ۲/۰ به S'_2 برسیم. چون مدل گذار از ابتدا معلوم نیست، عامل باید با تجربه کردن و گرفتن بازخورد آن را یاد بگیرد.

۲.۱۵ تفاوت با یادگیری نظارت‌شده و بدون‌ناظر

در Supervised Learning، داده‌ها برچسب دارند و مدل از رابطه میان input و label یاد می‌گیرد. اما در سیستم پاداش و یادگیری تقویتی، برچسب آماده وجود ندارد. عامل باید action انجام دهد، از محیط feedback بگیرد و بر اساس نتیجه تجربه یاد بگیرد.

در Unsupervised Learning نیز برچسب مستقیم وجود ندارد، اما مدل معمولاً از ساختار داده‌ها الگو استخراج می‌کند؛ مثلاً خوشه‌بندی می‌کند یا بازنمایی می‌آموزد. تفاوت Reinforcement Learning این است که مسئله فقط نبودن برچسب نیست. نکته اصلی تعامل فعال با محیط و دریافت reward پس از عمل است.

۳.۱۵ Temporal Difference Learning

در پایان جلسه، بحث به فرمول یادگیری رسید. فرض کنیم عامل در وضعیت فعلی S قرار دارد و با انجام عمل A به وضعیت بعدی S' می‌رود. عامل برای وضعیت فعلی یک ارزش قبلی دارد که آن را V_t می‌نامیم. پس از تجربه جدید، یک نمونه تازه از ارزش آن تجربه به دست می‌آید. اکنون سیستم باید ارزش قبلی را با توجه به تجربه جدید به‌روزرسانی کند.

نه دانش قبلی به‌تنهایی کافی است و نه نمونه جدید به‌تنهایی. بنابراین سیستم میان این دو درونیابی می‌کند. برای این کار از ضریب α استفاده می‌شود:

$$0 \leq \alpha \leq 1$$

α همان learning rate است. اگر $\alpha = 1$ باشد، فقط تجربه جدید مهم است و دانش قبلی نادیده گرفته می‌شود. اگر $\alpha = 0$ باشد، تجربه جدید اثری ندارد و سیستم چیزی یاد نمی‌گیرد.

فرمول کلی به‌روزرسانی چنین است:

$$V_{t+1} = V_t + \alpha(\text{Sample} - V_t)$$

ترم داخل پرانتز همان خطای پیش‌بینی پاداش است:

$$\text{Sample} - V_t = \text{RPE}$$

پس:

$$V_{t+1} = V_t + \alpha \times \text{RPE}$$

این منطق Temporal Difference Learning نامیده می‌شود، چون اختلاف میان دو زمان را محاسبه می‌کند: پیش از عمل در زمان t و پس از عمل در زمان $t + 1$. سیستم ارزش قبلی را به اندازه نرخ یادگیری و بر اساس خطای پیش‌بینی به‌روزرسانی می‌کند.

هدف نهایی این فرایند آن است که نمونه تجربه‌شده با ارزش پیش‌بینی‌شده همگرا شود:

$$\text{Sample} = V_t$$

در نتیجه:

$$RPE = 0$$

این همان ایده‌ای است که در بحث Free Energy Principle نیز دیده شد: کاهش غافلگیری و کاهش خطای پیش‌بینی.

Discount Factor ۴.۱۵

در این جلسه، هدف اصلی عمیق‌تر کردن مفاهیم جلسه‌های قبل بود؛ به‌ویژه Q-Learning، Active RL و ارتباط آن‌ها با سیستم پاداش. نقطه شروع این بود که عامل در یک فضای جست‌وجو حرکت می‌کند. ممکن است درباره بعضی حالت‌ها تجربه قبلی یا prior knowledge داشته باشد، و درباره بعضی حالت‌ها هیچ تجربه‌ای نداشته باشد؛ در این حالت، مقدار اولیه معمولاً صفر در نظر گرفته می‌شود.

فرض کنید عامل در یک حالت شروع قرار دارد و دو انتخاب دارد: رفتن به چپ یا رفتن به راست. اگر عامل فقط پاداش فوری را ببیند، ممکن است بگوید چپ پاداش ۱ دارد و راست فعلاً پاداش ۰ دارد؛ پس در حالت کاملاً greedy به چپ می‌رود. اما ممکن است رفتن به راست در آینده به پاداش بزرگ‌تری، مثلاً ۱۰، برسد. بنابراین عامل فقط نباید به پاداش فوری نگاه کند؛ باید تا حدی آینده را نیز در نظر بگیرد.

برای مدل کردن میزان اهمیت آینده از Discount Factor یا ضریب تخفیف استفاده می‌کنیم. این ضریب معمولاً با γ نشان داده می‌شود:

$$0 \leq \gamma \leq 1$$

اگر $\gamma = 0$ باشد، عامل فقط پاداش فوری را می‌بیند و کاملاً کوتاه‌نگر است. اگر $\gamma = 1$ باشد، عامل آینده را بدون تخفیف کامل لحاظ می‌کند. اگر $0 < \gamma < 1$ باشد، پاداش‌های آینده در محاسبه وارد می‌شوند، اما هرچه دورتر باشند اثر کم‌تری دارند:

$$r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$$

استاد اشاره کرد که مفهوم ضریب تخفیف را می‌توان تا حدی با سازوکارهای زیستی نیز مرتبط دانست. برای مثال درباره سروتونین گفته شد که اگر سطح آن بالاتر باشد، فرد ممکن است کم‌تر عجولانه و کم‌تر کاملاً greedy تصمیم بگیرد و توان بیشتری برای در نظر گرفتن آینده داشته باشد. البته این رفتار فقط به یک ماده زیستی وابسته نیست و ترکیبی از عوامل مختلف می‌تواند روی چیزی شبیه ضریب تخفیف اثر بگذارد.

Value Function ۵.۱۵

TD Learning هدف به‌روزرسانی مقدار یک حالت است:

$$V(s)$$

فرمول کامل‌تر به‌روزرسانی با ضریب تخفیف چنین است:

$$V_{t+1}(s) = (1 - \alpha)V_t(s) + \alpha[r + \gamma V_t(s')]$$

در این فرمول، $V_t(s)$ مقدار قبلی حالت s ، r پاداش دریافت‌شده، s' حالت بعدی، $V_t(s')$ مقدار حالت بعدی، α نرخ یادگیری و γ ضریب تخفیف است.

این فرمول نوعی درونیابی میان دانش قبلی و تجربه جدید است. بخش $(1 - \alpha)V_t(s)$ دانش قبلی را نگه می‌دارد و بخش $\alpha[r + \gamma V_t(s')]$ تجربه جدید را وارد می‌کند.

مثال کلاس سه تجربه داشت:

$$A \xrightarrow{a_1, r=-3} B$$

$$B \xrightarrow{a_1, r=4} A$$

$$A \xrightarrow{a_2, r=-4} A$$

فرض شد $\alpha = 0.9$ ، $\gamma = 1$ و در ابتدا $V(A) = 0$ و $V(B) = 0$.

در تجربه اول، از A به B با پاداش -3 می‌رویم:

$$V(A) = 0.9V(A) + 0.1[-3 + V(B)]$$

چون مقدارهای اولیه صفرند:

$$V(A) = 0.9(0) + 0.1[-3 + 0] = -0.3$$

پس مقدار A کاهش پیدا می‌کند. در تجربه دوم:

$$V(B) = 0.9V(B) + 0.1[4 + V(A)]$$

و چون $V(A) = -0.3$:

$$V(B) = 0.9(0) + 0.1[4 - 0.3] = 0.37$$

در تجربه سوم دوباره $V(A)$ به‌روزرسانی می‌شود:

$$V(A) = 0.9(-0.3) + 0.1[-4 - 0.37]$$

نکته اصلی این است که در TD Learning، در هر مرحله مقدار همان حالتی به‌روزرسانی می‌شود که عامل از آن حرکت کرده است.

۶.۱۵ Q Function

تا اینجا مقدار حالت‌ها را داشتیم، اما هدف نهایی عامل فقط دانستن ارزش حالت‌ها نیست. عامل باید در هر لحظه تصمیم بگیرد کدام عمل را انتخاب کند. بنابراین لازم است بدانیم اگر عامل در حالت s باشد و عمل a را انجام دهد، این انتخاب چقدر ارزش دارد.

این مقدار با $Q(s, a)$ نشان داده می‌شود. $Q(s, a)$ ارزش انجام عمل a در حالت s است. رابطه کلی میان V و Q چنین است:

$$V(s) = \max_a Q(s, a)$$

یعنی ارزش یک حالت برابر است با بیشترین مقدار Q میان عمل‌های ممکن در آن حالت.

در دنیای واقعی، اگر یک عمل را انتخاب کنیم، همیشه دقیقاً به همان حالتی که انتظار داریم نمی‌رسیم. برای مثال ممکن است عامل تصمیم بگیرد به چپ برود؛ با احتمال ۹/۰ واقعاً به خانه چپ برسد، اما با احتمال ۱/۰ به حالت دیگری منتقل شود. این احتمال گذار با تابع $T(s, a, s')$ نشان داده می‌شود:

$$T(s, a, s')$$

یعنی احتمال اینکه عامل از حالت s ، با انجام عمل a ، به حالت s' برود. در این حالت می‌توان مقدار Q را با جمع روی حالت‌های بعدی ممکن نوشت:

$$Q(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

در این رابطه، عامل هم پاداش فوری را در نظر می‌گیرد و هم ارزش حالت بعدی را، اما ارزش آینده با ضریب تخفیف وارد می‌شود.

۷.۱۵ Q-Learning

در Q-Learning به جای اینکه فقط $V(s)$ به‌روزرسانی شود، مستقیماً $Q(s, a)$ به‌روزرسانی می‌شود:

$$Q_{t+1}(s, a) = (1 - \alpha)Q_t(s, a) + \alpha \left[r + \gamma \max_{a'} Q_t(s', a') \right]$$

در این فرمول، s حالت فعلی، a عمل انجام‌شده، r پاداش دریافت‌شده، s' حالت بعدی، و a' عمل‌های ممکن در حالت بعدی هستند. عبارت $\max_{a'} Q_t(s', a')$ بهترین عمل ممکن در حالت بعدی را نشان می‌دهد.

تفاوت مهم Q-Learning با TD Learning ساده این است که در Q-Learning مستقیماً ارزش عمل‌ها در حالت‌ها یاد گرفته می‌شود، نه فقط ارزش خود حالت‌ها. به همین دلیل، Q-Learning یک گام به تصمیم‌گیری واقعی نزدیک‌تر است.

برای مثال، همان اپیزود قبل را در نظر می‌گیریم:

$$A \xrightarrow{a_1, r=-3} B, \quad B \xrightarrow{a_1, r=4} A, \quad A \xrightarrow{a_2, r=-4} A$$

فرض می‌کنیم $\alpha = 0.1$ ، $\gamma = 1$ ، و همه مقادارهای Q در ابتدا صفرند.

در تجربه اول، عامل از A با عمل ۱ به B می‌رود و پاداش -۳ می‌گیرد:

$$Q(A, 1) = 0.9Q(A, 1) + 0.1[-3 + \max_{a'} Q(B, a')]$$

چون هنوز درباره عمل‌های حالت B چیزی نمی‌دانیم، بیشینه صفر است:

$$Q(A, 1) = 0.1[-3] = -0.3$$

در تجربه دوم، عامل از B با عمل ۱ به A می‌رود و پاداش ۴ می‌گیرد:

$$Q(B, 1) = 0.9Q(B, 1) + 0.1[4 + \max_{a'} Q(A, a')]$$

در حالت A ، مقدار $Q(A, 1)$ برابر 0.3 - است و عمل‌های دیگر هنوز مقدار صفر دارند؛ پس بیشینه صفر می‌شود:

$$Q(B, 1) = 0.1[4 + 0] = 0.4$$

در تجربه سوم، عامل از A با عمل ۲ به A می‌رود و پاداش 4 - می‌گیرد:

$$Q(A, 2) = 0.9Q(A, 2) + 0.1[-4 + \max_{a'} Q(A, a')]$$

در این مرحله، بیشینه مقدارهای Q در A همچنان صفر است، پس:

$$Q(A, 2) = 0.1[-4 + 0] = -0.4$$

روند به همین شکل ادامه پیدا می‌کند. در هر تجربه، مقدار $Q(s, a)$ مربوط به همان حالت و همان عمل به‌روزرسانی می‌شود.

۸.۱۵ محیط‌های تصادفی و عدم قطعیت

در محیط قطعی، انجام یک عمل مشخص در یک وضعیت مشخص همیشه به نتیجه‌ای مشخص می‌رسد. اما در محیط تصادفی یا غیرقطعی، یک عمل می‌تواند نتایج مختلفی داشته باشد. مثال کلاس رانندگی بود: در یک محیط ایده‌آل، اگر راننده فرمان را به چپ بچرخاند، انتظار داریم ماشین به چپ برود؛ اما در محیطی غیرقطعی ممکن است ماشین کمی منحرف شود، متوقف شود یا رفتار دیگری نشان دهد.

بنابراین عامل باید با عدم قطعیت در گذارها کنار بیاید. یادگیری در این حالت یعنی تجربه کردن، دریافت بازخورد و اصلاح تدریجی مدل از اینکه هر عمل در هر وضعیت معمولاً به چه پیامدهایی منجر می‌شود.

۹.۱۵ Exploitation و Exploration

اگر عامل فقط از رابطه

$$V(s) = \max_a Q(s, a)$$

برای انتخاب عمل استفاده کند، همیشه عملی را انتخاب می‌کند که تا آن لحظه بهترین تجربه شناخته‌شده را داشته است. این حالت کاملاً greedy یا exploitative است؛ یعنی عامل فقط از تجربه‌های قبلی خود استفاده می‌کند.

اما در تصمیم‌گیری واقعی، همیشه فقط بهترین تجربه قبلی را تکرار نمی‌کنیم. استاد مثال خرید را مطرح کرد: معمولاً همیشه فقط از جایی خرید نمی‌کنیم که قبلاً بهترین تجربه را از آن داشته‌ایم؛ گاهی مکان‌های جدید را هم امتحان می‌کنیم، به‌ویژه اگر فکر کنیم شاید گزینه بهتری وجود داشته باشد.

Exploitation یعنی عامل از بهترین تجربه‌های قبلی خود استفاده کند: «تا الان این عمل بهترین بوده، پس دوباره همان را انتخاب می‌کنم.» در مقابل، Exploration یعنی عامل به عمل‌هایی هم فرصت بدهد که کم‌تر امتحان شده‌اند: «این عمل هنوز بهترین شناخته نشده، اما چون کم‌تر امتحان شده، شاید ارزش کشف کردن داشته باشد.» تصمیم‌گیری در دنیای واقعی معمولاً ترکیبی از این دو است.

۱۰.۱۵ Active RL و Passive RL

برای اینکه عامل فقط greedy نباشد، باید عدم قطعیت را هم وارد تصمیم‌گیری کنیم. اگر امتیاز یک عمل فقط برابر با مقدار Q باشد:

$$\text{Score}(s, a) = Q(s, a)$$

عامل بیشتر به سمت بهره‌برداری از تجربه‌های قبلی می‌رود. اما می‌توان جمله‌ای مربوط به عدم قطعیت نیز اضافه کرد:

$$\text{Score}(s, a) = Q(s, a) + \beta U(s, a)$$

در این رابطه، $Q(s, a)$ تجربه قبلی و پاداش یادگرفته‌شده را نشان می‌دهد، $U(s, a)$ میزان عدم قطعیت درباره آن عمل است، و β مشخص می‌کند عامل چقدر به عدم قطعیت اهمیت می‌دهد.

اگر عملی کمتر امتحان شده باشد، عدم قطعیت آن بیشتر است. بنابراین حتی اگر مقدار Q آن فعلاً خیلی بالا نباشد، ممکن است به خاطر عدم قطعیت بالا انتخاب شود تا عامل آن را کشف کند. این ایده به روش‌هایی مانند Upper Confidence Bound یا UCB شبیه است؛ یعنی عامل فقط میانگین تجربه‌شده را نگاه نمی‌کند، بلکه برای گزینه‌های کمتر امتحان‌شده یک حد بالایی خوش‌بینانه هم در نظر می‌گیرد.

در حالت passive، عامل بیشتر بر اساس تجربه‌های قبلی تصمیم می‌گیرد؛ بهترین گزینه‌های شناخته‌شده را انتخاب می‌کند، تمایل کمی به امتحان گزینه‌های جدید دارد و بیشتر exploit می‌کند. در حالت active، عامل علاوه بر تجربه‌های قبلی، عدم قطعیت را هم در نظر می‌گیرد؛ گزینه‌های کمتر امتحان‌شده را بررسی می‌کند و میان exploration و exploitation تعادل برقرار می‌کند.

استاد برای توضیح شهودی گفت می‌توان حالت active را شبیه جوانی دانست که بیشتر دوست دارد تجربه و کشف کند، و حالت passive را شبیه پیری دانست که بیشتر به تجربه‌های قبلی اتکا دارد. این تشبیه برای فهم شهودی است، نه یک ادعای زیستی دقیق.

فصل ۱۶

مبانی شبکه‌های عصبی اسپایکی

پس از بحث سیستم پاداش و خطای پیش‌بینی پاداش، کلاس وارد شبکه‌های عصبی اسپایکی یا Spiking Neural Networks شد. هدف این بود که ابتدا معماری کلی این شبکه‌ها فهمیده شود و سپس ببینیم چگونه می‌توان سیستم پاداش را از دریچه شبکه‌های اسپایکی مدل‌سازی کرد.

۱.۱۶ چرا شبکه‌های اسپایکی؟

در شبکه‌های عصبی مصنوعی معمولی یا ANN، ورودی‌ها اغلب اعداد پیوسته‌اند؛ برای مثال embedding یا بردارهای عددی. این ورودی‌ها در وزن‌ها ضرب می‌شوند، جمع وزن‌دار ساخته می‌شود، bias اضافه می‌شود، مقدار از activation function عبور می‌کند و در نهایت خروجی یا پیش‌بینی تولید می‌شود. سپس پیش‌بینی با پاسخ درست یا ground truth مقایسه می‌شود و اگر خطا وجود داشته باشد، وزن‌ها با backpropagation اصلاح می‌شوند.

اما این تصویر از نظر زیستی کاملاً واقع‌گرایانه نیست. دو تفاوت مهم در کلاس برجسته شد: مغز با اعداد پیوسته‌ای شبیه ورودی‌های معمولی ANN کار نمی‌کند، و مغز backpropagation کلاسیک و سراسری ندارد. در مغز، یادگیری بیشتر محلی است؛ هر نورون تا حد زیادی بر اساس ورودی‌ها و وضعیت خودش تصمیم می‌گیرد، نه اینکه کل شبکه منتظر خروجی نهایی بماند و سپس گرادیان سراسری به عقب برگردد.

شبکه‌های اسپایکی تلاش می‌کنند به این تصویر زیستی نزدیک‌تر شوند. در این شبکه‌ها، ارتباط نورون‌ها با رخداد‌های گسسته زمانی انجام می‌شود: اسپایک رخ می‌دهد یا رخ نمی‌دهد. به همین دلیل، این شبکه‌ها برای مدل‌سازی‌ای اهمیت دارند که می‌خواهد زمان، رویداد و مصرف انرژی را جدی بگیرد.

۲.۱۶ تفاوت ANN و SNN

در ANN معمولاً با مقدارهای پیوسته سروکار داریم. نورون‌ها در هر گام مقدار عددی دریافت می‌کنند و خروجی عددی تولید می‌کنند. اما در SNN، ورودی و خروجی نورون‌ها به شکل اسپایک است. اگر نورون اسپایک بزند، مقدار آن را می‌توان ۱ در نظر گرفت؛ اگر اسپایک نزند، مقدار ۰ است.

بنابراین در SNN، ورودی یک دنباله زمانی از صفر و یک‌هاست:

$$S = [0, 1, 0, 1, 1, 0, \dots]$$

وقتی مقدار ۱ باشد، یعنی اسپایک رخ داده است؛ و وقتی مقدار ۰ باشد، یعنی اسپایکی وجود ندارد. هر اسپایک در وزن سیناپسی ضرب می‌شود:

$$S_i \times W_i$$

اگر مجموع ورودی‌های وزن‌دار باعث شود ولتاژ نرون از آستانه عبور کند، نرون پس سیناپسی نیز اسپایک می‌زند.

۳.۱۶ نرون زیستی

برای فهم SNN، ابتدا ساختار ساده نرون زیستی مرور شد. یک نرون زیستی را می‌توان به صورت ساده شامل دندريت، بدنه سلولی، آکسون و سیناپس دانست.

۱.۳.۱۶ دندريت

دندريت محل دریافت ورودی است. سیگنال‌های نرون‌های دیگر از طریق سیناپس‌ها به دندريت می‌رسند.

۲.۳.۱۶ Soma

Soma یا بدنه سلولی بخش پردازشی نرون است. در توضیح ساده کلاس، اثر ورودی‌ها در این بخش جمع می‌شود و وضعیت نرون برای اسپایک زدن یا نزدن تعیین می‌شود.

۳.۳.۱۶ آکسون

آکسون خروجی نرون را به نرون‌های دیگر منتقل می‌کند. اگر نرون اسپایک بزند، این رخداد از مسیر آکسون به سمت سیناپس‌ها منتقل می‌شود.

۴.۳.۱۶ سیناپس

سیناپس محل اتصال آکسون یک نرون به دندريت نرون دیگر است. اگر نرونی به نرون دیگر اطلاعات بفرستد، نرون اول presynaptic neuron و نرون دوم postsynaptic neuron نامیده می‌شود. نرون پیش سیناپسی اطلاعات را می‌فرستد و نرون پس سیناپسی آن را دریافت می‌کند.

۴.۱۶ اسپایک و زمان محور بودن محاسبه

در SNN زمان بخشی اصلی از محاسبه است. در ANN معمولاً با یک بردار عددی برای یک شیء یا ورودی کار می‌کنیم، اما در SNN ورودی یک توالی زمانی است. مهم است کدام اسپایک زودتر آمده، کدام بعداً آمده و فاصله زمانی میان آن‌ها چقدر بوده است.

به همین دلیل، خروجی SNN نیز فقط یک مقدار عددی ساده نیست؛ بلکه معمولاً باید الگوی اسپایک‌ها در طول زمان تفسیر شود. هر اسپایک می‌تواند در همان لحظه بخشی از تصمیم‌گیری را فعال کند، بی‌آنکه لازم باشد همه نورون‌های شبکه درگیر محاسبه شوند.

۵.۱۶ مدل LIF

برای مدل‌سازی نورون اسپایکی، مدل‌های مختلفی وجود دارد؛ از جمله Leaky Integrate-and-Fire یا LIF، مدل Izhikevich و مدل Hodgkin-Huxley. در این جلسه تمرکز روی مدل LIF بود، چون هم ساده است و هم برخی رفتارهای مهم نورون زیستی را نشان می‌دهد.

فرمول ساده مدل چنین است:

$$V(t+1) = \lambda V(t) + I(t)$$

در این رابطه، $V(t)$ ولتاژ قبلی، λ ضریب نشت یا leak factor و $I(t)$ جریان ورودی است. جریان ورودی معمولاً از اسپایک‌های ورودی و وزن‌های سیناپسی ساخته می‌شود:

$$I(t) = \sum_i W_i S_i(t)$$

اگر ولتاژ از آستانه عبور کند:

$$V(t+1) \geq V_{\text{threshold}}$$

نورون اسپایک می‌زند.

۱.۵.۱۶ Integration

Integration یعنی اثر ورودی‌ها با هم جمع می‌شود. اگر نورون از چند نورون پیش‌سیناپسی ورودی بگیرد، اسپایک‌های ورودی در وزن‌های سیناپسی ضرب می‌شوند. سپس اثر آن‌ها در ولتاژ نورون تجمع پیدا می‌کند.

۲.۵.۱۶ Fire

Fire یعنی اگر ولتاژ تجمع‌یافته از آستانه عبور کند، نورون اسپایک می‌زند. در این لحظه خروجی نورون برابر با رخداد اسپایک است.

۳.۵.۱۶ Leak

Leak یعنی اگر ورودی کافی وجود نداشته باشد، ولتاژ نورون به تدریج کاهش پیدا می‌کند. این کاهش تدریجی همان نشأت است و باعث می‌شود اثر ورودی‌ها برای همیشه در نورون باقی نماند.

۴.۵.۱۶ Refractory Period

پس از اسپایک، نورون برای مدت کوتاهی وارد دوره‌ای می‌شود که در آن فعالیت ندارد یا به سادگی دوباره اسپایک نمی‌زند. این دوره refractory period نامیده می‌شود و از رفتار نورون‌های واقعی الهام گرفته شده است.

۶.۱۶ مصرف انرژی و پردازش رویدادمحور

یکی از ویژگی‌های مهم SNN این است که شبکه می‌تواند همیشه در حالت آماده‌باش یا on-call باشد، اما مصرف انرژی پایینی داشته باشد. دلیل آن این است که وقتی ورودی صفر است، محاسبه سنگین انجام نمی‌شود. نورون فقط در صورت وجود اسپایک و عبور از آستانه فعال می‌شود.

در ANN معمولاً در هر گام محاسباتی، لایه‌ها درگیر ضرب و جمع می‌شوند، حتی اگر تغییر مهمی در ورودی رخ نداده باشد. اما در SNN رخدادهای مهم، یعنی اسپایک‌ها، محاسبه را فعال می‌کنند. به همین دلیل، این شبکه‌ها برای سیستم‌هایی مناسب‌اند که باید همیشه فعال باشند اما مصرف انرژی کمی داشته باشند.

مثال‌های کلاس شامل دستیار صوتی همیشه‌روشن برای تشخیص wake-up word، سمک‌ها، سیستم‌های شنود کم‌مصرف، پهپادها، ربات‌هایی که باید نسبت به مانع یا حمله واکنش سریع نشان دهند، و سیستم‌هایی بود که به واکنش سریع و محلی نیاز دارند.

فصل ۱۷

یادگیری در شبکه‌های اسپایکی

پس از معرفی معماری SNN، بحث به این پرسش رسید که شبکه اسپایکی چگونه یاد می‌گیرد. در مغز، وقتی برخی مسیرهای نورونی بارها فعال می‌شوند، اتصال میان آن‌ها تقویت می‌شود. بنابراین یادگیری در این سطح را می‌توان به تغییر وزن‌های سیناپسی ربط داد:

$$\Delta W$$

وقتی یک مسیر بارها استفاده شود، وزن سیناپسی آن افزایش پیدا می‌کند و آن مسیر در شبکه تقویت می‌شود. استاد برای توضیح این ایده، مثال مسیر روی چمن را مطرح کرد. فرض کنید در یک دانشگاه مسیر رسمی برای عبور افراد طراحی شده است، اما پس از مدتی مسیر دیگری روی چمن بیشتر پا می‌خورد و سبزه‌هایش از بین می‌رود. این یعنی افراد بدون دستور مستقیم، مسیر دیگری را بارها انتخاب کرده‌اند. در شبکه نورونی نیز مسیری که بارها فعال شود، وزن بیشتری می‌گیرد و تقویت می‌شود.

۱.۱۷ قانون Hebbian Learning

ایده پایه برای یادگیری بدون ناظر در شبکه‌های اسپایکی، قانون Hebb است. عبارت مشهور آن چنین است:

Fire together, wire together

یعنی اگر دو نورون مرتب با هم فعال شوند، اتصال بین آن‌ها تقویت می‌شود.

در اینجا منظور از «با هم فعال شدن» الزاماً هم‌زمانی دقیق نیست. منظور این است که نورون پیش‌سیناپسی اسپایک بزند و نورون پس‌سیناپسی در ادامه فعال شود. اگر نورون پیش‌سیناپسی اسپایک بزند و بعد نورون پس‌سیناپسی نیز اسپایک بزند، وزن سیناپسی میان آن‌ها افزایش پیدا می‌کند.

این اتفاق مهم است، چون نشان می‌دهد میان آن دو نورون نوعی ارتباط اطلاعاتی وجود دارد. ممکن است هر دو نسبت به موضوعی مشابه حساس باشند، یا میان آن‌ها رابطه‌ای علی وجود داشته باشد؛ یعنی فعال شدن نورون پیش‌سیناپسی در فعال شدن نورون پس‌سیناپسی نقش داشته باشد.

STDP ۲.۱۷

STDP مخفف Spike-Timing-Dependent Plasticity است. یعنی تغییرپذیری وزن‌ها به زمان‌بندی اسپایک‌ها وابسته است. قانون Hebb می‌گوید اگر دو نورون با هم فعال شوند، اتصالشان تقویت می‌شود. اما STDP نکته دقیق‌تری اضافه می‌کند: ترتیب و فاصله زمانی اسپایک‌ها مهم است.

اگر نورون پیش‌سیناپسی پیش از نورون پس‌سیناپسی اسپایک بزند و به فعال شدن آن کمک کند، وزن سیناپسی باید تقویت شود. اما اگر نورون پس‌سیناپسی پیش از نورون پیش‌سیناپسی فعال شده باشد، احتمالاً فعال شدن پس‌سیناپسی به علت آن نورون پیش‌سیناپسی خاص نبوده است؛ بنابراین وزن میان آن دو باید تضعیف شود.

LTP ۱.۲.۱۷

LTP یا Long-Term Potentiation زمانی رخ می‌دهد که نورون پیش‌سیناپسی قبل از نورون پس‌سیناپسی اسپایک بزند:

$$t_{\text{pre}} < t_{\text{post}}$$

در این حالت رابطه علی-زمانی معقولی وجود دارد و وزن سیناپسی تقویت می‌شود:

$$\Delta W > 0$$

LTD ۲.۲.۱۷

LTD یا Long-Term Depression زمانی رخ می‌دهد که نورون پیش‌سیناپسی پس از نورون پس‌سیناپسی اسپایک بزند:

$$t_{\text{pre}} > t_{\text{post}}$$

در این حالت نورون پیش‌سیناپسی احتمالاً علت فعال شدن نورون پس‌سیناپسی نبوده است، پس وزن سیناپسی تضعیف می‌شود:

$$\Delta W < 0$$

۳.۲.۱۷ نقش اختلاف زمانی

در STDP فقط مثبت یا منفی بودن اختلاف زمانی مهم نیست؛ اندازه اختلاف زمانی نیز اهمیت دارد. اختلاف زمانی را می‌توان چنین تعریف کرد:

$$\Delta t = t_{\text{post}} - t_{\text{pre}}$$

اگر $\Delta t > 0$ ، یعنی نورون پیش‌سیناپسی زودتر از نورون پس‌سیناپسی فعال شده و این حالت به LTP مربوط است. اگر $\Delta t < 0$ ، یعنی نورون پس‌سیناپسی زودتر فعال شده و این حالت به LTD مربوط است.

هرچه $|\Delta t|$ کوچک‌تر باشد، دو اسپایک از نظر زمانی به هم نزدیک‌تر بوده‌اند و ارتباط میان آن‌ها قوی‌تر فرض می‌شود؛ پس تغییر وزن بیشتر خواهد بود. هرچه $|\Delta t|$ بزرگ‌تر باشد، ارتباط ضعیف‌تر است و تغییر وزن کم‌تر می‌شود. فرمول شهودی برای تقویت چنین بیان شد:

$$\Delta W = A_+ e^{-\Delta t/\tau_+}$$

و برای تضعیف:

$$\Delta W = -A_- e^{\Delta t/\tau_-}$$

در این فرمول‌ها، A_+ دامنه تقویت، A_- دامنه تضعیف، τ_+ و τ_- ثابت‌های زمانی، و Δt اختلاف زمانی اسپایک‌ها هستند.

اگر نوروں پیش‌سیناپسی قبل از پس‌سیناپسی و با فاصله کم اسپایک بزند، تقویت زیاد است. اگر همین ترتیب با فاصله زیاد رخ دهد، تقویت کم‌تر است. اگر پیش‌سیناپسی بعد از پس‌سیناپسی و با فاصله کم اسپایک بزند، تضعیف زیاد است؛ و اگر فاصله زیاد باشد، تضعیف کم‌تر است.

اختلاف زمانی زمانی محاسبه می‌شود که هر دو نوروں اسپایک زده باشند. اگر فقط نوروں پیش‌سیناپسی اسپایک بزند و نوروں پس‌سیناپسی فعال نشود، هنوز اختلاف زمانی معناداری برای STDP وجود ندارد.

۳.۱۷ Reward-Modulated STDP

تا اینجا دو جزء جداگانه داشتیم: از سیستم پاداش و TD Learning، مفهوم خطای پیش‌بینی پاداش را داشتیم:

$$\text{Reward Prediction Error} = R_{\text{actual}} - R_{\text{predicted}}$$

و از شبکه‌های اسپایکی، تغییر وزن بر اساس زمان‌بندی اسپایک‌ها را داشتیم:

$$\Delta W_{\text{STDP}}$$

هدف Reward-Modulated STDP این است که سیستم پاداش روی STDP سوار شود. یعنی هم اثر زمان‌بندی اسپایک‌ها لحاظ شود و هم اثر خطای پیش‌بینی پاداش.

فرم کلی تغییر وزن در کلاس چنین بیان شد:

$$\Delta W = \eta \times \delta \times E(t)$$

در این رابطه، η نرخ یادگیری، δ خطای پیش‌بینی پاداش، $E(t)$ اثر STDP یا eligibility trace، و ΔW تغییر وزن سیناپسی است.

در این مدل، STDP مشخص می‌کند کدام سیناپس‌ها از نظر زمانی در رفتار نقش داشته‌اند، و خطای پیش‌بینی پاداش مشخص می‌کند که نتیجه رفتار باید آن سیناپس‌ها را تقویت کند یا تضعیف. اگر سیناپسی از نظر STDP فعال و مرتبط بوده باشد و سپس سیستم پاداش بگوید نتیجه خوب بوده است، آن سیناپس بیشتر تقویت می‌شود. اما اگر نتیجه بدتر از انتظار باشد، همان سیناپس می‌تواند تضعیف شود.

این ایده همچنین به مسئله نسبت دادن علت‌ها کمک می‌کند. وقتی نورون پس‌سیناپسی اسپایک می‌زند، ممکن است از چند نورون پیش‌سیناپسی ورودی گرفته باشد و دقیقاً ندانیم کدام ورودی علت اصلی عبور از آستانه بوده است. STDP ردپای نورون‌هایی را که کمی قبل‌تر اسپایک زده‌اند نگه می‌دارد، و پاداش بعدی تعیین می‌کند این ردپا در جهت تقویت استفاده شود یا تضعیف.

۴.۱۷ از SNN پایه به Deep SNN

در جلسه‌های قبل، نورون اسپایکی با مدل LIF و یادگیری سیناپسی با STDP و Reward-Modulated STDP بررسی شد. در این جلسه، بحث از شبکه‌های کم‌عمق و پایه‌ای به سمت Deep SNN رفت؛ یعنی شبکه‌هایی که چندین لایه اسپایکی پشت سر هم دارند.

در یادآوری زیستی، نورون از دندریت، جسم سلولی، آکسون، پایانه آکسونی، شکاف سیناپسی و وزن سیناپسی میان نورون پیش‌سیناپسی و پس‌سیناپسی تشکیل می‌شود. وقتی نورون پیش‌سیناپسی اسپایک می‌زند، این رویداد از راه وزن سیناپسی روی نورون بعدی اثر می‌گذارد و به شکل جریان ورودی در نورون پس‌سیناپسی ظاهر می‌شود.

در مدل LIF، نورون جریان‌های ورودی را جمع می‌کند و ولتاژ غشایی آن افزایش می‌یابد. اگر ولتاژ از آستانه عبور کند، نورون اسپایک می‌زند:

$$V > \theta$$

پس از اسپایک، ولتاژ ریست می‌شود و نورون وارد دوره refractory می‌شود؛ یعنی برای مدتی دوباره فعال نمی‌شود یا فعال‌شدن آن دشوارتر است. خروجی ساده نورون را می‌توان چنین نوشت:

$$S = \begin{cases} 1 & V > \theta \\ 0 & V \leq \theta \end{cases}$$

بنابراین خروجی نورون اسپایکی معمولاً دودویی است: یا اسپایک داریم یا نداریم.

در ساده‌ترین حالت، اگر ورودی نورون x و وزن سیناپسی w باشد، جریان یا اثر سیناپسی چنین نوشته می‌شود:

$$I = xw$$

و در حالت چند ورودی:

$$V = x_1w_1 + x_2w_2 + x_3w_3 + \dots$$

اگر ولتاژ از آستانه عبور کند، خروجی ۱ می‌شود؛ در غیر این صورت خروجی صفر باقی می‌ماند.

در Deep SNN چندین لایه نورون اسپایکی پشت سر هم قرار می‌گیرند. خروجی هر لایه، یعنی دنباله‌ای از اسپایک‌ها، به لایه بعدی منتقل می‌شود و در آنجا دوباره وزن سیناپسی، جریان، ولتاژ و تصمیم اسپایک‌زدن یا نزدن محاسبه می‌شود. از این نظر، شبکه شبیه شبکه‌های عمیق معمولی چندلایه است؛ اما ورودی و خروجی هر لایه به شکل اسپایکی و دودویی است.

۵.۱۷ الهام زیستی از ساختار چندلایه مغز

ایده چندلایه بودن شبکه‌ها از ساختار مغز نیز الهام می‌گیرد. در سیستم بینایی انسان، ناحیه‌هایی مانند V1، V2، V3، V4، MT/V5 و IT در پردازش اطلاعات بینایی نقش دارند. در لایه‌های اولیه، ویژگی‌های ساده‌تری مانند لبه، خط، رنگ و جهت استخراج می‌شوند. هرچه پردازش جلوتر می‌رود، بازنمایی‌ها پیچیده‌تر و انتزاعی‌تر می‌شوند: از لبه و خط به شکل‌های پیچیده‌تر، از شکل‌ها به چهره، و در سطح‌های بالاتر به هویت فرد یا مفهوم شیء.

پس عمیق‌تر شدن شبکه با توانایی ساخت representation‌های پیچیده‌تر ارتباط دارد. البته در مغز، پردازش همیشه یک مسیر خطی واحد نیست. خروجی ناحیه‌های اولیه می‌تواند وارد مسیرهای مختلف شود؛ برخی لایه‌ها میان چند وظیفه مشترک‌اند و سپس مسیرها تخصصی‌تر می‌شوند. این تصویر به ایده multi-task learning شباهت دارد.

در سیستم بینایی، مسیر ventral بیشتر با تشخیص «چیستی» شیء و مسیر dorsal بیشتر با «کجایی»، حرکت و تعامل با شیء مرتبط دانسته می‌شود. اگر یک مسیر آسیب ببیند، ممکن است فرد شیء را ببیند اما درست با آن تعامل نکند؛ یا برعکس، بتواند با اشیا تعامل کند اما نتواند دقیق تشخیص دهد چه هستند.

این ساختار چندلایه فقط به بینایی محدود نیست. در سیستم لامسه نیز ناحیه‌هایی مانند S1 یا Primary Somatosensory Cortex و S2 یا Secondary Somatosensory Cortex وجود دارند. در S1 اطلاعات اولیه‌تری مانند لمس، درد و دما پردازش می‌شود و در S2 اطلاعات حسی ترکیب‌شده‌تر و انتزاعی‌تر شکل می‌گیرند.

۶.۱۷ مصرف انرژی و انگیزه Deep SNN

یکی از انگیزه‌های مهم برای مطالعه SNN، مصرف انرژی پایین مغز است. مغز انسان با وجود پیچیدگی زیاد، حدود ۲۰ وات انرژی مصرف می‌کند؛ در حالی که مدل‌های عمیق معمولی روی GPUها و سامانه‌های بزرگ می‌توانند انرژی بسیار بیشتری مصرف کنند.

در کلاس دو دلیل برای این بهینگی برجسته شد. نخست، اسپایکی بودن است: در مغز، ورودی و خروجی نورون‌ها پیوسته نیست و بیشتر به شکل رویدادهای صفر و یک دیده می‌شود:

۱ یا ۰

این دودویی و رویدادمحور بودن می‌تواند مصرف انرژی را کاهش دهد و سطحی از انتزاع ایجاد کند.

دلیل دوم، محلی بودن محاسبات است. در مغز، هر نورون تا حد زیادی بر اساس اطلاعات محلی خود تصمیم می‌گیرد. این وضعیت با شبکه‌های عمیق معمولی متفاوت است که در آن‌ها خطا به صورت سراسری محاسبه می‌شود و با backpropagation از انتها به ابتدا برمی‌گردد. محلی بودن نیز یکی از عواملی است که به کاهش مصرف انرژی کمک می‌کند.

سؤال اصلی این است که آیا می‌توان Deep SNN‌هایی ساخت که هم توانایی شبکه‌های عمیق را داشته باشند و هم در فاز اجرا به مزیت انرژی و رویدادمحوری مغز نزدیک‌تر شوند. برای پاسخ به این سؤال، باید روش یادگیری در شبکه‌های اسپایکی عمیق را بررسی کرد.

۷.۱۷ محدودیت‌های STDP در شبکه‌های عمیق

STDP برای شبکه‌های کوچک یا کم عمق می‌تواند مفید باشد، به‌ویژه وقتی یادگیری بدون ناظر است، برچسب و پاداش نداریم، و شبکه قرار است از الگوهای تکرارشونده ورودی یاد بگیرد. در Reward-Modulated STDP نیز همان ایده زمان‌بندی اسپایک با سیگنال پاداش ترکیب می‌شود تا پاداش تعیین کند ردپای سیناپسی باید تقویت شود یا تضعیف. اما وقتی تعداد لایه‌ها زیاد می‌شود، STDP به‌تنهایی کافی نیست. همگرایی سخت‌تر می‌شود، دقت پایین می‌آید، تنظیم وزن‌ها فقط با قوانین محلی دشوار می‌شود و شبکه برای fine-tune کردن وزن‌ها به روش‌های دقیق‌تری نیاز پیدا می‌کند. بنابراین در Deep SNN، مخصوصاً در وظایف نظارت‌شده، معمولاً به روش‌هایی شبیه backpropagation نیاز داریم.

۸.۱۷ Surrogate Gradient

در یادگیری نظارت‌شده، داده‌ها و برچسب‌ها مشخص‌اند. شبکه ورودی را می‌گیرد، خروجی تولید می‌کند و آن را با label مقایسه می‌کند. سپس وزن‌ها اصلاح می‌شوند. در شبکه‌های عمیق معمولی، این اصلاح با backpropagation انجام می‌شود.

مشکل اصلی در SNN این است که خروجی نرون یک تابع پله‌ای است:

$$S = \begin{cases} 1 & V > \theta \\ 0 & V \leq \theta \end{cases}$$

تابع پله‌ای تقریباً همه‌جا مشتق صفر دارد. بنابراین اگر بخواهیم backpropagation انجام دهیم، مشتق لازم برای به‌روزرسانی وزن‌ها از مسیر خروجی اسپایک به دست نمی‌آید. پرسش اصلی این است: چگونه می‌توان در شبکه اسپایکی backpropagation انجام داد، وقتی تابع اسپایک مشتق‌پذیر نیست؟

۱.۸.۱۷ مسئله مشتق‌پذیری اسپایک

راه‌حل رایج، استفاده از Surrogate Gradient است. ایده این است که در فاز forward، شبکه همچنان اسپایکی باقی بماند و خروجی نرون واقعاً صفر یا یک باشد؛ اما در فاز backward، برای محاسبه گرادیان، تابع پله‌ای با تابعی نرم‌تر و مشتق‌پذیر جایگزین شود.

۲.۸.۱۷ تابع جایگزین

تابع جایگزین یا surrogate function می‌تواند شکلی شبیه sigmoid یا تابع نرم دیگر داشته باشد. نقش آن این نیست که رفتار واقعی نرون را در فاز پیش‌رو تغییر دهد؛ نقش آن فقط فراهم کردن مشتقی است که در backpropagation بتوان از آن استفاده کرد.

پس در این چارچوب:

- در forward، تابع واقعی اسپایک استفاده می‌شود؛
- در backward، از تابع جایگزین مشتق‌پذیر برای محاسبه گرادیان استفاده می‌شود.

۳.۸.۱۷ تفاوت با Activation Function معمولی

در شبکه‌های عصبی معمولی، activation function در خود forward pass استفاده می‌شود. یعنی خروجی هر لایه واقعاً از تابعی مانند sigmoid، ReLU یا tanh عبور می‌کند.

اما در SNN با surrogate gradient، تابع جایگزین فقط برای backpropagation استفاده می‌شود. در فاز forward همچنان اسپایک واقعی داریم، ورودی و خروجی همچنان صفر و یک هستند، و تابع surrogate فقط برای محاسبه مشتق و اصلاح وزن‌ها وارد می‌شود. بنابراین surrogate function همان activation function معمولی نیست، چون نقش اصلی آن در فاز backward است، نه در فاز forward.

۹.۱۷ Deep SNN در Feedforward

در فاز feedforward، اسپایک‌های لایه قبلی به لایه بعدی منتقل می‌شوند. اگر خروجی لایه قبلی را S و وزن سیناپسی میان دو لایه را W بگیریم، جریان ورودی به نورون بعدی در ساده‌ترین حالت چنین است:

$$I = SW$$

در حالت زمانی، جریان می‌تواند اثر مقدار قبلی خود را نیز حفظ کند:

$$I[t+1] = \alpha I[t] + S[t]W$$

در این رابطه، $I[t]$ جریان قبلی، α ضریب کاهش یا decay جریان، $S[t]$ اسپایک ورودی و W وزن سیناپسی است. ولتاژ نورون نیز با توجه به ولتاژ قبلی، ضریب نشت و جریان ورودی به‌روزرسانی می‌شود:

$$V[t+1] = \beta V[t] + I[t+1]$$

که در آن، $V[t]$ ولتاژ قبلی، β ضریب نشت ولتاژ و $I[t+1]$ جریان ورودی جدید است. اگر ولتاژ از آستانه بیشتر شود، اسپایک تولید می‌شود:

$$S[t+1] = \begin{cases} 1 & V[t+1] > \theta \\ 0 & V[t+1] \leq \theta \end{cases}$$

پس از اسپایک، نورون باید ریست شود؛ یعنی ولتاژ و گاهی جریان آن کاهش یابد یا صفر شود.

۱.۹.۱۷ نقش reset

وقتی نورون اسپایک می‌زند، فقط به لایه بعدی سیگنال نمی‌فرستد؛ بلکه خودش نیز ریست می‌شود. بنابراین اسپایک دو اثر دارد: به لایه بعدی منتقل می‌شود و در وزن سیناپسی ضرب می‌شود، و هم‌زمان روی نورونی که اسپایک زده اثر می‌گذارد و ولتاژ آن را ریست می‌کند. به همین دلیل، در SNN زمان و وضعیت داخلی نورون اهمیت اساسی دارند.

۱۰.۱۷ Surrogate Gradient با Backpropagation

در انتهای شبکه، خروجی با هدف مقایسه می‌شود و خطا محاسبه می‌شود. برای نمونه، اگر از Mean Squared Error استفاده کنیم:

$$E = \frac{1}{2}(T - O)^2$$

که در آن T هدف و O خروجی شبکه است.

برای به‌روزرسانی وزن‌ها، مانند شبکه‌های معمولی از گرادینان استفاده می‌کنیم:

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial E}{\partial W}$$

در این رابطه، η نرخ یادگیری و $\frac{\partial E}{\partial W}$ مشتق خطا نسبت به وزن است.

با استفاده از قاعده زنجیره‌ای:

$$\frac{\partial E}{\partial W} = \frac{\partial E}{\partial O} \cdot \frac{\partial O}{\partial \text{Net}} \cdot \frac{\partial \text{Net}}{\partial W}$$

اگر:

$$\text{Net} = XW$$

آنگاه:

$$\frac{\partial \text{Net}}{\partial W} = X$$

و برای خطای مربعی:

$$\frac{\partial E}{\partial O} = O - T$$

بخش اصلی surrogate gradient در جمله میانی است:

$$\frac{\partial O}{\partial \text{Net}}$$

زیرا اگر O را خروجی تابع پله‌ای واقعی بگیریم، مشتق آن صفر می‌شود. بنابراین این مشتق را از تابع surrogate می‌گیریم.

۱.۱۰.۱۷ مثال عددی

فرض کنید:

$$X = 2, \quad W = 0.4, \quad \theta = 1, \quad T = 1, \quad V_0 = 0$$

ابتدا مقدار Net را حساب می‌کنیم:

$$Net = XW = 2 \times 0.4 = 0.8$$

چون:

$$0.8 < 1$$

نورون اسپایک نمی‌زند و بنابراین:

$$O = 0$$

اما هدف برابر ۱ بوده است:

$$T = 1$$

پس خروجی شبکه اشتباه است و وزن باید افزایش یابد.

مشتق خطا نسبت به خروجی:

$$\frac{\partial E}{\partial O} = O - T = 0 - 1 = -1$$

فرض شد مقدار مشتق تابع surrogate در نقطه $Net = 0.8$ تقریباً 0.25 است. همچنین:

$$\frac{\partial Net}{\partial W} = X = 2$$

پس:

$$\frac{\partial E}{\partial W} = (-1)(0.25)(2) = -0.5$$

اگر نرخ یادگیری $\eta = 0.1$ باشد:

$$W_{\text{new}} = 0.4 - 0.1(-0.5) = 0.45$$

بنابراین وزن از 0.4 به 0.45 افزایش پیدا می‌کند. این نتیجه منطقی است، چون خروجی باید ۱ می‌شد اما نورون به آستانه نرسیده بود؛ در نتیجه وزن افزایش می‌یابد تا دفعه بعد ولتاژ راحت‌تر از آستانه عبور کند.

۱۱.۱۷ محلی بودن، آموزش سراسری و ارزش عملی SNN

استفاده از surrogate gradient در فاز آموزش تا حدی ویژگی محلی بودن SNN را کاهش می‌دهد، چون خطا به صورت سراسری محاسبه می‌شود و backpropagation انجام می‌گیرد. اما نکته مهم این است که surrogate gradient فقط در فاز آموزش استفاده می‌شود.

پس از آموزش، شبکه همچنان اسپایکی است: ورودی و خروجی صفر و یک باقی می‌مانند، مدل می‌تواند روی سخت‌افزار neuromorphic اجرا شود، و دیگر نیازی به surrogate gradient یا backpropagation در فاز اجرا نیست. وزن‌ها پیش‌تر fine-tune شده‌اند.

بنابراین چارچوب اصلی SNN حفظ می‌شود. فقط برای اینکه بتوانیم وزن‌ها را در شبکه عمیق و نظارت‌شده تنظیم کنیم، در فاز آموزش از surrogate gradient کمک می‌گیریم.

ارزش عملی SNN بیشتر در فاز اجرا روشن می‌شود. در کاربردهایی مانند ربات، پهپاد، سامانه‌های embedded، سخت‌افزار neuromorphic و پردازش real-time با مصرف انرژی کم، شبکه آموزش دیده می‌تواند به صورت اسپایکی اجرا شود. در این حالت، مهم نیست وزن‌ها دقیقاً با چه روشی آموزش دیده‌اند؛ مهم این است که inference با اسپایک‌های صفر و یک انجام شود.

۱۲.۱۷ کاربرد STDP در برابر Surrogate Gradient

STDP مناسب‌تر است وقتی شبکه کوچک یا کم‌عمق است، یادگیری بدون ناظر است، یادگیری online لازم داریم، یا هدف، پیاده‌سازی زیستی‌تر است. در مقابل، surrogate gradient برای Deep SNN و وظایف نظارت‌شده مناسب‌تر است، جایی که باید خطای خروجی با هدف مقایسه شود و وزن‌ها در چندین لایه تنظیم شوند.

بخش ششم

مدل‌های اسپایکی مدرن

فصل ۱۸

SpikeFormer و Spiking Transformer

در آغاز این بخش، مسیر قبلی درس جمع‌بندی شد. بحث سیستم پاداش تا حد لازم پیش رفته بود و تمرکز درس به شبکه‌های اسپایکی عمیق منتقل شد. تا اینجا چند نکته روشن شده بود: SNNها از نظر ورودی رویدادمحور و مصرف انرژی به مغز نزدیک‌ترند، ورودی آنها معمولاً به صورت اسپایک‌های صفر و یک است، داده در آنها ماهیت زمانی دارد، و در مقایسه با شبکه‌های عمیق معمولی می‌توانند از نظر انرژی کم‌هزینه‌تر باشند.

در Deep SNN نیز اگر یادگیری نظارت‌شده داشته باشیم، شبکه برچسب را می‌بیند، خروجی خود را با هدف مقایسه می‌کند و بر اساس خطا وزن‌ها را به‌روزرسانی می‌کند. مسئله اصلی همان گسسته بودن خروجی‌های اسپایکی است؛ به همین دلیل برای آموزش، از ایده surrogate gradient استفاده می‌شود. اگر در یک Deep SNN از linear layer صحبت می‌کنیم، منظور این است که وزن‌ها در اسپایک‌ها ضرب می‌شوند و خروجی اسپایکی تازه‌ای برای لایه بعدی ساخته می‌شود:

$$W \times \text{spike}$$

۱.۱۸ چرا Transformer در SNN؟

وقتی بتوانیم معماری‌های عمیق اسپایکی بسازیم، پرسش طبیعی این است که آیا می‌توان ایده Transformer را نیز وارد SNN کرد. در شبکه‌های عمیق معمولی، Transformerها در بسیاری از حوزه‌ها معماری‌های بسیار موفقی هستند. بنابراین اگر بخواهیم یک SNN قدرتمندتر داشته باشیم، یکی از مسیرها ترکیب Transformer و Spiking Neural Network است؛ مسیری که در مدل‌هایی مانند SpikeFormer دنبال می‌شود.

برای ساختن Transformer اسپایکی، سه بخش اصلی باید روشن شود:

۱. کدگذاری داده ورودی؛

۲. Attention؛

۳. بلوک MLP.

اگر این سه بخش مشخص شوند، می‌توان گفت معماری‌ای شبیه Transformer اما مبتنی بر SNN داریم.

۲.۱۸ مسئله کدگذاری ورودی

در شبکه‌های معمولی، داده را می‌توان مستقیم به شبکه داد؛ اما در SNN داده باید به دنباله‌ای از اسپایک‌ها تبدیل شود. اگر داده اولیه شامل عددهای پیوسته یا floating-point باشد، باید این مقادارها به صفر و یک تبدیل شوند و این تبدیل در چند گام زمانی رخ دهد.

پس کدگذاری ورودی دو کار انجام می‌دهد: مقادارهای پیوسته را به اسپایک‌های صفر و یک تبدیل می‌کند، و آن‌ها را در چند گام زمانی پخش می‌کند. برای مثال، ممکن است داده را در چهار گام زمانی نمایش دهیم:

$$T = 4$$

در این حالت، هر مقدار عددی باید به یک دنباله اسپایکی در طول زمان تبدیل شود.

۱.۲.۱۸ کدگذاری تصویر

فرض کنید ورودی یک تصویر است و مقدار پیکسل‌ها در بازه ۰ تا ۲۵۵ قرار دارند. نخستین گام معمولاً نرمال‌سازی است:

$$x_{\text{norm}} = \frac{x}{255}$$

با این کار مقدار هر پیکسل به بازه صفر تا یک می‌رود.

برای نمونه، اگر چند مقدار نرمال‌شده داشته باشیم:

$$0.5, \quad 0.8, \quad 0.5, \quad 0.2$$

و بخواهیم آن‌ها را در چهار گام زمانی پخش کنیم، هر مقدار را در ۴ ضرب می‌کنیم:

$$0.5 \times 4 = 2, \quad 0.8 \times 4 = 3.2, \quad 0.5 \times 4 = 2, \quad 0.2 \times 4 = 0.8$$

سپس مقادارها را گرد می‌کنیم:

$$2, \quad 3, \quad 2, \quad 1$$

یعنی مقدار اول در کل زمان یک اسپایک تولید می‌کند، مقدار دوم سه اسپایک، مقدار سوم دو اسپایک و مقدار چهارم نیز دو اسپایک.

به این ترتیب، داده ثابت اولیه در طول چند گام زمانی پخش می‌شود و به دنباله‌ای اسپایکی تبدیل می‌گردد. استاد این روند را شبیه نوعی «برعکس integration» توضیح داد: عدد پیوسته به جای اینکه مستقیم نگه داشته شود، در طول زمان باز می‌شود.

۲.۲.۱۸ نکته محاسباتی در کدگذاری

پشت بسیاری از روش‌های کدگذاری، لزوماً معنای زیستی یا معنایی بسیار دقیقی وجود ندارد. این روش‌ها بیشتر مدل‌های محاسباتی‌اند. هدف اصلی این است که داده پیوسته به اسپایک تبدیل شود، نرخ اسپایک با مقدار عدد اولیه رابطه معنادار داشته باشد، و مدل در نهایت از نظر دقت و مصرف انرژی عملکرد قابل قبولی پیدا کند.

در شبکه‌های اسپایکی معمولاً یک مبادله وجود دارد:

$$\text{Energy / Cost} \leftrightarrow \text{Accuracy}$$

اگر از Transformer معمولی استفاده کنیم، معمولاً دقت بیشتر است، اما مصرف انرژی و هزینه محاسباتی نیز بیشتر می‌شود. اگر از SNN استفاده کنیم، مصرف انرژی کمتر است، اما ممکن است دقت کاهش یابد. به همین دلیل، در مقاله‌های مربوط به SpikeFormer معمولاً مدل را با SNN‌های دیگر مقایسه می‌کنند، نه لزوماً با Transformer‌های معمولی؛ چون از همان مرحله کدگذاری بخشی از اطلاعات پیوسته داده از دست می‌رود.

۳.۲.۱۸ کدگذاری متن

در متن نیز پس از ساخت embedding، با عددی‌های پیوسته سروکار داریم. برای نمونه، ممکن است یکی از ویژگی‌ها مقدار زیر را داشته باشد:

$$x = 0.8$$

این مقدار باید به دنباله‌ای اسپایکی تبدیل شود. یکی از ایده‌ها Rate Encoding است؛ یعنی مقدار عددی به نرخ اسپایک ربط داده می‌شود.

اگر:

$$x = 0.8, \quad T = 4$$

باشد، انتظار داریم تقریباً سه اسپایک داشته باشیم، چون:

$$0.8 \times 4 \approx 3$$

پس یک نمایش ممکن چنین است:

$$[1, 1, 1, 0]$$

یا هر چینش دیگری که سه اسپایک در چهار گام زمانی داشته باشد.

۴.۲.۱۸ کدگذاری با ایده Integration

روش دیگر این است که عدد در هر گام زمانی به پتانسیل غشا وارد شود و بررسی کنیم چه زمانی از آستانه عبور می‌کند. فرض کنید:

$$x = 0.8, \quad \theta = 1$$

در گام اول:

$$0.8 < 1$$

پس اسپایک نداریم. در گام دوم:

$$0.8 + 0.8 = 1.6$$

از آستانه عبور می‌کنیم، پس اسپایک داریم. پس از اسپایک، مقدار آستانه از پتانسیل کم می‌شود و باقی‌مانده ۰/۶ می‌ماند.

در گام بعد:

$$0.6 + 0.8 = 1.4$$

دوباره اسپایک داریم و پس از کم کردن ۱، باقی مانده 0.4 می شود. در گام بعد:

$$0.4 + 0.8 = 1.2$$

باز هم اسپایک داریم. پس خروجی می تواند چنین باشد:

$$[0, 1, 1]$$

در این روش نیز نرخ اسپایک معنای اصلی را حفظ می کند: عدد بزرگ تر، اسپایک بیشتری تولید می کند. حداقل معیار برای کدگذاری معنادار این است که نرخ اسپایک با مقدار عدد پیوسته رابطه مستقیم داشته باشد.

۳.۱۸ Attention در شبکه اسپایکی

در Transformer معمولی، هدف attention این است که بخش های مرتبط با هم در یک context به هم نزدیک تر شوند. برای مثال، اگر جمله ای درباره «شیر» و «جنگل» داشته باشیم، attention باید کمک کند بازنمایی کلمه «شیر» بیشتر به معنای حیوانی آن نزدیک شود، نه معنای نوشیدنی. در تصویر نیز attention باعث می شود بخش های مهم تصویر، با توجه به زمینه، وزن بیشتری بگیرند.

در SNN نیز همین هدف را داریم، اما ورودی ها دنباله های اسپایکی اند. بنابراین انتظار داریم در خروجی attention، نرخ اسپایک برای بخش هایی که به هم مرتبط ترند بیشتر شود.

۱.۳.۱۸ Value، Key و Query اسپایکی

در Transformer معمولی، embedding ورودی به سه بردار تبدیل می شود:

$$Q, \quad K, \quad V$$

در SNN نیز همین ایده وجود دارد، اما ورودی یک دنباله اسپایکی است. برای این کار سه ماتریس وزن داریم:

$$W_Q, \quad W_K, \quad W_V$$

دنباله اسپایکی ورودی در این وزن ها ضرب می شود و سه دنباله جدید Q ، K و V ساخته می شود. این وزن ها در طول آموزش، با همان ایده surrogate gradient به روزرسانی می شوند.

۲.۳.۱۸ حذف Softmax

در Transformer معمولی، نمره توجه از ضرب زیر به دست می آید:

$$QK^T$$

سپس معمولاً نرمال سازی و softmax انجام می شود:

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

و نتیجه در V ضرب می شود. هدف softmax این است که نمره های خام به نوعی توزیع وزن تبدیل شوند؛ یعنی وزن ها نرمال شوند و مجموعشان ۱ شود.

در SNN، داده ها پیوسته نیستند، بلکه اسپایک های صفر و یک اند. وقتی اسپایک ها در هم ضرب می شوند، خود این ضرب تا حدی نقش شباهت را بازی می کند: اگر هر دو اسپایک فعال باشند، خروجی ۱ می شود و اگر یکی یا هر دو فعال نباشند، خروجی ۰ می شود. بنابراین softmax همان معنای قبلی را ندارد، چون داده از فضای پیوسته وارد فضای زمانی اسپایک ها شده است.

در Spiking Self-Attention می توان ساختاری شبیه زیر داشت:

$$QK^TV$$

بدون اینکه الزاماً softmax اعمال شود. البته مقیاس گذاری مشابه Transformer معمولی، مانند تقسیم بر $\sqrt{d}$ ، می تواند همچنان در معماری وجود داشته باشد.

۳.۳.۱۸ تغییر ترتیب ضرب ها و کاهش پیچیدگی

وقتی softmax حذف شود، می توان ترتیب ضرب ها را تغییر داد. در Transformer معمولی معمولاً ابتدا QK^T محاسبه می شود. این کار ماتریسی با ابعاد وابسته به تعداد توکن ها می سازد و پیچیدگی بالایی دارد.

اما اگر softmax وجود نداشته باشد، می توان ابتدا K^TV را محاسبه کرد و سپس نتیجه را در Q ضرب کرد:

$$Q(K^TV)$$

این تغییر می تواند پیچیدگی محاسباتی را کاهش دهد. در مقاله های SpikeFormer نیز یکی از ادعاهای مهم همین کاهش مصرف انرژی و هزینه محاسباتی است. چون مدل اسپایکی معمولاً از نظر دقت به Transformer معمولی نمی رسد، باید مزیت خود را در انرژی و پیچیدگی نشان دهد.

۴.۳.۱۸ تفسیر Attention اسپایکی

در attention اسپایکی، وقتی توکنی مانند «شیر» به توکن دیگری مانند «جنگل» توجه می کند، معنایش این است که اسپایک های مربوط به «جنگل» اجازه پیدا می کنند روی پتانسیل یا بازنمایی مربوط به «شیر» اثر بگذارند. اگر نرخ اسپایک میان این دو بالا باشد، یعنی ارتباط بیشتری بین آن ها وجود دارد.

در نهایت، اگر وظیفه مثلاً next-token generation باشد، انتظار داریم logit مربوط به معنای درست افزایش پیدا کند. در زمینه «جنگل»، logit مربوط به معنای حیوانی «شیر» باید از معنای نوشیدنی بیشتر شود.

۴.۱۸ معماری کلی SpikeFormer

فرآیند کلی SpikeFormer چنین است: داده ورودی کدگذاری می شود، به دنباله اسپایکی تبدیل می شود، وارد Spiking Self-Attention یا SSA می شود، خروجی توجه وارد بلوک MLP می شود، و در اطراف بلوک های توجه و MLP اتصال های residual قرار می گیرند.

در بخش توجه، ورودی اسپایکی از linear layer عبور می کند و معمولاً پس از آن یک نورون LIF قرار می گیرد. خروجی LIF دوباره یک دنباله اسپایکی است. برای ساختن Q ، K و V ، سه وزن جداگانه داریم:

$$W_Q, \quad W_K, \quad W_V$$

و سپس دنباله های Q ، K و V با ضرب های ماتریسی یا ضرب داخلی ترکیب می شوند.

۵.۱۸ نقش LIF

در SNN، پس از بسیاری از linear layer ها یک نورون LIF یا Leaky Integrate-and-Fire قرار می گیرد. نقش LIF این است که پتانسیل غشا را در طول زمان جمع کند، نشی داشته باشد، اگر پتانسیل از آستانه عبور کرد اسپایک تولید کند، پس از اسپایک ریست شود، مدتی در دوره refractory بماند، و سپس دوباره شروع به جمع کردن ورودی کند. بنابراین خروجی LIF دوباره یک دنباله اسپایکی است.

۶.۱۸ MLP در Transformer اسپایکی

در Transformer معمولی، MLP معمولاً شامل دو linear layer و یک تابع فعال سازی میان آن هاست:

$$\text{Linear} \rightarrow \text{Activation} \rightarrow \text{Linear}$$

تابع فعال سازی می تواند ReLU، GELU، sigmoid یا تابعی مشابه باشد. نقش آن ایجاد غیرخطی بودن میان دو لایه خطی است تا مدل بتواند بازنمایی عمیق تر و انتزاعی تری از هر توکن بسازد.

در SNN نمی توان همان تابع های فعال سازی را به همان شکل روی اسپایک های صفر و یک استفاده کرد. در معماری اسپایکی، خود LIF نقش غیرخطی سازی را بازی می کند:

$$\text{Linear} \rightarrow \text{LIF} \rightarrow \text{Linear} \rightarrow \text{LIF}$$

پس LIF معادل دقیق activation function معمولی نیست، اما نقش اصلی آن، یعنی ایجاد non-linearity و تبدیل خروجی به دنباله اسپایکی، را انجام می دهد.

۷.۱۸ Residual Connection

در SpikeFormer نیز مانند Transformer معمولی، اطراف بلوک‌های مهم اتصال residual داریم؛ هم اطراف بلوک attention و هم اطراف بلوک MLP. دلیل استفاده از residual connection کمک به کاهش مشکل vanishing gradient، حفظ اطلاعات قبلی، و جلوگیری از نابودی بازنمایی پیشین است.

اگر یک بلوک چیز تازه‌ای یاد نگیرد، دست‌کم اطلاعات قبلی همچنان از مسیر residual باقی می‌ماند. استاد این اتصال را مانند یک پشتوانه توضیح داد: اگر مدل در یک بلوک موفق به ساخت بازنمایی بهتر نشود، بازنمایی قبلی کاملاً از دست نمی‌رود.

۸.۱۸ جمع‌بندی SpikeFormer

در این جلسه، سه بخش اصلی Transformer اسپایکی بررسی شد. نخست، کدگذاری: داده پیوسته باید به دنباله اسپایکی تبدیل شود و نرخ اسپایک با مقدار اولیه داده رابطه مستقیم داشته باشد. روش‌هایی مانند Rate Encoding و روش‌های مبتنی بر integration برای این هدف استفاده می‌شوند.

دوم، توجه: در attention اسپایکی همچنان ایده Q ، K و V وجود دارد، اما به دلیل صفر و یکی بودن اسپایک‌ها وزمانی بودن داده، softmax ضرورت قبلی را ندارد. حذف softmax معماری را ساده‌تر می‌کند، پیچیدگی محاسباتی را کاهش می‌دهد، اجازه تغییر ترتیب ضرب‌ها را می‌دهد و می‌تواند مصرف انرژی را کم کند.

سوم، MLP: در MLP معمولی از تابع‌هایی مانند GELU یا ReLU استفاده می‌شود. در SNN، LIF این نقش را بر عهده می‌گیرد، چون رفتار غیرخطی ایجاد می‌کند و خروجی را دوباره به دنباله‌ای اسپایکی تبدیل می‌کند.

فصل ۱۹

SpikeBERT و تقطیر دانش

جلسه قبل نشان داد که پس از Deep SNN می‌توان ایده Transformer را نیز به فضای اسپایکی برد. SpikeFormer نمونه‌ای از همین مسیر بود و بیشتر در حوزه بینایی ماشین مطرح شد. در این جلسه پرسش اصلی این است که آیا می‌توان همین ایده را برای متن و پردازش زبان طبیعی نیز به کار برد یا نه.

این دسته از کارها، از جمله SpikeFormer و SpikeBERT، بیشتر نقش Proof of Concept دارند: نشان می‌دهند تبدیل معماری‌های ANN به SNN ممکن است، اما هنوز چالش‌های جدی باقی است. در SpikeFormer سه ایده اصلی دیده شد: در بخش Attention، ضرب ماتریسی و softmax تا حدی با عملیات ساده‌تر در فضای اسپایکی جایگزین می‌شود؛ در بخش MLP، نورون‌هایی مانند LIF نقش غیرخطی‌سازی را بر عهده می‌گیرند؛ و برای آموزش، از Surrogate Gradient استفاده می‌شود.

۱.۱۹ چرا متن برای مدل اسپایکی دشوار است؟

وقتی ایده SpikeFormer از تصویر به متن منتقل می‌شود، مدل به سادگی همگرا نمی‌شود. دلیل اصلی این مشکل تفاوت ماهوی متن و تصویر است. متن هم از نظر تراکم اطلاعات، هم از نظر وابستگی‌های بلندمدت، و هم از نظر نمادین بودن، برای یک مدل اسپایکی دشوارتر از تصویر است.

۱.۱.۱۹ تراکم اطلاعات در متن

در متن، تراکم اطلاعات بسیار بالاست. گاهی یک تغییر کوچک می‌تواند معنای کل جمله را عوض کند. برای نمونه:

- «من رفتم»

- «من نرفتم»

اضافه شدن یک نشانه نفی، معنای جمله را تقریباً برعکس می‌کند. در تصویر معمولاً چنین نیست؛ چند پیکسل کوچک، در بسیاری از موقعیت‌ها، معنای کل تصویر را به‌طور کامل عوض نمی‌کنند، چون تصویر افزونگی بیشتری دارد.

در SNN، نمایش‌های پیوسته به دنباله‌هایی از صفر و یک تبدیل می‌شوند. اگر این تبدیل ظرافت‌های معنایی متن را از بین ببرد، شبکه نمی‌تواند به خوبی یاد بگیرد. به همین دلیل، اسپایکی کردن متن نسبت به اسپایکی کردن تصویر حساس‌تر است.

۲.۱.۱۹ وابستگی‌های بلندمدت

در زبان، معنای یک عبارت ممکن است به جمله‌ها یا کلمه‌هایی وابسته باشد که خیلی زودتر آمده‌اند. گاهی برای فهم یک جمله باید کل context پاراگراف در نظر گرفته شود.

نورون LIF ذاتاً نشتی دارد. نشتی یعنی پتانسیل غشا به تدریج کم می‌شود و این رفتار نوعی فراموشی ایجاد می‌کند. بنابراین یک شبکه اسپایکی ممکن است در نگهداری وابستگی‌های بلندمدت متن دچار مشکل شود. در تصویر، وابستگی‌های فضایی معمولاً چنین شکل زبانی و بلندمدتی ندارند و همین مسئله انتقال مستقیم ایده SpikeFormer به متن را دشوار می‌کند.

۳.۱.۱۹ ماهیت نمادین و انتزاعی زبان

متن از نمادهای گسسته و انتزاعی ساخته شده است. همین که این نمادها به embedding پیوسته تبدیل شوند، خود یک مسئله اصلی در NLP است. اگر پس از آن بخواهیم همین embedding‌های پیوسته را نیز اسپایکی کنیم، مسیر یادگیری پیچیده‌تر می‌شود.

هدف از SpikeBERT این نیست که حتماً از همان ابتدا به دقت مدل‌های زبانی معمولی برسد. هدف این است که مدل اسپایکی بتواند از مزیت‌هایی مانند ماهیت زمانی، مصرف انرژی کمتر و سرعت بالقوه بیشتر استفاده کند، در حالی که هنوز از پس وظایف اصلی NLP بربیاید.

۲.۱۹ Knowledge Distillation

ایده اصلی SpikeBERT استفاده از Knowledge Distillation است. اگر Student فقط در انتهای شبکه و فقط بر اساس برچسب نهایی آموزش ببیند، روی متن به خوبی همگرا نمی‌شود. بنابراین باید در بخش‌های مختلف شبکه به آن کمک کرد.

در این روش، مدل معمولی BERT نقش Teacher را دارد و مدل اسپایکی SpikeBERT نقش Student را. Student نه تنها با برچسب نهایی، بلکه با خروجی‌های میانی Teacher نیز مقایسه می‌شود. به این ترتیب، یادگیری فقط به یک loss نهایی محدود نمی‌ماند.

در SpikeBERT چند نوع خطا در کنار هم استفاده می‌شود:

- Embedding Loss
- Hidden Feature Loss

• Logit Loss

• Cross Entropy Loss روی برچسب واقعی

پس loss نهایی ترکیبی از چند loss است و برای هر بخش می توان ضریب وزنی جداگانه در نظر گرفت.

۳.۱۹ Student و Teacher

Teacher همان BERT معمولی است؛ مدلی که از قبل روی متن های خام آموزش دیده و نمایش های پیوسته زبانی تولید می کند. این نمایش ها حامل رابطه های معنایی اند. برای مثال، در فضای برداری زبان، رابطه «تهران» با «ایران» می تواند شبیه رابطه «پاریس» با «فرانسه» باشد.

Student همان SpikeBERT است. ورودی اولیه آن می تواند embedding های BERT باشد، اما این embedding ها باید به فضای اسپایکی برده شوند.

اگر ورودی embedding را به شکل زیر بنویسیم:

$$X \in \mathbb{R}^{N \times D}$$

در این رابطه، N تعداد token ها و D تعداد ویژگی های هر token است. برای نمونه، نمایش یک token می تواند چنین باشد:

$$[0.3, 0.7, 0]$$

این نمایش پیوسته باید به دنباله ای از اسپایک ها تبدیل شود تا بتواند وارد مدل اسپایکی شود.

۴.۱۹ تبدیل Embedding پیوسته به اسپایک

برای اسپایکی کردن embedding، یک ابرپارامتر مهم وارد می شود:

$$T = \text{Time Step}$$

اگر $T = 5$ باشد، هر مقدار پیوسته در طول پنج گام زمانی بازنمایی می شود. برای مثال، اگر مقدار ورودی 0.3 باشد و آستانه نورون LIF برابر ۱ در نظر گرفته شود، پتانسیل می تواند به این شکل جمع شود:

• گام اول: 0.3

• گام دوم: 0.6

• گام سوم: 0.9

• گام چهارم: ۱/۲، تولید اسپایک

پس از تولید اسپایک، پتانسیل reset یا کاهش داده می‌شود. به این ترتیب، مقدار پیوسته ۳/۰ در طول زمان به یک الگوی اسپایکی تبدیل می‌شود.

در نتیجه، شکل نمایش از حالت زیر:

$$N \times D$$

به حالت زیر تبدیل می‌شود:

$$T \times N \times D$$

یعنی همان نمایش زبانی در طول زمان کشیده می‌شود.

۵.۱۹ بازگرداندن اسپایک‌ها به نمایش پیوسته

چون خروجی‌های Teacher پیوسته‌اند، برای مقایسه Student با Teacher باید خروجی‌های اسپایکی دوباره به یک نمایش پیوسته تبدیل شوند. یک راه ساده این است که تعداد اسپایک‌ها یا نرخ اسپایک در چند گام زمانی به یک مقدار عددی پیوسته نگاشت شود.

جریان کلی چنین است:

۱. گرفتن embedding اولیه از BERT

۲. تبدیل embedding پیوسته به اسپایک

۳. پردازش در SpikeBERT

۴. تبدیل خروجی اسپایکی به مقدار پیوسته

۵. مقایسه با خروجی Teacher

با این حال، پیوسته‌کردن خروجی کافی نیست. فضای هندسی Student نیز باید با فضای Teacher قابل مقایسه شود.

۶.۱۹ Projection و هم‌ترازسازی بازنمایی

پس از تبدیل خروجی اسپایکی به مقدار پیوسته، باید آن را به فضایی منتقل کنیم که با خروجی Teacher قابل مقایسه باشد. یک راه ساده استفاده از یک لایه خطی است:

$$H' = HW + b$$

در SpikeBERT به جای یک لایه خطی ساده، از MLP استفاده می شود تا projection قوی تری انجام شود:

$$H' = MLP(H)$$

پس از این تبدیل، خروجی Student و خروجی Teacher با هم مقایسه می شوند و خطا محاسبه می شود.

Feature Loss ۷.۱۹

برای محاسبه Feature Loss یا Embedding Loss، خروجی Teacher و Student را می توان خانه به خانه مقایسه کرد. اگر بخشی از خروجی ها چنین باشد:

$$\text{Teacher} = [۱, ۲, ۰]$$

$$\text{Student} = [۰.۵, ۰.۳, ۱]$$

یک شکل ساده خطا چنین است:

$$L = (۱ - ۰.۵)^2 + (۲ - ۰.۳)^2 + (۰ - ۱)^2$$

همین منطق می تواند برای embedding loss و feature loss در لایه های مختلف استفاده شود. البته لازم نیست همه لایه ها با Teacher مقایسه شوند. انتخاب لایه هایی که در آنها هم ترازسازی انجام می شود، می تواند بخشی از طراحی مدل باشد.

۸.۱۹ دو مرحله آموزش در SpikeBERT

آموزش SpikeBERT را می توان در دو مرحله توضیح داد. مرحله اول انتقال دانش عمومی از Teacher به Student است و مرحله دوم آموزش مخصوص یک وظیفه مشخص.

۱.۸.۱۹ انتقال دانش عمومی

در مرحله اول، هدف این است که Student دانش عمومی Teacher را یاد بگیرد. این دانش شامل embedding ها و ویژگی های میانی شبکه است. در این بخش، embedding loss و hidden feature loss اهمیت بیشتری دارند، چون

مدل اسپایکی باید یاد بگیرد نمایش‌های داخلی خود را به نمایش‌های BERT معمولی نزدیک کند.

۲.۸.۱۹ آموزش مخصوص وظیفه

در مرحله دوم، مدل برای یک وظیفه مشخص، مانند Sentiment Analysis، تنظیم می‌شود. در اینجا خروجی‌های نهایی یا logit‌های Teacher و Student نیز با هم مقایسه می‌شوند. برای مثال، اگر Teacher برای دو کلاس خروجی زیر را بدهد:

$$[0.2, 0.1]$$

و Student چنین خروجی‌ای تولید کند:

$$[0.5, 0]$$

Logit Loss می‌تواند به شکل زیر محاسبه شود:

$$L_{\text{logit}} = (0.2 - 0.5)^2 + (0.1 - 0)^2$$

۹.۱۹ Cross Entropy Loss

در کنار مقایسه با Teacher، خروجی Student با برچسب واقعی یا Ground Truth نیز مقایسه می‌شود. اگر برچسب واقعی کلاس مثبت باشد، می‌توان آن را به شکل زیر نمایش داد:

$$[1, 0]$$

اگر خروجی Student مثلاً چنین باشد:

$$[0.6, 0.3]$$

برای خطای طبقه‌بندی از Cross Entropy Loss استفاده می‌شود. فرم کلی آن برای کلاس درست چنین است:

$$L_{\text{CE}} = -\log \left(\frac{e^{z_y}}{\sum_j e^{z_j}} \right)$$

در این رابطه، z_y logit مربوط به کلاس درست است.

در نهایت، همه خطاها با ضرایب مشخص جمع می‌شوند:

$$L_{\text{total}} = \lambda_1 L_{\text{embedding}} + \lambda_2 L_{\text{feature}} + \lambda_3 L_{\text{logit}} + \lambda_4 L_{\text{CE}}$$

سپس بر اساس این خطای نهایی، backpropagation انجام می‌شود. چون شبکه اسپایکی است، برای عبور گرادیان از اسپایک‌ها از Surrogate Gradient استفاده می‌شود. نکته مهم این است که حتی embedding اولیه SpikeBERT نیز در طول آموزش fine-tune می‌شود.

۱۰.۱۹ نقش Time Step

Time Step ابرپارامتری است که در شبکه‌های معمولی وجود ندارد، اما در شبکه‌های اسپایکی نقش اصلی دارد:

$$T = \text{Time Step}$$

پرسش این است که زیاد یا کم کردن T چه اثری روی دقت، همگرایی، زمان آموزش و مصرف حافظه دارد. در کلاس گفته شد که هرچه T بزرگ‌تر شود، همگرایی معمولاً دیرتر می‌شود، چون مدل باید گام‌های زمانی بیشتری را پردازش کند.

۱.۱۰.۱۹ رابطه Time Step و دقت

برای توضیح رابطه Time Step و دقت، استاد مثال تصمیم‌گیری انسانی را مطرح کرد. فرض کنید در خانه نشسته‌ایم و صدایی از پارکینگ می‌آید. بعد صدای آسانسور شنیده می‌شود، آسانسور به طبقه ما نزدیک می‌شود و سپس صدای قفل در می‌آید. در هر مرحله شواهد بیشتری جمع می‌شود و بهتر می‌توانیم حدس بزنیم چه کسی وارد خانه شده است.

این روند شبیه Evidence Accumulation در نورون‌های اسپایکی است: شواهد در طول زمان جمع می‌شوند تا از آستانه عبور کنند و تصمیم ساخته شود. بنابراین افزایش T می‌تواند تا حدی دقت را بیشتر کند، چون اطلاعات زمانی بیشتری در اختیار مدل قرار می‌گیرد.

۲.۱۰.۱۹ محدودیت‌های Time Step زیاد

افزایش Time Step همیشه مفید نیست. رابطه کلی T و دقت معمولاً ابتدا صعودی است، اما از یک نقطه به بعد به حالت اشباع می‌رسد. یعنی ابتدا با افزایش T ، دقت بهتر می‌شود؛ سپس مقدار بهبود کم می‌شود؛ ممکن است دقت ثابت بماند؛ و حتی ممکن است به دلیل توجه به نویز افت کند.

پس هدف این است که کمترین Time Step ای پیدا شود که بیشترین دقت عملی را می‌دهد. این نقطه همان دقت زمانی مناسب یا temporal resolution مناسب مدل است.

چند دلیل باعث می‌شود T خیلی بزرگ مشکل ساز شود:

- از یک حد به بعد، اطلاعات مفید تمام می‌شود و مدل ممکن است به نویز حساس شود.
- با افزایش T ، بار حافظه‌ای مدل بیشتر می‌شود. این نکته با بحث ظرفیت Working Memory در جلسات قبل نیز ارتباط دارد.
- هرچه T بزرگ‌تر باشد، آموزش و inference کندتر و سنگین‌تر می‌شوند.

خلاصه بحث چنین است:

$$T \uparrow \Rightarrow \text{Accuracy} \uparrow \quad \text{تا نقطه اشباع}$$

اما پس از آن:

$$T \uparrow \Rightarrow \text{هزینه بیشتر، نویز بیشتر، و بهبود کمتر یا حتی افت دقت}$$

۱۱.۱۹ بازگشت کوتاه به تمرین Spiking Attention

در ادامه جلسه، استاد به تمرین Attention اسپایکی برگشت. هدف تمرین این بود که اگر ورودی پیوسته با Rate Encoding به اسپایک تبدیل شود و سپس Attention محاسبه شود، همچنان بتوان تغییر معنادار در embedding را مشاهده کرد.

مثال مربوط به کلمه «شیر» بود که دو معنا دارد: شیر به معنای حیوان و شیر به معنای مایع. اگر این کلمه کنار «جنگل» بیاید، انتظار داریم نمایش آن به معنای حیوانی نزدیک‌تر شود.

برای ساده‌سازی، وزن‌های W_Q ، W_K و W_V همانی فرض شدند:

$$Q = K = V = X$$

مراحل تمرین چنین بود: ابتدا embedding پیوسته به اسپایک تبدیل می‌شود؛ سپس امتیازهای توجه محاسبه می‌شوند؛ خروجی دوباره پیوسته می‌شود؛ و در نهایت باید نشان داد که بعد حیوانی کلمه «شیر» افزایش و بعد مایع بودن آن کاهش می‌یابد.

اگر embedding کلمه «شیر» چنین باشد:

$$[0.5, 0.8]$$

با Rate Encoding و $T = 4$ ، مقدار 0.5 تقریباً دو اسپایک و مقدار 0.8 تقریباً سه اسپایک تولید می‌کند. در مثال کلاس، امتیاز توجه «شیر به شیر» برابر ۴ و امتیاز توجه «شیر به جنگل» برابر ۲ در نظر گرفته شد. پس:

$$4 + 2 = 6$$

$$\text{Attention}(\text{شیر به شیر}) = \frac{4}{6} = \frac{2}{3}$$

$$\text{Attention}(\text{شیر به جنگل}) = \frac{2}{6} = \frac{1}{3}$$

بنابراین نمایش جدید «شیر» از ترکیب valueهای «شیر» و «جنگل» ساخته می‌شود:

$$V'_{\text{شیر}} = \frac{2}{3}V_{\text{شیر}} + \frac{1}{3}V_{\text{جنگل}}$$

نتیجه مورد انتظار این است که زمینه «جنگل» بعد حیوانی «شیر» را تقویت کند و بعد مایع بودن آن کاهش یابد.

فصل ۲۰

SpikeGPT، RWKV و تولید متن اسپایکی

پس از SpikeFormer و SpikeBERT، آخرین معماری مهمی که در ادامه بحث شبکه‌های اسپایکی بررسی شد SpikeGPT است. ایده کلی این است که برای بیشتر معماری‌های رایج یادگیری عمیق، دست‌کم یک طرح اولیه برای پیاده‌سازی اسپایکی مطرح شده است. SpikeGPT اهمیت ویژه دارد، چون نشان می‌دهد چگونه می‌توان مدل‌های زبانی خودرگرسیو مانند GPT را به سمت معماری‌های اسپایکی برد.

۱.۲۰ تفاوت طبقه‌بندی و تولید

برای فهم SpikeGPT، ابتدا باید تفاوت دو نوع وظیفه اصلی را روشن کرد: classification و generation.

در classification، مدل باید کل ورودی را بفهمد و در نهایت یک برچسب به آن اختصاص دهد. برای نمونه، در BERT، چون مدل باید هم به کلمه‌های قبل و هم به کلمه‌های بعد توجه کند، معماری معمولاً bidirectional است. BERT یک معماری encoder-only است: کل دنباله را encode می‌کند و سپس از خروجی آن برای طبقه‌بندی استفاده می‌شود. به همین دلیل، در SpikeBERT نمی‌شد به سادگی از attention صرف‌نظر کرد، چون طبقه‌بندی به فهم کامل متن از دو جهت وابسته است.

در generation، هدف تولید token بعدی است. مدل در زمان تولید فقط به گذشته دسترسی دارد، نه آینده. اگر یک prompt داده شود، مدل کلمه‌ها را یکی‌یکی تولید می‌کند: کلمه‌های قبلی را می‌بیند، کلمه بعدی را تولید می‌کند، همان کلمه به context اضافه می‌شود، و سپس token بعدی تولید می‌شود.

پس GPT یک مدل علی و خودرگرسیو است. معمولاً با masked self-attention کار می‌کند؛ یعنی attention فقط اجازه دارد به گذشته نگاه کند.

در آموزش تولید متن، یک دنباله ورودی داریم:

$$x_1, x_2, x_3, \dots, x_s$$

برچسب مطلوب هر موقعیت، token بعدی است. اگر جمله نمونه «آسمان آبی است.» باشد، ورودی «آسمان» باید به برچسب «آبی» برسد، ورودی «آبی» باید به «است» برسد، و ورودی «است» باید نقطه را پیش‌بینی کند. در SpikeGPT

نیز خروجی مدل با برجسب‌های جابه‌جا شده مقایسه می‌شود و معمولاً از cross entropy loss استفاده می‌شود. برای آموزش بخش‌های اسپایکی نیز مانند قبل از surrogate gradient کمک گرفته می‌شود.

تفاوت استفاده از SpikeGPT در تولید و طبقه‌بندی نیز مهم است. در تولید، خروجی نهایی باید به vocabulary وصل شود تا مدل بتواند logit مربوط به token‌های مختلف را بسازد و token بعدی را انتخاب کند. اما در طبقه‌بندی، الزاماً نیازی به head تولید token نیست. ورودی از بدنه مدل عبور می‌کند، hidden state‌های میانی گرفته می‌شوند، مثلاً روی آن‌ها average pooling انجام می‌شود، و سپس کلاس نهایی پیش‌بینی می‌شود.

۲.۲۰ چرا تبدیل مستقیم Transformer کافی نیست؟

فرض پیش‌فرض در کلاس این بود که اگر خود Transformer را مستقیم برداریم و فقط آن را اسپایکی کنیم، معمولاً نتیجه خوبی نمی‌گیریم. در SpikeBERT هم دیده شد که حتی برای BERT، که از نظر تولید متن ساده‌تر از GPT است، تبدیل مستقیم کافی نبود و به Teacher و Distillation نیاز داشت.

بنابراین برای GPT، که باید generation انجام دهد، مسئله سخت‌تر است. نمی‌توان فقط با کارهایی مانند گذاشتن encoder یا decoder اسپایکی، تبدیل نمایش پیوسته به اسپایک و اسپایک به نمایش پیوسته، یا حذف و تغییر تابع‌های فعال‌سازی انتظار داشت مدل به خوبی کار کند. SpikeGPT به همین دلیل مسیر دیگری را انتخاب می‌کند و به جای اتکا به attention معمولی، از ایده RWKV استفاده می‌کند.

۳.۲۰ شباهت RNN و GPT

GPT ذاتاً decoder-only است و فقط به گذشته نگاه می‌کند. این ویژگی آن را از یک جهت به RNN نزدیک می‌کند. RNN نیز ترتیبی است و اطلاعات گذشته را در یک state نگه می‌دارد.

در RNN، مدل در هر لحظه token فعلی و state قبلی را می‌گیرد، اطلاعات جدید را وارد state می‌کند، state را به‌روزرسانی می‌کند و خروجی جدید می‌دهد. به جای اینکه مانند attention همه token‌ها دوبه‌دو با هم مقایسه شوند، یک state وجود دارد که خلاصه‌ای از گذشته را نگه می‌دارد.

برای مثال، اگر ابتدا کلمه «شیر» دیده شود، state ممکن است هم معنای حیوانی و هم معنای مایع را در خود نگه دارد. اگر بعداً کلمه «جنگل» بیاید، state به‌روزرسانی می‌شود و معنای حیوانی تقویت می‌شود.

۴.۲۰ مشکل RNN‌های کلاسیک

مشکل اصلی RNN‌های کلاسیک vanishing gradient و فراموشی اطلاعات بلندمدت است. علت آن تا حدی به استفاده از تابع‌های غیرخطی مانند tanh روی state برمی‌گردد. وقتی دنباله طولانی می‌شود، گرادیان به تدریج کوچک می‌شود و مدل نمی‌تواند وابستگی‌های بلندمدت را خوب حفظ کند.

Attention این مشکل را به همان شکل ندارد، چون بخش عمده آن بر ضرب و جمع خطی متکی است. اما هزینه محاسباتی بالایی دارد. برای n token، ماتریس attention معمولاً با پیچیدگی حدودی $O(n^2)$ ساخته می‌شود. بنابراین برای مدل اسپایکی، بهتر است راهی پیدا شود که هم خاصیت ترتیبی و نگهداری گذشته را داشته باشد و هم هزینه attention کامل را تحمیل نکند.

۵.۲۰ معماری RWKV

RWKV ترکیبی از ایده‌های RNN و Transformer است. از یک طرف، مانند RNN یک state دارد؛ از طرف دیگر، تلاش می‌کند وابستگی‌های بلندمدت را بهتر از RNN کلاسیک حفظ کند. state بازگشتی آن خطی‌تر است و تابع غیرخطی سنگین مستقیماً روی state اعمال نمی‌شود. بنابراین مشکل vanishing gradient نسبت به RNN کلاسیک کمتر می‌شود.

در SpikeGPT، به جای استفاده مستقیم از attention، از RWKV استفاده می‌شود. هدف این است که معماری سبک‌تر و برای اسپایکی شدن مناسب‌تر باشد.

۱.۵.۲۰ Receptance

Receptance یا R نقش gate را دارد و مشخص می‌کند چه مقدار از اطلاعات از state عبور کند و وارد خروجی شود.

۲.۵.۲۰ Weight یا عامل فراموشی

Weight یا W در این بحث نقش عامل فراموشی را دارد. این مقدار تعیین می‌کند چه بخشی از اطلاعات قبلی در state حفظ شود و چه مقدار کاهش پیدا کند.

۳.۵.۲۰ Key

Key یا K مشابه key در attention است و اهمیت token فعلی را نشان می‌دهد.

۴.۵.۲۰ Value

Value یا V مقدار یا نمایش محتوایی مربوط به token فعلی است. در RWKV دیگر query نداریم، چون قرار نیست برای هر token همه token‌های دیگر را جداگانه query کنیم. به جای آن، یک state وجود دارد که گذشته را نگه می‌دارد.

۶.۲۰ ایده State در RWKV

در هر لحظه، مدل یک state دارد که خلاصه‌ای از چیزهایی است که تا آن زمان دیده است. برای token فعلی، K اهمیت token را مشخص می‌کند، V محتوای آن را نشان می‌دهد، W تعیین می‌کند از گذشته چقدر فراموش شود، و R تعیین می‌کند چه مقدار از state فعلی وارد خروجی شود.

استاد برای ساده‌سازی، یک مثال یک‌بعدی مطرح کرد. فرض کنیم برای token اول:

$$K_1 = 2$$

$$V_1 = 10$$

در نتیجه صورت اولیه چنین می‌شود:

$$K_1 V_1 = 2 \times 10 = 20$$

و مخرج برابر است با:

$$K_1 = 2$$

پس state یا مخزن اولیه:

$$\frac{20}{2} = 10$$

اکنون token دوم دیده می‌شود:

$$K_2 = 3, \quad V_2 = 4, \quad R_2 = 0.9$$

و عامل فراموشی:

$$W = 0.5$$

اطلاعات قبلی با عامل فراموشی کم می‌شود و اطلاعات جدید اضافه می‌شود. صورت جدید:

$$0.5 \times 20 + 3 \times 4 = 10 + 12 = 22$$

مخرج جدید:

$$0.5 \times 2 + 3 = 1 + 3 = 4$$

پس state جدید:

$$\frac{22}{4} = 5.5$$

خروجی با gate یا همان R ساخته می‌شود:

$$\text{Output} = R \times \text{State}$$

$$\text{Output} = 0.9 \times 5.5 = 4.95$$

این خروجی همان نمایش جدیدی است که به لایه بعدی داده می‌شود.

۷.۲۰ تفاوت Time Step در SNN و RWKV

در فرمول‌های RWKV، t همان گام زمانی دنباله token‌هاست؛ یعنی مثلاً token قبلی و token فعلی. این t نباید با بعد زمانی در شبکه‌های اسپایکی اشتباه گرفته شود.

در شبکه اسپایکی، خود embedding ممکن است در چند time step اسپایکی کدگذاری شود. اما در RWKV، منظور از t ، موقعیت token در sequence است. بنابراین SpikeGPT هم با زمان زبانی یا ترتیبی سروکار دارد و هم با زمان اسپایکی.

۸.۲۰ معماری کلی SpikeGPT

روند کلی در SpikeGPT چنین است:

۱. ابتدا embedding ورودی به حالت باینری یا اسپایکی تبدیل می‌شود.
 ۲. این تبدیل معمولاً با نورون‌هایی مانند LIF انجام می‌شود.
 ۳. ورودی اسپایکی وارد بلوک RWKV می‌شود.
 ۴. خروجی RWKV دوباره از یک لایه اسپایکی عبور می‌کند.
 ۵. سپس خروجی وارد بخش feed-forward اصلاح‌شده، یعنی R-FFN، می‌شود.
 ۶. خروجی R-FFN نیز دوباره اسپایکی می‌شود.
 ۷. بسته به وظیفه، خروجی برای تولید متن یا طبقه‌بندی استفاده می‌شود.
- بنابراین در ابتدا و انتهای هر بلوک تلاش می‌شود نمایش دوباره به حالت اسپایکی برگردد.

Time-Mix ۱.۸.۲۰

در Transformer کلاسیک، بخش attention مسئول فهم context است؛ یعنی مدل می‌فهمد token فعلی در کنار token‌های قبلی چه معنایی دارد. در SpikeGPT این نقش با RWKV جایگزین می‌شود. به بلوک RWKV معمولاً Time-Mix گفته می‌شود، چون اطلاعات زمانی و token‌های قبلی را با token فعلی ترکیب می‌کند.

Channel-Mix ۲.۸.۲۰

در Transformer کلاسیک، بخش feed-forward یا MLP روی خود token فعلی عمیق‌تر می‌شود و ویژگی‌های پیچیده‌تر استخراج می‌کند. در SpikeGPT این بخش با R-FFN جایگزین می‌شود. به این بخش Channel-Mix گفته می‌شود، چون تمرکز اصلی آن دیگر روی زمان نیست، بلکه روی ویژگی‌ها و کانال‌های خود token فعلی است.

R-FFN ۳.۸.۲۰

R-FFN از نظر نقش کلی شبیه feed-forward در Transformer است. در Transformer کلاسیک، معمولاً ابتدا بعد نمایش بزرگ‌تر می‌شود، سپس یک تابع فعال‌سازی مانند ReLU اعمال می‌شود، بعد بعد دوباره فشرده می‌شود و نمایش بهتر ساخته می‌شود.

در R-FFN نیز همین ایده وجود دارد. ورودی ابتدا expand می‌شود تا مدل بتواند الگوهای پیچیده‌تری استخراج کند، سپس از تابع فعال‌سازی عبور می‌کند و دوباره compress می‌شود. در این بخش سه ماتریس مطرح می‌شود: یک ماتریس برای gate یا G ، یک ماتریس برای مقدار اصلی یا P ، و یک ماتریس برای projection نهایی. بخشی از مسیر بزرگ‌تر می‌شود، از ReLU عبور می‌کند و دوباره فشرده می‌شود. سپس با مقدار اولیه به صورت ضرب عنصر به عنصر یا Hadamard product ترکیب می‌شود:

$$[a_1, a_2, a_3] \odot [b_1, b_2, b_3] = [a_1b_1, a_2b_2, a_3b_3]$$

هدف این است که به ویژگی‌های مختلف وزن داده شود.

۹.۲۰ پیوستگی محاسبات میانی و اسپایکی بودن مدل

یک پرسش مهم این است: اگر پس از اسپایک دوباره ضرب ماتریسی انجام می‌شود و خروجی پیوسته می‌شود، چه تفاوتی با یک شبکه کاملاً غیر اسپایکی دارد؟

پاسخ کلاس این بود که در شبکه‌های اسپایکی لازم نیست همه نقاط معماری همیشه صفر و یک باشند. در بسیاری از بخش‌ها مقدارهای پیوسته دیده می‌شوند، اما این پیوستگی با پیوستگی در شبکه‌های معمولی یکسان نیست.

نخست، خروجی لایه‌های اسپایکی بسیار sparse است. بسیاری از مقدارها صفرند و فقط وقتی اسپایک رخ می‌دهد، محاسبه فعال می‌شود. همین sparsity می‌تواند محاسبات و مصرف انرژی را کاهش دهد.

دوم، مقدارهای پیوسته‌ی میانی در اینجا بیشتر به پتانسیل غشایی یا membrane potential مربوط‌اند، نه به همان معنای معمول ویژگی‌های پیوسته در ANN. اطلاعات روی زمان کدگذاری شده است. بنابراین اگرچه از نظر ظاهری عدد پیوسته دیده می‌شود، ماهیت آن با مقدار پیوسته در شبکه غیر اسپایکی تفاوت دارد.

۱۰.۲۰ مصرف انرژی و محدودیت اجرای SNN روی GPU

SNN‌ها ذاتاً برای سخت‌افزارهای نورومورفیک طراحی شده‌اند، در حالی که ANN‌ها معمولاً روی GPU یا TPU اجرا می‌شوند. اگر SNN روی GPU اجرا شود، ممکن است بخشی از مزیت اصلی آن دیده نشود؛ چون GPU برای محاسبات متراکم و موازی طراحی شده است، نه برای محاسبات sparse و event-driven.

شبکه اسپایکی را می‌توان شبیه یک گراف sparse دید: گره‌های زیادی وجود دارد، اما فقط بعضی ارتباط‌ها در زمان‌های خاص فعال می‌شوند. شبکه غیر اسپایکی بیشتر شبیه یک گراف متراکم است که ارتباط‌های زیادی همیشه فعال‌اند. بنابراین اجرای SNN روی GPU الزاماً مزیت واقعی آن را نشان نمی‌دهد؛ مزیت اصلی زمانی روشن‌تر می‌شود که مدل روی سخت‌افزار نورومورفیک اجرا شود.

برای مقایسه منصفانه مصرف انرژی، معمولاً به جای اجرای مستقیم روی دو سخت‌افزار متفاوت، تعداد عملیات‌ها شمرده می‌شود. در شبکه غیر اسپایکی، عملیات اصلی معمولاً MAC است:

$$\text{MAC} = \text{Multiply} + \text{Accumulate}$$

اما در شبکه اسپایکی، چون ورودی‌ها صفر و یکی هستند، بسیاری از عملیات‌ها به جمع کاهش پیدا می‌کنند:

$$\text{AC} = \text{Accumulate}$$

انرژی MAC از AC بیشتر است. بنابراین اگر تعداد MAC‌ها در Vanilla GPT و تعداد AC‌ها در SpikeGPT شمرده شود، می‌توان مصرف انرژی تقریبی دو مدل را مقایسه کرد. طبق توضیح استاد، مقاله SpikeGPT کاهش انرژی حدوداً ۳۲ برابری را گزارش می‌کند.

جمع‌بندی این جلسه چنین است: GPT مدلی decoder-only و خودرگرسیو است؛ برای تولید متن فقط به گذشته نیاز دارد؛ masked self-attention کامل می‌تواند برای نسخه اسپایکی ناکارآمد باشد؛ تبدیل مستقیم Transformer به SNN معمولاً کافی نیست؛ و SpikeGPT به همین دلیل از RWKV استفاده می‌کند. در این معماری، Time-Mix نقش فهم زمینه را دارد، Channel-Mix یا R-FFN ویژگی‌های عمیق‌تر را از token فعلی استخراج می‌کند، و خروجی بلوک‌ها با LIF یا لایه‌های مشابه دوباره اسپایکی می‌شود.

بخش هفتم

کدگذاری عصبی، هم‌ترازی مغز و استدلال

فصل ۲۱

کدگذاری و رمزگشایی عصبی

۱.۲۱ Neural Decoding و Neural Encoding

پس از پایان بحث شبکه‌های اسپایکی، مسیر کلاس به سمت Neural Encoding و Neural Decoding رفت. هدف این بخش این است که روشن شود اطلاعات چگونه در مغز بازنمایی می‌شوند و چگونه می‌توان از روی فعالیت مغزی، رفتار، فرمان یا متن را بازسازی کرد.

در علوم اعصاب، Encoding به این معناست که یک محرک خارجی وارد سیستم عصبی می‌شود و به نوعی فعالیت یا فرمان در مغز تبدیل می‌گردد. صدا، تصویر، لمس یا هر محرک بیرونی دیگر می‌تواند چنین مسیری را آغاز کند. در مقابل، Decoding یعنی از روی فعالیت‌های مغزی بتوانیم یک خروجی خارجی را بازسازی کنیم؛ برای مثال یک فرمان حرکتی، یک رفتار، یک متن یا حتی نیت فرد را حدس بزنیم. پس مسیر کلی را می‌توان چنین خلاصه کرد:

فرمان یا رفتار → فعالیت مغزی → محرک خارجی

در Encoding از محرک به فعالیت مغزی می‌رسیم، و در Decoding مسیر را برعکس طی می‌کنیم: از فعالیت مغزی تلاش می‌کنیم محرک، نیت یا فرمان را بازسازی کنیم.

۲.۲۱ نظریه‌های اصلی Neural Coding

سه نظریه اصلی درباره کدگذاری عصبی در کلاس به ترتیب تاریخی بررسی شدند:

۱. Grandmother Cell Theory

۲. Population Coding

۳. Sparse Coding

هیچ کدام از این نظریه‌ها در کلاس به عنوان پاسخ مطلق معرفی نشدند. هر کدام بخشی از شهود لازم برای فهم مغز و مدل‌های محاسباتی امروزی را فراهم می‌کنند.

۱.۲.۲۱ Grandmother Cell

نظریه Grandmother Cell حدود دهه ۱۹۶۰ مطرح شد. طبق این دیدگاه، برای هر مفهوم، شیء یا موجودیت خارجی، یک سلول یا یک گروه بسیار محدود و اختصاصی از نورون‌ها وجود دارد. مثال کلاسیک این است که برای مفهوم «مادر بزرگ»، یک نورون یا گروهی خاص از نورون‌ها مسئول تشخیص همان مفهوم باشد.

نکته اصلی این نظریه انحصار است. منظور لزوماً این نیست که دقیقاً فقط یک نورون وجود دارد، بلکه ایده این است که مسئولیت شناخت یک مفهوم به شکل اختصاصی در اختیار یک سلول یا گروه کوچک قرار می‌گیرد.

این نظریه چند اشکال روشن دارد. نخست، اگر آن سلول یا گروه خاص آسیب ببیند، باید کل مفهوم از دست برود؛ در حالی که حافظه و شناخت معمولاً تا این اندازه به یک واحد منفرد وابسته نیستند. دوم، اگر برای هر چهره، صدا، شیء، مفهوم و حالت جهان به یک نورون یا گروه اختصاصی نیاز باشد، ظرفیت لازم بسیار بزرگ و از نظر زیستی و محاسباتی نامعقول می‌شود.

با این حال، ایده Grandmother Cell کاملاً بی‌فایده نیست. در برخی سطوح مدل‌های عصبی، به ویژه در لایه خروجی شبکه‌های طبقه‌بندی، شباهتی با این ایده دیده می‌شود. اگر یک شبکه سه کلاس داشته باشد و برای هر کلاس یک نود خروجی تعریف شود، هر نود تا حدی نقشی شبیه یک بازنمایی اختصاصی برای آن کلاس پیدا می‌کند. این تشبیه، نسخه زیستی و مطلق Grandmother Cell را اثبات نمی‌کند، اما نشان می‌دهد ایده «نماینده اختصاصی» در برخی معماری‌ها ظاهر می‌شود.

۲.۲.۲۱ Population Coding

Population Coding در برابر دیدگاه اختصاصی Grandmother Cell قرار می‌گیرد و حدوداً از دهه ۱۹۸۰ برجسته‌تر شد. در این نظریه، یک سلول یا گروه کوچک مسئول یک مفهوم نیستند؛ بلکه کل جمعیت نورون‌ها در تصمیم‌گیری نقش دارند.

این ایده را می‌توان به یک نظرسنجی جمعی تشبیه کرد. خروجی نهایی از مشارکت تعداد زیادی نورون ساخته می‌شود و هر نورون، بسته به فعالیت و وزن خود، سهمی در تصمیم دارد. در مدل‌های عصبی مصنوعی نیز شکل ساده این ایده را در جمع وزن‌دار می‌بینیم:

$$\sum_i W_i X_i$$

در اینجا X_i می‌تواند فعالیت یا اسپایک نورون باشد و W_i وزن سیناپسی یا وزن یادگرفته شده آن. همه نورون‌ها به یک اندازه تعیین‌کننده نیستند، اما تصمیم نهایی حاصل مشارکت جمعی آن‌هاست.

برای توضیح این نظریه، استاد به آزمایش‌هایی روی حرکت میمون‌ها اشاره کرد. در این نوع آزمایش‌ها، میمون برای حرکت یا پرتاب به جهت‌های مختلف مانند چپ، راست، بالا و پایین آموزش داده می‌شود و سپس فعالیت نورون‌های مغزی

هنگام اجرای حرکت بررسی می‌گردد. مشاهده مهم این است که هنگام حرکت به یک جهت خاص، فقط یک نورون یا یک ناحیه کاملاً اختصاصی فعال نمی‌شود؛ بلکه تعداد زیادی نورون با شدت‌های متفاوت درگیر می‌شوند. هنگام حرکت به چپ، نورون‌هایی که با جهت چپ سازگارترند فعالیت بیشتری دارند، اما نورون‌های دیگر هم می‌توانند با شدت کمتر فعال باشند.

مزیت نخست Population Coding پایداری است. اگر یک نورون آسیب ببیند، کل مفهوم یا فرمان از بین نمی‌رود، چون اطلاعات در یک جمعیت بزرگ‌تر توزیع شده است. مزیت دوم دقت بالاتر است؛ چون تصمیم جمعی می‌تواند از تصمیم یک واحد منفرد پایدارتر باشد، هرچند این گزاره مطلق نیست. مزیت سوم ظرفیت محاسباتی بیشتر است: به جای اینکه برای هر زاویه، رنگ یا شیء یک نورون اختصاصی داشته باشیم، ترکیب فعالیت نورون‌ها می‌تواند حالت‌های بسیار متنوعی را بسازد.

ضعف اصلی این نظریه مصرف انرژی و تفسیرپذیری پایین‌تر است. اگر جمعیت بزرگی از نورون‌ها در بسیاری از تصمیم‌ها مشارکت کنند، هزینه انرژی بالا می‌رود. از طرف دیگر، دیگر نمی‌توان گفت یک نورون مشخص مسئول یک خروجی مشخص است. به همین دلیل، Population Coding از نظر تفسیرپذیری به یک black box نزدیک می‌شود.

۳.۲.۲۱ Sparse Coding

Sparse Coding را می‌توان حالتی میانی میان دو نظریه قبلی دانست. این دیدگاه حدوداً از دهه ۱۹۹۰ پررنگ‌تر شد. در Sparse Coding، نه همه اطلاعات در اختیار یک سلول یا گروه بسیار کوچک است، و نه همه نورون‌ها در همه وظایف فعال می‌شوند. برای هر وظیفه، فقط زیرمجموعه‌ای نسبتاً کوچک و مرتبط از نورون‌ها فعال می‌شود.

در این حالت، ماتریس فعالیت نورونی تنک است. ممکن است برای یک کار خاص فقط ۲۰ درصد نورون‌ها، یا حتی ۵ درصد آن‌ها، فعال باشند و بقیه خاموش بمانند. این روش هم مصرف انرژی را کاهش می‌دهد و هم تفسیرپذیری را نسبت به Population Coding بهتر می‌کند.

استاد Sparse Coding را با مفهوم سلسله‌مراتب نیز مرتبط کرد. در پردازش تصویر، لایه‌های ابتدایی ممکن است لایه‌ها را تشخیص دهند، لایه‌های میانی رنگ، شکل یا ترکیب‌های پیچیده‌تر را بازنمایی کنند، و لایه‌های انتهایی به مفهوم یا شیء نهایی نزدیک شوند. در چنین معماری‌ای، گروه‌های مختلف نورونی در سطوح‌های متفاوتی از انتزاع فعال می‌شوند.

مزیت Sparse Coding این است که از یک طرف مانند Population Coding به یک نورون منفرد وابسته نیست و از طرف دیگر مانند آن هم مصرف و کم‌تفسیر نیست. به همین دلیل، می‌توان آن را ترکیبی از پایداری نسبی و تفسیرپذیری نسبی دانست.

۳.۲۱ ترکیب نظریه‌های کدگذاری در مدل‌های امروزی

در مدل‌های محاسباتی امروز، بهتر است این سه نظریه را رقیب‌های مطلق ندانیم. در بسیاری از معماری‌ها می‌توان هر سه سطح را هم‌زمان دید:

- Population Coding → لایه‌های ابتدایی
 Sparse Coding → لایه‌های میانی
 Grandmother-like Representation → لایه‌های انتهایی

در لایه‌های ابتدایی، اطلاعات خام‌تر و کم‌انتزاع‌تر است. در پردازش صدا، این بخش می‌تواند به ویژگی‌هایی مانند واج، فرکانس یا الگوهای آوایی مربوط باشد. در بینایی نیز نواحی اولیه مانند V1 تا V4 ویژگی‌هایی مثل لبه، جهت، رنگ و فرم‌های ابتدایی را پردازش می‌کنند. این نمایش‌ها قدرتمندند، اما تفسیر هر بعد آن‌ها دشوار است؛ شبیه embedding‌های مدل‌های یادگیری عمیق.

در لایه‌های میانی، ویژگی‌ها مشخص‌تر می‌شوند. ابتدا لبه‌ها و ویژگی‌های ساده‌تر دیده می‌شوند، سپس شکل‌ها و ترکیب‌ها و در ادامه ویژگی‌هایی که به اشیاء یا مفاهیم نزدیک‌ترند. این سطح به Sparse Coding نزدیک‌تر است، چون همه واحدها برای همه ورودی‌ها فعال نیستند.

در لایه‌های انتهایی، خروجی معمولاً به کلاس‌ها یا مفاهیم مشخص نگاشت می‌شود. اگر مدل چند کلاس داشته باشد، هر نود خروجی تا حدی مسئول یک کلاس است. این وضعیت شبیه یک بازنمایی Grandmother-like است، با این تأکید که این شباهت به معنای پیاده‌سازی دقیق نظریه زیستی Grandmother Cell نیست.

۴.۲۱ نسبت Symbolic AI با نظریه‌های کدگذاری

در کلاس این پرسش مطرح شد که اگر بخواهیم Symbolic AI را با نظریه‌های کدگذاری عصبی مقایسه کنیم، به کدام نظریه نزدیک‌تر است. پاسخ اولیه یکی از دانشجویان Grandmother Cell بود، اما توضیح استاد این بود که Symbolic AI بیشتر به Sparse Coding نزدیک است.

دلیل این مقایسه آن است که در Symbolic AI معمولاً با سیستم‌های قانون‌محور، ویژگی‌های استخراج‌شده و ساختارهای نمادین سروکار داریم. تصمیم‌نهایی نه کاملاً توزیع‌شده و همه‌گیر مانند Population Coding است، و نه کاملاً انحصاری مانند Grandmother Cell. مجموعه‌ای از قواعد یا ویژگی‌های مشخص در تصمیم‌دخالت می‌کنند؛ بنابراین شباهت آن با Sparse Coding بیشتر است.

۵.۲۱ Brain-to-Text

بخش بعدی جلسه به مقاله‌ای از Meta در سال ۲۰۲۵ درباره Brain-to-QWERTY یا Brain-to-Text اختصاص داشت. در چارچوب کلی Brain-to-Text هدف این است که از فعالیت مغزی به متن برسیم، اما در Brain-to-QWERTY دامنه محدودتر می‌شود: سیستم تلاش می‌کند بفهمد فرد هنگام تایپ چه کلیدهایی را روی صفحه‌کلید QWERTY فشار داده یا قصد فشار دادن آن‌ها را داشته است.

پس هدف این مقاله خواندن تمام افکار فرد نیست. دامنه مسئله به سناریوی تایپ محدود شده است، چون هرچه دامنه محدودتر باشد، decoding فعالیت مغزی عملی‌تر می‌شود.

۱.۵.۲۱ فاز آموزش

در فاز آموزش، فرد در دستگاه قرار می‌گیرد و فعالیت مغزی او هنگام خواندن و تایپ ثبت می‌شود. در توضیح کلاس، روش‌هایی مانند EEG و MEG یا سیگنال‌های مشابه به‌عنوان مسیرهای ممکن ثبت فعالیت مغزی مطرح شدند. روند کلی چنین است:

۱. متنی به فرد نشان داده می‌شود.
۲. فرد ابتدا متن را می‌خواند.
۳. سپس از او خواسته می‌شود متن را با صفحه‌کلید QWERTY تایپ کند.
۴. هنگام تایپ، سیگنال‌های مغزی ثبت می‌شوند.
۵. سیستم می‌داند متن ورودی چه بوده و فرد قرار بوده چه چیزی تایپ کند.
۶. مدل یاد می‌گیرد فعالیت مغزی را به کلیدها یا متن تایپ‌شده نگاشت کند.

در این مرحله، خروجی مدل با متن اصلی یا کلیدهای مورد انتظار مقایسه می‌شود و loss محاسبه می‌گردد. سپس وزن‌های بخش‌های مختلف مدل بر اساس همین خطا به‌روزرسانی می‌شوند. این فرایند معمولاً برای همان فرد تکرار می‌شود تا مدل روی سیگنال‌های مغزی او کالیبره شود. معماری کلی که در کلاس توصیف شد چنین بود:

CNN → سیگنال مغزی هنگام تایپ

- ویژگی‌های میانی
- Transformer Decoder-only
- Language Model
- متن بازسازی‌شده

۲.۵.۲۱ نقش CNN

سیگنال مغزی خام ابتدا به یک Convolutional Neural Network داده می‌شود. نقش CNN استخراج ویژگی از سیگنال خام است. یعنی داده اولیه مغزی به یک representation یا مجموعه‌ای از ویژگی‌های میانی تبدیل می‌شود تا بخش‌های بعدی مدل بتوانند با آن کار کنند.

۳.۵.۲۱ نقش Transformer

ویژگی‌های استخراج‌شده به یک Transformer داده می‌شوند. این Transformer در توضیح کلاس از نوع decoder-only معرفی شد. وظیفه آن مدل کردن توالی زبانی یا توالی کلیدهای تایپ‌شده از روی ویژگی‌های مغزی است؛ برای

مثال تشخیص دهد فرد در حال تایپ چه حرف‌ها یا چه کلمه‌هایی بوده است.

۴.۵.۲۱ نقش Language Model

Language Model در انتهای مسیر می‌تواند خطاهای خروجی را اصلاح کند. اگر بخش‌های قبلی مدل خروجی‌ای شبیه «سلام» تولید کنند، مدل زبانی با توجه به زمینه می‌تواند آن را به «سلام» نزدیک کند. بنابراین نقش آن شبیه یک تصحیح‌گر کانتکتستی یا autocorrect است.

۵.۵.۲۱ فاز آزمون

در فاز آزمون، متن ورودی برای مدل معلوم نیست. فرد چیزی را تایپ می‌کند یا قصد تایپ آن را دارد و مدل فقط از روی فعالیت مغزی تلاش می‌کند بفهمد او چه نوشته یا قصد نوشتن چه چیزی داشته است. هدف نهایی این است که از سیگنال مغز بتوان به متن، فرمان یا نیت فرد رسید.

۶.۲۱ کاربرد Brain-to-Text و مقایسه با Neuralink

کاربرد مثبت چنین سیستم‌هایی کمک به افرادی است که ذهن فعال دارند اما توانایی حرکت یا ارتباط فیزیکی ندارند؛ برای مثال افرادی که نمی‌توانند تایپ کنند، دچار آسیب نخاعی اند یا محدودیت شدید حرکتی دارند. اگر فعالیت مغزی آن‌ها decode شود، نیت یا دستور ذهنی می‌تواند به متن یا فرمان خارجی تبدیل شود و اجرای فرمان به سیستم‌های دیگر سپرده شود.

در همین زمینه، استاد به Neuralink نیز اشاره کرد. Neuralink نمونه‌ای از ایمپلنت مغزی است که با جراحی در مغز قرار می‌گیرد و هدف آن تبدیل فعالیت مغزی، به ویژه نیت حرکتی، به فرمان خارجی است. برای مثال فرد به حرکت نشانگر ماوس فکر می‌کند و سیستم تلاش می‌کند همان نیت را به حرکت ماوس تبدیل کند.

تفاوت مهم این است که Brain-to-QWERTY دامنه محدودتری دارد و به تایپ روی صفحه‌کلید مربوط است، اما در Neuralink هدف می‌تواند مستقیم‌تر باشد: تبدیل فکر یا نیت حرکتی به حرکت ماوس، کلیک یا کنترل یک ابزار.

۷.۲۱ مقایسه روش‌های ثبت فعالیت مغزی

یکی از چالش‌های اصلی Brain-to-Text محدودیت سخت‌افزار و روش ثبت فعالیت مغزی است. در کلاس سه دسته روش مقایسه شد: EEG، fMRI و روش‌های تهاجمی مانند ایمپلنت‌های مغزی.

۱.۷.۲۱ fMRI

fMRI بر اساس تغییرات جریان خون کار می‌کند. وقتی بخشی از مغز فعال می‌شود، جریان خون در آن بخش تغییر می‌کند و fMRI این تغییر را ثبت می‌کند.

مزیت fMRI رزولوشن مکانی خوب آن است؛ یعنی می‌توان نسبتاً دقیق‌تر دید کدام ناحیه مغز فعال شده است. ضعف آن رزولوشن زمانی پایین است، چون تغییرات جریان خون با تأخیر رخ می‌دهد. استاد تأکید کرد که fMRI در مقیاس ثانیه معنا دارد، نه میلی ثانیه.

۲.۷.۲۱ EEG

EEG فعالیت الکتریکی مغز را از روی پوست سر ثبت می‌کند. مزیت آن رزولوشن زمانی خوب است، چون تغییرات سریع الکتریکی را می‌تواند در حد میلی ثانیه ثبت کند. ضعف آن رزولوشن مکانی پایین‌تر است؛ سیگنال باید از بافت‌ها، جمجمه و پوست سر عبور کند و بنابراین تشخیص دقیق محل تولید سیگنال دشوارتر می‌شود.

۳.۷.۲۱ ایمپلنت‌های مغزی

در روش‌های تهاجمی، حسگر یا تراشه با جراحی وارد مغز می‌شود. چون سنسور به نورون‌ها نزدیک‌تر است، هم رزولوشن زمانی خوب است و هم رزولوشن مکانی نسبت به روش‌های غیرتهاجمی بهتر می‌شود. اما مشکل اصلی، تهاجمی بودن این روش‌هاست: نیاز به جراحی، مسائل ایمنی، پایداری بلندمدت و ملاحظات اخلاقی. بنابراین مقایسه کلی چنین است:

- fMRI: غیرتهاجمی، رزولوشن مکانی خوب، رزولوشن زمانی پایین، مبتنی بر جریان خون.
- EEG: غیرتهاجمی، رزولوشن زمانی خوب، رزولوشن مکانی پایین‌تر، مبتنی بر فعالیت الکتریکی از روی پوست سر.
- ایمپلنت‌های مغزی: تهاجمی، با رزولوشن زمانی و مکانی بهتر. نمونه‌ای مانند Neuralink نیازمند جراحی است و چالش‌های پزشکی و اخلاقی دارد.

۸.۲۱ چالش‌های Brain-to-Text

سه چالش اصلی که در کلاس برجسته شد عبارت‌اند از plasticity مغز، تعمیم‌پذیری بین افراد و چندوجهی بودن فعالیت مغزی. این چالش‌ها باعث می‌شوند ساخت یک مدل عمومی و پایدار برای Brain-to-Text بسیار دشوارتر از یک مسئله ساده طبقه‌بندی باشد.

۱.۸.۲۱ Plasticity

مغز انسان ثابت نیست. هر فرد در طول زمان یاد می‌گیرد، فراموش می‌کند، خسته می‌شود و وضعیت ذهنی‌اش تغییر می‌کند. بنابراین مدلی که امروز روی سیگنال‌های مغزی یک فرد کالبره شده، الزاماً در روزهای بعد دقیقاً همان ورودی‌ها را نمی‌بیند.

استاد این مسئله را چنین توضیح داد: ما مدلی را روی مغز یک فرد آموزش می‌دهیم، اما خود آن مغز هم هم‌زمان در حال آموزش دیدن و تغییر کردن است. بنابراین مدل باید بتواند با plasticity مغز کنار بیاید.

۲.۸.۲۱ تعمیم‌پذیری بین افراد

نقشه مغزی هر فرد با فرد دیگر متفاوت است؛ استاد آن را به اثر انگشت تشبیه کرد. مدلی که روی یک فرد آموزش دیده الزاماً روی فرد دیگر خوب کار نمی‌کند. در پروژه‌هایی مانند Neuralink یا Brain-to-QWERTY معمولاً مدل برای یک فرد خاص آموزش و آزمون می‌شود.

چالش اصلی این است که آیا می‌توان یک foundation model برای مغز ساخت که روی افراد مختلف تعمیم پیدا کند یا نه. در مدل‌های زبانی تا حدی به چنین مدل‌هایی رسیده‌ایم، اما در مغز مسئله پیچیده‌تر است، چون باید فعالیت مغزی به زبان، حرکت، تصویر یا فرمان نگاشت شود.

۳.۸.۲۱ Multimodality

فعالیت مغزی ذاتاً چندوجهی است. حتی اگر آزمایش فقط روی متن تمرکز کند، مغز هم‌زمان از تصویر، صدا، حافظه، توجه، خستگی، احساسات و عوامل دیگر تأثیر می‌گیرد. به همین دلیل در آزمایش‌ها تلاش می‌شود دامنه محدود شود؛ برای مثال فقط متن نشان داده شود و فرد فقط تایپ کند.

اما هرچه سیستم به شرایط واقعی نزدیک‌تر شود، چندوجهی‌تر می‌شود و decoding دشوارتر خواهد بود.

۹.۲۱ مسیر آینده: سنسورهای پوشیدنی و سخت‌افزار Neuromorphic

مسیر ایده‌آل این حوزه حرکت به سمت سنسورهایی است که غیرتهاجمی، پوشیدنی، کم‌مصرف و دقیق باشند؛ یعنی هم رزولوشن زمانی خوب داشته باشند، هم رزولوشن مکانی مناسب، هم برای بدن قابل تحمل باشند و هم گرمای زیادی تولید نکنند. در کلاس، سنسورهای کوانتومی به‌عنوان نمونه‌ای از مسیرهای مورد توجه مطرح شدند.

مسئله انرژی در اینجا حیاتی است. اگر سخت‌افزاری نزدیک بدن یا داخل بدن قرار بگیرد، مصرف انرژی زیاد می‌تواند گرما تولید کند و برای بافت مغز خطرناک باشد. بنابراین سخت‌افزار باید هم دقیق و هم کم‌مصرف باشد.

یکی از ایده‌های مهم برای این هدف، سخت‌افزار neuromorphic است. این سخت‌افزارها از ساختار و عملکرد مغز الهام می‌گیرند و معمولاً event-driven هستند. در سیستم‌های time-based، محاسبه بر اساس کلاک انجام می‌شود؛

چه رویدادی رخ داده باشد و چه رخ نداده باشد. اما در سیستم‌های event-driven، فقط وقتی رویدادی اتفاق بیفتد پردازش انجام می‌شود. در شبکه‌های اسپایکی، این رویداد همان اسپایک است:

پردازش انجام می‌شود $\rightarrow$ وقتی spike رخ دهد

چون مغز نیز تا حد زیادی رویدادمحور عمل می‌کند، سخت‌افزارهای neuromorphic برای این حوزه گزینه‌ای طبیعی به نظر می‌رسند. این نکته به‌ویژه در Brain-to-Text و ایمپلنت‌های مغزی مهم است.

۱۰.۲۱ ترجیح مکانی، قانون Hebb و مقداردهی اولیه

در پایان بخش علمی، استاد به این نکته برگشت که در مدل‌های عصبی خروجی معمولاً از ترکیب وزن‌ها و فعالیت‌ها ساخته می‌شود:

$$\sum_i W_i X_i$$

اما این وزن‌ها فقط از یک عامل نمی‌آیند. سه عامل مهم در شکل‌گیری وزن نهایی مطرح شد: Hebbian Learning، ترجیح مکانی و مقداردهی اولیه.

طبق قانون Hebb، نورون‌هایی که با هم فعال می‌شوند، ارتباطشان تقویت می‌شود. این قانون در شکل‌گیری وزن‌های سیناپسی نقش دارد. علاوه بر آن، موقعیت مکانی یا ترجیح عملکردی نورون‌ها نیز مهم است. برای مثال، در آزمایش حرکت میمون، هنگام حرکت به سمت چپ، نورون‌هایی که از نظر مکانی یا عملکردی به جهت چپ نزدیک‌ترند بیشتر فعال می‌شوند.

عامل سوم مقداردهی اولیه است. افراد از ابتدا نیز تفاوت‌هایی در وزن‌ها یا تنظیمات اولیه مغزی دارند و همین تفاوت‌ها در پاسخ نهایی نقش پیدا می‌کند. بنابراین وزن نهایی را می‌توان حاصل ترکیب این عوامل دانست:

$$\text{مقداردهی اولیه} + \text{ترجیح مکانی} + \text{Hebbian Learning} = \text{وزن نهایی}$$

جمع‌بندی این جلسه این است که سه نظریه Grandmother Cell، Population Coding و Sparse Coding سه نگاه متفاوت به بازنمایی عصبی ارائه می‌کنند. مدل‌های امروزی را می‌توان ترکیبی از این سه سطح دید: embedding اولیه به Population Coding نزدیک‌اند، ویژگی‌ها و hidden state‌های میانی به Sparse Coding، و لایه خروجی به بازنمایی Grandmother-like. در ادامه، Brain-to-QWERTY به‌عنوان یک نمونه عملی از decoding مغزی بررسی شد و چالش‌های آن، از تغییرپذیری مغز تا سخت‌افزار و انرژی، روشن شد.

فصل ۲۲

Brain Score و Brain Alignment

۱.۲۲ مسئله بازنمایی

بحث Brain Alignment پس از مباحث Brain-to-QWERTY و Brain Decoding مطرح شد. در جلسه قبل دیدیم که می‌توان از روی سیگنال‌های مغزی، تا حدی قصد تایپ یا فرمان ذهنی فرد را بازسازی کرد. این کاربردها از یک طرف برای کمک به افراد دارای محدودیت حرکتی بسیار مهم‌اند، و از طرف دیگر نگرانی‌های جدی درباره ذهن‌خوانی، سوءاستفاده از داده‌های ذهنی و افشای ناخواسته افکار ایجاد می‌کنند.

برای ورود به Brain Alignment، استاد مسئله را از زاویه بازنمایی توضیح داد. در یک نگاه فلسفی، می‌توان میان ماده، صورت یا Form، و سطحی مجردتر تمایز گذاشت. آنچه در جهان مادی می‌بینیم، بازنمایی یا representation چیزی عمیق‌تر است.

در همین چارچوب، خروجی یک مدل زبانی، سیگنال fMRI هنگام فکر کردن، تصویر، متن، صدا یا رفتار انسان را می‌توان بازنمایی‌هایی متفاوت از یک معنا یا حقیقت واحد دانست. ممکن است چند بازنمایی متفاوت داشته باشیم، اما همه آن‌ها به یک مفهوم مشترک اشاره کنند. مسئله اصلی این است که آیا بازنمایی‌های مدل و بازنمایی‌های مغز انسان به یکدیگر نزدیک‌اند یا نه.

۲.۲۲ آیا مدل واقعاً می‌فهمد؟

استاد تأکید کرد که نمی‌توان به صورت صددرصدی و الگوریتمی ثابت کرد یک مدل «حقیقت» را فهمیده است یا نه. این مسئله تا حدی شبیه مسئله hallucination است: هیچ الگوریتم قطعی‌ای وجود ندارد که تضمین کند مدل هیچ‌وقت دچار hallucination نمی‌شود، چون درست یا غلط بودن پاسخ به زمینه وابسته است.

یک جمله ممکن است در زمینه‌ای تخیلی درست باشد، در زمینه‌ای پزشکی غلط باشد، یا از نگاه افراد مختلف تفسیرهای متفاوتی پیدا کند. بنابراین «فهمیدن» یا «نفهمیدن» مدل در معنای مطلق، مسئله‌ای کاملاً حل شده نیست.

با این حال، می‌توان بررسی کرد که مدل تا چه حد از پوسته ظاهری داده فاصله گرفته و به معنا نزدیک‌تر شده است. مثال کلاس، سیب بود. اگر مدل فقط یاد گرفته باشد سیب معمولاً گرد، قرمز یا زرد است، ممکن است با دیدن یک سیب

فاسد یا کپک‌زده دچار مشکل شود. اما انسان حتی اگر قبلاً سیب کپک‌زده ندیده باشد، مفهوم «فاسد شدن» را می‌فهمد و می‌تواند بگوید هنوز با یک سیب، اما یک سیب فاسد، روبه‌روست.

پس فهم انسانی صرفاً حفظ ظاهر نیست. مدل نیز اگر واقعاً به معنا نزدیک شده باشد، باید بتواند از ویژگی‌های سطحی عبور کند و ساختار انتزاعی‌تری از مفهوم را بازنمایی کند.

۳.۲۲ ایده اصلی Brain Alignment

هدف Brain Alignment این است که بررسی کنیم آیا بازنمایی‌های مدل با بازنمایی‌های مغز انسان هم‌راستا هستند یا نه. برای قضاوت درباره کیفیت یک مدل، فقط مقایسه خروجی نهایی کافی نیست. باید بینیم لایه‌های میانی مدل چه چیزی را encode کرده‌اند، مغز انسان هنگام همان task چه بازنمایی‌هایی دارد، و آیا این دو مجموعه بازنمایی قابل نگاشت به هم هستند یا نه.

اگر مدلی فقط خروجی درست بدهد، ممکن است شبیه طوطی الگوهای را حفظ کرده باشد. اما اگر بازنمایی‌های میانی‌اش با بازنمایی‌های انسانی هم‌راستا باشند، احتمال بیشتری دارد که مدل به سطحی عمیق‌تر از معنا نزدیک شده باشد.

استاد همچنین از روایت‌های تجربه‌های نزدیک به مرگ برای توضیح دشواری نگاشت میان سطح‌های مختلف بازنمایی استفاده کرد. در چنین روایت‌هایی، فرد گاهی تجربه‌ای غیرمادی را با مفاهیم مادی توضیح می‌دهد؛ مثلاً می‌گوید «درخت‌هایی بودند که حرف می‌زدند». یعنی تجربه را با واژگان جهان مادی بیان می‌کند، اما همان واژه‌ها ویژگی‌هایی پیدا می‌کنند که در ماده عادی وجود ندارد. این مثال نشان می‌دهد نگاشت میان بازنمایی‌ها همیشه ساده و یک‌به‌یک نیست.

۴.۲۲ طراحی آزمایش هم‌ترازی

سؤال عملی این است: چگونه بفهمیم مدل از سطح ظاهر عبور کرده و به سطح معنا نزدیک شده است؟

مقایسه مدل با انسان یک مسیر طبیعی است، اما اگر انسان را ground truth بگیریم، باید مشخص کنیم چه چیزی را از انسان جمع‌آوری می‌کنیم. در بسیاری از آزمایش‌ها تا امروز، خروجی انسان و مدل مقایسه شده است: انسان task را انجام می‌دهد، مدل هم همان task را انجام می‌دهد، و خروجی‌ها مقایسه می‌شوند. اما برای Brain Alignment فقط خروجی کافی نیست؛ بازنمایی‌های میانی نیز باید بررسی شوند.

یک مسیر دیگر استفاده از بازنمایی چندوجهی است. اگر حقیقت واحدی مانند مفهوم سیب از مسیرهای مختلفی مثل تصویر، متن، صدا و رفتار انسانی بررسی شود، و مدل بتواند در همه این modality‌ها همان مفهوم را تشخیص دهد، احتمالاً بازنمایی عمیق‌تری ساخته است.

مسیر سوم انتخاب task‌هایی است که معنایی‌تر و دورتر از پوسته مادی‌اند؛ مثل task‌های فلسفی، استدلالی، علی و معلولی، فیزیکی، کشف قانون، استنتاج و قضاوت در وضعیت‌های جدید.

استاد مثال نیوتون و افتادن سیب را مطرح کرد: اگر مدلی را زیر درخت بگذاریم و قبلاً قانون جاذبه را به آن نگفته باشیم،

آیا می‌تواند از افتادن سیب تعجب کند و به وجود یک قانون برسد؟ در چنین taskهایی مطلوب است که مدل پاسخ را از قبل ندیده باشد، قانون به صورت صریح به آن داده نشده باشد، و مجبور شود رابطه علی و معلولی بسازد.

هدف نهایی Brain Alignment اثبات قطعی فهم مدل نیست. هدف عملی تر این است که بفهمیم مدل کجا را نمی‌فهمد تا بتوانیم آن را اصلاح کنیم. مسئله سخت «آیا مدل حقیقت را فهمیده است؟» به مسئله قابل بررسی تری کاهش داده می‌شود: دو جعبه سیاه داریم، مغز انسان و مدل هوش مصنوعی؛ خروجی هایشان تا حدی مشابه است. حالا می‌خواهیم ببینیم لایه‌های میانی‌شان نیز با هم هم‌راستا هستند یا نه.

۱.۴.۲۲ تعریف لایه‌های میانی مغز

در مدل عصبی مصنوعی، لایه‌ها و hidden stateها مشخص‌اند. اما در مغز باید تصمیم بگیریم که «لایه میانی» را چگونه تعریف کنیم. این پرسش به مسئله Brain Parcellation می‌رسد: مغز را چگونه به بخش‌ها یا parcelهای مختلف تقسیم کنیم تا بتوان بازنمایی‌های آن را با مدل مقایسه کرد؟

۲.۴.۲۲ انتخاب Task

باید مشخص شود چه taskهایی برای سنجش هم‌ترازی مناسب‌اند. taskهای زبانی، پیش‌بینی، استدلالی، معنایی و چندوجهی می‌توانند گزینه باشند. هرچه task فقط به خروجی سطحی وابسته نباشد و مدل را مجبور به ساختن بازنمایی معنایی کند، برای بررسی alignment مناسب‌تر است.

۳.۴.۲۲ تعریف Score

پس از جمع‌آوری داده از مغز و مدل، باید مشخص شود هم‌ترازی چگونه اندازه‌گیری می‌شود. معیار می‌تواند similarity، correlation، loss یا ترکیبی از چند معیار باشد. مهم این است که یک معیار عملی داشته باشیم که نشان دهد بازنمایی مدل تا چه حد می‌تواند داده مغزی یا رفتاری انسان را پیش‌بینی کند.

۵.۲۲ مقاله Brain Score

Brain Score به عنوان نمونه‌ای از این خط پژوهشی مطرح شد. تمرکز این مقاله بیشتر روی سؤال سوم است: چگونه برای alignment یک benchmark و score تعریف کنیم؟

ایده کلی آزمایش چنین است: انسان داخل دستگاه fMRI قرار می‌گیرد، یک محرک مثل متن به او داده می‌شود، همان متن به مدل نیز داده می‌شود، از مغز انسان سیگنال fMRI گرفته می‌شود، و از مدل بازنمایی‌های میانی یا hidden vectors استخراج می‌گردد. سپس یک مدل ساده یاد می‌گیرد بازنمایی مدل را به فعالیت مغزی نگاشت کند. در مرحله آزمون بررسی می‌شود این نگاشت تا چه حد می‌تواند فعالیت مغز را پیش‌بینی کند.

۱.۵.۲۲ اجزای آزمایش

در این آزمایش دو جعبه سیاه داریم: مغز انسان و مدل، مثلاً یک LLM. ورودی یا stimulus می‌تواند متن باشد. برای مدل، متن وارد شبکه می‌شود و از لایه‌های میانی بردارهایی مانند v_1 ، v_2 و v_3 گرفته می‌شود. مدل اصلی freeze می‌ماند، چون هدف ارزیابی آن است، نه آموزش دوباره آن. برای انسان، همان متن نمایش داده می‌شود و از مغز او fMRI گرفته می‌شود. ممکن است رفتارهایی مثل سرعت خواندن نیز ثبت شوند. سپس یک مدل کوچک و ساده آموزش داده می‌شود تا نداشت میان بازنمایی‌های مدل و سیگنال‌های مغزی یا رفتاری را یاد بگیرد.

۲.۵.۲۲ مدل Mapping

مدل اصلی، مثلاً LLM، دست‌کاری نمی‌شود. اما یک مدل کمکی آموزش داده می‌شود. وظیفه این مدل کمکی این است که اگر بردار میانی مدل را دریافت کند، پیش‌بینی کند کدام الگوی fMRI متناظر با آن است؛ یا اگر confidence مدل در next token prediction را دریافت کند، آن را به رفتار انسانی مثل سرعت خواندن نگاشت کند.

این مدل کمکی معمولاً می‌تواند خطی باشد. هدف ساختن یک مدل پیچیده جدید نیست؛ هدف این است که ببینیم آیا بازنمایی‌های موجود مدل با مغز قابل نگاشت هستند یا نه.

یکی از ایده‌های مهم، مقایسه رفتار انسان با confidence مدل است. اگر جمله برای انسان قابل پیش‌بینی و روان باشد، سریع‌تر خوانده می‌شود. اگر جمله غیرمنتظره باشد، انسان مکث می‌کند. برای مدل هم اگر next token قابل پیش‌بینی باشد، confidence بالاتر است؛ و اگر عبارت غیرمنتظره باشد، confidence پایین‌تر می‌آید. بنابراین می‌توان confidence مدل را با سرعت خواندن انسان مرتبط کرد.

نمونه کلاس این بود که «پردازش زبان طبیعی» عبارتی آشناست و ادامه آن آسان‌تر پیش‌بینی می‌شود، اما «پردازش زبان فیزیک» عجیب‌تر است و می‌تواند باعث مکث شود.

۳.۵.۲۲ مرحله آموزش و آزمون

در مرحله آموزش، متن‌هایی به انسان و مدل داده می‌شود. بازنمایی‌های مدل استخراج می‌گردد، fMRI و رفتار انسان ثبت می‌شود، و مدل mapping یاد می‌گیرد این‌ها را به هم وصل کند.

در مرحله آزمون، متن جدید به مدل داده می‌شود. مدل mapping از روی بازنمایی مدل پیش‌بینی می‌کند که fMRI یا رفتار انسانی چه خواهد بود. سپس این پیش‌بینی با داده واقعی انسان مقایسه می‌شود و از این مقایسه score نهایی ساخته می‌شود.

Brain Score می‌تواند از چند مؤلفه تشکیل شود؛ برای مثال هم‌ترازی با fMRI، هم‌ترازی با خروجی یا پیش‌بینی، و هم‌ترازی با رفتارهایی مثل reading time. در توضیح کلاس، معیار اصلی برای مقایسه می‌تواند correlation باشد.

۴.۵.۲۲ اهمیت Next Word Prediction

در Brain Score چند task بررسی شده‌اند؛ از جمله sentence entailment، coherence، question answering و similarity و next word prediction. نکته جالبی که استاد برجسته کرد این بود که همبستگی مثبت و معنادار بیشتر در task مربوط به next word prediction دیده شده است.

این نتیجه با مباحث قبلی ترم سازگار است: مغز انسان دائماً در حال پیش‌بینی است. بنابراین task‌هایی که حالت predictive دارند، احتمالاً برای Brain Alignment مناسب‌ترند. Task‌هایی که صرفاً به خروجی نهایی مربوط‌اند، ممکن است بیشتر لایه آخر را درگیر کنند؛ اما task‌های پیش‌بینی‌کننده بازنمایی‌های میانی را بهتر آشکار می‌کنند.

۶.۲۲ Brain Parcellation

Brain Parcellation پاسخی به این پرسش است که لایه‌ها یا بخش‌های میانی مغز را چگونه تعریف کنیم. در این روش، مغز یا یک ناحیه مورد علاقه از مغز به واحدهای کوچک‌تر و مجزا تقسیم می‌شود.

۱.۶.۲۲ Voxel

در fMRI، مغز به واحدهای کوچک مکعبی تقسیم می‌شود که به هرکدام voxel گفته می‌شود. Voxel‌ها مرزهای طبیعی و زیستی مغز نیستند؛ آن‌ها واحدهایی‌اند که ما برای اندازه‌گیری تعریف می‌کنیم.

۲.۶.۲۲ Parcel

چند voxel که در یک گروه قرار می‌گیرند، یک parcel می‌سازند:

یک atlas یا parcellation → چند parcel → یک parcel → چند voxel

۳.۶.۲۲ Region of Interest

گاهی لازم نیست کل مغز بررسی شود. بسته به task، فقط یک ناحیه خاص انتخاب می‌شود. برای مثال اگر task زبانی باشد، می‌توان فقط ناحیه‌های مرتبط با زبان را بررسی کرد. به چنین ناحیه‌ای Region of Interest یا ROI می‌گوییم.

۴.۶.۲۲ سیگنال BOLD

در fMRI معمولاً از شاخصی به نام BOLD استفاده می‌شود:

BOLD = Blood Oxygen Level Dependent

این شاخص تغییرات سطح اکسیژن خون را نشان می‌دهد. چون فعالیت عصبی با مصرف اکسیژن و جریان خون مرتبط است، تغییرات BOLD به عنوان نشانه‌ای از فعالیت مغزی استفاده می‌شود.

برای هر voxel، در طول زمان یک time series داریم. اگر یک parcel شامل چند voxel باشد، می‌توان برای آن parcel یک بردار میانگین یا بازنمایی کلی ساخت. برای مثال، اگر یک parcel سه voxel داشته باشد و در چهار زمان اندازه‌گیری انجام شود، داده اولیه یک ماتریس 4×3 است که می‌توان آن را به بردار میانگین parcel تبدیل کرد.

۷.۲۲ روش‌های Parcellation

دو رویکرد کلی برای parcellation مطرح شد: تقسیم‌بندی بر اساس آناتومی و تقسیم‌بندی بر اساس کارکرد.

۱.۷.۲۲ بر اساس آناتومی

در این روش، مغز بر اساس ساختار آناتومیک تقسیم می‌شود. در بینایی، نمونه‌هایی مانند V1، V2، V4 و IT مطرح‌اند. در اینجا تقسیم‌بندی بر اساس محل و ساختار فیزیکی مغز است.

۲.۷.۲۲ Functional Parcellation

در بحث Brain Alignment، functional parcellation معمولاً مهم‌تر است. در این روش، مهم نیست بخش‌ها از نظر فیزیکی دقیقاً کجا هستند؛ مهم این است که کدام بخش‌ها با هم کار می‌کنند. ممکن است دو نقطه از مغز از نظر مکانی دور باشند، اما چون هنگام یک کارکرد خاص با هم فعال می‌شوند، در یک شبکه قرار بگیرند.

استاد این وضعیت را به کارمندانی تشبیه کرد که در طبقه‌های مختلف یک سازمان هستند، اما برای یک کار مشخص مدام با هم ارتباط دارند. از روی ارتباط کاری می‌فهمیم در یک تیم‌اند، نه صرفاً از روی محل فیزیکی‌شان.

۳.۷.۲۲ Atlas و Localizer

برای انتخاب ناحیه‌های مغزی دو راه وجود دارد. راه اول استفاده از atlas‌های موجود است؛ یعنی parcellation‌هایی که قبلاً در مقاله‌های دیگر ساخته شده‌اند. پژوهشگر می‌گوید مبنای من فلان atlas معروف است.

راه دوم ساختن localizer مخصوص همان task است. برای مثال اگر task زبانی باشد، محرک زبانی به فرد داده می‌شود و دیده می‌شود کدام بخش‌های مغز فعال می‌شوند. سپس همان بخش‌ها بررسی می‌گردند. در این حالت پژوهشگر لزوماً از قبل نمی‌گوید ناحیه زبان دقیقاً کجاست؛ بلکه بر اساس داده همان task مشخص می‌کند کدام بخش‌ها فعال شده‌اند.

۸.۲۲ شبکه‌های هفت‌گانه Yeo

Yeo 7-Network Parcellation یکی از تقسیم‌بندی‌های معروف کارکردی است. در این تقسیم‌بندی، مغز به هفت شبکه کارکردی تقسیم می‌شود. این تقسیم‌بندی functional است؛ یعنی بر اساس کارکرد و هم‌فعال‌شدن نواحی مغز ساخته شده، نه صرفاً آناتومی.

۱.۸.۲۲ Visual Network

Visual Network مربوط به پردازش دیداری است. محل اصلی آن در بخش پس‌سری مغز است؛ یعنی تقریباً پشت سر، در راستای پردازش ورودی چشم‌ها.

۲.۸.۲۲ Somatomotor Network

Somatomotor Network به حرکت و حس بدنی مربوط است؛ از جمله حرکت دست و پا، هماهنگی عضلات، انقباض عضلات و کنترل حرکتی بدن.

۳.۸.۲۲ Dorsal Attention Network

Dorsal Attention Network به توجه درونی و ارادی مربوط است. وقتی خودمان تصمیم می‌گیریم روی چیزی تمرکز کنیم، مثلاً می‌خواهیم ساکت باشیم و متنی را بخوانیم، این شبکه درگیر می‌شود. این نوع توجه از درون هدایت می‌شود.

۴.۸.۲۲ Ventral Attention Network

Ventral Attention Network به توجه بیرونی و ناگهانی مربوط است. وقتی کسی در می‌زند، صدایی ناگهانی می‌شنویم، یا چیزی از بیرون توجه ما را می‌گیرد، این شبکه فعال می‌شود. این توجه از تصمیم درونی نمی‌آید، بلکه با یک محرک بیرونی تحریک می‌شود.

برای توضیح dorsal و ventral، استاد گفت اگر بدن یا مغز را در وضعیت جنینی تصور کنیم، dorsal به سمت پشت یا بالا و ventral به سمت شکم یا پایین اشاره دارد. در مغز نیز dorsal معمولاً به بخش‌های بالاتر و پشتی‌تر و ventral به بخش‌های پایین‌تر و جلویی‌تر اشاره می‌کند.

۵.۸.۲۲ Frontoparietal Network

Frontoparietal Network در بخش‌هایی از ناحیه پیشانی و آهیانه‌ای فعال است. کارکرد اصلی آن مدیریت و هماهنگی چند task، تصمیم‌گیری، مرتب‌سازی و کنترل رفتار است. نام frontoparietal به این معنا نیست که کل بخش پیشانی

و کل بخش آهیانه‌ای درگیر است؛ بلکه فقط بخش‌هایی از این نواحی که در این کارکرد فعال می‌شوند، در شبکه قرار می‌گیرند.

۶.۸.۲۲ Default Mode Network

Default Mode Network زمانی فعال‌تر می‌شود که task مشخصی انجام نمی‌دهیم؛ مثل استراحت، تفکر آزاد، خیال‌پردازی، رؤیای‌پردازی یا ایده‌پردازی آزاد. استاد توضیح داد گاهی وقتی از یک task خسته می‌شویم و آن را رها می‌کنیم، ناگهان ایده‌های بهتری به ذهنمان می‌آید. این می‌تواند با فعال‌شدن Default Mode Network مرتبط باشد، چون مغز خاموش نشده و وارد حالت پردازش آزادتر شده است.

۷.۸.۲۲ Limbic Network

Limbic Network با احساسات، هیجان‌ها و پردازش عاطفی مرتبط است.

۹.۲۲ فعالیت شبکه‌ها و Resting-State fMRI

استاد تأکید کرد که همه شبکه‌های مغز نمی‌توانند هم‌زمان با حداکثر شدت فعال باشند. مغز درون مجموعه‌ای محدود قرار دارد و جریان خون نیز محدود است. فعالیت مغزی شبیه پینگ‌پنگ میان شبکه‌هاست: بعضی شبکه‌ها فعال‌تر می‌شوند، بعضی پایین می‌آیند، و بسته به task الگوی فعالیت تغییر می‌کند. برای مثال نمی‌توان هم‌زمان هم درگیر تفکر آزاد بود و هم با تمرکز کامل یک task مشخص را انجام داد.

برای توضیح محدودیت فیزیکی مغز، استاد به جراحی مغز اشاره کرد. در برخی جراحی‌ها، بعد از باز کردن مجموعه، ممکن است نتوان بلافاصله استخوان را مثل قبل سر جای خود گذاشت، چون مغز دچار تورم می‌شود. این مثال نشان می‌دهد محدودیت فیزیکی مجموعه و جریان خون در فعالیت مغز نقش واقعی دارند.

در parcellation معروف Yeo از resting-state fMRI استفاده شده است. در resting-state fMRI فرد task خاصی انجام نمی‌دهد، اما این به معنای خاموش بودن مغز نیست. مغز حتی هنگام استراحت یا خواب فعال است، انرژی زیادی مصرف می‌کند، همچنان پیش‌بینی انجام می‌دهد و شبکه‌های آن کاملاً خاموش نمی‌شوند.

۱۰.۲۲ همبستگی در Parcellation

برای parcellation، time series سیگنال BOLD و کسل‌ها بررسی می‌شود. اگر دو voxel یا دو ناحیه مغزی در طول زمان با هم بالا و پایین شوند، یعنی همبستگی مثبت بالایی داشته باشند، احتمالاً در یک شبکه کارکردی قرار دارند. روند ساده چنین است:

۱. برای هر voxel یک time series گرفته می‌شود.
 ۲. همبستگی میان time series ها محاسبه می‌شود.
 ۳. voxel هایی که همبستگی بالایی دارند cluster می‌شوند.
 ۴. هر cluster به عنوان یک parcel یا network در نظر گرفته می‌شود.
- در اینجا clustering همان parcellation است.

۱۱.۲۲ Task-Negative و Task-Positive

استاد در پایان بخش parcellation شبکه‌ها را با دو مفهوم توضیح داد. Task-positive networks هنگام انجام یک task فعال‌تر می‌شوند. شبکه‌های Dorsal Attention، Ventral Attention و Frontoparietal معمولاً در این دسته‌اند.

در مقابل، task-negative networks شبکه‌هایی‌اند که هنگام انجام task کاهش فعالیت دارند و در حالت عدم انجام task فعال‌ترند. این دسته به‌ویژه با Default Mode Network مرتبط است.

۱۲.۲۲ بازگشت به Brain Alignment

پس از parcellation می‌توان بازنمایی‌های مغزی را با بازنمایی‌های مدل مقایسه کرد. روند کلی چنین است:

۱. مغز به parcel های کارکردی تقسیم می‌شود.
 ۲. برای هر parcel سیگنال BOLD یا بازنمایی زمانی استخراج می‌شود.
 ۳. از مدل، hidden state یا بردارهای لایه‌های میانی گرفته می‌شود.
 ۴. میان بازنمایی‌های مدل و مغز correlation یا similarity محاسبه می‌شود.
 ۵. هرچه این ارتباط بیشتر باشد، می‌توان گفت هم‌ترازی بیشتر است.
- با این حال، این همچنان اثبات فهم حقیقی نیست؛ فقط راهی عملی برای نزدیک شدن به مسئله است.

۱۳.۲۲ خط پژوهشی آینده: استنتاج و مغالطه

در پایان، استاد گفت اگر بخواهیم واقعاً بررسی کنیم مدل به معنا و فهم انسانی نزدیک شده یا نه، باید از task های سخت‌تر استفاده کنیم: طراحی task های استنتاجی، ساختن شرایطی که مدل در آن به اشتباه بیفتد، بررسی مغالطه‌ها، و سنجش reasoning انسانی و خطاهای انسانی در مدل.

مغالطه پدیده‌ای انسانی است. اگر مدلی بتواند مغالطه را تشخیص دهد یا حتی ساختار مغالطه را بازتولید کند، یعنی تا حدی ساختار استدلال را فهمیده است. استاد اشاره کرد این مسیر، یعنی چالش کردن مدل از نظر استنتاجی، ادامه بحث جلسه بعد خواهد بود.

فصل ۲۳

استدلال، قیاس و هم‌ترازی خطای انسان و مدل

۱.۲۳ چرا پاسخ درست کافی نیست؟

جلسه قبل نشان داد که در پس داده‌ها می‌توان یک حقیقت یا مفهوم پنهان را در نظر گرفت؛ چیزی که در کلاس با Z نشان داده شد. خود Z را مستقیماً در اختیار نداریم، اما بازنمایی‌هایی از آن داریم: بازنمایی در مغز انسان، بازنمایی در لایه‌های میانی مدل، و خروجی نهایی مدل.

در Brain Alignment هدف این است که بفهمیم آیا مدل واقعاً به آن مفهوم پنهان نزدیک شده است یا فقط از نظر آماری خروجی‌هایی شبیه داده‌های آموزشی تولید می‌کند. چون Z را مستقیم نداریم، مغز انسان به عنوان مرجع عملی استفاده می‌شود. Brain Score نیز در همین چارچوب می‌پرسد لایه‌های میانی مدل تا چه حد با لایه‌های میانی مغز انسان هم‌بستگی دارند.

اما نکته مهم این جلسه این بود که هم‌ترازی نباید فقط بر اساس پاسخ درست سنجیده شود. اگر انسان‌ها در یک موقعیت خاص خطا می‌کنند، مدلی که واقعاً شبیه انسان می‌فهمد باید در الگوی خطا نیز تا حدی شبیه انسان باشد. بنابراین سؤال فقط این نیست که مدل چند درصد پاسخ درست داده است؛ سؤال‌های مهم‌تر این‌اند:

- آیا مدل همان جاهایی اشتباه می‌کند که انسان‌ها اشتباه می‌کنند؟
- آیا خطاهای شایع انسانی در مدل نیز شایع‌اند؟
- آیا ترجیحات، زمان پاسخ، و توزیع خطاهای مدل به انسان شبیه است؟

پس هم‌ترازی می‌تواند چند سطح داشته باشد: هم‌بستگی لایه‌های میانی مدل با مغز، هم‌بستگی خروجی مدل با خروجی انسان، شباهت در زمان پاسخ، شباهت در ترجیحات، و شباهت در خطاها و بایاس‌های انسانی.

۲.۲۳ یادگیری از خطای انسان

در نگاه اول شاید عجیب باشد که بخواهیم مدل خطاهای انسان را نیز یاد بگیرد. در برخی کاربردها، مثل جراحی رباتیک، هدف دقیقاً این است که مدل خطای انسان را پوشش دهد و بهتر از انسان عمل کند. اما موضوع این جلسه ساختن چنین

سیستم‌هایی نبود؛ موضوع این بود که آیا مدل واقعاً مفهوم پشت داده را فهمیده است یا نه. اگر مغز انسان نزدیک‌ترین مرجع ما به فهم مفهوم باشد، خطاهای انسانی نیز بخشی از همان مرجع می‌شوند. از این زاویه، سوگیری شناختی الزاماً باگ نیست. بسیاری از این خطاها در طول تکامل برای بقا مفید بوده‌اند. برای مثال، انسانی که در موقعیت خطرناک بیش از حد صبر کند تا همه چیز را منطقی اثبات کند، ممکن است زنده نماند. در مقابل، تصمیم‌گیری سریع بر اساس پیش‌فرض‌های ذهنی می‌تواند شانس بقا را افزایش دهد. گاهی خطا داشتن یعنی ساده‌سازی و تصمیم‌گیری سریع‌تر.

نمونه دیگر مصرف انرژی مغز است. مغز نمی‌خواهد همه اطلاعات محیط را با جزئیات کامل پردازش کند؛ بنابراین فیلتر می‌گذارد و فقط بخشی از اطلاعات را برجسته می‌کند. وقتی استاد به کلاس نگاه می‌کند، به همه صندلی‌ها و جزئیات محیط توجه نمی‌کند، بلکه بیشتر روی چهره‌ها، واکنش‌ها و صداها تمرکز می‌کند. این فیلترکردن از یک نظر خطاست، اما برای عملکرد طبیعی شناخت لازم است.

حتی طیف بینایی انسان نیز همین منطق را نشان می‌دهد. انسان فقط نور مرئی را می‌بیند، نه کل طیف الکترومغناطیسی. اگر چشم انسان مادون قرمز را نیز می‌دید، شاید دائماً جریان گرما، تغییرات دمای بدن افراد و حرکت هوای گرم و سرد را می‌دید و توجهش مختل می‌شد. حذف برخی اطلاعات ضعف مطلق نیست؛ نوعی بهینه‌سازی است.

۳.۲۳ Pareidolia و تشخیص الگو

Pareidolia یعنی دیدن چهره یا الگوی آشنا در چیزهایی که واقعاً چنین الگوی معناداری ندارند؛ مثل ابرها، نقش مبل، گلدوزی یا اشیای بی‌جان. استاد مثال زد که کودکان خردسال گاهی روی نقش مبل یا اشیای دیگر چهره می‌بینند و آن را به انسان نسبت می‌دهند.

این رفتار الزاماً آموخته شده نیست، بلکه بخشی از سیستم ادراکی انسان است. از نظر منطقی، دیدن چهره در نقش بی‌جان خطاست؛ اما از نظر تکاملی می‌تواند مفید باشد، چون انسان باید چهره‌ها، نگاه‌ها و واکنش‌های دیگران را سریع تشخیص دهد. اگر کسی وارد کلاس شود و اصلاً به چهره افراد واکنش نشان ندهد، برای ما ناخوشایند و غیرطبیعی به نظر می‌رسد.

پس Pareidolia نمونه‌ای از خطای ادراکی است که در سطح بقا و تعامل اجتماعی می‌تواند معنا داشته باشد.

۴.۲۳ System 1، System 2 و تفکر انتقادی

در بحث استدلال دوباره به تفاوت System 1 و System 2 اشاره شد. System 1 سریع، کم‌هزینه و مبتنی بر prior knowledge یا دانش قبلی است. این سیستم بر اساس تجربه‌های پیشین و الگوهای آماری سریع تصمیم می‌گیرد.

System 2 کندتر، پرهزینه‌تر و تحلیلی‌تر است. وقتی فعال می‌شود، تلاش می‌کند مسئله را واقعاً بررسی و اثبات کند. تفکر انتقادی بیشتر به System 2 مربوط است؛ یعنی فرد در موقعیت‌های لازم می‌تواند از پاسخ سریع و پیش‌فرض ذهنی عبور کند و وارد تحلیل عمیق‌تر شود.

Reasoning ۵.۲۳ انواع

Reasoning یا استدلال انواع مختلفی دارد. در کلاس سه نوع اصلی نام برده شد:

- Deductive Reasoning
- Inductive Reasoning
- Abductive Reasoning

تمرکز این جلسه روی شکل خاصی از Deductive Reasoning بود: Syllogistic Reasoning. در فارسی می‌توان آن را قیاس یا «صغرا و کبرا چیدن» نامید.

Syllogism ۶.۲۳

Syllogism ساختاری صوری برای استدلال قیاسی است. در ساده‌ترین حالت، دو مقدمه و یک نتیجه داریم:

۱. Premise 1

۲. Premise 2

۳. Conclusion

۱.۶.۲۳ ساختار قیاس

در این ساختار یک Middle Term یا ترم میانی وجود دارد که در هر دو مقدمه مشترک است و اجازه می‌دهد میان دو گزاره ارتباط برقرار کنیم. اگر ترم مشترک وجود نداشته باشد، اصولاً قیاسی شکل نمی‌گیرد، چون چیزی وجود ندارد که دو مقدمه را به هم وصل کند.

Mode A ۲.۶.۲۳

Mode A رابطه کلی مثبت است:

All A are B

یعنی همه A ها B هستند.

Mode I ۳.۶.۲۳

Mode I رابطه جزئی مثبت است:

Some A are B

یعنی بعضی از A ها B هستند.

Mode E ۴.۶.۲۳

Mode E رابطه کلی منفی است:

No A are B

یعنی هیچ A ای B نیست.

Mode O ۵.۶.۲۳

Mode O رابطه جزئی منفی است:

Some A are not B

یعنی بعضی از A ها B نیستند.

Figure ۶.۶.۲۳

علاوه بر mode ها، در Syllogism چند figure مختلف نیز داریم. Figure یعنی جایگاه ترم‌ها، به‌ویژه ترم میانی، در دو مقدمه تغییر کند. با چهار mode و چهار figure، در مجموع $4 \times 4 \times 4 = 64$ فرم ممکن برای Syllogism ساخته می‌شود. بعضی از این فرم‌ها نتیجه معتبر دارند و بعضی ندارند.

Valid Conclusion ۷.۶.۲۳

یک نتیجه زمانی valid است که در همه حالت‌های ممکن درست باشد و هیچ مثال نقضی نداشته باشد. فرم معروف Barbara یا ۱ – (AAA) از ساده‌ترین فرم‌های معتبر است:

All A are B
 All B are C
 $\therefore$ All A are C

این نتیجه همیشه معتبر است. اما بسیاری از فرم‌ها نتیجه معتبر ندارند. ممکن است در بعضی حالت‌ها یک نتیجه درست از آب دربیاید، اما اگر همیشه درست نباشد، valid محسوب نمی‌شود. چنین مواردی می‌توانند منشأ مغالطه باشند.

۷.۲۳ استفاده از قیاس برای Brain Alignment

پرسش اصلی این است که ساختارهای قیاسی چه ربطی به Brain Alignment و مدل‌های زبانی دارند. می‌توان ۶۴ فرم Syllogism را به انسان‌ها داد و دید در کدام فرم‌ها درست پاسخ می‌دهند و در کدام فرم‌ها خطا می‌کنند. سپس همان فرم‌ها را به مدل زبانی داد و خروجی مدل را با انسان مقایسه کرد.

در اینجا دقت کلی کافی نیست. مهم‌تر این است که توزیع خطاهای مدل شبیه توزیع خطاهای انسان باشد. اگر انسان‌ها در یک فرم خاص زیاد اشتباه می‌کنند، ولی مدل همیشه درست جواب می‌دهد، این الزاماً نشانه هم‌ترازی نیست. ممکن است مدل فقط ساختارهای صوری را از داده آموزشی یاد گرفته باشد، بدون اینکه شبیه انسان استدلال کند.

۸.۲۳ نتایج انسانی در Syllogistic Reasoning

در مقاله‌ای که استاد نشان داد، ۶۴ فرم Syllogism به انسان‌ها داده شده بود. از میان این ۶۴ فرم، ۲۷ فرم valid conclusion داشتند و ۳۷ فرم no valid conclusion بودند.

برای هر فرم، توزیع پاسخ انسان‌ها بررسی شده بود. حتی در فرم‌های ساده‌ای مثل Barbara هم همه افراد پاسخ درست نمی‌دهند: تعداد زیادی درست جواب می‌دهند، اما همچنان بخشی از افراد پاسخ اشتباه تولید می‌کنند. این نشان می‌دهد که حتی در قیاس‌های صوری ساده نیز انسان‌ها همیشه کاملاً منطقی عمل نمی‌کنند.

در کلاس این پرسش مطرح شد که چرا با وجود شناخته‌شدن این فرم‌ها در طول قرن‌ها، انسان‌ها همچنان در بسیاری از آن‌ها اشتباه می‌کنند. چند پاسخ ممکن مطرح شد: انسان در زندگی روزمره به همه این فرم‌ها نیاز نداشته، برخی فرم‌ها کمتر در تجربه واقعی استفاده شده‌اند، بعضی فرم‌های غلط یا مغالطه‌آمیز ممکن است در تعاملات اجتماعی کاربرد داشته باشند، و مغز انسان برای حل همه فرم‌های منطقی بهینه نشده، بلکه برای تصمیم‌گیری سریع در جهان واقعی بهینه شده است.

بنابراین ممکن است انسان بتواند با تمرین همه این فرم‌ها را یاد بگیرد، اما مغز به‌صورت طبیعی خودش را درگیر تمام حالت‌های ممکن نکرده است.

در یک فرم سخت‌تر، دانشجویان چند پاسخ مختلف پیشنهاد کردند، اما پاسخ معتبر ابتدا به راحتی پیدا نشد. استاد با نمودار ون توضیح داد که برای valid بودن، نتیجه باید در همه حالت‌ها درست باشد. اگر فقط در بعضی حالت‌ها درست باشد، نتیجه معتبر نیست.

در Syllogism های صوری، معمولاً بحث مجموعه تهی وارد نمی‌شود، چون هدف بررسی قیاس رسمی است، نه همه پیچیدگی‌های زبان طبیعی یا هستی‌شناسی. اما وقتی این ساختارها به زبان طبیعی برده می‌شوند، مسائلی مانند تهی بودن یک مجموعه می‌توانند مهم شوند.

۹.۲۳ از فرم صوری به زبان طبیعی

در حالت صوری، جمله‌ها به شکل‌هایی مانند $All\ A\ are\ B$ و $Some\ B\ are\ C$ نوشته می‌شوند. اما برای مقایسه واقعی با انسان و مدل زبانی، بهتر است این‌ها به Natural Language تبدیل شوند؛ یعنی به جای A ، B و C ، جمله‌های طبیعی و معنادار داشته باشیم.

در این حالت کار سخت‌تر می‌شود، چون مدل باید ترم میانی را تشخیص دهد، رابطه منطقی میان دو مقدمه را پیدا کند، مغالطه‌های زبانی را تشخیص دهد، و نتیجه معتبر یا نبود نتیجه معتبر را تولید کند.

۱۰.۲۳ دیتاست Avicenna

استاد به دیتاستی به نام Avicenna اشاره کرد که در دوره دکتری روی آن کار شده بود. این دیتاست حدود ۶۰۰۰ نمونه Syllogism در زبان طبیعی دارد.

در هر نمونه معمولاً این اجزا وجود دارد:

- Premise 1
- Premise 2
- Syllogistic Relation
- Conclusion یا No Valid Conclusion

Syllogistic Relation مشخص می‌کند آیا واقعاً بین دو مقدمه ترم میانی معناداری وجود دارد یا نه. گاهی دو جمله از نظر واژگانی کلمات مشترک دارند، اما این اشتراک برای استدلال کافی نیست. مثلاً ممکن است در یک جمله «شیر جنگل» و در جمله دیگر «شیر آب» آمده باشد. واژه مشترک است، اما ترم میانی واقعی وجود ندارد؛ پس استدلال معتبر شکل نمی‌گیرد.

۱۱.۲۳ تفاوت Syllogism و Entailment

در Entailment ممکن است فقط یک گزاره داشته باشیم و با کمک دانش عمومی نتیجه‌ای بگیریم. برای مثال، اگر جمله‌ای درباره سیگار و آرسنیک باشد، ممکن است با دانش قبلی بدانیم آرسنیک ماده‌ای خاص است و از آن نتیجه‌ای بگیریم.

اما در Syllogism، دو مقدمه باید به واسطه یک ترم میانی به هم وصل شوند و نتیجه از ترکیب آن دو مقدمه به دست بیاید. پس هر نتیجه‌گیری زبانی الزاماً Syllogism نیست.

۱۲.۲۳ مغالطه‌ها و استدلال‌های روزمره

برای نشان دادن نقش prior knowledge و احساسات در استدلال، دو مثال روزمره مطرح شد.
مثال اول:

۱. دولت‌های فاسد تورم ایجاد می‌کنند.

۲. کشور ایران تورم دارد.

۳. پس دولت ایران فاسد است.

این نتیجه از نظر منطقی معتبر نیست، چون تورم می‌تواند علت‌های دیگری هم داشته باشد. این ساختار شبیه مغالطه تأیید تالی است. اما اگر فرد از قبل نسبت به دولت نارضایتی داشته باشد، نتیجه برای او پذیرفتنی به نظر می‌رسد. در اینجا prior knowledge و احساسات قبلی باعث می‌شوند System 1 فعال شود و فرد وارد بررسی دقیق System 2 نشود.

مثال دوم:

۱. هر کشوری که تولید ناخالص داخلی اش رشد کند، اقتصادش بزرگ‌تر شده است.

۲. طبق آمار جهانی، تولید ناخالص داخلی ایران رشد کرده است.

۳. پس اقتصاد ایران بزرگ‌تر شده است.

از نظر ساختار قیاسی، این نتیجه معتبر است. اما ممکن است فرد به دلیل تجربه شخصی یا وضعیت اقتصادی اطرافش آن را نپذیرد. اینجا نتیجه منطقی معتبر است، اما prior knowledge یا تجربه زیسته باعث مقاومت در برابر پذیرش آن می‌شود.

۱۳.۲۳ مقایسه مدل‌های زبانی و انسان در قیاس

در بخش پایانی، مقاله‌ای بررسی شد که مدل‌های زبانی را روی فرم‌های صوری Syllogism با انسان‌ها مقایسه کرده بود. مدل‌ها با فرم‌هایی مانند $\text{Some } B \text{ are } C, \text{ All } A \text{ are } B$ و حالت‌های مشابه آزمون شده بودند، نه الزاماً با زبان طبیعی پیچیده.

نتیجه مهم این بود که با بزرگ‌تر شدن مدل‌ها، دقت آن‌ها در برخی موارد از انسان بیشتر می‌شود. اما این به معنای هم‌ترازی کامل با انسان نیست. مدل‌های بزرگ ممکن است پاسخ درست‌تری بدهند، ولی توزیع خطاهایشان شبیه انسان نباشد. یعنی جاهایی که انسان‌ها اشتباه می‌کنند، مدل الزاماً اشتباه نمی‌کند، و جاهایی که مدل خطا دارد، الزاماً با خطاهای انسانی منطبق نیست.

پس عملکرد بهتر از انسان لزوماً به معنای فهم انسانی یا هم‌ترازی با انسان نیست. ممکن است مدل به دلیل حافظه بهتر، داده آموزشی بیشتر یا دیدن نمونه‌های مشابه در آموزش، پاسخ درست بدهد. این با فهم انسانی یکی نیست.

مدل‌های زبانی معمولاً با متن‌های درست، استدلال‌های رسمی و داده‌های گسترده آموزش می‌بینند. در نتیجه ممکن است فرم‌های درست را زیاد دیده باشند. اما انسان در جهان واقعی با مغالطه، ابهام، تجربه شخصی، بایاس و تصمیم‌گیری سریع روبه‌روست. بنابراین اگر مدل فقط در فرم‌های بهتر عمل کند، هنوز نمی‌توان گفت مثل انسان استدلال می‌کند.

۱۴.۲۳ حافظه، آمار و استدلال

در گفت‌وگوی پایانی، این سؤال مطرح شد که آیا مدل فقط حفظ کرده است یا واقعاً استدلال می‌کند. استاد تأکید کرد که «حفظ کردن» تعبیر دقیقی نیست. هم انسان و هم مدل به‌نوعی آماری یاد می‌گیرند. انسان نیز بر اساس تجربه‌های قبلی، priorها و توزیع‌های آماری جهان تصمیم می‌گیرد.

اما آمار مدل با آمار تجربه انسانی یکی نیست. اگر بخواهیم مدل واقعاً به انسان نزدیک‌تر شود، باید توزیع داده آموزشی و خطاهای آن را به سمت خطاها و الگوهای انسانی تنظیم کنیم.

یکی از راه‌ها Adversarial Data Augmentation است: ببینیم مدل در چه نوع نمونه‌هایی شکست می‌خورد، سپس همان نوع نمونه‌ها را بیشتر به داده اضافه کنیم تا مدل روی آن‌ها بهتر شود. در دیتاست Avicenna نیز چنین کاری انجام شده بود: فرم‌هایی که مدل در آن‌ها مشکل داشت شناسایی شدند و داده‌های مشابه آن‌ها تقویت شدند. در نتیجه عملکرد مدل بهتر شد، اما این بهبود صرفاً با افزایش تصادفی داده حاصل نشده بود؛ بلکه بر اساس تحلیل خطا انجام شده بود.

جمع‌بندی جلسه این است که برای فهم واقعی مدل، نباید فقط دقت نهایی را سنجید. باید دید آیا مدل در مسیر رسیدن به پاسخ، شبیه انسان عمل می‌کند یا نه. در استدلال قیاسی، انسان‌ها الگوی خطای خاصی دارند و این خطاها تصادفی نیستند؛ بلکه به prior knowledge، System 1، System 2، تجربه، بایاس و ساختار شناختی انسان مربوط‌اند.

اگر مدل در پاسخ نهایی از انسان بهتر باشد اما توزیع خطایش با انسان هم‌تراز نباشد، هنوز نمی‌توان گفت واقعاً با انسان aligned است. بنابراین در Brain Alignment علاوه بر پاسخ درست، باید به خطاهای انسان، خطاهای مدل، توزیع خطاها، توضیح یا مسیر استدلال انسان، لایه‌های میانی مدل، اثر prior knowledge و تفاوت میان reasoning صوری و reasoning در جهان واقعی توجه کرد.